# shAnalysis Workflow & Theory Guide

GRACE / GRACE-FO spherical-harmonic processing in base MATLAB — theory, best practices, and step-by-step recipes

| | |
|---|---|
| Toolbox version | shAnalysis v5.7.1 |
| Edition | 5 (adds: ICGEM time-series download, the standard chain and custom filter design as single entry points, and what real provider files actually look like — FORTRAN D-exponents, the ICGEM 2.0 column order, ragged record groups) |
| Scope | ICGEM I/O, spectral diagnostics, synthesis & analysis, destriping, Gaussian / DDK / tvANS filtering, VCE, basin averages & deconvolution, climatology & AR(1) significance, GIA, degree-1/C20 chain, Slepian localization, SINEX covariances, mascon comparison, load deformation, gradient tensor, multi-center combination, sea-level fingerprints, Love-number I/O incl. frame conversion, ICGEM series download, pole-tide conventions, pointwise synthesis at satellite altitude |
| Dependencies | base MATLAB only (no toolboxes); Java optionally for gz streaming |
| Validation | every numerical method independently validated in Python (numpy/scipy) before MATLAB implementation; 169 unit tests |
| License | MIT (see LICENSE; bundled DDK3 Wbd file MIT from its upstream repository; ITSG/GFZ fixtures remain their providers' property) |
| Provenance | Developed by Matthias Weigelt with the help of Claude (Fable 5), 2026-08-07 |

**How to read this guide.** Part I develops the theory exactly as implemented (equation forms match the code, including storage conventions and numerically validated tolerances). Part II condenses the best practices the implementation enforces or enables. Part IV is the complete API reference — every public function and method with typed, dimensioned inputs/outputs, defaults and a real-data example, generated from the source arguments blocks at build time. Part III gives copy-paste step-by-step recipes for the standard products. The HTML reference (`doc shAnalysis`) documents every function signature; this guide explains why and in which order.

# Installation and data

The toolbox itself needs nothing beyond base MATLAB. One call puts it on the path and, optionally, fetches the correction files the standard chain uses:

```
cd shAnalysis
setup_shAnalysis(Permanent = true, Download = "core")
% levels: "none" (path only) | "core" (TN-13/TN-14) | "filters" (+DDK)
%         | "starter" (+ a few ITSG months);  DryRun = true shows the plan
shLowLevel.makeTutorials(Cases = "core");   % Live Scripts to open and run
```

Downloads land under a persistent data folder (`shLowLevel.dataFolder`), set once with `DataFolder=`. The fetchers throttle themselves and verify every file by parsing it before it replaces anything, so an interrupted download is resumable and never leaves a corrupt file in place.

## Opt-in test data

Two checks need data far too large to ship as fixtures, so they read an environment variable and filter themselves out when it is unset. This is not tidiness: the fixture suite cannot see the class of failure that only appears on real data of real length — three such bugs were found the first time the toolbox was run against a full 24-year series, and the mascon reader's dimension order was only settled by a real product file.

| Variable | What it enables | Where to get the data |
|---|---|---|
| `SHX_SERIES_FOLDER` | the trend regression in `testScience`: a folder of monthly .gfc files, more than 24 epochs, GSM level | `shLowLevel.fetchITSG` or `fetchICGEM(Type="temporal")` |
| `SHX_MASCON_FILE` | the `readMascon` check against a REAL product — the synthetic fixture writes (lon, lat, time), the JPL file writes (time, lat, lon), and a size check passes either way | CSR (no login), GSFC, or JPL PO.DAAC (free Earthdata login) |

```
% set them for this session, or pass them to setup:
setup_shAnalysis(SeriesFolder = "D:/grace/itsg", ...
                 MasconFile = "D:/grace/GRCTellus_JPL.nc");
% for good, put the same two lines in your startup.m:
setenv("SHX_SERIES_FOLDER", "D:/grace/itsg");
setenv("SHX_MASCON_FILE", "D:/grace/GRCTellus_JPL.nc");
```

`SHX_SERIES_FOLDER` is deliberately NOT wired to the persistent data folder. A routine acceptance run must never reach for a network or archive drive, so pointing at that data is a decision the user makes explicitly. Neither variable is required: without them the suite runs complete and the two tests report themselves as filtered rather than passing silently.

# Part I — Theory as implemented

## 0. What GRACE measures, for readers starting from zero

Mass attracts. Every redistribution of water on and in the Earth — a monsoon soaking a river basin, an ice sheet shedding meltwater, groundwater pumped from an aquifer — changes the gravity field by a minute amount. The GRACE mission (2002–2017) and its successor GRACE-FO (2018–) measure exactly this: two satellites chase each other about 220 km apart on a near-polar orbit, and a microwave (GRACE) or laser (GRACE-FO) link measures their separation to micrometers or better. Flying toward extra mass, the leading satellite accelerates first and the gap stretches; the gap history maps the gravity field, month after month. The magic number to remember: converted to water, GRACE resolves changes of roughly a centimeter of water spread over a few hundred kilometers — enough to weigh continents breathing.

The monthly product is not a map but a set of a few thousand numbers: **Stokes coefficients**, the spherical-harmonic spectrum of the field (Section 1 explains the mathematics; for now: degree l is inverse wavelength, about 20000/l km, so degree 60 means ~330 km features). Everything in this toolbox starts from such coefficient files. The product vocabulary, once and for all:

- GSM — the monthly gravity solution, with atmosphere and ocean dynamics already subtracted by background models. GSM over land = water storage; GSM over the ocean = nearly nothing, ON PURPOSE.
- GAA/GAB/GAC/GAD — monthly means of exactly those subtracted background models (atmosphere / ocean / both / ocean bottom pressure). To study ocean mass you must ADD the right one back (Section 23); shLowLevel.fetchGAX provisions them.
- Monthly SINEX — the full normal-equation matrix of a solution (ITSG only), i.e. its complete error description; the key to the VDK filter (Sections 21–22). One month is ~460 MB.
- Daily Kalman solutions — low-resolution (n40) daily fields from ITSG, the basis of near-real-time flood tracking (Section 26).
- Mascons — an alternative representation on mass tiles with built-in regularization; readMascon reads CSR and JPL products.

Two truths shape all processing. First, the raw monthly fields are STRIPED: the along-track sampling makes north–south errors much smaller than east–west ones, and unfiltered maps drown in meridional stripes — hence the whole filtering industry of Sections 7–9 and 21. Second, low degrees are special: the satellites cannot see degree 1 (geocenter) at all and measure C20 poorly, so both are completed from external series (Section 16). Skipping either correction silently biases every trend you compute afterwards.

## 1. Spherical harmonics, normalization, storage

The disturbing potential is expanded in fully (4π-)normalized real spherical harmonics:

$$V(r,\vartheta,\lambda)=\frac{GM}{R}\sum_{n=0}^{N}\left(\frac{R}{r}\right)^{n+1}\sum_{m=0}^{n}\left[\bar{C}_{nm}\cos m\lambda+\bar{S}_{nm}\sin m\lambda\right]\bar{P}_{nm}(\cos\vartheta)$$

with the normalization $\int|Y_{nm}|^2\,d\Omega = 4\pi$, the ICGEM convention. Throughout the toolbox coefficients live in lower-triangular matrices with 1-based indexing `C(n+1, m+1)`; vectorized orderings are always mediated by `shLowLevel.shIndex` (fields n, m, cs, P; Nmin defaults to 2 for GRACE work, 0 for analysis). GM and R ride with the data (read from the gfc header); the defaults (3.986004415e14, 6378136.3) are overridable, never baked-in physics. Load Love numbers $k_n$ for water-storage kernels must be supplied explicitly — the toolbox refuses to guess them.

## 2. Legendre functions: scaled Holmes–Featherstone recursion

Fully normalized associated Legendre functions are computed by the standard order-diagonal seed plus degree recursion. The naive diagonal seed underflows near the poles at high degree; `shLowLevel.legendreALF` therefore implements the globally scaled Holmes–Featherstone variant (scale $10^{-280}$, rescaling on the fly), verified stable to degree 2190: the sum rule and parity

$$\sum_{m=0}^{n}\bar{P}_{nm}^{2}(\cos\vartheta)=2n+1\quad(\sim 10^{-11}\ \text{at}\ n=2190),\qquad \bar{P}_{nm}(-\varphi)=(-1)^{n+m}\,\bar{P}_{nm}(\varphi)$$

hold across all latitudes; the parity relation halves the recursion cost on hemispherically symmetric grids (the parity trick).

## 3. Synthesis

Spatial synthesis evaluates, per latitude, the order sums

$$A_m(\varphi) = \sum_{n=m}^{N} f_n \, \bar{C}_{nm} \, \bar{P}_{nm}(\varphi), \qquad B_m(\varphi) \text{ analogously with } \bar{S}_{nm},$$

then the longitude sum

$$g(\varphi, \lambda) = \sum_{m=0}^{N} [A_m(\varphi)\cos m\lambda + B_m(\varphi)\sin m\lambda].$$

For uniform full-circle longitude grids this last sum is an inverse FFT after folding aliased orders into mod(m, $n_{lon}$) bins — exact, validated against the direct product to 3e-13 (`Method="auto"|"fft"|"direct"`).

**Memory model (v2.2).** The Legendre stack ($n_{max}$+1)² × $n_{lat}$ × 8 bytes is the memory driver: 7 GB at $n_{max}$ 2190 over 181 latitudes. Above the 'MaxMemGB' budget (default 4), synthesis streams latitude bands: compute the Legendre block, fold into $A_m$/$B_m$, discard. Chunked equals monolithic exactly (Python-validated, max deviation 0.0); the class-level Legendre cache is bypassed automatically when the stack would not fit.

## 4. Quantity kernels

A single degree factor $f_n$ maps Stokes coefficients to the target quantity (`shLowLevel.kernelFactors`):

| quantity | $f_n$ | unit |
|---|---|---|
| geoid | $R$ | m |
| potential | $GM/R$ | m²/s² |
| gravity disturbance | $\frac{GM}{R^2}(n+1)$ | m/s² |
| gravity anomaly | $\frac{GM}{R^2}(n-1)$ | m/s² |
| EWH (water) | $R\,\frac{\rho_{ave}}{3\rho_w}\,\frac{2n+1}{1+k_n}$ | m |
| surface density * | $R\,\frac{\rho_{ave}}{3}\,\frac{2n+1}{1+k'_n}$ | kg/m² |
| bottom pressure * | $g_0 R\,\frac{\rho_{ave}}{3}\,\frac{2n+1}{1+k'_n}, \quad g_0 = \frac{GM}{R^2}$ | Pa |
| vertical deformation * | $R\,\frac{h'_n}{1+k'_n}$ | m |
| gravity gradient $T_{rr}$ * | $\frac{GM}{R^3}(n+1)(n+2)$ | 1/s² |

(* new in v2.3.) The 'Height' option continues potential-type kernels upward by $(R/r)^p$ (p = n+1 potential, n+2 anomaly/disturbance, n+3 $T_{rr}$; validated against -d/dr and d²/dr² of the continued potential) — evaluate directly at satellite altitude. Surface quantities reject Height by contract.

The EWH kernel divides by (1+$k_n$): the observed potential contains the solid-Earth loading response, which must be removed to recover surface mass. $k_n$ is always user-supplied (e.g. Wahr et al. 1998 or your loading-theory set of choice).

## 5. Elastic load deformation (v2.3)

A surface load deforms the elastic Earth; the displacement at the surface, expressed through the load-induced (residual) Stokes coefficients, is (Wahr et al. 1998):

$$u_{\text{up}} = R\sum_{n,m}\frac{h'_n}{1+k'_n}\,\Delta\bar{C}_{nm}\,\bar{Y}_{nm}, \qquad u_{\text{north}} = R\sum_{n,m}\frac{l'_n}{1+k'_n}\,\Delta\bar{C}_{nm}\,\frac{\partial\bar{Y}_{nm}}{\partial\varphi}, \qquad u_{\text{east}} = R\sum_{n,m}\frac{l'_n}{1+k'_n}\,\Delta\bar{C}_{nm}\,\frac{1}{\cos\varphi}\,\frac{\partial\bar{Y}_{nm}}{\partial\lambda}$$

The vertical is a pure degree factor ('deformation_up'); the horizontals need the exact spherical-harmonic gradient. `shLowLevel.legendreALFDeriv` provides dPbar/dϕ through a frozen identity — the (n,m) coefficient pattern (with √2 corrections at m=0 and m=1 from the normalization of $Pbar_{n0}$/$Pbar_{n1}$) was calibrated by least squares against numerical differentiation and then frozen; residuals sit at the finite-difference floor (~1e-9). `shLowLevel.shSynthesisDeformation` / `g.deformation` evaluate all three components on grids or GNSS station lists (Mode="points", LatType="geodetic"); north/east validated point-wise against brute-force gradients to ~5e-10. Degree 0 is excluded; degree 1 is opt-in and only meaningful with consistent geocenter (CF/CE frame) handling; all three Love-number sets must come from ONE loading model. East is NaN at the poles (undefined direction).

## 6. Analysis (the inverse problem)

`shLowLevel.shAnalysisGrid` / `shCoefficients.analysis` estimate coefficients from data. Two regimes:

- **Ring grids** (uniform full-circle longitudes, arbitrary latitude rings): an FFT along longitude decouples the normal equations per order; each order solves a small least-squares over latitude rings. Exact recovery of band-limited input to ~8.7e-15 when the sampling theorem is met ($n_{lat} \geq n_{max}+1$, $n_{lon} \geq 2n_{max}+1$).
- **Scattered points**: chunked accumulation of full normal equations. Under-determined or ill-conditioned samplings (polar gaps, regional patches) need Kaula regularization: a diagonal prior

$$\sigma^2_{\text{prior}}(n) = \left(\frac{K}{n^2}\right)^2$$

stabilizes the inversion at the cost of damping unconstrained coefficients toward zero. For regional data the mathematically cleaner route is Slepian localization (Section 12) — estimate only the ~N concentrated linear combinations the data can actually see.

## 7. Destriping and Gaussian smoothing

GRACE monthly solutions carry correlated north–south stripes: within one order and parity, coefficients of successive degrees correlate. The classic Swenson–Wahr destriper (`shLowLevel.shDestripe`) fits and removes a low-order polynomial across same-parity degree series per order (defaults: orders ≥ 8, polynomial order 2, window 5, all configurable). Gaussian smoothing (`shLowLevel.shGaussianFilter`) multiplies degree-wise by Jekeli weights from the recursion

$$b = \frac{\ln 2}{1-\cos(r/R)}, \qquad W_0 = 1, \qquad W_1 = \frac{1+e^{-2b}}{1-e^{-2b}} - \frac{1}{b}, \qquad W_{n+1} = -\frac{2n+1}{b}\,W_n + W_{n-1}$$

with the half-response radius r as the single tuning knob. Both are provided primarily as the baseline against which tvANS and DDK are judged.

## 8. DDK anisotropic decorrelation (v2.2)

The DDK filters (Kusche 2007; Kusche et al. 2009: DDK1–DDK8, ordered strong→weak) replace the scalar Gaussian weight by a full matrix per (order, C/S) block, derived from a GRACE error covariance and a signal prior — anisotropic, order-dependent smoothing that suppresses stripes with less signal attenuation than an equivalent Gaussian. `shLowLevel.readDDK` parses the released binary Wbd files natively (Rietbroek/Kusche BIN format; endian autodetect; verified to 4.4e-16 against the repository-documented DDK3 reference values; DDK1..8 = a_1d14p_4, 1d13, 1d12, 5d11, 1d11, 5d10, 1d10, 5d9; Nmax= truncates an Lmax-120 filter to n96 series), plus .mat containers and an ASCII exchange layout; `applyDDK` applies block-wise c → M·c, passing uncovered degrees through unchanged. Formal errors are invalidated by the filter (coefficients become correlated); propagate uncertainties with `shLowLevel.mcPropagate`.

**Note.** The real DDK3 file (Wbd_2-120.a_1d12p_4, MIT-licensed, from github.com/strawpants/GRACE-filter) ships in tests/test_data; the unit tests pin the parser to the repository-documented pack values. Degrees below the filter's Lmin (2) pass through unfiltered, as in the reference implementation.

## 9. The tvANS filter (time-variable anisotropic noise/signal Wiener filter)

The toolbox's flagship decorrelation filter. Model per month t: x = signal + noise with signal covariance S (estimated from the data themselves) and noise covariance $s_t$N (a shape N, e.g. from a SINEX covariance or the built-in order/parity model, scaled monthly by VCE factors $s_t$). The Wiener estimate of the residual field is

$$\hat{x}_t = W_t\, x_t, \qquad W_t = S\,(S + s_t N)^{-1}$$

Implementation: one generalized symmetric eigendecomposition gives, for every month simultaneously,

$$SU = NU\Lambda \quad \Rightarrow \quad W_t = V\,\mathrm{diag}\!\left(\frac{\lambda_i}{\lambda_i + s_t}\right)U^{\mathsf{T}}, \qquad V = U^{-\mathsf{T}}$$

— O(P²) per month instead of a fresh P³ solve. The deterministic model (bias/trend/annual/semi-annual) is removed first and restored after filtering; VCE factors are estimated from high-degree residuals via a robust median $\chi^2$ estimator (`shLowLevel.vceRescale`). With a block-diagonal N (order/parity blocks), the whole machinery runs per block (Blocks="auto"), tractable to Lmax ≈ 120. External noise models join the block path when they are block-diagonal in the (order, C/S, parity) partition (verified, not assumed — v2.2.2); dense covariances fall back to the full path (auto) or error loudly (on).

**Per-order-band VCE (v2.2).** One $s_t$ per month is coarse: striping strength varies with order. VCEBands=[0 16 33 61] estimates independent monthly factors per order band (block path only — band-wise scaling breaks the single global eigendecomposition, which the code asserts). The per-block factors are honored consistently in the filter

application, the posterior sigmas, and the basin deconvolution covariance (identity of the banded block filter with the directly constructed W validated to 5.6e-16).

**Posterior uncertainty.** The filtered-residual variance

$$\mathrm{diag}((I - W_t)\,S) = (V \circ V)\,\tfrac{s_t \lambda}{\lambda + s_t}$$

(validated 3.5e-16) is exposed as per-coefficient/month sigma stacks (`tsF.sigmaCs/Ss`). With hard constraints (v2.5) the EXACT diagonal of (W–I)S(W–I)' + s·WNW' is used (O(P²q) in the eig basis, validated 2e-15 against the brute-force covariance; the constrained-direction identity Ac'·Cov·Ac = s·Ac'NAc holds exactly). Honest correction: the earlier formula UNDERESTIMATED along constrained directions (ratios down to ~0.72) — the pre-v2.5 'upper bound' note had the direction wrong.

## 10. Basin averages and deconvolution

A basin kernel b (Section 11) gives the filtered basin series $a_t = b^T x_t / (b^T b)$. Because filtering attenuates and leaks, `shLowLevel.basinDeconvolve` solves the small linear system that undoes the filter's action on a set of basin kernels B:

$$c_t = A^{-1} B^T \hat{x}_t, \quad A = B^T W_t B \;\; (K \times K); \qquad \mathrm{cov}(c_t) = A^{-1}\left[G_v^T\,\mathrm{diag}(s_{\mathrm{vec}}\, g^2)\, G_v\right] A^{-T}$$

where g are the spectral gains and $s_{vec}$ the (possibly band-wise) monthly noise factors — the v2.2 form is exactly the v2.1 formula when bands are off. Ill-conditioned kernel sets accept a relative ridge (λ = Ridge·||A||), with the covariance consistently using the regularized inverse. Since v2.5 the basin sigma also carries the OLS parameter uncertainty of the restored deterministic part (design leverage $h_t$ × per-coefficient residual variance, carried on the operator; Monte-Carlo: emp/pred 1.18 without the term, 1.00 with it, 1.26 → 1.08 at seasonal leverage peaks), and with constraints the deconvolution noise covariance uses the exact constrained operator. The deterministic-part errors remain correlated across epochs through the shared fit (pointwise sigma; documented).

## 11. Basin kernels: buffering and tapering (v2.2)

`shLowLevel.basinKernel` builds $b = Y^T(w \cdot \mathrm{mask})$ by exact Gauss–Legendre quadrature from a polygon, mask, or f(lat,lon) handle. Two leakage controls:

- **BufferKm**: grow/shrink the region by exact great-circle distance before quadrature (trade leakage-in against leakage-out; computed in latitude chunks, practical to Lmax ~ 96).
- **TaperKm**: multiply the kernel spectrally by Jekeli Gaussian weights — soft edges suppress the ringing of the truncated indicator. Rule of thumb: taper radius ≈ the coefficient filter's half-response radius.

The quadrature area fraction is returned; b(1) equals it exactly ($Y_{00}$ = 1), which the tests pin down.

## 12. Slepian localization (v2.2)

For a region R, the concentration problem maximizes the energy ratio over band-limited g = Yx:

$$\lambda = \frac{\int_R g^2\, d\Omega}{\int_\Omega g^2\, d\Omega} \;\rightarrow\; \max, \qquad K = Y^T \mathrm{diag}(w \cdot \mathrm{mask})\, Y, \qquad N_{\mathrm{Shannon}} = \sum_i \lambda_i = \tfrac{A_R}{4\pi}\, P$$

Eigenvectors (Slepian tapers) are orthonormal; eigenvalues are concentrations in [0,1]; the Shannon number (exact by the addition theorem — the trace identity the tests verify) counts the tapers the region supports. Regional analysis then estimates ~N Slepian coefficients $a = G^T x$ instead of P Stokes coefficients — well-posed by construction, no Kaula prior needed. Validated on the 30° polar cap: λ bounds to 4e-16, orthonormality 3e-15.

## 13. Climatology, alias periods, AR(1) significance (v2.2)

`ts.climatology()` fits bias, trend, annual and semi-annual terms (plus optional extra periods) per coefficient. GRACE tidal aliasing puts S2 at 161 d, K2 at 3.66 yr, K1 at 7.48 yr: Periods=[161/365.25, 3.66, 7.48] keeps these out of trend and semi-annual estimates. Per-coefficient 1-sigma significance comes from the OLS covariance (weighted and robust variants available) and rides on every component accessor, enabling significance-masked maps.

**AR(1) correction.** GRACE residuals are temporally correlated; white-noise sigmas are optimistic. ARCorrect=true estimates the lag-1 residual autocorrelation $r_1$ per coefficient and inflates sigmas by

$$\sigma_{\mathrm{AR(1)}} = \sigma\,\sqrt{\tfrac{1 + r_1}{1 - r_1}}$$

— the standard first-order effective-sample-size correction, since v2.5 applied to the Kendall bias-corrected $r_1' = r_1 + (1+3r_1)/T$. Monte-Carlo validation: uncorrected sigmas underestimate the empirical trend scatter by 2.00× ($\phi = 0.6$, $T = 120$); with the raw-$r_1$ inflation the ratio is 1.065, with the bias-corrected $r_1$ 1.028 (1.152 → 1.079 at T = 60). Still first-order and slightly conservative — the residual few percent stems from the regression absorbing low-frequency noise.

## 14. GIA correction (v2.2)

Secular Stokes trends mix present-day mass change with glacial isostatic adjustment. `ts.removeGIA(gia)` subtracts rate·(t - $T_0$) per epoch; `clim.removeGIA(gia)` corrects a fitted trend directly — both routes agree identically (tested to 1e-13). GIA models (ICE-6G_D, Caron et al., A/W13, ...) are user-supplied gfc rate files, never embedded. The model is treated as exact: its uncertainty, often the dominant regional trend error (Laurentia, Fennoscandia, Antarctica), is not propagated — difference several models as a spread estimate.

## 15. Geocentric vs. geodetic latitude (v2.2)

All SH mathematics here runs in geocentric latitude; maps, GIS products and mascon grids are geodetic:

$$\tan\varphi' = (1-f)^2 \tan\varphi$$

with the WGS84 flattening as overridable default. The difference peaks at 0.192° (~21 km) near 45° — large enough to visibly bias basin averages and point comparisons. Synthesis, analysis and shAnalysisGrid accept LatType="geodetic" and convert internally.

## 16. The degree-1 / C20 completion chain

GRACE does not observe geocenter motion (degree 1), and its C20 is weak: the standard product chain is GSM + TN-14 (C20/C30 from SLR) + TN-13 (degree-1). `shLowLevel.readTN14` and `shLowLevel.readTN13` parse the official technical notes (TN-13 verified against the real GFZ RL06 file: 256 paired months 2002-04 to 2026-04, including the 2017.4–2018.4 gap); `ts.addDegree1` and the C20/C30 replacement close the chain with epoch matching and provenance notes in the history.

## 17. Normal field and (GM, R) conventions (v2.4)

Geoid maps need the DISTURBING field: subtract the ellipsoidal normal potential. `shLowLevel.normalFieldCS` computes the even zonals from the WGS84/GRS80 defining constants (GM, a, f, ω) via the Heiskanen–Moritz closed form ($q_0$ series → $J_2$ → $J_{2n}$) — no coefficient tables; validated against NIMA TR8350.2 to all published digits. Crucially, WGS84 GM (3.986004418e14) and a (6378137.0) DIFFER from the ICGEM conventions (3.986004415e14, 6378136.3): `g.subtractNormalField` rescales the ellipsoid field to the model's (GM, R) first via

$$\bar{C}'_{nm} = \bar{C}_{nm}\,\frac{GM_1}{GM_2}\left(\frac{R_1}{R_2}\right)^n$$

(`shLowLevel.rescaleGMR` / `g.toReference`; the physical field is invariant — verified by synthesizing the potential at a common radius from both representations). Skipping the rescaling costs millimetres. The arithmetic operators refuse GM/R-mismatched operands (shCoefficients:operands:constantsMismatch) instead of silently mixing conventions. Permanent tide: the ellipsoid is tide-free by construction; the model's C20 tide system (ICGEM header) is NOT converted silently — zero-tide vs tide-free differ by ~4.2e-9 in $\bar{C}_{20}$.

## 18. Multi-center combination (v2.4)

Several centers observe the same monthly field with different noise. One variance factor per (center, month) — the COST-G granularity — estimated by Foerstner iteration with PARTIAL redundancies:

$$\hat{x} = \left(\sum_c w_c N_c^{-1}\right)^{-1} \sum_c w_c N_c^{-1} x_c, \qquad s_c^2 = \frac{v_c^{\mathrm{T}} N_c^{-1} v_c}{r_c}, \qquad r_c = P - \mathrm{tr}\left(H\, w_c N_c^{-1}\right)$$

Without $r_c$ the factors bias low ($v_c$ is correlated with x■). Noise shapes $N_c$ come from each center's own climatology residuals as (order, C/S, parity) blocks; the whole iteration runs block-wise. Robust=true replaces the global ratio by the median of per-block unbiased estimates $q_b/r_b$. The empirical shapes are trace-normalized, so the factors carry each center's absolute noise power and are directly comparable (weights invariant; v2.4.1). Two limits are structural: common-mode errors (shared K-band data, AOD1B, tides) are invisible to inter-center VCE — posterior sigmas are a LOWER bound — and inter-center correlations violate independence; info.interCenterCorr quantifies the optimism. Combine GSM before TN-14/TN-13, then apply the replacements once.

## 19. Elastic sea-level fingerprints (v2.4)

A melting ice mass does not raise the ocean uniformly: the ocean surface follows the perturbed geoid minus the deformed crust, under global mass conservation (Farrell & Clark 1976, elastic limit):

$$S = \mathcal{O}\,(N - U + \kappa), \qquad \int_{\text{ocean}} \rho_w\, S\, dA = -\int_{\text{land}} \sigma\, dA$$

`shLowLevel.seaLevelFingerprint` iterates this on the toolbox's exact Gauss–Legendre quadrature pair. Validation: mass conserved to machine precision, ~11 iterations, and the classic pattern — sea level FALLS near the melting mass (lost self-attraction plus rebound; validation run: near field -0.3 mm vs far field +0.8 mm about a +0.6 mm eustatic mean per unit load). Elastic only, no rotational feedback, fixed coastlines.

## 20. Gravity gradient tensor (v2.4)

`shLowLevel.shSynthesisGradientTensor` synthesizes all six NEU components at altitude; second Legendre derivatives come from the frozen first-derivative stencil applied twice (exact identity). Every call self-checks the Laplace invariant trace(G) = 0, validated at 7e-16 of max|G|; $G_{uu}$ is bit-consistent with the 'gravity_gradient_rr' kernel route. 1 Eotvos = 1e-9 $s^{-2}$.

## 21. Time-variable decorrelation: the VDK/VADER filter (v3.13)

Sections 8–9 introduced the Wiener family: damp each coefficient by how much you trust it. The DDK filter fixes ONE synthetic error model for the whole mission; tvANS estimates one empirical structure from the series itself. Both assume the error STRUCTURE never changes — but it does: orbit height decays, ground tracks enter and leave short repeat cycles, instruments age. The VDK filter (Horvath et al. 2018; VADER in the paper) uses the actual monthly normal-equation matrix N from the ITSG SINEX:

$$x_\alpha = (N + \alpha M)^{-1} N x$$

with M the inverse signal variance (a Kaula rule $al^b$ per calendar month, estimated by signalVarianceKaula with an exact log-chi-square bias correction) and $\alpha$ the strength knob. With $S = M^{-1}$ this is ALGEBRAICALLY the tvANS form $S(S+\alpha N^{-1})^{-1}$ — same family, different inputs, and the difference is worth 15% median error and an order of magnitude in bad months (paper Table 2). The price: the one-eigendecomposition trick of tvANS requires a stationary structure, so VDK pays one Cholesky factorization per month instead (seconds at n96). vdkApply never forms the filter matrix; tools/dev/run_vdk_series.m drives the archive-scale batch.

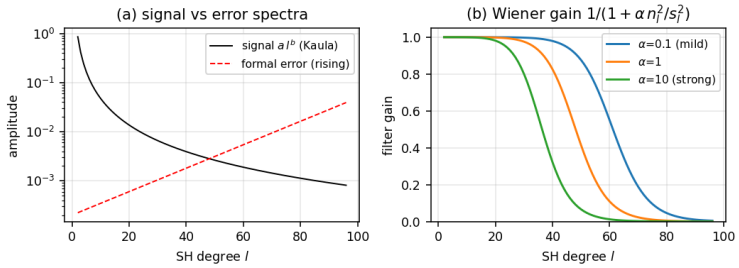



Figure T4 — the Wiener principle behind DDK, tvANS and VDK (computed illustration). (a) Signal follows a falling Kaula law while GRACE errors rise with degree; (b) the resulting gain damps degrees where noise wins. VDK recomputes the gain every month from the measured N.

```
Idx= shLowLevel.shIndex(96, Nmin= 2);
f = shLowLevel.fetchITSGSINEX("2018-06", Nmax = 96);   % ~460 MB, deliberate
snx = shLowLevel.readSINEX(f(1), Index = idx);     % N + x in idx order
[ab, ~] = shLowLevel.signalVarianceKaula(ts.Cs, ts.Ss, ts.epochs);
xf = shLowLevel.vdkApply(snx.x, snx.M, idx.n, ab(6, :), Alpha = 1);
```

## 22. Normal equations and SINEX, demystified

A monthly solution is a least-squares estimate: the satellites provide millions of range observations, the unknowns are the ~9400 Stokes coefficients (n96, degrees 2–96), and the NORMAL-EQUATION MATRIX $N = A^T P A$ condenses the whole adjustment: its inverse is the error covariance, its structure encodes which coefficient combinations that month's orbit could or could not see. SINEX is simply the exchange format that ships x, N and metadata in labelled blocks; the ITSG monthly files carry SOLUTION/NORMAL_EQUATION_MATRIX as a packed triangle — hence 460 MB. readSINEX streams the blocks and reorders everything onto a shIndex layout, verified against exactly this product. COST-G distributes no per-month SINEX (their combination happens internally), so ITSG is the only public source.

## 23. Ocean mass: restoring GAX, barystatic sea level and OBP (v3.10–3.11)

Because GSM has ocean dynamics removed by construction, the honest recipe for ocean-mass questions is: restore what was subtracted, then average. oceanChain adds the monthly GAD field (ocean bottom pressure) back, subtracts GAA over the ocean to avoid double-counting atmosphere, filters, masks ($|\text{lat}| \le 66°$ away from the crude coasts) and area-averages to the barystatic curve — the ocean-mass part of sea-level rise, +1.41±0.03 mm/yr in the shipped acceptance on COST-G (published range ~1.6–2.2; Section 24 explains the gap's main suspect). obpChain keeps the FIELDS instead (GravIS OBP recipe: GAD before the filter, reference period 2002/04–2020/03, land NaN). The pixel residuals still contain real ocean circulation; SeparateCirculation splits them by EOF with a Marchenko–Pastur significance limit — modes are only removed where they rise above what pure noise eigenvalues could produce.

```
[out, rep] = shLowLevel.oceanChain(ser, kn = kn, OceanMask = oc, ...
    GADFolder = gadF, GAAFolder = gaaF, SeparateCirculation = true);
[out.trend, out.nModes, out.sigMon]
```

## 24. Coastal leakage: buffers and support separation (v3.15)

A band-limited field has no sharp edges: truncate any coastline at degree 60–96 and it rings; filter it and land signal smears hundreds of kilometers into the ocean. Greenland alone biases the North Atlantic mean visibly. Two controls, both quantified before they were built. CoastBufferKm erodes the ocean mask by a great-circle distance (shLowLevel.erodeMask): for gauss445, 300 km removes 54% of a coastal point-source leak, 500 km 82%, at under 1% ocean-area loss. RemoveLandLeakage attacks the cause: separate the field into land-supported and ocean-supported band-limited components by two-sided alternating projections (Papoulis–Gerchberg style) and subtract the land part BEFORE filtering. The pre-validation taught a hard lesson worth stating in bold: a naive ONE-step subtraction makes the leak WORSE, because the reconstruction of the outside field rings INTO the mask. The two-sided iteration (LandLeakIter = 5) cuts an adjacent-source leak of 47% of the ocean mean by 94%.

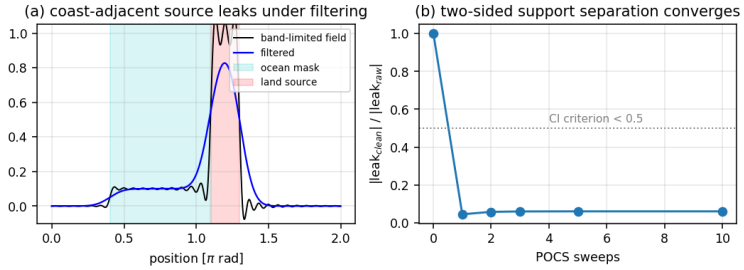



Figure T5 — the 1D reference experiment behind the v3.15 leakage controls (computed, identical mathematics to the sphere). (a) A band-limited land source adjacent to the ocean mask leaks under filtering; (b) two-sided support separation converges within a few sweeps to a small fraction of the raw leak — the CI test freezes the <0.5 criterion, the converged value is ~0.06.

## 25. Hydrological extremes: standardized indices (v3.14)

A TWS anomaly of –12 cm means nothing by itself: is that a dry month for THIS place and season, or business as usual? Standardized indices answer by comparing against the local seasonal climatology. The GRACE-DSI (Zhao et al. 2017) is the z-score per calendar month, binned into the 11 USDM-style classes D4 (exceptional drought, z ≤ –2) through W4; it agrees with USDM, PDSI, SPEI, NDVI and in-situ groundwater in the validation literature — while seeing the FULL storage column including groundwater, which no precipitation-based index does. Two policies are measured, not assumed: DETREND FIRST (a 0.5 cm/yr GIA or depletion trend weakens a planted exceptional-drought month from –2.41 to –1.44 — out of its class — and turns 45% of the late decade spuriously wet), and a robust sigma option (one corrupt month inflates the classical sigma sixfold; the 1.4826·MAD not at all). For floods, the causal Reager storage deficit $S_{def}(t)$ = max of the past minus the last state measures how much water the basin can still take; with a precipitation grid it becomes the full Flood Potential Index behind multi-month lead times (Reager et al. 2014).

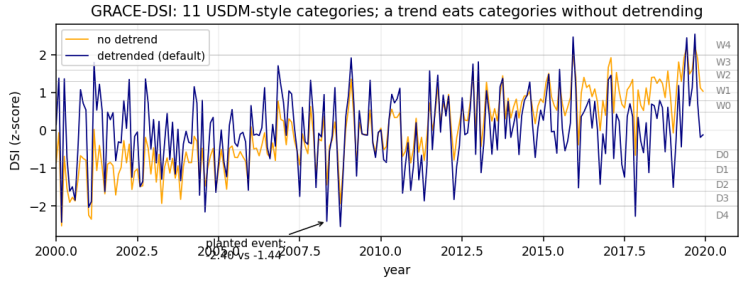



Figure T7 — the DSI construction on the pre-validation experiment (computed). The planted –2.5σ event keeps its D4 class only when the trend is removed first; the class edges are the USDM convention.

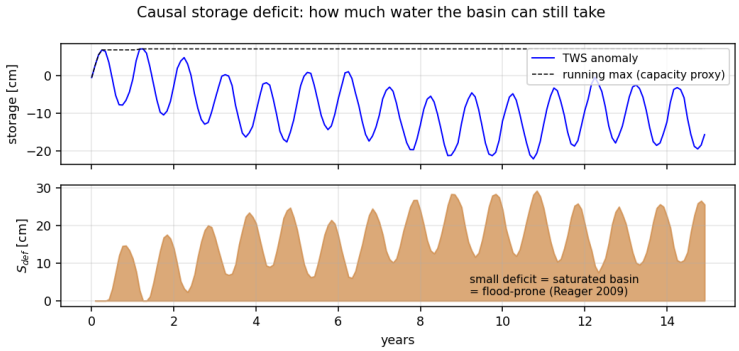



Figure T8 — the causal storage deficit (computed illustration): zero at a running maximum, growing as the basin dries; small deficit means flood-prone.

```
[tws, rep] = shLowLevel.twsChain(ser, kn = kn);
[Z, ih] = shLowLevel.hydroExtremeIndex(tws.grid, tws.epochs);
ih.categoryNames(ih.category(iLat, iLon, end) + 6)
```

## 26. Daily solutions and near-real-time monitoring (v3.14 stage 2)

Monthly fields cannot see a two-week flood wave. The ITSG daily Kalman solutions (n40, one gfc per day, fetchITSG Product = "daily") can — Gouweleeuw et al. (2018) tracked major Ganges–Brahmaputra floods this way. hydroExtremeIndex switches automatically to a day-of-year climatology on daily input: means per exact DOY, sigma from the RESIDUALS in a circular 31-day window that wraps December into January, with the sqrt(n/(n-1)) correction for the estimated mean. The construction is measured, not assumed: window sigma on RAW values leaks the seasonal cycle (1.99 instead of 1.5), the two-stage form does not; a planted +3σ January-2nd flood scores z = 2.9 straight across the year boundary.

## 27. Temporal smoothing: Kalman filtering with VAR process models (v3.21–3.26)

Every method so far treats months independently. But mass redistribution has memory: the field of one epoch predicts the next. Kurtenbach (2012) and Kvas (2019) built the ITSG daily solutions on exactly this idea — a vector-autoregressive (VAR) process model inside a Kalman filter — and the module implements their chain end to end: estimateVAR, kalmanFilter, rtsSmoother, buildCondFun, neqCombine, with kalmanChain as the single point of access.

### The process and observation model

The reduced (climatology-removed) coefficient vector $x_t$ follows a VAR(p); observations are either solutions with covariance (l = x + v) or normal equations (N x = b) consumed in information form. For p = 1 the Yule–Walker solution is Kurtenbach's closed form:

$$B = \Sigma(1)\,\Sigma(0)^{-1}, \qquad Q = \Sigma(0) - B\,\Sigma(0)\,B^{\mathsf{T}}$$

Higher orders embed into the companion form; the filter starts stationary (x■■ = 0, P■■ = Σ(0)), which makes the forward filter plus RTS smoother an EXACT solver of the joint block-tridiagonal least-squares adjustment over all epochs (Kvas Sec. 2.3) — pinned in the test suite to 1e-15 (Python) and 1e-9 (MATLAB) against the explicitly built batch system. Gap months become prediction-only and relax toward the prior at the rate the estimated dynamics dictate.

### Choosing the process model: measured, not assumed

The first live VAR table (ITSG monthly n40, T = 257, 36-month holdout, one-step coefficient RMS, computed through the shipped estimator) settled the defaults:

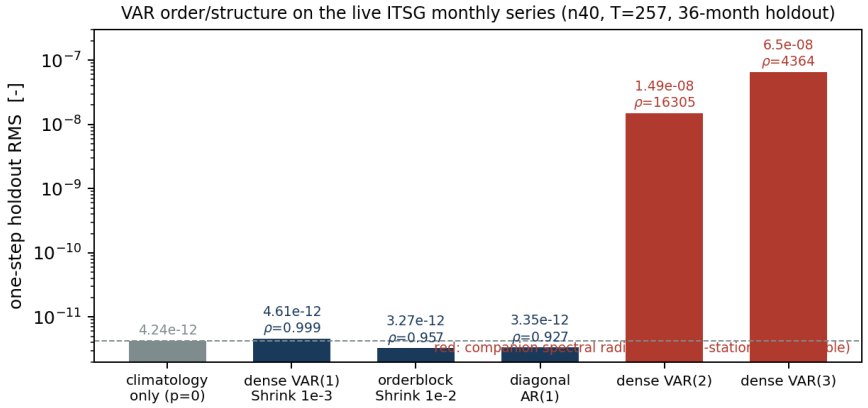



Figure: one-step holdout RMS per process model on the live monthly series. The dense VAR(1) at weak shrink is WORSE than no dynamics at all (overfit at T ≈ 0.13 P); the structured estimators beat climatology-only by 21–23%; p ≥ 2 explodes. Order = 1 confirmed.

Consequence in the API: estimateVAR Structure = "full" | "diagonal" | "orderblock". For MONTHLY series use "orderblock" (one block per order m and C/S — kalmanChain builds the partition itself) or "diagonal"; the dense default targets the DAILY regime, where Kurtenbach's production choice is dense plus spatial conditioning (buildCondFun: the Kvas EWH-domain land/ocean masks with the exp(–ψ/ψ■) taper, PSD by the Schur product theorem — it turns a singular empirical Σ(0) into a usable one, measured rcond 0 → 5.7e-2).

### What the filter does to real data

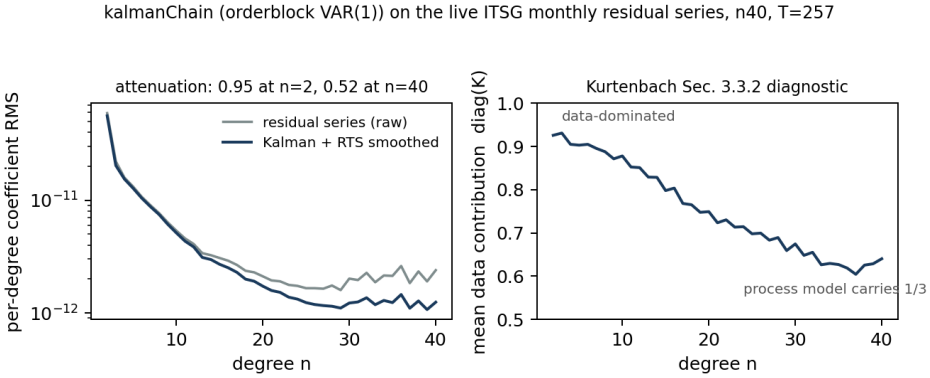



Figure: the live run on the monthly residual series (orderblock VAR(1), formal sigmas as R). Left: the smoother leaves the signal-dominated low degrees at 95% and halves the noise-dominated n = 40 band. Right: Kurtenbach's contribution diagnostic diag(K) — the share of each estimate coming from the DATA; the process model carries a third of the estimate at high degrees.

Honest scope note: an 11°-cap basin average of the same run changes by 2% RMS only (measured ratio 0.979) — basin averaging is itself a low-pass, so on monthly data the module's value sits at coefficient level, in the propagated sigmas, and in the machinery below. The DAILY regime it targets is different: there the process model is the difference between unusable and usable fields (Kurtenbach), and a live acceptance against the published ITSG daily series is on the roadmap, waiting on public daily normal equations.

### Robustness machinery, each decided on measurement

- Innovation QC (Kvas Sec. 3.3, v3.21): per-epoch chi-square test T = d'S■¹d (solution form, dof = P) or u'(N P■ N + N)■ u (NEQ form, dof = rank(N)); identical when N = R■¹. QC = "reject" turns a failing epoch into prediction-only — essential in a RECURSIVE estimator, where one accepted blunder propagates into every later state (test: rejecting one 50-sigma epoch cuts downstream error by > 20%). Thresholds from the base-MATLAB Wilson–Hilferty quantile, accuracy measured against scipy.
- Joseph-stabilized update (v3.22): decided on a 50-digit-reference A/B, not textbook habit — with R small against a wide-spectrum prior (the strong-daily-data regime) the standard (I–K)P update loses six orders of covariance accuracy (1.3e-6 vs 1.9e-13). Exact-arithmetic identical otherwise.
- Multi-center (v3.23): a string array of SINEX folders is grouped by epoch and combined by neqCombine (per-epoch VCE or fixed weights) BEFORE the update; rep.sigma2 reports the factors.

• Bounded memory and latency (v3.19/v3.24): StoreCov = "matfile" keeps year-long full-covariance runs flat in RAM (bit-identical to in-memory); rtsSmoother Lag = L bounds the latency for near-real-time production (Lag = 0 is the filter, Lag ≥ T−1 the full smoother to rounding). With QC = "reject" this is the complete bounded-latency chain; the daily output feeds hydroExtremeIndex directly (chapter 26).

**The one-call form**

```
Idx= shLowLevel.shIndex(40);
ts   = shSeries.fromFolder(folder, Truncate = 40);
[tsK, rep] = shLowLevel.kalmanChain(ts, ModelSeries = ts, ...
    Structure = "orderblock", Shrink = 1e-2, QC = "reject");
plot(rep.epochs, rep.qcStat); yline(shLowLevel.chi2Quantile(0.999, idx.P));
```

For SINEX normal equations replace the first argument by the folder (or a string array of folders, one per center); kalmanChain reads, epoch-groups, VCE-combines and filters in information form, and returns the smoothed shSeries with propagated sigmas plus the full report (model, contribution, QC record).

## 28. References


• Chambers, D.P., Willis, J.K. (2010): A global evaluation of ocean bottom pressure from GRACE, OMCT, and steric-corrected altimetry. J. Atmos. Oceanic Technol. 27, 1395–1402.

• Cheng, M., Tapley, B.D., Ries, J.C. (2013): Deceleration in the Earth's oblateness. J. Geophys. Res. Solid Earth 118, 740–747.

• Gerchberg, R.W. (1974): Super-resolution through error energy reduction. Optica Acta 21, 709–720. (With Papoulis 1975, the support-projection principle behind Section 24.)

• Gouweleeuw, B.T. et al. (2018): Daily GRACE gravity field solutions track major flood events in the Ganges–Brahmaputra Delta. Hydrol. Earth Syst. Sci. 22, 2867–2880.

• Horvath, A., Murboeck, M., Pail, R., Horwath, M. (2018): Decorrelation of GRACE time variable gravity field solutions using full covariance information. Geosciences 8, 323.

• Jekeli, C. (1981): Alternative methods to smooth the Earth's gravity field. Rep. 327, Dept. of Geodetic Science, Ohio State University.

• Kusche, J. (2007): Approximate decorrelation and non-isotropic smoothing of time-variable GRACE-type gravity field models. J. Geodesy 81, 733–749.

• Kurtenbach, E. (2012): Entwicklung eines Kalman-Filters zur Bestimmung kurzzeitiger Variationen des Erdschwerefeldes aus Daten der Satellitenmission GRACE. DGK Reihe C, Nr. 683, Bonn. (The VAR(1) closed form and the contribution diagnostic of chapter 27.)

• Kvas, A. (2019): Estimation of high-frequency mass variations from GRACE inter-satellite ranging. PhD thesis, TU Graz. (The batch-equivalence, innovation QC and EWH conditioning of chapter 27.)

• Kvas, A. et al. (2019): ITSG-Grace2018: overview and evaluation of a new GRACE-only gravity field time series. J. Geophys. Res. Solid Earth 124, 9332–9344.

• Mayer-Guerr, T. et al.: ITSG GRACE/GRACE-FO monthly, daily and SINEX products. TU Graz, ftp.tugraz.at/pub/ITSG/GRACE.

• Papoulis, A. (1975): A new algorithm in spectral analysis and band-limited extrapolation. IEEE Trans. Circuits Syst. 22, 735–742.

• Reager, J.T., Famiglietti, J.S. (2009): Global terrestrial water storage capacity and flood potential using GRACE. Geophys. Res. Lett. 36, L23402.

• Reager, J.T., Thomas, B.F., Famiglietti, J.S. (2014): River basin flood potential inferred using GRACE gravity observations at several months lead time. Nature Geosci. 7, 588–592.

• Sasgen, I. et al. (2012): Timing and origin of recent regional ice-mass loss in Greenland. Earth Planet. Sci. Lett. 333–334, 293–303.

• Sinha, D., Syed, T.H., Famiglietti, J.S., Reager, J.T., Thomas, R.C. (2017): Characterizing drought in India using GRACE observations of terrestrial water storage deficit. J. Hydrometeorol. 18, 381–396.

• Swenson, S., Wahr, J. (2002): Methods for inferring regional surface-mass anomalies from GRACE. J. Geophys. Res. 107, 2193.

• Swenson, S., Wahr, J. (2006): Post-processing removal of correlated errors in GRACE data. Geophys. Res. Lett. 33, L08402.

• Swenson, S., Chambers, D., Wahr, J. (2008): Estimating geocenter variations from a combination of GRACE and ocean model output. J. Geophys. Res. 113, B08410.

• Wahr, J., Molenaar, M., Bryan, F. (1998): Time variability of the Earth's gravity field. J. Geophys. Res. 103, 30205–30229. (The EWH kernel of Section 4.)

• Zhao, M., Geruo, A., Velicogna, I., Kimball, J.S. (2017): A global gridded dataset of GRACE drought severity index. J. Hydrometeorol. 18, 2117–2129.

Data providers to cite alongside the methods: ITSG (TU Graz), COST-G, GFZ/GravIS, CSR and JPL (mascons), ICGEM as the archive. The shipped acceptance numbers (–224.6 Gt/yr Greenland vs the published –231.1; +1.41 mm/yr barystatic) are reproduced against these products with the exact recipes of this guide.

# Part II — Best practices

## Processing chain order

The order below is not arbitrary; each step assumes the previous ones:

- **1.** Read GSM monthlies (shSeries.read, gz transparent, ICGEM 1.0/2.0 incl. gfct).
- **2.** Replace C20 (and C30 in the accelerometer-degraded years) from TN-14; add degree-1 from TN-13 (addDegree1). Do this BEFORE filtering — filters see the stripes, not the low-degree completions.
- **3.** Bridge the GRACE↔GRACE-FO gap explicitly: dropNaN / select; never fit a climatology across NaNs.
- **4.** Filter: tvANS (or DDK for benchmarking; Gaussian as baseline).
- **5.** Remove GIA (trend work only).
- **6.** Fit climatology with tidal alias periods and ARCorrect=true; or form basin series with deconvolution.
- **7.** Synthesize maps / averages; propagate sigmas (analytic where derived, mcPropagate everywhere else).

## Choosing a filter

| situation | recommendation |
|---|---|
| global EWH maps, standard use | tvANS (Blocks="auto"); Gaussian 300 km only as sanity baseline |
| cross-paper comparability | add DDK3-DDK6 variants via readDDK (binary Wbd, Nmax=96) + applyDDK |
| strong order-dependent striping | tvANS with VCEBands=[0 16 33 61] (v2.2) |
| small basins (< ~200,000 km²) | expect leakage; taper kernels, consider basinDeconvolve with several neighboring kernels |
| regional studies | Slepian basis instead of filtering a global field (Part III, G4) |

## Uncertainty accounting — what is honest and what is approximate

- Formal per-coefficient sigmas (readers) → exact as published.
- tvANS posterior sigmas → exact for the unconstrained AND (v2.5) the constrained filter.
- Basin sigmas → noise part consistent with (banded) VCE and the exact (constrained) operator; deterministic-fit parameter uncertainty included since v2.5 (pointwise; epoch-correlated through the shared fit).
- Climatology sigmas → OLS/weighted/robust; AR(1) correction first-order with Kendall-corrected $r_1$ (v2.5; residual ~3% optimism from regression absorption).
- GIA → model treated exact; use model spread.
- Multi-center combination → posterior sigmas are a LOWER bound (common mode invisible; see info.interCenterCorr).
- DDK-filtered fields → no analytic sigmas; use mcPropagate.
- Anything else → mcPropagate: independent-sigma or full SINEX covariance sampling validates every analytic formula empirically (the toolbox's own tests do exactly this).

## Validation habits the toolbox is built around

- Every numerical method was validated in Python (numpy/scipy) before the MATLAB implementation; the validation numbers are quoted in the help texts. Keep the habit for your own extensions.
- Ring-grid analysis↔synthesis roundtrips are exact to ~1e-14: use them as a self-test on your grids (Part III, G1 step 5).
- Real provider files beat synthetic fixtures: the test suite auto-discovers any *.gfc / *.snx(.gz) you drop into tests/test_data and sanity-checks them (large SINEX are streamed, v2.2).
- runAllTests is the gate: run it after every toolbox update; perf_log.csv accumulates machine-local timing history.

## Common silent errors this toolbox guards against

| error | guard |
|---|---|
| geodetic latitudes fed to SH math (~21 km bias) | LatType="geodetic" + conversion helpers (v2.2) |
| hardcoded Love numbers | kn is a required explicit input for EWH |
| climatology across the 2017-2018 gap NaNs | assertClean errors loudly; dropNaN/select |

| error | guard |
|---|---|
| trend maps without GIA | removeGIA one-liner (v2.2) |
| optimistic trend sigmas | ARCorrect=true (v2.2) |
| tidal alias leakage into trends | Periods=[161/365.25, 3.66, 7.48] |
| zero-variance rows collapsing the noise floor | positive-diagonal floor + shLowLevel:buildNoiseCov:allZeroVariance |
| 8-column ICGEM 1.0 acos/asin ambiguity | EIGEN-style period read; documented in README |
| sine sectorals $S_{nn}$ blanked in difference triangles | fixed v2.4.2; rendered-CData regression test |

# Part III — Step-by-step guides

## G1. Monthly EWH anomaly maps from ITSG files

```
% 1. read the monthly GSM series (gz ok, ICGEM 1.0/2.0)
ts = shSeries.read("data/ITSG-Grace2018_n96_*.gfc");

% 2. low-degree completion BEFORE filtering
tn14 = shLowLevel.readTN14("TN-14_C30_C20_SLR_GSFC.txt");
ts   = ts.applyTN14(tn14);                  % C20 (+C30 where flagged)
tn13 = shLowLevel.readTN13("TN-13_GEOC_GFZ_RL06.txt");
ts   = ts.addDegree1(tn13);

% 3. bridge the mission gap explicitly
ts = ts.dropNaN();

% 4. anomalies w.r.t. a mean field, then filter
ts  = ts - ts.mean();                       % anomalies
tsF = ts.filter("tvANS", Blocks="auto");    % or .gaussian(300)

% 5. EWH synthesis (Love numbers user-supplied!)
kn  = readmatrix("loadLoveNumbers.txt");    % your set, degrees 0..nmax
lat = -89.5:1:89.5; lon = 0.5:1:359.5;
ewh = tsF.at(24).synthesis(lat, lon, Quantity="ewh", kn=kn);  % [m]

% self-test: analysis roundtrip on a ring grid should be ~1e-14
gChk = shCoefficients.analysis(ewh, lat, lon, tsF.nmax, ...
          Quantity='ewh', kn=kn);
```

Interpretation: values are equivalent water height anomalies relative to the removed mean, in meters. The FFT synthesis path engages automatically on this uniform grid; check `tsF.at(24).history` for the full provenance chain.

## G2. Basin time series with uncertainties

```
Idx= shLowLevel.shIndex(ts.nmax, Nmin=2);
% kernel from a polygon [lat lon], buffered + tapered (v2.2)
amazon = readmatrix("amazon_polygon.txt");
[b, info] = shLowLevel.basinKernel(idx, amazon, BufferKm=150, TaperKm=350);

% filtered series + filter operator
[tsF, op] = ts.filter("tvANS", Blocks="auto", VCEBands=[0 16 33 61]);

% deconvolved basin average with 1-sigma (banded-VCE-consistent, v2.2)
[avg, out] = shLowLevel.basinDeconvolve(b, op);
shLowLevel.plotBasinSeries(op.tYears, avg(1,:)', out.sigma(1,:)', Units="cm");
```

The deconvolution undoes the filter's attenuation of the kernel; the sigma comes from the posterior covariance with per-band noise factors. For several basins pass a kernel matrix B — the joint solve also redistributes mutual leakage. (Edition-2 erratum fixed: the signature is `basinDeconvolve(B, op)`, epochs live in `op.tYears`, averages in the first output.)

## G3. Significance-masked trend maps (GIA- and AR(1)-honest)

```
gia = shCoefficients.read("ICE-6G_D_rates.gfc");       % rate [1/yr]
tsG = tsF.removeGIA(gia);
clim = tsG.climatology(Periods=[161/365.25, 3.66, 7.48], ...
                       ARCorrect=true);
tr   = clim.trend();                        % shCoefficients WITH sigmas
kn   = readmatrix("loadLoveNumbers.txt");
rate = tr.synthesis(lat, lon, Quantity="ewh", kn=kn);         % [m/yr]
mc   = shLowLevel.mcPropagate(@(g) g.synthesis(lat, lon, ...
          Quantity="ewh", kn=kn), tr, N=300, Seed=1);
sigma = mc.sigma;
mask = abs(rate) > 2 * sigma;               % 2-sigma significance
rate(~mask) = NaN; imagesc(lon, lat, rate);
```

Compare against a second GIA model (Caron) and show the difference as the model-spread contribution to the error budget — usually dominant over formal errors in formerly glaciated regions.

## G4. Regional analysis with Slepian functions

```
Idx= shLowLevel.shIndex(60, Nmin=0);
region = @(la,lo) double(la>-90 & la<-60);      % Antarctica-ish belt
[G, lam, infoS] = shLowLevel.slepianBasis(idx, region);
fprintf("Shannon %.1f -> keeping %d tapers\n", infoS.shannon, numel(lam));

x  = shLowLevel.vecFromCS(g.C, g.S, idx);              % one month, idx order
a  = G' * x;                                    % Slepian coefficients
xR = G * a;                                     % regional part of x
[Cr, Sr] = shLowLevel.csFromVec(xR, idx);
gR = shCoefficients(Cr, Sr, GM=g.GM, R=g.R);
```

Estimating a (dimension ≈ Shannon number) instead of all P Stokes coefficients is well-posed on regional data by construction; concentrations λ near 1 tell you which tapers the region genuinely constrains.

## G5. Working with released covariances (SINEX)

```
Idx= shLowLevel.shIndex(96, Nmin=2);
% full read (monthly solutions to ~30 MB): covariance in idx order
snx = shLowLevel.readSINEX("ITSG-..._n96_2008-04.snx.gz", ...
                    Output="covariance", Index=idx);
N   = snx.M;                                    % noise shape for tvANS
[tsF, op] = ts.filter("tvANS", NoiseCov=N, Blocks="off");

% multi-100-MB NEQ SINEX: stream just the estimates in seconds (v2.2)
est = shLowLevel.readSINEX("ITSG-..._operational_n96_2020-01.snx.gz", ...
                    Only="estimate");

% propagate the full covariance through ANY functional (v2.2)
out = shLowLevel.mcPropagate(@(gs) b' * shLowLevel.vecFromCS(gs.C,gs.S,idx), ...
                      g0, Cov=snx.M, Idx=idx, N=2000);
```

## G6. Comparing against mascons

```
mas = shLowLevel.readMascon("GRCTellus.JPL...MSCNv03CRI.nc");
% mascon lats are GEODETIC -> one switch avoids the 21-km bias (v2.2)
kn = readmatrix("loadLoveNumbers.txt");
k  = find(abs(mas.epoch - 2010.29) < 0.05, 1);     % match an epoch
ewhSH = tsF.at(k).synthesis(mas.lat', mas.lon', ...
          Quantity="ewh", kn=kn, LatType="geodetic");  % [m]
d = 100*ewhSH - mas.ewh(:,:,k);                    % mascon units: cm
fprintf("RMS diff %.2f cm\n", sqrt(mean(d(:).^2, 'omitnan')));
```

Expect coastal and small-basin differences: mascons carry their own regularization. Compare basin-integrated series rather than pointwise fields for a fair statement; use the same GIA convention on both sides (JPL mascons ship GIA-corrected — remove GIA from the SH side too).

## G7. GNSS loading comparison (v2.3)

```
% residual monthly series (mean removed), filtered
ts  = ts - ts.mean();                       % anomalies
tsF = ts.filter("tvANS", Blocks="auto");

% load Love numbers k', h', l' - ONE consistent model, user-supplied;
% v2.5 reader: layouts, sparse pchip interpolation, CF-frame conversion
LN = shLowLevel.readLoveNumbers("loadLoveNumbers_PREM.txt", ...
    Columns="n k h l", Nmax=tsF.nmax, Interp="pchip", ...
    InFrame="CE", OutFrame="CF");               % match TN-13 (CF)
kn = LN.kn; hn = LN.hn; ln = LN.ln;

% GNSS stations (geodetic coordinates!)
sta = readmatrix("stations.txt");               % [lat lon]
T = tsF.nEpochs;
up = zeros(size(sta,1), T); no = up; ea = up;
for k = 1:T
    [up(:,k), no(:,k), ea(:,k)] = tsF.at(k).deformation( ...
        sta(:,1)', sta(:,2)', kn=kn, hn=hn, ln=ln, ...
        Mode="points", LatType="geodetic");
end
% compare up(i,:) against the GNSS vertical of station i (same epochs);
% typical GRACE-elastic amplitudes: mm to ~1 cm vertical, ~1/3 horizontal
```

Degree 1 is excluded by default; include it only when both the GRACE side (TN-13 in the CF frame) and the Love numbers use the same frame convention. The elastic prediction excludes GIA and poroelastic effects — detrend both series or remove GIA explicitly before comparing trends.

## G7b. Data management (v2.4.1, temporal catalogue v2.4.2)

```
shLowLevel.dataFolder("D:/geodata/shLowLevel");      % once; persists (getpref)
shLowLevel.fetchDDK(1:8);                    % all released DDK filters
W5 = shLowLevel.readDDK("DDK5");             % by name, from the data folder
T = shLowLevel.listICGEM();                  % 180+ static models as a table
f = shLowLevel.fetchICGEM("EGM2008");        % -> dataFolder/icgem/EGM2008.gfc
g = shCoefficients.read(f);
shLowLevel.fetchITSG(2010:2016);             % -> dataFolder/itsg_series
shLowLevel.fetchITSG("2008-04", Product="daily"); % Kalman daily n40 (v2.4.1)

% FULL temporal catalogue: all ~70+ series, all centers
Tt = shLowLevel.listICGEM(Type="temporal");
% fetch a WHOLE monthly series (v3.1.1) - one row of that catalogue
fs = shLowLevel.fetchICGEM(17, Type="temporal");
ts = shSeries.fromFolder(fileparts(fs(1)));   % straight into a series
```

The ICGEM static catalogue is parsed from icgem.gfz.de (fixture-tested offline). The temporal catalogue lists every series of every center — groups 01_GRACE, 02_COST-G, 03_other, 04_SLR — with columns group, center, series, path, url and zip. Since v3.1.1 a temporal row is not just browsable but fetchable: `fetchICGEM(idx, Type="temporal")` downloads the whole series into `<dataFolder>/icgem/series/<group>_<center>_<series>/`, which `shSeries.fromFolder` and `shLowLevel.standardChain` consume directly. For ITSG monthlies `shLowLevel.fetchITSG` remains the convenient route.

**Mode: one request, not three hundred**

The default `Mode="auto"` takes the server's whole-series ZIP in a SINGLE request and unpacks it. This is not an optimization but a courtesy requirement: icgem.gfz.de rate-limits, and fetching a 283-file series one file at a time earns HTTP 429 and a tarpit (field-observed as "too many connections"). If the archive is unavailable the fetcher falls back to per-file mode automatically — resumable, throttled with `Pause=` and `Retries=` exponential backoff, each file verified by parse before it replaces anything. `Mode="archive"|"files"` force either path; `info.mode` reports which one actually ran, so a script can tell a fresh download from a no-op. `Files=` filters the series, and `FileList=` injects a catalogue table for offline mirrors and subsets.

```
% forced per-file, filtered to a window, gentle on the server
fs = shLowLevel.fetchICGEM(17, Type = "temporal", Mode = "files", ...
        Files = "*2008*.gfc", Pause = 3, Retries = 3);
[ts, rep] = shLowLevel.standardChain(fileparts(fs(1)), Filter = "DDK3");
```

## G8. Multi-center combination (v2.4)

```
shLowLevel.fetchITSG(2010:2016);             % download once (websave)
tsI = shSeries.fromFolder("ITSG/");    % GSM level, same nmax
tsC = shSeries.fromFolder("CSR/");
tsG = shSeries.fromFolder("GFZ/");
[tsComb, info] = shLowLevel.combineCenters({tsI, tsC, tsG}, Robust=true);

figure; plot(info.epochs, info.weights');     % weight time series
legend(info.centers); ylabel("relative weight");
figure; imagesc(info.interCenterCorr);        % honesty diagnostic
% THEN degree-1 / C20-C30 replacements, once:
tsComb = tsComb.applyTN14("TN-14.txt");
tsComb = tsComb.addDegree1("TN-13.txt");
```

Watch the weight series for regime changes (single-accelerometer era, GRACE-FO transition) and treat posterior sigmas as lower bounds — the common mode is invisible.

## G9. Sea-level fingerprint (v2.4)

```
Idx= shLowLevel.shIndex(96, Nmin=0);
LN = readmatrix("loadLoveNumbers.txt"); kn = LN(:,2); hn = LN(:,3);
greenland = @(la,lo) -280 * inGreenland(la, lo);   % kg/m^2/yr, say
oceanF = @(la,lo) oceanMaskFun(la, lo);            % 1 over ocean
[S, grid, info] = shLowLevel.seaLevelFingerprint(greenland, oceanF, idx, ...
    kn=kn, hn=hn);
shLowLevel.plotSHMap(info.S2D / info.eustatic, grid.latDeg, grid.lonDeg, ...
    Units="S / eustatic", Title="Greenland fingerprint");
```

Plot S normalized by the eustatic mean: near-field values drop below zero, far-field plateaus around 1.1–1.3 — if not, check the ocean mask and that idx uses Nmin= 0.

## G10. The standard chain in one call

```
% the whole post-processing chain, in the ONE correct order
gia = shCoefficients.read("ICE-6G_D_trend.gfc");   % a TREND field [1/yr]
[ts, rep] = shLowLevel.standardChain("D:/grace/monthly", ...
    TN14File = "TN-14_C30_C20_SLR_GSFC.txt", ...   % TN14 itself is a FLAG
    Degree1 = "CSR", ...                           % provider, not a path
    GIA = gia, Filter = "DDK3");
disp(rep.steps')          % what was applied, in order, with provenance
```

Order is not a matter of taste here. The SLR C20/C30 replacement and the degree-1 restoration must happen on the UNFILTERED coefficients — a filter mixes degrees, so replacing a coefficient afterwards inserts an unfiltered value into a filtered field and the low degrees no longer mean anything consistent. GIA is a trend in the same (corrected) coefficients, so it comes next; filtering is last. `standardChain` encodes exactly that order and returns REP, a provenance record naming every step, the files it used and the toolbox version that ran — the thing you paste into a paper's data section. Every stage is optional: leave `GIA=` out and the report says so, rather than silently skipping a step you assumed had happened.

Two contracts worth reading before the first call: `TN14` is a logical FLAG and the path goes in `TN14File`, while `Degree1` names the TN-13 provider ("GFZ" | "CSR" | "JPL" | "none") with the path in `Degree1File`; and `GIA` takes an `shCoefficients` TREND field in [1/yr], not a filename — it is applied as (t – GIAEpoch) × GIA. Left at their defaults the files come from the persistent data folder that `setup_shAnalysis` populates.

## G11. Designing your own anisotropic filter

```
% a DDK-class filter built from YOUR error covariance
g  = shCoefficients.read("ITSG-Grace2018_n60_2008-04.gfc");
W  = shLowLevel.designFilter(g.sigmaC, g.sigmaS, Kaula = 1e-5);
[Cf, Sf] = shLowLevel.applyDDK(g.C, g.S, W);   % same block format as readDDK
```

The released DDK filters are built from a specific center's normal equations. If you have your own sigmas or a released covariance, `designFilter` builds the same kind of operator, $W = (N + a\,S^{-1})^{-1}\,N$, in the block format `readDDK` produces — so it drops into `applyDDK`, `g.applyDDK` and `standardChain(Filter=W)` unchanged. Note that `Noise=` expects a COVARIANCE, not sigmas: passing standard deviations where a variance is expected is the classic way to get a filter that looks plausible and smooths by the square root of what you intended.

## G12. Correcting leakage

```
% forward modelling: only the FILTER has to be known
ewh = g.synthesis(-89:89, 0:359, Quantity= "ewh", kn = kn);
[m, info] = shLowLevel.leakageCorrect(ewh, -89:89, 0:359, ...
    Filter = "gauss300", Mask = basinMask, Gain = 2);

% or scaling factors, if you trust a model's spatial PATTERN
k = shLowLevel.gridScaling(modelEWH, -89:89, 0:359, Filter = "gauss300");
corrected = k .* ewh;                % NaN where the model is blind
```

A filter removes real signal and spreads the rest across basin boundaries; a 6° disc under a 500 km Gaussian keeps about half its peak. The two corrections differ in what you have to BELIEVE, which is how to choose between them. Forward modelling (`leakageCorrect`) needs only the filter and, usefully, a mask saying where mass can exist — it then has to explain the observation with mass inside that region, so leakage outside is removed rather than redistributed. Scaling factors (`gridScaling`) need a model series, and inherit the correctness of its spatial pattern: the factors are invariant to the model's amplitude but not to its shape.

Two behaviours worth knowing before you read the output. Convergence is judged on the change of the SOLUTION, not on the residual: with a mask the problem is inconsistent, so the residual settles at a floor while the

solution is converged. And `gridScaling` returns NaN where the model carries no signal instead of a ratio of two numerical zeros — a model that does not reach your basin shows up as missing coverage rather than as noise multiplying your data.

## G13. What real provider files look like

Format specifications describe what files ought to contain. The reader is written against what they actually contain, and three separate field failures are worth knowing about, because they are silent — you get numbers, just not the right ones.

- **FORTRAN D-exponents.** Providers switch from `1.23E-10` to `1.23D-10` partway through large files (EIGEN-6C4 does it above degree ~370). MATLAB's `str2double` returns NaN for those, so a line-by-line parser quietly corrupts the high degrees of exactly the models where the high degrees are the point. The reader normalizes D/d to E/e before conversion.
- **ICGEM 2.0 column order.** The real-world layout of an `acos`/`asin` line is `... sigC sigS t0 t1 period` — period LAST, verified against CNES/GRGS files. A period-first assumption mis-parses every such file and produces a time-variable model whose seasonal terms are wrong without ever raising an error.
- **Ragged record groups.** EIGEN-5S and 5C carry a single `gfc` line with a trailing epoch among thousands of uniform ones. Bulk `sscanf(txt, '%f', [nc Inf])` does not complain about that — it silently re-flows the whole group by one column and asks for a coefficient matrix indexed by a date. The reader checks rows-out == lines-in per group, subgroups by numeric width when ragged, and keeps an n/m sanity net.

The practical rule: when a new provider or a new release enters your pipeline, read one file and check `g.C(3,1)` and the highest degrees against the provider's own published values before you trust a thousand of them. Bulk parsing is fast — EIGEN-6C4 at 177.7 MB and degree 2190 in about 5 s, a 73.6 MB GRGS mean field with 674k variable terms in about 7 s — but fast and correct are different properties.

# Validation against GravIS

This chapter is a worked, reproducible comparison of the toolbox against a PUBLISHED product: the GravIS Greenland ice-mass basin averages (https://gravis.gfz.de/gis). It is written out in full because the numbers only mean something together with the choices behind them — and because two of those choices cost a factor that would otherwise have been mistaken for a bug in the toolbox.

## V1. The reference

GravIS publishes seven Greenland drainage basins as ASCII time series in Gt. Summing them and fitting bias, trend, annual and semi-annual gives the reference trend. It must be recomputed over the SPAN YOU ACTUALLY USE: over the full record COST-G RL02 gives –220.8 Gt/yr, but over the shorter span for which the auxiliary correction tables were available it is –231.1 Gt/yr (that span: 2002-04..2023-02, n = 217 monthly solutions - QUOTE THE DATES with the number; an independent audit measured ~10 Gt/yr of reference drift over two years of end date). Comparing against the wrong span is a 5% error before any processing starts.

## V2. Matching the GravIS processing

GravIS Level-3 basin averages come from Level-2B coefficients, which are Level-2 plus four corrections (https://gravis.gfz.de/corrections). Applying anything less makes the comparison meaningless. All four are read with `shLowLevel.readSHM` or as plain tables:

```
% (1) their low-degree replacement: C20, C30, C21, S21
% (2) their geocenter (Swenson approximation, NOT TN-13)
% (3) their mean field, 2002/04-2020/03
% (4) GIA: ICE-6G_D (VM5a), a GRDOTA rate model in 1/yr
Mn  = shLowLevel.readSHM("GRAVIS-2B_..._MEAN_2002095-2020091_NFIL.gz", Nmax = 90);
Gi  = shLowLevel.readSHM("GRAVIS-2B_..._GIA_ICE-6G_D_VM5a.gz", Nmax = 90);

ts = shSeries.fromFolder(costgFolder);
ts = ts.select(ts.epochs <= maxTableEpoch + 0.05);   % tables trail the data
% ... insert their C20/C30/C21/S21 and C10/C11/S11 per epoch ...
for k = 1:ts.nEpochs                    % mean field and GIA, per epoch
    Cs(:,:,k) = Cs(:,:,k) - Mn.C - (ts.epochs(k) - 2011) * Gi.C;
    Ss(:,:,k) = Ss(:,:,k) - Mn.S - (ts.epochs(k) - 2011) * Gi.S;
end
ts = ts.removeAlias();                  % (5) the S2 161-day alias
```

## V3. The basin average

```
Idx= shLowLevel.shIndex(ts.nmax, Nmin= 0);
[b, info] = shLowLevel.basinKernel(idx, gisPolygon);   % NaN-separated rings
area = info.areaFraction * 4*pi*6371^2;                % NOT b(1)
kf   = shLowLevel.kernelFactors("ewh", ts.nmax, GM, R, kn = kn);
avg  = (b .* kf(idx.n+1))' * X / (b'*b);               % EWH [m] per epoch
mass = avg(:) * area * 1e6 * 1000 / 1e12;              % -> Gt
```

## V4. Results, and what each step is worth

| Method | Gt/yr | vs published |
|---|---|---|
| naive kernel average | –196.4 | –15% |
| naive grid integral | –170.3 | –26% |
| forward modelling, **Greenland-only** mask | –259.8 | +12% |
| forward modelling, **union** mask | –240.2 | +4% |
| union mask + S2 alias removed | –240.0 | +4% |
| **GravIS COST-G RL02, matching span** | **–231.1** | — |

## V4b. Describe the filter the FORWARD MODEL sees

The table above hides the single largest error I made. GravIS basin averages start from UNFILTERED Level-2B coefficients, so I passed `Filter="none"` to `leakageCorrect`. But Dahle et al. (2025), Sect. 2.2.1, list the basin-average steps: spectral masking, then **low-pass filtering with a Wiener optimal filter, approximately a Gaussian of 4° latitude spatial half-width**, then the conversion to surface mass, then the least-squares

adjustment. The forward model is filtered IDENTICALLY to the observations — that is the whole point of the method — so the filter to declare is the Wiener one, not "none".

```
r = 4 * 111.195;                          % 4 deg latitude ~ 445 km
wG = shLowLevel.shGaussianWeights(nmax, r);
obs = shCoefficients(Ct .* wG(:), St .* wG(:)).synthesis( ...
        lat, lon, Quantity= "ewh", kn = kn, Nmin= 0);
m = shLowLevel.leakageCorrect(obs, lat, lon, Filter = "gauss445", ...
        Mask = unionMask, Gain = 2, MaxIter = 300);
```

| Declared filter | Gt/yr | vs published |
|---|---|---|
| `"none"` (wrong) | –234.9 | +1.6% |
| `"gauss445"` (Wiener equivalent) | –**227.4** | –**1.6%** |
| **GravIS COST-G RL02, matching span** | –**231.1** | — |

Declaring the filter correctly turns a 1.6% overshoot into a 1.6% undershoot, and the iteration also drifts less with continued iteration (2.0 Gt/yr from 300 to 900 steps, against 2.5 unfiltered) because the filter regularises the inverse problem. At that point the agreement is as close as this reproduction can meaningfully be read: the remaining difference is smaller than the spread between GravIS product versions and smaller than the effect of the stopping point.

The paper also specifies the mask GravIS uses, which is NOT the bare drainage-basin outline: the spectral mask is 1 until 200 km outside the grounding line and tapers smoothly to 0 by 600 km. That is `basinKernel(..., BufferKm = 200, TaperKm = 400)`, not a hard boundary.

## V4c. The regularised number, and the noise it needs

Every figure above was produced by stopping the iteration at a fixed count, which is not a criterion. Rerun with the discrepancy principle, two further things have to be right — and getting either wrong is worse than the leakage it was meant to fix.

- **The residual must be measured where the model is responsible.** With a Greenland mask the model describes one region while the data contains Antarctica, Alaska and global hydrology, so the GLOBAL residual never falls to the noise level: 0.0085 globally against 0.0008 near the mask, with a noise level of 0.0024. The principle silently never fires and the run stops at MaxIter. `ResidualRegion` (default: the mask) fixes this.
- **The noise level must match the quantity being inverted.** The open-ocean RMS of a TREND field is 0.0024 m/yr — but most of that is real barystatic sea-level rise, not error. Using it stops the iteration after 4 steps and gives –201.0 Gt/yr, 13% low. The trend's actual noise is the monthly noise propagated through the fit, sigma_monthly / sqrt(Sxx) = 0.0115 / sqrt(8353) = 0.000125 m/yr — nineteen times smaller.

```
% the concrete definitions this chapter previously left implicit
% (audit 2026-08: the recipe was not runnable as printed):
lat = (-89.5:89.5)';  lon = (0.5:359.5)';       % 1-degree grid
[LO, LA] = meshgrid(lon, lat);  lonW = mod(LO+180, 360) - 180;
box = @(a,b,c,d) LA>=a & LA<=b & lonW>=c & lonW<=d;
unionMask = gisMask ...                          % basin polygon pixels
    | box(60, 84, -128, -60) ...                 % Canadian Arctic
    | box(63, 67,  -25, -13) ...                 % Iceland
    | box(76, 81,   10,  34);                    % Svalbard
% Greenland mass rate from the corrected trend grid m [m/yr EWH]:
A  = (6371e3)^2 .* cosd(LA) * deg2rad(1)^2;      % cell areas [m^2]
Gt = sum(m(gisMask) .* A(gisMask), 'omitnan') * 1000 / 1e12;
```

```
% monthly noise from the residuals about the deterministic fit ...
sigMon = shLowLevel.oceanRMS(residGrid, lat, lon, openOcean);
% ... propagated to the TREND: sigma_trend = sigma_monthly / sqrt(Sxx)
sigTrend = sigMon / sqrt(sum((epochs - mean(epochs)).^2));
[m, info] = shLowLevel.leakageCorrect(trendGrid, lat, lon, ...
    Filter = "gauss445", Mask = unionMask, NoiseLevel = sigTrend);
info.stoppedBy      % "discrepancy", after 82 iterations
```

| Stopping rule | Gt/yr | vs published |
|---|---|---|
| fixed 300 iterations (no criterion) | –227.4 | –1.6% |
| discrepancy, trend-field oceanRMS (wrong noise) | –201.0 | –13% |

| Stopping rule | Gt/yr | vs published |
|---|---|---|
| **discrepancy, propagated trend noise** | **−224.6** | **−2.8%** |
| **GravIS COST-G RL02, matching span** | **−231.1** | **—** |

The regularised answer is 2.8% from the published one, slightly further than the 1.6% obtained by stopping at an arbitrary iteration count. That is the honest outcome: the closer number was a coincidence of where the iteration happened to be, and a result that depends on an arbitrary choice is not a result. This one follows from a stated criterion and a noise estimate that can be recomputed.

## V5. The lessons

**The mask must cover every region that can hold mass, not only the one you are measuring.** A Greenland-only mask tells the inversion that mass exists nowhere else, so Canadian Arctic, Iceland and Svalbard signal is forced into Greenland: +12% instead of +4%. In the union solution the neighbours absorb −112 Gt/yr — mass the first mask had nowhere to put. The union mask also converges faster. This is a usage rule, not a bug: `Mask=` always accepted a union.

**The filter you declare must be the filter the FORWARD MODEL sees, not the filter the input file carries.** The inputs here are genuinely unfiltered, and declaring that was still wrong, because the method filters both sides. This cost more than the mask, the polygon resolution and the alias put together.

**The S2 alias removal is worth 0.2 Gt/yr on a twenty-year trend — that is, nothing.** A 161-day harmonic is very nearly orthogonal to a linear trend over two decades, so removing it barely moves the trend even though it is a real and necessary correction for the MONTHLY series. Reported here because it was expected to explain part of the residual and did not: a hypothesis removed is worth as much as one confirmed.

## V6. What the remaining 4% is, honestly

- GravIS inverts SEVEN drainage basins jointly (Sasgen et al. 2012); this recipe constrains one region against coarse neighbour boxes.
- The basin outline used here is decimated to 97.4% of the published area, and the neighbour outlines are rectangles.
- The iteration has not converged: step 3.2e-4 against Tol 1e-4 after 900 iterations, and the estimate was still drifting (−240.2 at 300, −242.5 at 900). Call it ±5 Gt/yr.
- Nothing here was tuned to close the gap. A 4% agreement with an independently produced product, reached by matching its documented processing step for step, is the result; making it smaller by adjusting a mask until the number matched would not be.

## V7. Antarctica: where the one-map inversion meets its limit

The same chain, run unchanged on the same corrected, GIA-reduced series with the 25 GravIS AIS basin polygons as the mask (span 2002-04..2023-02, n = 217), gives −125.7 Gt/yr against the official AWI basin-average product's −146.9 — a 14% attenuation, an order of magnitude worse than Greenland's 2.8%. Three controls say the chain itself is healthy: the GIA field alone carries +52.2 Gt/yr of apparent mass over the AIS and the uncorrected chain lands at −34.5 (the correction does exactly its advertised work); the wrong-noise control collapses to −108 in 8 iterations with the same signature as Greenland; and an 8-fold sweep of the noise level moves the answer by 2 Gt/yr — the stopping rule is NOT the problem.

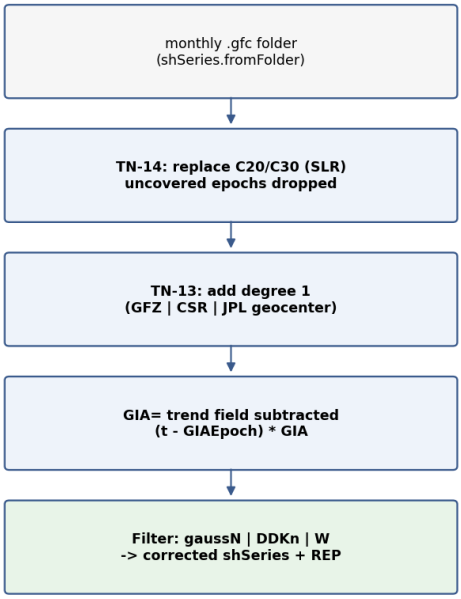
The basin-by-basin comparison locates the gap: the inversion damps the EXTREMES of both signs (Amundsen and Getz come out 7–11 Gt/yr too weakly negative, the Dronning-Maud-sector accumulation basins 12–16 too weakly positive; 5.9 Gt/yr RMS per basin). The decisive cross-check is the SECOND official product: integrating the TU Dresden tailored-sensitivity-kernel GRID product (Groh & Horwath 2021) over the same basin polygons gives −112.8 Gt/yr with an 8.2 Gt/yr per-basin RMS and a 0.905 correlation to this chain's difference pattern. Every map-based estimator damps the same basins the same way; the Sasgen forward-modelling inversion (25 basins jointly, Sasgen et al. 2013) is a different method class, and the 14% is the distance between the classes, not a defect of the chain. A one-map inversion of a continent with a 113-pixel peninsula also reproduces every stopping pathology this guide warns about: the Peninsula alone never reaches the noise floor (maxIter), and the three sub-sheet inversions sum to −172, not −126 — V2's mask lesson, live.

## V8. Terrestrial water storage: 0.03 cm/yr against GravIS

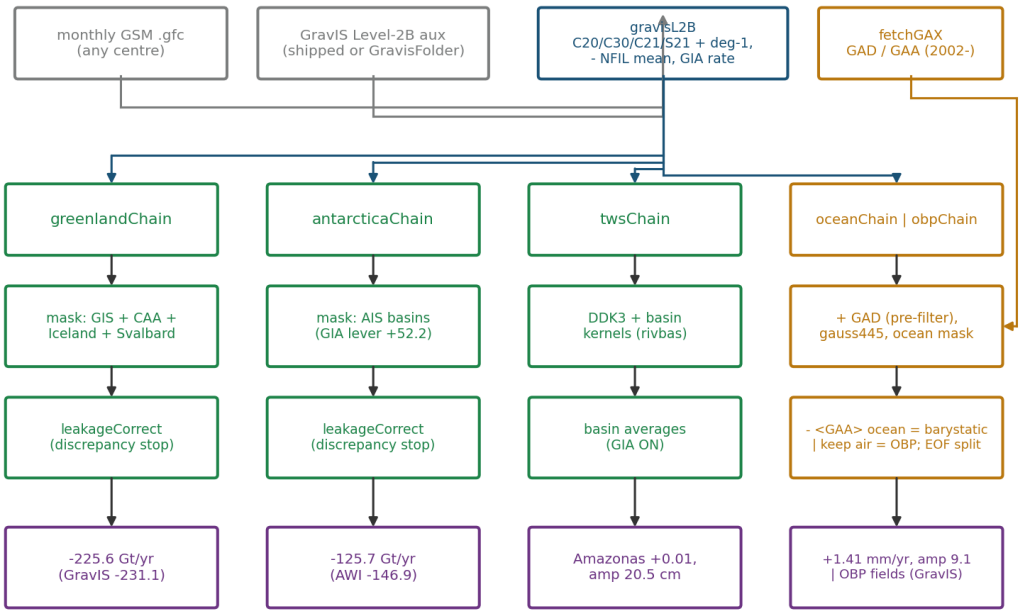
The hydrology test inverts the roles: small signals, seasonal amplitudes, and a reference (the GravIS TWS portal series, COST-G) whose averaging convention was first pinned by reproducing the official gridded product's basin means to two decimals from the same polygons. The chain — corrections, minus the NFIL mean, DDK3 as the

declared stand-in for GravIS' VDK3/VDK5 blend, cos-weighted 1° basin means over eleven major river basins, full 252-month span — reproduces the seasonal amplitudes at a median ratio of 1.001 (0.98..1.04). The trends came out at 0.225 cm/yr RMS with a pattern any geodesist will recognise: Mississippi –0.49 against +0.07, Ob/Yenisei/Lena all too positive — the Laurentide and Fennoscandian GIA signature. Subtracting the same ICE-6G_D rate model the ice products use drops the trend RMS to **0.032 cm/yr** (Mississippi +0.09 vs +0.07, Ganges –1.24 vs –1.22, Amazonas +0.01 vs +0.01). Two findings travel beyond this toolbox: the GravIS TWS Technical Note (2025-08-14) does not mention a GIA correction, but the product plainly carries one — a 7-fold RMS reduction is not a coincidence; and the residual Ganges difference is consistent with GravIS' empirical removal of the 2004 Sumatra earthquake steps, which this chain deliberately does not model. Declare the filter, declare the GIA, and a 90-degree spherical harmonic series reproduces an independently produced hydrology product to three hundredths of a centimetre per year.

## V9. The chains: the validation as an API




The generic core: shLowLevel.standardChain applies the classic TN-14/TN-13 corrections in the only correct order, with the same drop-and-record policy for uncovered epochs. gravisL2B (below) is its GravIS-flavoured sibling: same shape, GravIS aux tables and NFIL mean instead of the TN files.




*The chain family at v3.11.0: one shared gravisL2B core; the ocean lane forks after GAD into barystatic (oceanChain, with EOF circulation split) and bottom pressure (obpChain).*

The chain family at v3.11.0. gravisL2B is the single shared correction core; the ice chains share one engine (gravisRegionChain), the TWS chain filters and averages in per-basin boxes, and the ocean lane forks after the GAD restoration: oceanChain removes the atmospheric ocean mean for the barystatic series (with an optional EOF split of residual circulation from noise), obpChain keeps the air column and delivers GravIS-style bottom-pressure fields. Defaults ARE the tested configuration; the shipped data/gravis folder makes the calls below run as written.

Every number in V1–V8 is reproducible in one call. The shipped `data/gravis` copies (frozen 2026-08-12) cover the aux tables, the NFIL mean, the ICE-6G_D GIA rate and the GravIS basin polygons (`shLowLevel.gravisDataFolder` returns the path); only the monthly series folder and the Love numbers are yours to supply. Greenland, with the guide-V4c ocean mask:

```
kn = readmatrix("loadLoveNumbers_Gegout97.txt", FileType = "text", ...
    NumHeaderLines = 2);
oc = @(la, lo) abs(la) <= 55 & ~(la < -60 | ...      % crude continents
    (la > -35 & la < 37 & mod(lo+180,360)-180 > -17 & mod(lo+180,360)-180 < 52));
[gt, rep] = shLowLevel.greenlandChain("E:/series/COSTG", ...
    kn = kn, OceanMask = oc);                        % -> -225.6 Gt/yr
disp(rep.steps')
```

Antarctica adds the 25-basin table, and `TrendGrid=` re-feeds a previous report so a sensitivity sweep costs seconds, not synthesis:

```
[gt, rep] = shLowLevel.antarcticaChain("E:/series/COSTG", ...
    kn = kn, OceanMask = oc);                        % -> -125.7 Gt/yr
disp(rep.basins)                                     % 301..325 [Gt/yr]
gt2 = shLowLevel.antarcticaChain("E:/series/COSTG", kn = kn, ...
    TrendGrid = rep, NoiseLevel = 2 * rep.sigTrend); % noise sweep, ~20 s
```

TWS runs full-span with GIA ON by default (the V8 finding); switching it off reproduces the 7× worse trend RMS:

```
[out, rep] = shLowLevel.twsChain("E:/series/COSTG", kn = kn, ...
    Basins = ["Amazonas", "Ganges", "Mississippi River"], ...
    DDKFolder = "E:/DDK");                           % amp 20.5/14.3/6.1 cm
plot(out.epochs, out.series); legend(out.name)
noGia = shLowLevel.twsChain("E:/series/COSTG", kn = kn, ...
    Basins = "Mississippi River", DDKFolder = "E:/DDK", GIAFile = "");
```

The two ice chains share one engine (`gravisRegionChain`); every input above is exchangeable, every deviation from the defaults is a declared sensitivity study, and REP records each step with the toolbox version. Data fetches behind these chains and all fetchers retry politely on 429/5xx: `httpFetch` honours the server's Retry-After header and `httpRetryDelay` supplies the capped, jittered backoff otherwise — the policy the GravIS portal's 503 bursts

demanded during the V7/V8 data collection. Spelling convention throughout: Capitalized names for arguments-block options (`OceanMask=`), lowercase for the legacy inputParser names and all quantity strings; both mechanisms tolerate any casing (regression-tested), and `quantity='none'` is the dimensionless passthrough for synthesizing raw coefficient fields.

## V10. Ocean and Slepian: closing the chain family

The ocean is the missing member: `oceanChain` runs the same gravisL2B core, applies the ICE-6G_D ocean-floor GIA rate (a measured +0.89 mm/yr lever on the barystatic trend) and gauss445, and returns the area-weighted ocean-mean series. Acceptance run on COST-G RL02.1 (252 months, |lat|≤66°, crude-continent mask): **+1.49 ± 0.02 mm/yr (+312 Gt/yr)**, annual amplitude 8.0 mm. The ocean MEAN is filter-invariant to 0.01 mm/yr while unfiltered pixel residuals are stripe-dominated (1.62 m measured) — sigMon = 0.0121 m after the trend+seasonal separation is the honest noise proxy the V6 finding demanded. GAD/GAA restoration is a declared limitation (`GADFolder=` adds GAD where available; AOD1B carries no secular trend by construction).

```
[out, rep] = shLowLevel.oceanChain("E:/series/COSTG", kn = kn, ...
    OceanMask = oc);                      % -> +1.49 mm/yr, +312 Gt/yr
plot(out.epochs, out.oceanMean)           % barystatic curve [cm]
noGia = shLowLevel.oceanChain("E:/series/COSTG", kn = kn, ...
    OceanMask = oc, GIAFile = "");        % the +0.89 mm/yr lever
```

Slepian analysis gets its application half: project any coefficient series onto the ~Shannon leading tapers of a region and analyze those few well-posed modes instead of all P coefficients. The localization kernel is cross-validated against an independent Python ring quadrature (30° cap, λ■ = 0.999981), and ICGEM `dot` secular lines now read as `trnd` synonyms.

Full ocean restoration (Chambers & Willis 2010) is one fetch away: `fetchGAX` downloads the AOD1B monthly means as ICGEM-converted .gfc from the GFZ series pages (163 GRACE-era + 88 GRACE-FO files per product, verified live; the AOD1B products are centre-independent, so they serve COST-G too). `GADFolder=` adds the model ocean signal back on the coefficient level, `GAAFolder=` subtracts the per-epoch ocean mean of the atmospheric product - the land-ocean air-mass term GRACE sees but that is not water. Epoch coverage is reported, never silent: REP carries nGadRestored and nGaaApplied. Accepted on the full COST-G RL02.1 series (252 months, 251 covered): the annual amplitude rises from 8.0 to 9.1 mm - the published manometric range - while the trend moves only 0.08 mm/yr (+1.41 ± 0.03), confirming that AOD1B is trend-free by construction; the residual RMS grows from 1.21 to 1.49 cm because GAD puts REAL sub-annual ocean variability back into the field.

**Bottom pressure versus barystatic - one step apart.** The GravIS Level-3 OBP product is GSM plus GAD with the standard corrections: bottom pressure is the weight of the FULL column (water plus air), so nothing is subtracted and the ocean mean contains the mean atmospheric mass. `obpChain` reproduces that recipe as gridded EWH anomalies against the GravIS reference window 2002/04-2020/03, with GAD added BEFORE the filter (the GravIS order - the filter note in both chain helps) and land set to NaN. `oceanChain` continues one step further and removes the ocean mean of GAA (Chambers & Willis 2010) to isolate water mass.

```
[obp, rep] = shLowLevel.obpChain(ser, kn = kn, OceanMask = oc, ...
    GADFolder = "E:/DATAPOOL/GravityField/GAX/GAD");
size(obp.grid)                % nLat x nLon x T, land NaN
rep.nRefEpochs                % epochs inside the GravIS window
```

**Residual circulation, separated instead of ignored.** After the trend + seasonal fit the ocean pixels still carry coherent interannual dynamics - real circulation, not error. `eofSeparate` splits the residual stack with an area-weighted EOF and the North et al. (1982) rule, keeping modes while their eigenvalue is separated by more than its sampling uncertainty. The physical limit is stated, not hidden: modes at or below the Marchenko-Pastur noise bulk (about sigma^2 (1+sqrt(Q/T))^2) are not separable by ANY eigenvalue criterion - the Python pre-validation demonstrated exactly that failure before the synthetic amplitudes were dimensioned above the bulk. Passing SeparateCirculation to oceanChain applies the split and turns sigMon into the de-circulated noise RMS.

```
[out, rep] = shLowLevel.oceanChain(ser, kn = kn, OceanMask = oc, ...
    GADFolder = gadF, GAAFolder = gaaF, SeparateCirculation = true);
[out.nModes, out.circulationRMS, out.sigMon]   % modes, circ, noise
```

**The fetch family, one function per source and product.** fetchITSG and fetchICGEM download solution series; fetchGAX the AOD1B GAA/GAB/GAC/GAD monthly means from the ICGEM/GFZ pages; fetchITSGSINEX the ITSG monthly normal-equation SINEX from TU Graz - the ONLY public per-month SINEX source, and heavy: one n96 month is about 460 MB gzipped (verified live), so months are a required argument and there is no convenience "all"; fetchITSGBackground the ITSG monthly background models (dealiasing, tides, and in the GRACE era also

atmosphere/ocean splits, c20, degree-1, GIA, hydrology). The ITSG background means are NOT the AOD1B GAX split - dealiasing is the closest GAC counterpart and no GAD substitute. All fetchers share one robust download loop: skip-if-present, MaxFiles/BudgetSec/MaxFailures caps, polite pauses, a single capped 429 retry, and a websave transport fallback.

```
f = shLowLevel.fetchITSGBackground(["2018-06", "2018-07"]);
s = shLowLevel.fetchITSGSINEX("2018-06", Nmax = 96);   % ~460 MB, deliberate
snx = shLowLevel.readSINEX(s(1), Only = "estimate");
```

**VDK decorrelation - the reason the SINEX fetcher exists.** With the monthly normal-equation matrices the VDK/VADER filter of Horvath et al. (2018) becomes buildable: $x_f = (N + \alpha M)^{-1} N x$ with the FORMAL monthly N (structure changes with orbit, repeat cycles and instrument state) and a cyclostationary Kaula signal model sigma_M(l) = a l^b estimated per calendar month from a pre-filtered series (signalVarianceKaula, with the exact log-chi-square bias correction - without it a biases 6%% low). Algebraically this is the SAME family as the tvANS Wiener filter ($S = M^{-1}$, Python-verified to 1e-15); the difference is the input, and it is worth 15%% median cumulative geoid error - an order of magnitude in short-repeat-cycle months (paper Table 2). tvANS remains the tool for series without released covariances. The full series is a deliberate batch (~460 MB SINEX per month): tools/dev/run_vdk_series.m is the resumable driver - downloads skip present files, months skip existing output, the signal model is cached, and the paper's closed-form identity runs as a built-in self-test.

```
Idx= shLowLevel.shIndex(96, Nmin= 2);
snx = shLowLevel.readSINEX(f, Index = idx);        % N + x, idx order
[ab, ~] = shLowLevel.signalVarianceKaula(ts.Cs, ts.Ss, ts.epochs);
xf = shLowLevel.vdkApply(snx.x, snx.M, idx.n, ab(mo, :), Alpha = 1);
```

**Flood and drought indices per cell or basin.** hydroExtremeIndex turns any TWS stack - twsChain grids or basinAverage series - into published extreme indices: the GRACE-DSI of Zhao et al. (2017), the standardized anomaly per calendar month with the 11-class USDM-style categories (validated in the literature against USDM, PDSI, SPEI, NDVI and in-situ groundwater); the WSDI deficit standardization; and the Reager & Famiglietti (2009) storage deficit - the CAUSAL flood predisposition (how much the store can still take), which with a precipitation grid becomes the full Flood Potential Index, the quantity behind multi-month flood lead times (Reager et al. 2014, Nature Geoscience). One policy is quantified rather than assumed: detrending. The pre-validation planted an exceptional-drought month under a 0.5 cm/yr trend - undetrended it weakens from -2.41 to -1.44, OUT of its class, while 45%% of the late decade turns spuriously wet; Detrend = linear is therefore the DSI/WSDI default, while StorageDeficit keeps the physical Reager convention (none). A robust sigma option (1.4826 MAD) survives single corrupt months that inflate the classical sigma sixfold. STAGE 2 is live too: on the ITSG daily Kalman solutions (fetchITSG Product = daily, 365 files/yr, n40) the index switches to a day-of-year climatology - per-DOY means, sigma from the RESIDUALS in a circular 31-day window that wraps December into January, with the sqrt(n/(n-1)) estimated-mean correction. The construction matters and was measured: window sigma on RAW values leaks the seasonal cycle (1.99 instead of 1.5), the two-stage form does not (1.52). Daily solutions track short-lived floods that monthly fields miss (Gouweleeuw et al. 2018, Ganges-Brahmaputra) - a planted +3-sigma January-2nd flood scores z = 2.9 across the year wrap.

```
[tws, rep] = shLowLevel.twsChain(ser, kn = kn);
[Z, inf1] = shLowLevel.hydroExtremeIndex(tws.grid, tws.epochs);
inf1.categoryNames(inf1.category(iLat, iLon, end) + 6)   % cell class
[Sd, ~] = shLowLevel.hydroExtremeIndex(basinSeries, ep, ...
    Mode = "StorageDeficit");                % flood predisposition
```

**Offline by design.** No chain ever fetches: greenland/antarctica/tws/ocean/obp and every filter consume LOCAL paths only. The GravIS correction tables (C20/C30/C21/S21, degree 1, NFIL mean, GIA rate, ice basins) ship frozen inside the toolbox (data/gravis), so a fresh clone plus one local GSM series folder runs the ice and TWS chains completely offline; the ocean chains additionally need the GAD/GAA folders on disk - however they got there (fetchGAX, a browser, a copied drive: the chains cannot tell and do not care). The fetch* functions are one-time provisioners with skip-if-present semantics - re-running them against a populated folder is a no-op, and the VDK batch runs fully offline once its SINEX archive exists. Path resolution is three layers, explicit first: a passed argument always wins; below it the setup_shAnalysis preferences (SeriesFolder, MasconFile, GravisFolder, DDKFolder - setpref-persistent) and dataFolder() as the root for fetch targets; at the bottom the shipped freezes. The on-disk layout since v3.16 is uniform: temporal products under series/ (series/itsg/monthly|daily|sinex|background, series/series/icgem/<name>lt;icgem-nameseries/icgem/<name>gt;, series/GAX/<product>), static models under static/, auxiliaries under DDK/ and TN/; the fetchers warn once when they find data in a pre-v3.16 location instead of silently re-downloading. The single deliberate exception is gravisL2B with an explicit GravisFolder= requesting fresh tables - and even there the shipped freeze is the default, precisely so reproducibility never hangs on a server (the

ISDC 503 episode was the confirmation).

**Coastal leakage, controlled twice.** The crude ocean mask plus gauss445 lets land signal - Greenland above all - smear across the coast into the ocean mean. Two independent controls, both quantified before implementation: `CoastBufferKm` (the public helper `shLowLevel.erodeMask`) erodes the mask by a great-circle distance from every coast (the Python point-source experiment for gauss445: 300 km removes 54%% of the leak, 500 km 82%%, at under 1%% ocean-area loss - 300 km is the Chambers-style standard), and `RemoveLandLeakage` separates the outside-mask sources by two-sided alternating support projections (Papoulis-Gerchberg style, LandLeakIter sweeps) and subtracts the converged land component before the filter. The pre-validation taught the hard lesson here: a naive ONE-step subtraction makes the leak WORSE, because the reconstruction of the outside field rings INTO the mask; the two-sided iteration cuts an adjacent-source leak of 47%% of the ocean mean by 94%% (1D reference, frozen in CI on the band-limited spherical experiment). Since v3.16.1 the 300 km buffer IS the default - the machine re-baseline moved the barystatic trend +1.407 to +1.557 mm/yr (500 km: +1.647), into the published ~1.6-2.2 band; CoastBufferKm = 0 reproduces the pre-v3.16.1 numbers. Changing a validated number happened here the only allowed way: measured first, then decided, then frozen.

```
[out, rep] = shLowLevel.oceanChain(ser, kn = kn, OceanMask = oc, ...
    GADFolder = gadF, GAAFolder = gaaF, ...
    CoastBufferKm = 300, RemoveLandLeakage = true);
```

```
f = shLowLevel.fetchGAX("E:/DATAPOOL/GravityField/GAX");
[out, rep] = shLowLevel.oceanChain(ser, kn = kn, OceanMask = oc, ...
    GADFolder = "E:/DATAPOOL/GravityField/GAX/GAD", ...
    GAAFolder = "E:/DATAPOOL/GravityField/GAX/GAA");
[rep.nGadRestored, rep.nGaaApplied]        % coverage, reported
```

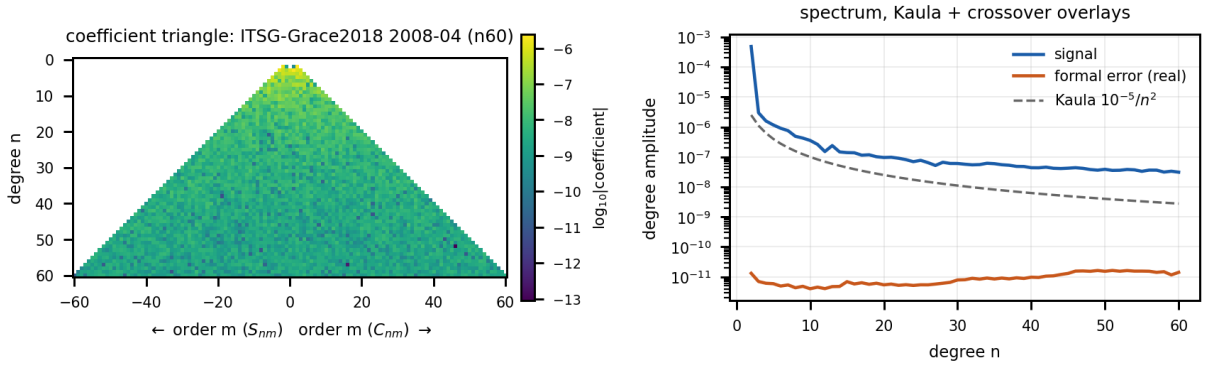
```
Idx= shLowLevel.shIndex(30, Nmin= 0);
[G, lam, info] = shLowLevel.slepianBasis(idx, @(la, lo) double(la > 60));
K = round(info.shannon);                   % ~N useful modes
a = shLowLevel.slepianProject(ts.Cs, ts.Ss, G(:, 1:K), idx);
plot(ts.epochs, a')                        % regional modes vs time
```

# Demo gallery

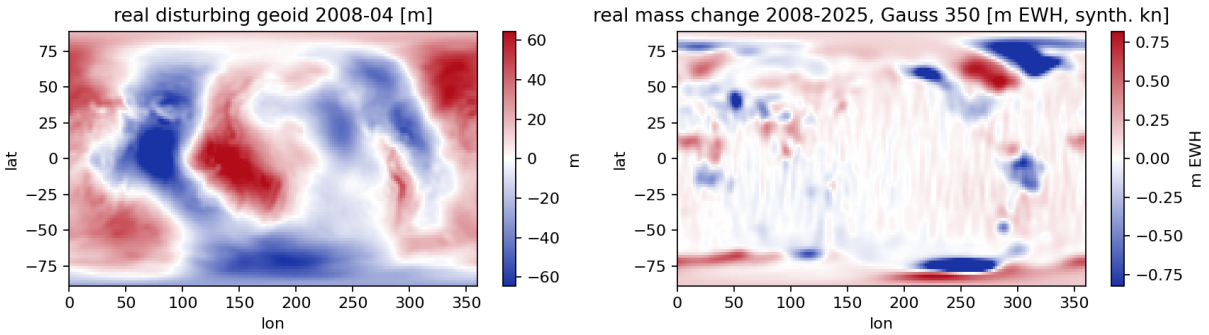
`demo_shAnalysis` is a case registry: `demo_shAnalysis("list")` prints the table below, `demo_shAnalysis("all")` runs everything, `demo_shAnalysis(["D04","D15"])` runs a selection, and Visible=false keeps figures hidden for smoke tests. Real files in tests/test_data (ITSG month, DDK3 Wbd) are used when present; every case falls back to synthetic data. Love numbers in the demos are SYNTHETIC — real work requires a real loading model.

| ID | Case | Exercises |
|---|---|---|
| D01 | Read & spectral diagnostics | shReadGFC, shDegreeRMS, triangle, spectrum |
| D02 | Synthesis quantities & maps | shSynthesis, kernelFactors, plotSHMap, Height |
| D03 | Normal field → geoid | normalFieldCS, subtractNormalField, toReference |
| D04 | Filter comparison | gaussian, fan, destripe, DDK, triangle diff |
| D05 | Climatology & basin series | climatology, basinKernel, plotBasinSeries |
| D06 | tvANS pipeline | buildNoiseCov, tvANSFilter, basinDeconvolve |
| D07 | Analysis (inverse) | shAnalysisGrid, Kaula |
| D08 | Basin tools | basinKernel taper, deconvolve vs scaling |
| D09 | Uncertainty | errorMap, mcPropagate, plotCovariance |
| D10 | Load deformation | shSynthesisDeformation, Mode=points |
| D11 | Gradient tensor | shSynthesisGradientTensor |
| D12 | EOF analysis | eofAnalysis |
| D13 | Trend breakpoints | trendBreaks, F-test |
| D14 | Multi-center combination | combineCenters, VCE weights |
| D15 | Sea-level fingerprint | seaLevelFingerprint, evalMask |
| D16 | Export | writeGrid netCDF, writeAnimation MP4 |

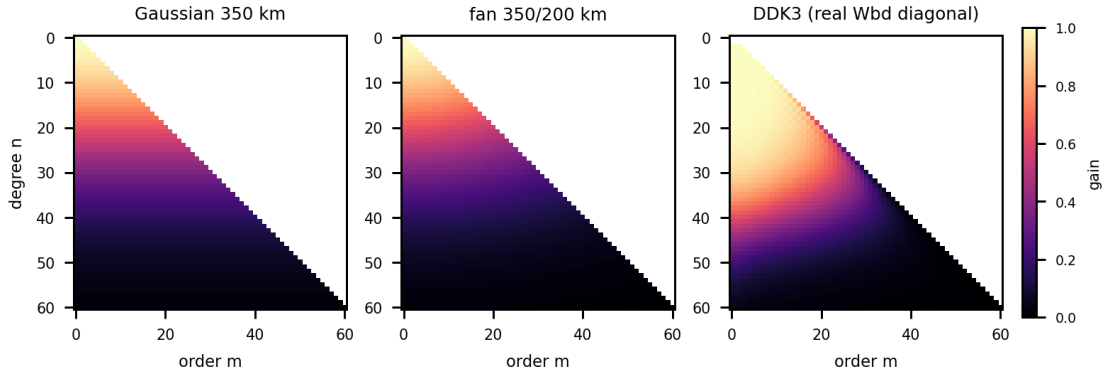
**Note.** Provenance of the figures below: rendered with the toolbox's Python validation port — the same algorithms that gate every MATLAB implementation, numerically equivalent by construction. D01/D02 use the REAL ITSG files shipped in tests/test_data (and D04 the real DDK3 Wbd binary); the remaining figures use synthetic data with known ground truth, because recovery demonstrations (VCE factors, EOF modes, F-test calibration) require a truth to recover. `shLowLevel.fetchITSG(years)` extends the real-data demos to full monthly series. The MATLAB demo cases produce the interactive originals.



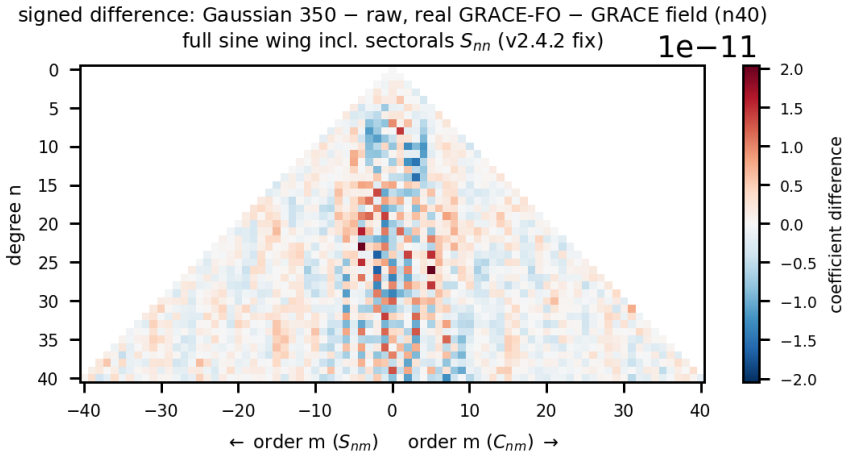

**D01 — REAL DATA:** the shipped ITSG-Grace2018 2008-04 month (rendered by the rebuilt Python port, v2.5). Coefficient triangle (apex up; C00/C20 removed for display) and the degree-amplitude spectrum with the REAL formal errors from the gfc file. At this n60 truncation the formal errors stay ~3 orders below the signal - the signal/error crossover (the resolution limit) lies beyond the truncation degree; Kaula decay as the yardstick.



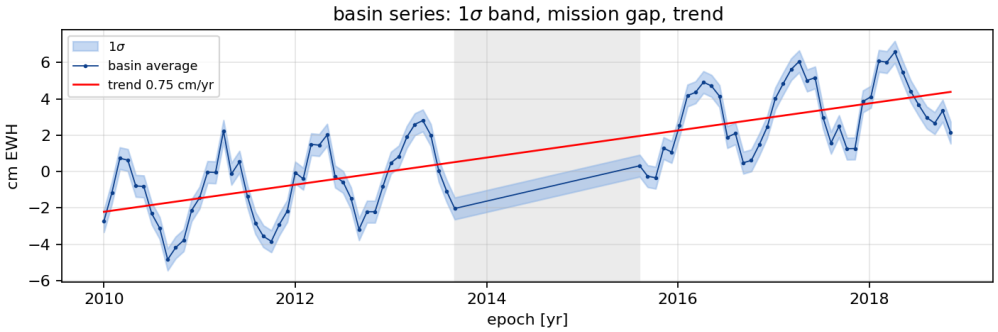

**D02 — REAL DATA.** Left: the disturbing geoid of 2008-04 after subtracting the WGS84 normal field computed from defining constants and rescaled to the ITSG (GM, R) — the classic ±100 m map (Indian Ocean low, New Guinea high). Right: the real 2008-2025 mass change (GRACE-FO minus GRACE, Gaussian 350 km, EWH): Greenland and West Antarctic mass loss stand out; residual meridional stripes at low latitudes are the real GRACE error structure. Honest caveats: synthetic Love numbers, no GIA correction, degree 1 absent — patterns are real, amplitudes approximate.



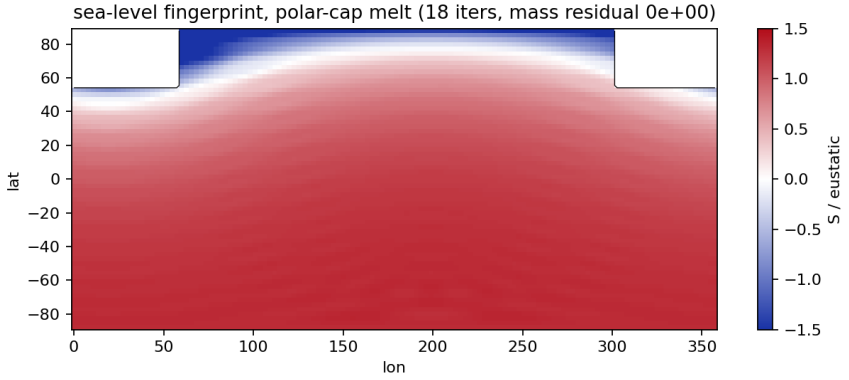

**D04 — Filter gain over the (degree, order) triangle.** Gaussian: isotropic in degree. Fan (Han): the order Gaussian additionally damps high orders — the striping direction. DDK3 (diagonal of the real Wbd blocks): anisotropic and order-dependent — near-zonal signal survives to n~35 while high orders are suppressed hard; exactly the structure a GRACE error covariance demands.



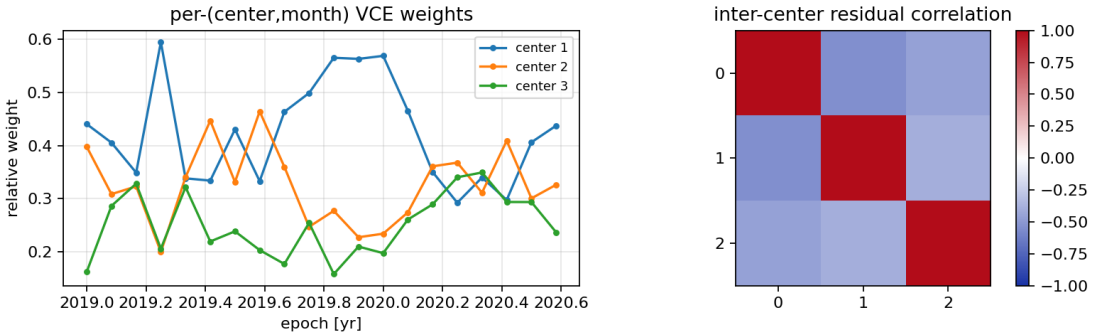

**D04b — REAL DATA, v2.4.2 fix:** the signed difference triangle (Gaussian 350 – raw) on the real GRACE-FO – GRACE field at n40 — the removed signal concentrates at the low orders where the real stripes and secular signal live. The sine wing now spans the full n ≥ m ≥ 1 support including the sectorals $S_{nn}$: before v2.4.2 the difference mode blanked the sectoral edge and every diff triangle rendered the left wing one column narrower than the right (regression-tested on the rendered CData since).



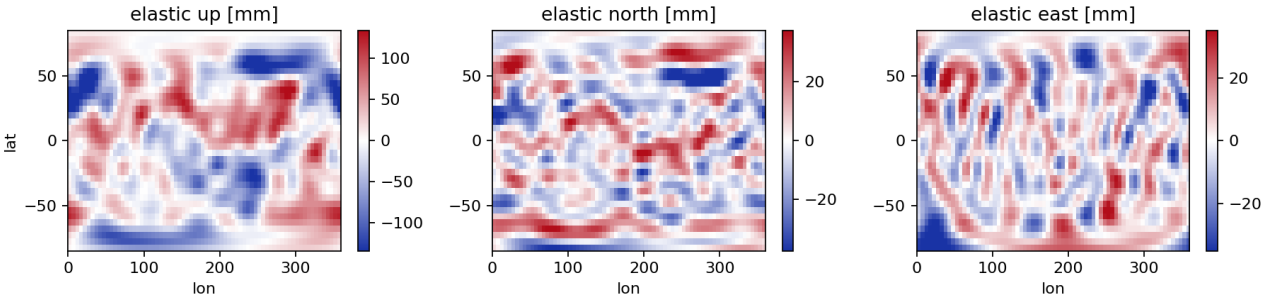

**D05/D06 — The standard basin figure** from plotBasinSeries: 1-sigma band, grey mission-gap patch, AR(1)-corrected trend annotation. With tvANS + basinDeconvolve the band comes from propagated posterior covariances instead of an empirical scatter.



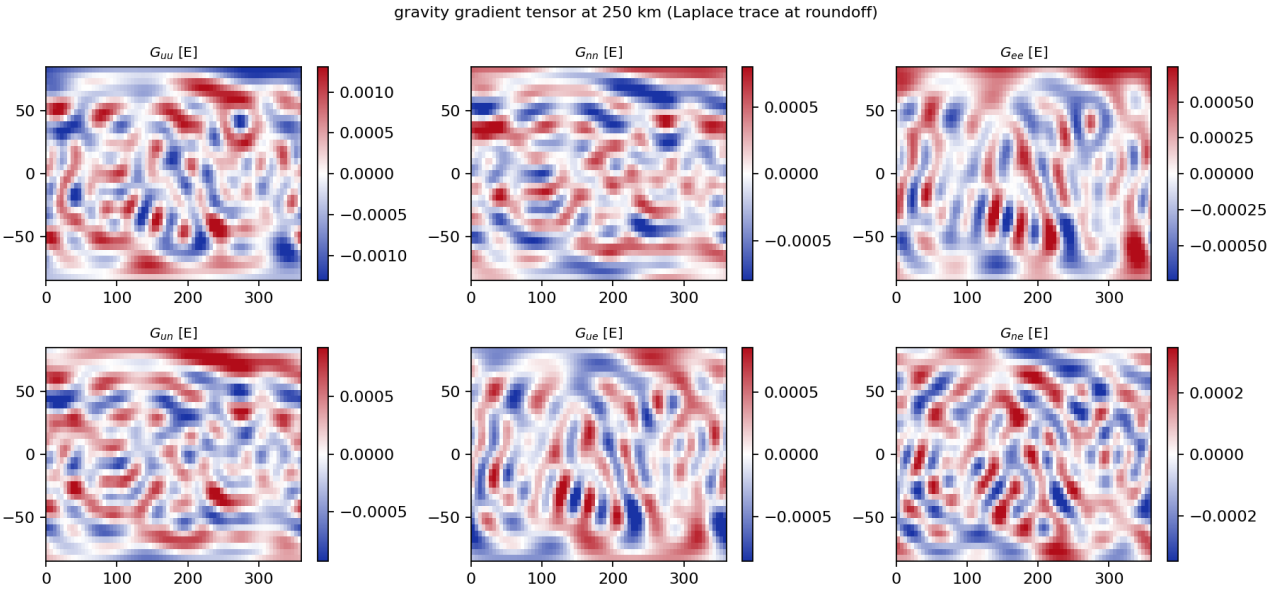

**D15 — Sea-level fingerprint** of a polar-cap melt, normalized by the eustatic mean: the near field FALLS (blue; lost self-attraction plus crustal rebound), the far field exceeds eustatic by ~20-30%. Mass conserved to machine precision — the printed residual is the built-in check.



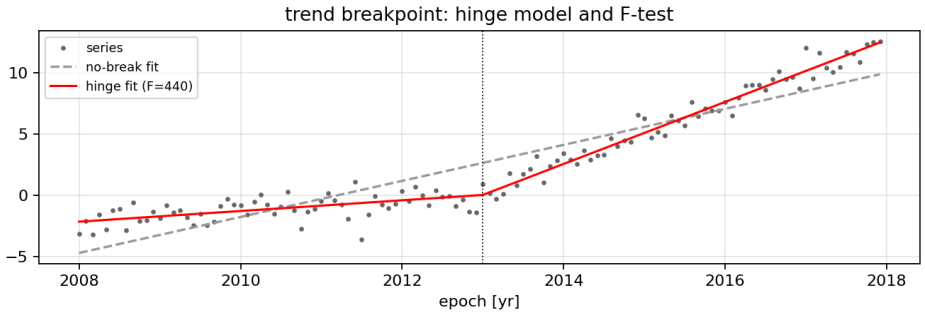

**D14 — Multi-center VCE:** per-(center, month) weights track the time-varying noise levels; the inter-center residual correlation matrix is the honesty diagnostic — VCE assumes independence, GRACE centers share data and background models.




**D10 — Elastic load deformation:** up, north, east from the same residual field. Horizontals are ~1/3 of the vertical (validation run: 0.38) and require the exact SH gradient (legendreALFDeriv); east is undefined at the poles.




**D11 — All six gradient tensor components** at 250 km in Eotvos. The Laplace invariant $G_{nn} + G_{ee} + G_{uu} = 0$ holds at roundoff (4e-15 of max|G| here) — a full-chain check of first and second Legendre derivatives, trig derivatives and frame terms at once.




**D13 — Trend breakpoint:** continuous piecewise-linear hinge model vs the no-break fit; the F-test (p via betainc, base MATLAB) quantifies the break. Null rejection is calibrated at the nominal level (5.1% at 5%).

# Part IV — API reference

Generated from the source: the **arguments blocks are the ground truth** for input names, dimensions, classes and defaults (default 'required' = mandatory positional input); name-value options are passed as Name=Value. Conventions throughout: coefficient triangles are addressed C(n+1, m+1); vectors follow shLowLevel.shIndex ordering (Nmin= 2 unless stated); latitudes are geocentric degrees; radii in km; epochs in decimal years. Every entry closes with a real-data example from the canonical GRACE chain; all examples are lint- and API-gated at build time.

## shCoefficients class

Spherical harmonic (Stokes) coefficient set - single epoch. The single point of access for one gravity field: a GRACE/GRACE-FO monthly solution (GSM), a dealiasing product (GAA/GAB/GAC/GAD), a static model, or any derived field (difference, filtered, climatology component). Immutable value class: every operation returns a NEW object and appends to .history (full provenance).

| property | size | type | default | notes |
|---|---|---|---|---|
| C | — | double | — | |
| S | — | double | — | |
| sigmaC | — | double | [] | |
| sigmaS | — | double | [] | |
| GM | 1 x 1 | double | 3.986004415e14 | |
| R | 1 x 1 | double | 6378136.3 | |
| epoch | 1 x 1 | double | NaN | |
| productType | 1 x 1 | string | "unknown" | |
| tideSystem | 1 x 1 | string | "unknown" | |
| name | 1 x 1 | string | "" | |
| header | — | struct | struct() | |
| variableTerms | — | — | [] | |
| history | — | string | string.empty(0,1) | |
| nmax | — | — | — | |

### shCoefficients.shCoefficients

```
obj = g.shCoefficients(C, S, Name=Value)
```

Construct from coefficient matrices.

| input | size | type | default | description |
|---|---|---|---|---|
| C | nmax+1 x nmax+1 | double | 0 | double cosine coefficients, C(n+1, m+1) layout |
| S | nmax+1 x nmax+1 | double | 0 | double sine coefficients, same layout (S(:,1) = 0) |
| SigmaC | n x m | double | [] | formal 1-sigma of the cosine coefficients, same layout as C ([]: none attached) |
| SigmaS | n x m | double | [] | formal 1-sigma of the sine coefficients |
| GM | 1 x 1 | double | 3.986004415e14 | gravitational constant times Earth mass [m^3/s^2] (default 3.986004415e14, overridable) |
| R | 1 x 1 | double | 6378136.3 | reference radius [m] (default 6378136.3, overridable) |
| Epoch | 1 x 1 | double | NaN | decimal year of the solution; required by every epoch-matched operation (applyTN14, addDegree1, restore) |

| input | size | type | default | description |
|---|---|---|---|---|
| ProductType | 1 x 1 | string | "unknown" | provider product code, e.g. "GSM", "GAD", "GAA" - arithmetic uses it to catch mixing levels |
| TideSystem | 1 x 1 | string | "unknown" | "tide_free" \| "zero_tide" \| "mean_tide" \| "unknown"; carried through and written back out, never converted silently |
| Name | 1 x 1 | string | "" | model label used in displays and plot titles |
| Header | n x m | struct | struct() | ) parsed gfc header fields, kept verbatim so writeGFC can round-trip them |
| VariableTerms | n x m | — | [] | ICGEM 2.0 gfct terms (trend/periodic per coefficient) evaluated by evalAt |
| History | 1 x 1 | string | string.empty(0, 1) | ) initial processing history; every operation appends one line |

| output | description |
|---|---|
| obj | (1 x 1) shCoefficients immutable coefficient set with GM (3.986004415e14), R (6378136.3), epoch, sigmas, history |

### shCoefficients.plus

```
out = g.plus(a, b)
```

Coefficient addition (e.g. GSM + GAD background restore).

| input | size | type | default | description |
|---|---|---|---|---|
| a | — | — | required | — |
| b | — | — | required | — |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients sum; sigmas RSS; epochs must match |

### shCoefficients.minus

```
out = g.minus(a, b)
```

Coefficient difference (e.g. monthly change gB - gA).

| input | size | type | default | description |
|---|---|---|---|---|
| a | — | — | required | — |
| b | — | — | required | — |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients difference; sigmas RSS |

### shCoefficients.uminus

```
out = g.uminus(a)
```

Unary minus: negate all coefficients.

| input | size | type | default | description |
|---|---|---|---|---|
| a | — | — | required | — |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients negated coefficients (sigmas kept) |

### shCoefficients.mtimes

```
out = g.mtimes(a, b)
```

Scalar scaling.

| input | size | type | default | description |
|---|---|---|---|---|
| a | — | — | required | — |
| b | — | — | required | — |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients scalar-scaled; sigmas scale by \|a\| |

### shCoefficients.compare

```
[rep, h] = g.compare(other, varargin)
```

Full comparison against another solution (v2.6.0). REP = g.compare(G2, ...) delegates to shLowLevel.compareSolutions with OBJ as the first solution; all options pass through (Plot=true for the 4-panel figure).

| input | size | type | default | description |
|---|---|---|---|---|
| other | — | — | required | — |
| varargin | — | — | required | — |

| output | description |
|---|---|
| rep | (1,1) struct as returned by shLowLevel.compareSolutions, which documents every field |
| h | (1 x 1) graphics handle figure (Plot = true) |

### shCoefficients.times

```
out = g.times(a, b)
```

Elementwise scaling .* (scalar or triangle matrix). W .* G scales every coefficient by the matching entry of the (nmax+1 x nmax+1) matrix W (per-coefficient weighting or tapering); a scalar factor delegates to mtimes. Sigmas scale by |factor| elementwise (v2.5.1).

| input | size | type | default | description |
|---|---|---|---|---|
| a | — | — | required | — |
| b | — | — | required | — |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients elementwise-scaled coefficients |

### shCoefficients.truncate

```
out = g.truncate(nmaxNew)
```

Reduce the maximum degree.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| nmaxNew | 1 x 1 | double | required | double new maximum degree (must not exceed the current nmax) |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients truncated to the new nmax |

### shCoefficients.setCoefficient

```
out = g.setCoefficient(n, m, Cval, Sval, Name=Value)
```

Replace a single (n,m) coefficient pair.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| n | 1 x 1 | double | required | spherical harmonic degree |
| m | 1 x 1 | double | required | spherical harmonic order |
| Cval | 1 x 1 | double | required | double new C(n+1, m+1) value |
| Sval | 1 x 1 | double | NaN | double new S(n+1, m+1) value (NaN keeps the current one) |
| SigmaC | 1 x 1 | double | NaN | formal 1-sigma for the coefficient being set (NaN leaves the existing sigma untouched) |
| SigmaS | 1 x 1 | double | NaN | same for the sine coefficient |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients copy with C/S(n+1, m+1) replaced |

### shCoefficients.applyTN14

```
out = g.applyTN14(tn, Name=Value)
```

Replace C20 (and C30 where provided) from a TN-14 table. OUT = obj.applyTN14(TN) with TN from shLowLevel.readTN14 (or a filename). Requires a finite obj.epoch; the nearest TN epoch within opts.Tolerance (default 0.05 yr) is used.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| tn | n x m | — | required | correction-table struct as returned by the matching reader |
| Tolerance | 1 x 1 | double | 0.05 | epoch matching tolerance [yr] |
| ReplaceC30 | 1 x 1 | string ∈ ["auto","never","always"] | "auto" | "auto" replaces C30 when the table carries it, "always"/"never" force the choice opts.ReplaceC30: "auto" (default: replace when the table has a non-NaN C30 f... |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients C20 (and C30 when available) replaced by the SLR values, sigmas updated |

### shCoefficients.addDegree1

```
out = g.addDegree1(tn, Name=Value)
```

Insert TN-13 geocenter (degree-1) coefficients. OUT = obj.addDegree1(TN) with TN from shLowLevel.readTN13 (or a filename) sets C10, C11, S11 (with their sigmas) from the TN-13 record nearest to obj.epoch (within opts.Tolerance (0.05), default 0.05 yr). Without degree 1, EWH/mass grids are systematically biased - this completes the standard GSM + TN-14 + TN-13 processing chain.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| tn | n x m | — | required | correction-table struct as returned by the matching reader |
| Tolerance | 1 x 1 | double | 0.05 | epoch matching tolerance [yr] |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients degree-1 row set from the TN-13 record nearest to obj.epoch |

### shCoefficients.destripe

```
out = g.destripe(Name=Value)
```

Swenson & Wahr (2006) / P3M6 decorrelation. OUT = obj.destripe(MinOrder=6, PolyOrder=3, WindowLength=[]) Formal errors are passed through UNCHANGED (the destriping operator is data-dependent; rigorous error propagation is not defined for it - use the tvANS filter on an shSeries for a filter with proper covariance semantics).

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| MinOrder | 1 x 1 | double | 6 | lowest order the decorrelation touches |
| PolyOrder | 1 x 1 | double | 3 | order of the fitted/removed polynomial |
| WindowLength | n x m | double | [] | sliding-window length; [] uses the Swenson-Wahr degree-dependent width |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients Swenson-Wahr decorrelated |

### shCoefficients.gaussian

```
out = g.gaussian(radiusKm)
```

Jekeli (1981) isotropic smoothing; sigmas scaled by Wn.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| radiusKm | 1 x 1 | double | required | double Gaussian filter half-response radius [km] |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients Gaussian-smoothed; sigmas scaled by Wn |

### shCoefficients.degreeRMS

```
spec = g.degreeRMS(Name=Value)
```

Degree variance/RMS/amplitude spectrum (+ error spectrum).

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| Nmin | 1 x 1 | double | 0 | first degree included in the spectrum (renamed from N0 in v4.0.0) |

| output | description |
|---|---|
| spec | (1,1) struct the degree spectrum, including the formal-error curve, as returned by shLowLevel.shDegreeRMS |

### shCoefficients.crossover

```
[n0, nInterp] = g.crossover(Name=Value)
```

Signal-vs-formal-error crossover degree.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| Nmin | 1 x 1 | double | 2 | first degree searched for the signal/error crossover (renamed from N0 in v4.0.0) |

| output | description |
|---|---|
| nInterp | (1,1) double interpolated crossing degree |

### shCoefficients.spectrum

```
h = g.spectrum(varargin)
```

Plot the degree-amplitude spectrum. h = obj.spectrum(...)

| input | size | type | default | description |
|---|---|---|---|---|
| varargin | — | — | required | — |

| output | description |
|---|---|
| h | (1 x 1) graphics handle spectrum plot |

### shCoefficients.triangle

```
h = g.triangle(varargin)
```

Plot the degree/order coefficient triangle. h = obj.triangle(...)

| input | size | type | default | description |
|---|---|---|---|---|
| varargin | — | — | required | — |

| output | description |
|---|---|
| h | (1 x 1) graphics handle coefficient triangle plot |

### shCoefficients.synthesis

```
[grid, lat, lon] = g.synthesis(latVec, lonVec, Name=Value)
```

Spatial synthesis (geoid, gravity, potential, EWH). [GRID, LAT, LON] = obj.synthesis(LATVEC, LONVEC, Quantity ("geoid")=..., kn ([])=..., Nmin (0)=..., Nmax (NaN)=..., UseCache (true)=true). The Legendre functions are served from a verified process-wide cache (shLowLevel.legendreCached), so monthly time series on a fixed grid pay the recursion only once - no manual 'P' passing. EWH requires explicitly supplied load Love numbers kn (never hardcoded).

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| latVec | 1 x : | double | required | geocentric latitude vector of the grid [deg] |
| lonVec | 1 x : | double | required | longitude vector of the grid [deg], [0, 360) |
| Quantity | 1 x 1 | string | "geoid" | functional of the field, see shLowLevel.kernelFactors |
| kn | n x m | double | [] | load Love numbers, (degree, kn) table or column vector - always user-supplied, no reference frame is assumed |
| hn | n x m | double | [] | vertical-deformation Love numbers, degrees 0..nmax (user-supplied) |
| Height | 1 x 1 | double | 0 | evaluation height [m] above the evaluation surface; applies the upward continuation (R/r)^p_n |
| Surface | 1 x 1 | string ∈ ["sphere","ellipsoid"] | "sphere" | "sphere" \| "ellipsoid": evaluate on the sphere r = R or on the ellipsoid r(phi), see shLowLevel.shSynthesis. It is the SAME ellipsoid the Flattening option d... |
| SemiMajorAxis | 1 x 1 | double | 6378137 | ellipsoid semi-major axis [m], WGS84 |
| Nmin | 1 x 1 | double | 0 | minimum spherical harmonic degree |
| Nmax | 1 x 1 | double | NaN | maximum spherical harmonic degree |
| rho_ave | 1 x 1 | double | 5517 | mean Earth density [kg/m^3] in the EWH kernel |
| rho_water | 1 x 1 | double | 1000 | water density [kg/m^3] in the EWH kernel |
| UseCache | 1 x 1 | logical | true | reuse the Legendre cache between calls |
| Method | 1 x 1 | string ∈ ["auto","direct","fft"] | "auto" | "auto" \| "direct" \| "fft": FFT along longitude is much faster on uniform full-circle longitude vectors; "auto" uses it whenever it is applicable |

| input | size | type | default | description |
|---|---|---|---|---|
| LatType | 1 x 1 | string ∈ ["geocentric", "geodetic"] | "geocentric" | "geocentric" \| "geodetic": how the latitude inputs are interpreted; geodetic values are converted with Flattening before evaluation |
| Flattening | 1 x 1 | double | 1/298.257223563 | flattening of the reference ellipsoid for the geodetic/geocentric conversion and, with Surface = "ellipsoid", of the evaluation surface (WGS84; overridable, ... |
| MaxMemGB | 1 x 1 | double | 4 | memory budget [GB]; above it the synthesis streams in latitude bands instead of allocating the full design matrix (this is what makes nmax 2190 tractable) |

| output | description |
|---|---|
| grid | (nlat x nlon) double synthesized field |
| lat | (1 x nlat) double the latitudes AS GIVEN, in the convention named by LatType - not the internally converted ones (changed in v3.31.0) |
| lon | (1 x nlon) double longitudes |

### shCoefficients.toReference

```
out = g.toReference(Name=Value)
```

Convert to another (GM (NaN), R (NaN)) reference (exact). OUT = g.toReference(GM=3.986004418e14, R=6378137.0) re-expresses the coefficients via shLowLevel.rescaleGMR; the physical field is invariant (Python-validated). Sigmas rescale identically; GM/R properties are updated.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| GM | 1 x 1 | double | NaN | gravitational constant times Earth mass [m^3/s^2] (default 3.986004415e14, overridable) |
| R | 1 x 1 | double | NaN | reference radius [m] (default 6378136.3, overridable) |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients rescaled to the given GM/R |

### shCoefficients.subtractNormalField

```
out = g.subtractNormalField(Name=Value)
```

Remove the ellipsoidal normal field. OUT = g.subtractNormalField() % WGS84 OUT = g.subtractNormalField(System ("WGS84")="GRS80") Computes the even zonals from the DEFINING constants (shLowLevel.normalFieldCS, Heiskanen-Moritz closed form; validated against NIMA TR8350.2 to all published digits), rescales them from the ellipsoid's own (GM (NaN), a (NaN)) to the object's (GM, R) - the WGS84 GM and a DIFFER from the ICGEM values, skipping this step costs millimetres - and subtracts them including the deg...

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| System | 1 x 1 | string ∈ ["WGS84","GRS80"] | "WGS84" | normal-field system, e.g. 'GRS80' or 'WGS84' |
| GM | 1 x 1 | double | NaN | gravitational constant times Earth mass [m^3/s^2] (default 3.986004415e14, overridable) |
| a | 1 x 1 | double | NaN | semi-major axis of the normal ellipsoid [m] |
| f | 1 x 1 | double | NaN | flattening of the normal field (NaN: the System default; give GM, a, f and omega together to define your own) |
| omega | 1 x 1 | double | NaN | angular velocity [rad/s] of the normal field |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients disturbing field (even zonals of the normal field removed) |

### shCoefficients.map

```
H = g.map(latDeg, lonDeg, Name=Value)
```

Synthesize and draw in one call, through geoMap. H = g.map(-89:89, 0:359, Quantity ("ewh")=..., kn ([])=...) builds the handover struct with g.toGrid and hands it to geo.map. It draws NOTHING itself.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| latDeg | 1 x : | double | required | double latitudes [deg] |
| lonDeg | 1 x : | double | required | double longitudes [deg] |
| Quantity | 1 x 1 | string | "ewh" | any kernelFactors quantity |
| kn | n x m | double | [] | load Love numbers k'_n (mass quantities; never hardcoded) |
| hn | n x m | double | [] | load Love numbers h'_n (deformation_up only) |
| Nmin | 1 x 1 | double | 0 | degrees below nmin are zeroed |
| Nmax | 1 x 1 | double | NaN | truncation degree; NaN uses the object's nmax |
| Height | 1 x 1 | double | 0 | evaluation height [m] above the evaluation surface |
| rho_ave | 1 x 1 | double | 5517 | mean Earth density [kg/m^3] |
| rho_water | 1 x 1 | double | 1000 | water density [kg/m^3] |
| LatType | 1 x 1 | string ∈ ["geocentric", "geodetic"] | "geodetic" | "geodetic" \| "geocentric"; coastlines are geodetic, so a map wants geodetic |
| Flattening | 1 x 1 | double | 1/298.257223563 | ellipsoid flattening (WGS84), for LatType and for Surface = "ellipsoid" |
| Surface | 1 x 1 | string ∈ ["sphere","ellipsoid"] | "sphere" | "sphere" \| "ellipsoid": evaluate on the sphere r = R or on the ellipsoid r(phi), as g.synthesis |
| SemiMajorAxis | 1 x 1 | double | 6378137 | ellipsoid semi-major axis [m], WGS84 |
| CRS | 1 x 1 | string | "equirectangular" | projection, forwarded to geo.map |
| Basemap | 1 x 1 | struct | struct() | ) forwarded to geo.map's Basemap element; a robust symmetric CLim and the Divergent flag are inserted unless you set them yourself |
| GeoArgs | n x m | cell | {} | name-value pairs forwarded to geo.map verbatim |

| output | description |
|---|---|
| H | (1,1) struct the graphics handle struct, as returned by geoMap's map function - this toolbox draws no map of its own |

*Example*

```
    H = g.map(-89.5:89.5, 0.5:359.5, Quantity = "ewh", kn = kn, ...
            CRS = "robinson");
```

### shCoefficients.fan

```
out = g.fan(rDegKm, rOrdKm)
```

Han fan filter (degree x order Gaussian) - shLowLevel.shFanFilter.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |

| input | size | type | default | description |
|---|---|---|---|---|
| rDegKm | 1 x 1 | double | required | double fan filter degree-direction radius [km] |
| rOrdKm | 1 x 1 | double | required | double fan filter order-direction radius [km] |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients fan-filtered |

### shCoefficients.deformation

```
[up, north, east] = g.deformation(latDeg, lonDeg, Name=Value)
```

Elastic load deformation up/north/east [m]. [UP, NORTH, EAST] = g.deformation(LAT, LON, kn=, hn=, ln=, Mode ("grid")="grid"|"points", Nmin (1)=1, LatType ("geocentric")="geocentric") from the object's (residual!) coefficients - remove a mean field first. Love numbers are always user-supplied; see shLowLevel.shSynthesisDeformation for formulas and validation.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| latDeg | 1 x : | double | required | double latitudes [deg, geocentric] |
| lonDeg | 1 x : | double | required | double longitudes [deg] |
| kn | n x m | double | required | load Love numbers, (degree, kn) table or column vector - always user-supplied, no reference frame is assumed |
| hn | n x m | double | required | load Love numbers h_n, same layout as kn |
| ln | n x m | double | required | load Love numbers l_n (horizontal), same layout as kn |
| Nmin | 1 x 1 | double | 1 | minimum spherical harmonic degree |
| Mode | 1 x 1 | string ∈ ["grid","points"] | "grid" | operating mode of the routine (see the function help) |
| LatType | 1 x 1 | string ∈ ["geocentric", "geodetic"] | "geocentric" | 'geocentric' (native) or 'geodetic' (converted via Flattening) |
| Flattening | 1 x 1 | double | 1/298.257223563 | ellipsoid flattening for the geodetic/geocentric conversion of the station latitudes |

| output | description |
|---|---|
| up | (nlat x nlon \| npts) double vertical deformation [m] |
| north | same size north component [m] |
| east | same size east component [m] |

### shCoefficients.gradient

```
[dq, info] = g.gradient(latDeg, lonDeg, Name=Value)
```

North/east/up gradient of a synthesized quantity. [DQ, INFO] = g.gradient(LAT, LON, Quantity ("geoid")=..., Units ("perMeter")=..., Mode ("grid")=...) is the exact spatial derivative of what g.synthesis would return under the SAME options - the vector field to draw over a scalar map, or to advect tracers through. DQ.up is EMPTY for surface quantities (ewh, geoid, surface_density, bottom_pressure, deformation_up, none), which have no radial law; INFO.hasUp says so. See shLowLevel.shSynthesisGradient for the formu...

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| latDeg | 1 x : | double | required | double latitudes [deg] |
| lonDeg | 1 x : | double | required | double longitudes [deg] |
| Quantity | 1 x 1 | string | "geoid" | any kernelFactors quantity |

| input | size | type | default | description |
|---|---|---|---|---|
| kn | n x m | double | [] | load Love numbers k'_n (mass quantities; never hardcoded) |
| hn | n x m | double | [] | load Love numbers h'_n (deformation_up only) |
| Nmin | 1 x 1 | double | 0 | degrees below nmin are zeroed |
| Nmax | 1 x 1 | double | NaN | truncation degree; NaN uses the object's nmax |
| Height | 1 x 1 | double | 0 | evaluation height [m] above the evaluation surface; surface quantities reject it |
| Surface | 1 x 1 | string ∈ ["sphere","ellipsoid"] | "sphere" | "sphere" \| "ellipsoid", see shLowLevel.shSynthesisGradient; the components stay in the local spherical frame either way |
| SemiMajorAxis | 1 x 1 | double | 6378137 | ellipsoid semi-major axis [m], WGS84 |
| rho_ave | 1 x 1 | double | 5517 | mean Earth density [kg/m^3] |
| rho_water | 1 x 1 | double | 1000 | water density [kg/m^3] |
| Mode | 1 x 1 | string ∈ ["grid","points"] | "grid" | "grid" \| "points" (pairwise LAT/LON) |
| Units | 1 x 1 | string ∈ ["perMeter","perRadian"] | "perMeter" | "perMeter" \| "perRadian" (= r x perMeter) |
| MaxMemGB | 1 x 1 | double | 4 | Legendre memory budget [GB] |
| LatType | 1 x 1 | string ∈ ["geocentric", "geodetic"] | "geocentric" | "geocentric" \| "geodetic": how the latitude inputs are read. NOTE that only the evaluation POINT is converted - the components stay in the geocentric local f... |
| Flattening | 1 x 1 | double | 1/298.257223563 | ellipsoid flattening for that conversion and for Surface = "ellipsoid" (WGS84; overridable, never silently assumed) |

| output | description |
|---|---|
| dq | (1,1) struct as returned by shLowLevel.shSynthesisGradient: north, east and up, each (nlat x nlon) double or (npts x 1) for Mode="points"; up is [] for surface quantities |
| info | (1,1) struct as returned by shLowLevel.shSynthesisGradient (quantity, units, r, surface, nmin, nmax, hasUp, lat, lon, latType), except that lat and latType are overwritten here when geodetic input was given, so the convention travels with the numbers |

*Example*

```
lat = -89:2:89; lon = 0:2:358;
[dq, info] = g.gradient(lat, lon, Quantity = "ewh", kn = kn);
info.hasUp        % false - EWH is a surface quantity
```

### shCoefficients.toGrid

`b = g.toGrid(latDeg, lonDeg, Name=Value)`

Synthesized field plus the metadata a map needs to draw it. B = g.toGrid(LAT, LON, Quantity ("ewh")=..., kn ([])=..., LatType ("geodetic")=...) synthesizes exactly as g.synthesis does and returns the values TOGETHER WITH what they are: units, latitude convention, product type, epoch, provenance, and whether the field is a full field or an anomaly.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| latDeg | 1 x : | double | required | double latitudes [deg], convention per LatType |
| lonDeg | 1 x : | double | required | double longitudes [deg] |

| input | size | type | default | description |
|---|---|---|---|---|
| Quantity | 1 x 1 | string | "ewh" | any kernelFactors quantity; the default is the one a map of a GRACE field almost always wants |
| kn | n x m | double | [] | load Love numbers k'_n (mass quantities; never hardcoded) |
| hn | n x m | double | [] | load Love numbers h'_n (deformation_up only) |
| Nmin | 1 x 1 | double | 0 | degrees below nmin are zeroed |
| Nmax | 1 x 1 | double | NaN | truncation degree; NaN uses the object's nmax |
| Height | 1 x 1 | double | 0 | evaluation height [m] above the evaluation surface |
| rho_ave | 1 x 1 | double | 5517 | mean Earth density [kg/m^3] |
| rho_water | 1 x 1 | double | 1000 | water density [kg/m^3] |
| LatType | 1 x 1 | string ∈ ["geocentric", "geodetic"] | "geodetic" | "geodetic" \| "geocentric" |
| Flattening | 1 x 1 | double | 1/298.257223563 | ellipsoid flattening (WGS84), for LatType and for Surface = "ellipsoid" |
| Surface | 1 x 1 | string ∈ ["sphere","ellipsoid"] | "sphere" | "sphere" \| "ellipsoid", as g.synthesis |
| SemiMajorAxis | 1 x 1 | double | 6378137 | ellipsoid semi-major axis [m], WGS84 |
| Method | 1 x 1 | string ∈ ["auto","direct","fft"] | "auto" | "auto" \| "direct" \| "fft" |
| MaxMemGB | 1 x 1 | double | 4 | synthesis memory budget [GB] |

| output | description |
|---|---|
| b | (1,1) struct the handover bundle, as built by shLowLevel.gridHandover, which documents every field: Z (nlat x nlon) double values; lat (nlat x 1) AS GIVEN; lon (1 x nlon); the string fields Units, LatType, Quantity, ProductType, Source, Name and FieldKind; the scalars Epoch, C00, Zmin, Zmax and Zmean; and History |

*Example*

```
b = g.toGrid(-89.5:89.5, 0.5:359.5, Quantity = "ewh", kn = kn);
b.Units          % "m"
b.FieldKind      % "anomaly" for a residual field
```

### shCoefficients.basinAverage

`avg = g.basinAverage(B)`

Basin averages of one field from basin kernels. AVG = g.basinAverage(B) is the naive basin average B'*x ./ diag(B'*B), one value per basin, with x the coefficient vector of G. It is the one-epoch case of shSeries.basinAverage and delegates to it, so the two cannot disagree. The exact deconvolution (Deconvolve = true) needs the stored operator of a filtered SERIES, so it stays on shSeries.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| B | P x K | double | required | double basin kernels in shLowLevel.shIndex ordering for this nmax, exactly as shSeries.basinAverage takes them (from a grid mask with shLowLevel.synthesisMa... |

| output | description |
|---|---|
| avg | (K x 1) double basin averages, one per kernel column |

*Example*

```
avg = g.basinAverage(B);   % B: P x K, as for ts.basinAverage
```

### shCoefficients.errorMap

`[sig, latDeg, lonDeg] = g.errorMap(latDeg, lonDeg, Name=Value)`

Formal-error map of a synthesized quantity. SIG = g.errorMap(LAT, LON) is the pointwise 1-sigma of what g.synthesis returns on the grid LAT x LON, propagated from G's own formal sigmas (sigmaC, sigmaS) taken as a DIAGONAL covariance. SIG = g.errorMap(LAT, LON, Covariance = M, Index = IDX) propagates a full P x P covariance in IDX ordering instead, e.g. from shLowLevel.readSINEX(..., Output = "covariance"). Either way it is shLowLevel.errorMap with G's GM and R filled in. Quantity, kn, hn and Height go to shLowLe...

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| latDeg | 1 x : | double | required | double geocentric latitudes [deg] |
| lonDeg | 1 x : | double | required | double longitudes [deg] |
| Covariance | n x m | double | [] | (P x P) double covariance in Index ordering; [] builds diag(sigma.^2) from sigmaC and sigmaS |
| Index | n x m | — | [] | shLowLevel.shIndex struct Covariance is ordered by; [] uses shLowLevel.shIndex(nmax) |
| Quantity | lane default | — | required | the quantity whose error is mapped |
| kn | lane default | — | required | load Love numbers, for mass quantities |
| hn | lane default | — | required | vertical-deformation Love numbers |
| Height | lane default | — | required | evaluation height above R [m] |

| output | description |
|---|---|
| sig | (nlat x nlon) double 1-sigma of the field, in the unit of Quantity |
| latDeg | (1,nlat) double latitudes used [deg], as passed in |
| lonDeg | (1,nlon) double longitudes used [deg], as passed in |

*Example*

```
    sig = g.errorMap(-89:2:89, 0:2:358, Quantity = "ewh", kn = kn);
```

### shCoefficients.synthesisPoints

`val = g.synthesisPoints(latDeg, lonDeg, r, Name=Value)`

Field values at scattered 3-D points. VAL = g.synthesisPoints(LAT, LON, RGEO) evaluates G pointwise at equal-length vectors of geocentric latitude and longitude [deg] and geocentric radius [m] - stations, satellite positions, profiles - with the upward continuation (R/r)^n that a grid on the sphere cannot give. It is shLowLevel.synthesisPoints with G's coefficients, GM and R; that function documents the quantities, the two summations and why "auto" means "clenshaw". Its options go there only when given, so "(lan...

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| latDeg | : x 1 | double | required | double geocentric latitudes [deg] |
| lonDeg | : x 1 | double | required | double longitudes [deg] |
| r | : x 1 | double | required | double geocentric radii [m] |
| Quantity | lane default | — | required | the quantity evaluated |
| Nmin | lane default | — | required | first degree included |

| input | size | type | default | description |
|---|---|---|---|---|
| Method | lane default | — | required | which summation |
| ClenshawFrom | lane default | — | required | point count where "auto" switches |
| MaxMemGB | lane default | — | required | memory budget of the direct path [GB] |

| output | description |
|---|---|
| val | (P,1) double the quantity at each point |

*Example*

```
    r = shLowLevel.ellipsoidRadius(lat) + 450e3;
    dg = g.synthesisPoints(lat, lon, r, Quantity = "disturbance");
```

### shCoefficients.gradientTensor

`[comps, info] = g.gradientTensor(latDeg, lonDeg, Name=Value)`

Full gravity gradient tensor in the local NEU frame. [COMPS, INFO] = g.gradientTensor(LAT, LON) synthesizes the six independent components of the disturbing gravity gradient tensor - uu, nn, ee, un, ue, ne - the GOCE-style diagnostic. It is shLowLevel.shSynthesisGradientTensor with G's coefficients, GM and R; that function documents the formulas, the Laplace self-check INFO carries, and the validation. Its options go there only when given, so "(lane default)" below is that function's - and its Height defaults to...

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| latDeg | 1 x : | double | required | double geocentric latitudes [deg] |
| lonDeg | 1 x : | double | required | double longitudes [deg] |
| Height | lane default | — | required | evaluation altitude above R [m] |
| Nmin | lane default | — | required | degrees below Nmin are zeroed |

| output | description |
|---|---|
| comps | (1,1) struct as returned by shLowLevel.shSynthesisGradientTensor, which documents every field: the six components, each (nlat x nlon) |
| info | (1,1) struct as returned by shLowLevel.shSynthesisGradientTensor, which documents every field |

*Example*

```
    [G, info] = g.gradientTensor(-89:89, 0:359, Height = 255e3);
    info.maxTraceResidual     % the Laplace self-check
```

### shCoefficients.vdkApply

`[out, info] = g.vdkApply(N, sigM, Name=Value)`

Apply the VDK decorrelation filter with a month's normals. [OUT, INFO] = g.vdkApply(N, SIGM) filters G with its own monthly normal-equation matrix N, xf = (N + alpha M) \ (N x) (Horvath et al. 2018), through shLowLevel.vdkApply, which documents the filter, its relation to tvANS and its cost. G's coefficients are taken in Index ordering - by default shLowLevel.shIndex(nmax, Nmin = 2), the ordering of shLowLevel.readSINEX(..., Index = idx) - filtered, and put back. Coefficients outside Index pass through unchanged...

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| N | P x P | double | required | double normal-equation matrix in Index ordering |

| input | size | type | default | description |
|---|---|---|---|---|
| sigM | n x m | double | required | double the signal sigma per parameter, or [a, b] for sigma = a * l^b, as shLowLevel.vdkApply takes it |
| Index | n x m | — | [] | shLowLevel.shIndex struct N is ordered by; [] uses shLowLevel.shIndex(nmax, Nmin = 2) |
| Alpha | lane default | — | required | filter strength |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients the filtered field |
| info | (1,1) struct as returned by shLowLevel.vdkApply, which documents every field |

*Example*

```
idx = shLowLevel.shIndex(96, Nmin = 2);
snx = shLowLevel.readSINEX(f, Index = idx);
gf = g.vdkApply(snx.M, ab(mo, :), Index = idx, Alpha = 1);
```

### shCoefficients.applyDDK

`out = g.applyDDK(W)`

Apply a DDK (order-block-diagonal) decorrelation filter. OUT = obj.applyDDK(W) with W from shLowLevel.readDDK (struct, .mat, or documented ASCII exchange format). Degrees not covered by the filter pass through unchanged; sigmas are NOT propagated through the filter (set to NaN - use shLowLevel.mcPropagate for filtered uncertainties).

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| W | n x m | — | required | filter container from shLowLevel.readDDK / designFilter, or degree weights |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients DDK-decorrelated |

### shCoefficients.evalAt

`out = g.evalAt(epoch)`

Evaluate gfct time-variable terms at an epoch.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| epoch | 1 x 1 | double | required | epoch [decimal years] |

| output | description |
|---|---|
| out | (1 x 1) shCoefficients gfct variable terms evaluated at the epoch; static models pass through unchanged |

### shCoefficients.write

`g.write(filename, Name=Value)`

Export to an ICGEM .gfc file (round-trip tested).

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| filename | n x m | — | required | char/string path of the file to read/write (gzipped .gz accepted where documented) |
| Comment | 1 x 1 | string | "" | free-text comment written into the gfc header |

| input | size | type | default | description |
|---|---|---|---|---|
| Sidecar | 1 x 1 | logical | true | also write "<filename>.provenance.json". Set false when exporting into a folder that will be read back as a series: the sidecar is metadata, and a loose read... |

### shCoefficients.vec

```
x = g.vec(idx)
```

Pack into a coefficient vector in shLowLevel.shIndex ordering.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| idx | 1 x 1 | struct | required | index struct from shLowLevel.shIndex (fields n, m, cs, P, Lmax, minDegree, pos) |

| output | description |
|---|---|
| x | (P x 1) double coefficients in idx ordering |

### shCoefficients.disp

```
g.disp()
```

Compact display: size, epoch, GM/R, processing history. disp(G) prints one header line with the model name, one contract line (nmax, epoch, product type, tide system, GM, R), whether formal sigmas are attached and how many gfct variable terms are carried, followed by the processing history. Called automatically when an shCoefficients object is shown without a semicolon.

*Example*

```
    g = shCoefficients.read("ITSG-Grace2018_n60_2008-04.gfc");
    disp(g)                           % degree, epoch, history
```

### shCoefficients.read (static)

```
obj = shCoefficients.read(filename, Name=Value)
```

Read an ICGEM .gfc/.gfct/.gfc.gz file into an shCoefficients. G = shCoefficients.read(FILE) parses product type and mid-epoch from standard GRACE L2 filenames (GSM/GAA/GAB/GAC/GAD-2_...); override with Epoch (NaN)=..., ProductType ("")=... when needed.

| input | size | type | default | description |
|---|---|---|---|---|
| filename | n x m | — | required | char/string path of the file to read/write (gzipped .gz accepted where documented) |
| Epoch | 1 x 1 | double | NaN | epoch [decimal years] |
| ProductType | 1 x 1 | string | "" | product type string, e.g. 'gravity_field' |

| output | description |
|---|---|
| obj | (1 x 1) shCoefficients parsed gfc/gfct(.gz) with GM, R, sigmas, variable terms, Epoch= as given |

### shCoefficients.fromVec (static)

```
obj = shCoefficients.fromVec(x, idx, like)
```

Unpack a shLowLevel.shIndex-ordered vector into an shCoefficients. G = shCoefficients.fromVec(X, IDX, LIKE) copies GM/R and other metadata from the template LIKE (an shCoefficients).

| input | size | type | default | description |
|---|---|---|---|---|
| x | : x 1 | double | required | coefficient stack, (P x T) in index ordering |
| idx | 1 x 1 | struct | required | index struct from shLowLevel.shIndex (fields n, m, cs, P, Lmax, minDegree, pos) |

| input | size | type | default | description |
|---|---|---|---|---|
| like | 1 x 1 | shCoefficients | required | template shCoefficients supplying GM, R and metadata |

| output | description |
|---|---|
| obj | (1 x 1) shCoefficients rebuilt from x with sizes/GM/R/epoch of the template |

### shCoefficients.analysis (static)

```
[obj, info] = shCoefficients.analysis(grid, latVec, lonVec, nmax, Name=Value)
```

Spherical harmonic analysis: gridded/scattered data -> Stokes coefficients (the inverse of synthesis).

| input | size | type | default | description |
|---|---|---|---|---|
| grid | n x m | double | required | field values on the (lat, lon) grid |
| latVec | n x m | double | required | geocentric latitude vector of the grid [deg] |
| lonVec | n x m | double | required | longitude vector of the grid [deg], [0, 360) |
| nmax | 1 x 1 | double | required | maximum spherical harmonic degree |
| Quantity | 1 x 1 | string | "geoid" | physical quantity of the grid values; its kernel is divided out, see shLowLevel.kernelFactors |
| Method | 1 x 1 | string | "auto" | "auto" \| "rings" \| "ls", see shAnalysisGrid |
| Weights | 1 x 1 | string | "none" | ("none" \| "coslat") latitude weighting |
| Kaula | 1 x 1 | double | 0 | (1 x 1) Kaula regularization for "ls" |
| kn | n x m | double | [] | load Love numbers, degrees 0..nmax, required for Quantity="ewh" (user-supplied, never hardcoded) |
| hn | n x m | double | [] | vertical-deformation Love numbers, degrees 0..nmax |
| rho_ave | 1 x 1 | double | 5517 | mean Earth density [kg/m^3] in the EWH kernel |
| rho_water | 1 x 1 | double | 1000 | water density [kg/m^3] in the EWH kernel |
| GM | 1 x 1 | double | 3.986004415e14 | , R (6378136.3), Epoch (NaN) stored on the returned object |
| R | 1 x 1 | double | 6378136.3 | reference radius [m] (default 6378136.3, overridable) |
| Epoch | 1 x 1 | double | NaN | epoch [decimal years] |
| Name | 1 x 1 | string | "analysis" | (1 x 1) string stored as the object name |
| LatType | 1 x 1 | string ∈ ["geocentric", "geodetic"] | "geocentric" | "geocentric" \| "geodetic": how latVec is interpreted; geodetic values are converted with Flattening before evaluation |
| Flattening | 1 x 1 | double | 1/298.257223563 | flattening of the reference ellipsoid for that conversion (WGS84; overridable, never silently assumed) |

| output | description |
|---|---|
| obj | (1 x 1) shCoefficients Stokes coefficients estimated from the grid (exact on ring grids; Kaula for scattered points) |
| info | (1,1) struct the estimation diagnostics, as returned by shLowLevel.shAnalysisGrid |

## shSeries class

Time series of spherical harmonic coefficient sets. Wraps a stack of shCoefficients over epochs and provides everything with a time dimension: the mean field, the climatology (bias / trend / annual / semi-annual), GAX background restoration, the classic destripe+Gaussian chain applied per epoch, and the tvANS time-variable anisotropic Wiener filter with exact basin deconvolution. Immutable value class with provenance in .history.

| property | size | type | default | notes |
|---|---|---|---|---|
| Cs | — | double | — | |
| Ss | — | double | — | |
| sigmaCs | — | double | [] | |
| sigmaSs | — | double | [] | |
| epochs | : x 1 | double | — | |
| GM | 1 x 1 | double | 3.986004415e14 | |
| R | 1 x 1 | double | 6378136.3 | |
| productType | 1 x 1 | string | "unknown" | |
| names | : x 1 | string | string.empty(0,1) | |
| history | — | string | string.empty(0,1) | |
| nmax | — | — | — | |
| nEpochs | — | — | — | |

### shSeries.shSeries

```
obj = ts.shSeries(in, Name=Value)
```

Construct from an shCoefficients array or raw stacks. ts = shSeries(objArray) ts = shSeries(Cs, Epochs ([])=..., Ss ([])=...) (raw-stack form via opts)

| input | size | type | default | description |
|---|---|---|---|---|
| in | nmax+1 x nmax+1 x T | — | required | double or (1,T) shCoefficients coefficient stacks or object array |
| Ss | n x m | double | [] | stacked sine coefficients, S(n+1, m+1, T) |
| Epochs | : x 1 | double | [] | epochs [decimal years] |
| GM | 1 x 1 | double | 3.986004415e14 | gravitational constant times mass [m^3/s^2] of the series (overridable default) |
| R | 1 x 1 | double | 6378136.3 | reference radius [m] of the series |
| ProductType | 1 x 1 | string | "unknown" | provider product code shared by all epochs, e.g. "GSM" |
| Names | : x 1 | string | string.empty(0, 1) | ) display labels, one per solution/series |
| SigmaCs | n x m | double | [] | formal 1-sigma stack for Cs, same size |
| SigmaSs | n x m | double | [] | formal 1-sigma stack for Ss, same size |
| History | 1 x 1 | string | string.empty(0, 1) | ) initial processing history; every operation appends one line |

| output | description |
|---|---|
| obj | (1 x 1) shSeries epoch-sorted monthly stack with sigmas and history |

### shSeries.at

```
g = ts.at(k)
```

Extract epoch k as an shCoefficients.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| k | 1 x 1 | double | required | epoch index (1-based) or decimal year (nearest match) |

| output | description |
|---|---|
| g | (1 x 1) shCoefficients month k with epoch and sigmas |

### shSeries.mean

```
[g, info] = ts.mean(Name=Value)
```

Mean field, over the whole series or a user-given window. G = ts.mean() averages every epoch and returns an shCoefficients whose sigmas are the standard error of the temporal scatter, divided by the number of VALID epochs per coefficient (before v3.29.0 it divided by nEpochs, which understated the sigmas of any series holding NaN months).

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| Range | n x m | — | [] | (1 x 2) double decimal years [t1 t2], or a 1 x 2 datetime; [] uses the full series |
| Estimator | 1 x 1 | string ∈ ["arithmetic", "model"] | "arithmetic" | (1 x 1) string, "arithmetic" (equal weights) or "model" (fit and integrate, see above) |
| Periods | 1 x : | double | double.empty(1,0) | ) (1 x K) double extra harmonic periods [yr] for Estimator = "model", beyond the built-in annual and semi-annual - e.g. 161/365.25 for the S2 alias |
| Omitnan | 1 x 1 | logical | true | (1 x 1) logical, ignore NaN coefficients ("arithmetic") or drop NaN epochs ("model"); false makes a NaN month poison the result / raise shSeries:stack:nanInS... |
| MinEpochs | 1 x 1 | double | 1 | (1 x 1) double, minimum number of epochs inside the window; below it the call errors |

| output | description |
|---|---|
| g | (1 x 1) shCoefficients mean field over the window, with sigmas (scatter-based for "arithmetic", propagated through the fit for "model") |
| info | (1,1) struct fields estimator (1,1) string which estimator ran range (1,2) double the window requested nUsed (1,1) double epochs that contributed nTotal (1,1) double epochs in the series nDropped (1,1) double epochs dropped as empty span (1,2) double first and last epoch used coverage (1,1) double used*dt / window length, capped at 1; NaN below two epochs maxGapYears (1,1) double largest gap between two used epochs; NaN below two epochs centroidOffset (1,1) double mean(used) minus the window centre - how lopsided the sample is epoch (1,1) double the epoch stamped on the returned field attenuation (K+2 x 2) double [period factor]; Estimator="model" only |

*Example*

```
gBase = ts.mean(Range = [2004 2010]);
[gGap, info] = ts.mean(Range = [2015 2021], Estimator = "model");
```

### shSeries.climatology

```
[clim, resid] = ts.climatology(Name=Value)
```

Bias/trend/annual/semi-annual (+extra periods) fit. [CLIM, RESID] = ts.climatology(Robust=false, T0 (NaN)=mean(epochs), Periods (double.empty(1,0))=[], Weights ([])=[]) CLIM is an shClimatology; RESID the residual shSeries. Periods: extra periodic terms [years] - e.g. the GRACE tidal alias periods [161/365.25, 3.66, 7.48] (S2, K2, K1); without them, trend and semi-annual estimates absorb tidal aliasing. Weights: T x 1 per-epoch weights (e.g. 1./info.vce from the tvANS filter). CLIM carries per-coefficient 1-sigm...

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| Robust | 1 x 1 | logical | false | (1 x 1) logical Huber re-weighting |
| T0 | 1 x 1 | double | NaN | (1 x 1) reference epoch; NaN uses the mean |
| Periods | 1 x : | double | double.empty(1,0) | (1 x K) extra harmonic periods [yr] beyond the built-in annual + semiannual |

| input | size | type | default | description |
|---|---|---|---|---|
| Weights | : x 1 | double | [] | (T x 1) epoch weights |
| ARCorrect | 1 x 1 | logical | false | (1 x 1) AR(1) correction of the sigmas |

| output | description |
|---|---|
| clim | (1 x 1) shClimatology fitted bias/trend/annual/semiannual (+Periods=) with coefficient sigmas |
| resid | (1 x 1) shSeries residual series about the fit |

### shSeries.trendBreaks

```
out = ts.trendBreaks(Name=Value)
```

Piecewise-linear trends with break significance. OUT = ts.trendBreaks(Breaks=[2016.0], Periods (double.empty(1,0))=[], T0 (NaN)=, ARCorrect (false)=false) fits the standard climatology design PLUS hinge terms max(0, t - tb) per break epoch (continuous piecewise-linear trend) to every coefficient, and tests the break's significance with an F-test against the no-break model (p-values via betainc - base MATLAB, no toolbox). Python-validated: 5.1% null rejection at the 5% level; F = 23 for a 0.02/yr break in 0.05-si...

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| Breaks | 1 x : | double | required | (1 x B) trend break epochs [decimal years] |
| Periods | 1 x : | double | double.empty(1,0) | (1 x K) extra harmonic periods [yr] |
| T0 | 1 x 1 | double | NaN | (1 x 1) reference epoch; NaN uses the mean |
| ARCorrect | 1 x 1 | logical | false | (1 x 1) AR(1) correction of the sigmas |

| output | description |
|---|---|
| out | (1,1) struct NOT an shCoefficients - the coefficient matrices are plain (n1 x n1) doubles - fields breaks (1 x B) double the break epochs, as given [decimal years] t0 (1,1) double reference epoch used trend1C, trend1S (n1 x n1) double pre-break rate [units/yr], cosine and sine sigmaTrend1C, sigmaTrend1S (n1 x n1) double their 1-sigma hingeC, hingeS (n1 x n1 x B) double rate CHANGE at each break; the post-break rate is trend1 + cumsum over the hinges sigmaHingeC, sigmaHingeS (n1 x n1 x B) double 1-sigma of the rate changes trendNoBreakC (n1 x n1) double rate of the no-break model, the F-test's null sigmaTrendNoBreakC (n1 x n1) double its 1-sigma F, pValue (n1 x n1) double F statistic and p-value per cosine coefficient FS, pValueS (n1 x n1) double the same for the sine coefficients |

### shSeries.fan

```
out = ts.fan(rDegKm, rOrdKm)
```

Han fan filter on every epoch (degree x order Gaussian). OUT = ts.fan(RDEG, RORD) - see shLowLevel.shFanFilter.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| rDegKm | 1 x 1 | double | required | double fan filter degree-direction radius [km] |
| rOrdKm | 1 x 1 | double | required | double fan filter order-direction radius [km] |

| output | description |
|---|---|
| out | (1 x 1) shSeries fan-filtered per month |

### shSeries.applyTN14

```
out = ts.applyTN14(tn, Name=Value)
```

Replace C20 (and flagged C30) per month from TN-14. OUT = ts.applyTN14("TN-14.txt") - series-level convenience around shCoefficients.applyTN14 (v2.4.1; the per-month epoch selects the TN entry). All options pass through.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| tn | n x m | — | required | correction-table struct as returned by the matching reader |
| Tolerance | 1 x 1 | double | 0.05 | maximum \|epoch difference\| [yr] accepted when matching a table entry to this epoch |
| ReplaceC30 | 1 x 1 | string | "auto" | "auto" (replace when the table has a non-NaN C30 for that month) \| "never" \| "always" |

| output | description |
|---|---|
| out | (1 x 1) shSeries C20/C30 replaced epoch-matched |

### shSeries.addDegree1

```
out = ts.addDegree1(tn, Name=Value)
```

Insert TN-13 geocenter degree-1 per month. OUT = ts.addDegree1("TN-13.txt") - series-level convenience around shCoefficients.addDegree1 (v2.4.1).

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| tn | n x m | — | required | correction-table struct as returned by the matching reader |
| Tolerance | 1 x 1 | double | 0.05 | maximum \|epoch difference\| [yr] accepted when matching a table entry to this epoch |

| output | description |
|---|---|
| out | (1 x 1) shSeries degree 1 completed epoch-matched |

### shSeries.removeAlias

```
out = ts.removeAlias(Name=Value)
```

Remove a tidal alias harmonic from every epoch. TS2 = ts.removeAlias() fits, per coefficient, a harmonic at the S2 tidal alias period (161 days) TOGETHER WITH bias, trend, annual and semi-annual, and subtracts only that harmonic. This is the correction GravIS applies to its Level-2B products (https://gravis.gfz.de/corrections): errors in the ocean-tide background model alias into the monthly fields at this period, and leaving them in puts a spurious ~161-day signal into every derived series.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| Period | 1 x 1 | double | 161/365.25 | alias period [years]. The default is the S2 alias; the K2 and K1 aliases (3.66 and 7.48 years) are longer than most records and are better handled as climato... |
| PhaseOffset | 1 x 1 | double | 100 | phase offset [degrees] applied to epochs at or after SplitEpoch |
| SplitEpoch | 1 x 1 | double | 2018.0 | mission boundary [decimal year]; the GRACE-FO record starts mid-2018 |
| T0 | 1 x 1 | double | NaN | reference epoch of the fit (NaN: the mean epoch) |

| output | description |
|---|---|
| out | (1,1) shSeries copy with the alias harmonic removed from every epoch; the operation is appended to the history |

*Example*

```
    ts = ts.removeAlias();                  % S2, 100 deg offset
    ts = ts.removeAlias(SplitEpoch = 2018.4);
```

### shSeries.removeGIA

`[out, info] = ts.removeGIA(gia, Name=Value)`

Subtract a glacial isostatic adjustment model. OUT = ts.removeGIA(GIA, T0 (NaN)=mean(epochs)) subtracts the linear GIA signal giaRate * (t - T0) from every epoch. GIA is an shCoefficients object holding RATE coefficients [1/yr] - read your model (ICE-6G_D, Caron et al., ...) from its gfc file with shCoefficients.read; models are ALWAYS user-supplied, never embedded. Trends computed after this are GIA-corrected. The GIA model is truncated to the series nmax; a model with smaller nmax is zero-padded with a note ...

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| gia | 1 x 1 | shCoefficients | required | GIA rate field: shCoefficients, coefficient struct or .gz/.gfc file path (subtracted as rate times (t - Epoch)) |
| T0 | 1 x 1 | double | NaN | reference epoch of the linear model; the series mean epoch when not given |
| Sigma | n x m | — | [] | shCoefficients of per-coefficient 1-sigma RATE uncertainties [1/yr]; defaults to gia's own sigmaC/sigmaS |

| output | description |
|---|---|
| out | (1 x 1) shSeries GIA trend removed about the series mean epoch |
| info | (1 x 1) struct what was propagated, fields t0 (1,1) double reference epoch used rateSigmaC (n1 x n1) double rate 1-sigma, cosine rateSigmaS (n1 x n1) double rate 1-sigma, sine propagated (1,1) logical false when no sigma was available and none was added sigmaWasEmpty (1,1) logical the series carried no sigmas, so the stacks hold the GIA term alone correlated (1,1) logical always true - the GIA term is fully correlated across epochs; see the note above |

### shSeries.applyDDK

`out = ts.applyDDK(W)`

Apply a DDK decorrelation filter to every epoch. OUT = ts.applyDDK(W), W from shLowLevel.readDDK. Sigma stacks are invalidated (NaN) - the filter correlates coefficients.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| W | n x m | — | required | filter container from shLowLevel.readDDK / designFilter, or degree weights |

| output | description |
|---|---|
| out | (1 x 1) shSeries DDK-decorrelated per month |

### shSeries.compare

`[rep, h] = ts.compare(others, varargin)`

Temporal comparison against other series (v2.6.0). REP = ts.compare(TS2) or ts.compare({TS2, TS3, ...}) delegates to shLowLevel.compareSeries with OBJ as the reference; all options pass through (Basin=, Names=, Plot=, ...).

| input | size | type | default | description |
|---|---|---|---|---|
| others | — | — | required | — |
| varargin | — | — | required | — |

| output | description |
|---|---|
| rep | (1,1) struct as returned by shLowLevel.compareSeries, which documents every field |
| h | (1 x 1) graphics handle figure (Plot = true) |

### shSeries.restore

```
out = ts.restore(gax, Name=Value)
```

Add a background (GAA/GAB/GAC/GAD) series, epoch-matched. OUT = ts.restore(GAXSERIES, Tolerance=0.05, AllowMissing=false) Each epoch of OBJ is matched to the nearest GAX epoch within Tolerance [yr]. Missing matches error (or, with AllowMissing=true, leave those epochs unchanged with a note).

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| gax | 1 x 1 | shSeries | required | GAX product to add back: shSeries, folder of monthly .gfc, or file list (epoch-matched) |
| Tolerance | 1 x 1 | double | 0.05 | epoch matching tolerance [yr] |
| AllowMissing | 1 x 1 | logical | false | tolerate epochs absent from some centres |

| output | description |
|---|---|
| out | (1 x 1) shSeries background model added back epoch-matched (e.g. GSM + GAD) |

### shSeries.minus

```
out = ts.minus(a, b)
```

Subtract a field (shCoefficients) or an epoch-matched series.

| input | size | type | default | description |
|---|---|---|---|---|
| a | — | — | required | — |
| b | — | — | required | — |

| output | description |
|---|---|
| out | (1 x 1) shSeries per-month difference (series or single field) |

### shSeries.destripe

```
out = ts.destripe(Name=Value)
```

Swenson & Wahr / P3M6, applied per epoch.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| MinOrder | 1 x 1 | double | 6 | lowest order to destripe; coefficients at m < minOrder pass through unchanged (below it the signal is real, not stripes) |
| PolyOrder | 1 x 1 | double | 3 | order of the polynomial fitted and removed per order/parity sequence |
| WindowLength | n x m | double | [] | [] fits ONE polynomial over the whole sequence; an odd integer >= polyOrder+2 uses a centered moving window instead (WindowLength=6 with PolyOrder=3 is the c... |

| output | description |
|---|---|
| out | (1 x 1) shSeries destriped per month |

### shSeries.gaussian

```
out = ts.gaussian(radiusKm)
```

Jekeli smoothing per epoch (vectorized over the stack).

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| radiusKm | 1 x 1 | double | required | double Gaussian filter half-response radius [km] |

| output | description |
|---|---|
| out | (1 x 1) shSeries Gaussian-smoothed per month |

### shSeries.truncate

```
out = ts.truncate(nmaxNew)
```

Reduce the maximum degree of the whole series.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| nmaxNew | 1 x 1 | double | required | double new maximum degree (must not exceed the current nmax) |

| output | description |
|---|---|
| out | (1 x 1) shSeries truncated per month |

### shSeries.dropNaN

```
out = ts.dropNaN()
```

Remove epochs containing any non-finite coefficient. OUT = ts.dropNaN() drops every epoch whose C or S stack has a NaN/Inf anywhere - typically placeholder months across the GRACE / GRACE-FO gap (2017.5-2018.5) or corrupt reads. The remaining series is irregularly sampled, which climatology and the tvANS filter handle natively (least-squares fit on the actual epochs; per-month VCE).

| output | description |
|---|---|
| out | (1 x 1) shSeries gap months removed |

### shSeries.select

```
out = ts.select(which, Name=Value)
```

Subset the series by index, mask, or time range. OUT = ts.select(WHICH) with WHICH one of logical (1,T) keep masked epochs indices keep listed epochs (in the given order) [tMin tMax] keep epochs with tMin <= t <= tMax

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| which | n x 1 | — | [] | logical \| double mask, index list, or [tMin tMax]; omit it when passing Range |
| Range | n x m | — | [] | (1 x 2) double decimal years, or a 1 x 2 datetime; mutually exclusive with WHICH |

| output | description |
|---|---|
| out | (1 x 1) shSeries the selected epochs, in the order given; the operation is appended to the history (immutable value-class pattern) |

*Example*

```
    tsFO = ts.select(ts.epochs > 2018.5);
    tsB  = ts.select(Range = [2004 2010]);
```

### shSeries.filter

```
[out, op, info] = ts.filter(method, Name=Value)
```

Optimal time-variable filtering of the series. [TSF, OP, INFO] = ts.filter("tvANS", Constraints ([])=..., NoiseCov ([])=..., SignalMode ("isotropic")="isotropic"|"inhomogeneous", NIterSignal (3)=3, Robust (false)=false, Nmin (2)=2) runs the tvANS chain (see shLowLevel.tvANSFilter): deterministic-model separation, empirical or supplied noise covariance, monthly VCE scaling, data-driven signal covariance, per-month anisotropic Wiener filter, optional hard constraints. OP is the stored linear operator - required fo...

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| method | 1 x 1 | string | required | string filter chain to run; currently "tvANS" (the simple filters are their own methods: gaussian / fan / destripe / applyDDK) |
| NoiseCov | n x m | double | [] | (P x P \| struct) noise covariance from shLowLevel.buildNoiseCov; [] uses a scaled identity |
| Constraints | n x m | double | [] | (P x P \| struct) prior constraint matrix or a designFilter constraint struct added to the normals |
| SignalMode | 1 x 1 | string | "isotropic" | "isotropic" \| "inhomogeneous" signal covariance model (see shLowLevel.buildSignalCov) |
| NIterSignal | 1 x 1 | double | 3 | (1 x 1) signal-covariance re-estimation iterations |
| Robust | 1 x 1 | logical | false | (1 x 1) logical, Huber re-weighting of epoch residuals |
| Nmin | 1 x 1 | double | 2 | (1 x 1) lowest degree entering the filter |
| Blocks | 1 x 1 | string ∈ ["auto","on","off"] | "auto" | ("auto" \| "on" \| "off") order-block usage in the normal matrix |
| Shrinkage | 1 x 1 | double | [] | (1 x 1) shrink the empirical signal covariance toward its diagonal, 0..1; [] auto-selects |
| VCEMinDegree | 1 x 1 | double | [] | (1 x 1) lowest degree in the VCE bands; [] follows Nmin |
| VCEBands | 1 x : | double | [] | (1 x B) variance-component band edges; [] uses the built-in banding |

| output | description |
|---|---|
| out | (1 x 1) shSeries tvANS-filtered series with sigmaCs/Ss posterior stacks (exact incl. constraints, v2.5) |
| op | (1,1) struct the stored linear operator, as returned by shLowLevel.tvANSFilter, for shLowLevel.basinDeconvolve and shLowLevel.resolutionMap |
| info | (1,1) struct the noise, signal and VCE diagnostics, as returned by shLowLevel.tvANSFilter |

### shSeries.basinAverage

```
[avg, out] = ts.basinAverage(B, Name=Value)
```

Basin averages, naive or exactly deconvolved. AVG = ts.basinAverage(B) - naive averages B'*x ./ diag(B'*B), K x T, with B (P x K) basin kernels in shLowLevel.shIndex ordering (build from a grid mask via shLowLevel.synthesisMatrix: b = Y' * (w .* mask)). [AVG, OUT] = ts.basinAverage(B, Deconvolve (false)=true, Op (struct())=op) removes filter attenuation and inter-basin leakage exactly using the stored operator from ts.filter (see shLowLevel.basinDeconvolve). OUT.sigma (K x T) is the 1-sigma noise uncertainty of ...

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| B | n x m | double | required | basin-kernel coefficient vector from basinKernel |
| Deconvolve | 1 x 1 | logical | false | apply shLowLevel.basinDeconvolve to the basin series (undo kernel attenuation) |
| Op | n x m | struct | struct() | linear operator/kernel the map is derived from |

| input | size | type | default | description |
|---|---|---|---|---|
| Ridge | 1 x 1 | double | 0 | Tikhonov ridge added to the deconvolution normal matrix; raise it when the kernel matrix is ill-conditioned (out.condA reports the condition number) |

| output | description |
|---|---|
| avg | (K x T) double basin averages (deconvolved when Deconvolve=true) |
| out | (1,1) struct on the deconvolution path, as returned by shLowLevel.basinDeconvolve (sigma, c, attn, condA, avgNaive); empty otherwise |

### shSeries.toGrid

```
b = ts.toGrid(latDeg, lonDeg, Name=Value)
```

Synthesize every epoch into one stack, with its metadata. B = ts.toGrid(LAT, LON, Quantity ("ewh")=..., kn ([])=..., LatType ("geodetic")=...) is shCoefficients.toGrid over a whole series: B.Z is (nlat x nlon x nEpochs) and B.Epoch carries the decimal years, so an animation can label its own frames from the data rather than from a filename.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| latDeg | 1 x : | double | required | double latitudes [deg], convention per LatType |
| lonDeg | 1 x : | double | required | double longitudes [deg] |
| Quantity | 1 x 1 | string | "ewh" | any kernelFactors quantity |
| kn | n x m | double | [] | load Love numbers k'_n (mass quantities; never hardcoded) |
| hn | n x m | double | [] | load Love numbers h'_n (deformation_up only) |
| Nmin | 1 x 1 | double | 0 | degrees below nmin are zeroed |
| Nmax | 1 x 1 | double | NaN | truncation degree; NaN uses the series nmax |
| Height | 1 x 1 | double | 0 | evaluation height [m] above the evaluation surface |
| rho_ave | 1 x 1 | double | 5517 | mean Earth density [kg/m^3] |
| rho_water | 1 x 1 | double | 1000 | water density [kg/m^3] |
| LatType | 1 x 1 | string ∈ ["geocentric", "geodetic"] | "geodetic" | "geodetic" \| "geocentric" |
| Flattening | 1 x 1 | double | 1/298.257223563 | ellipsoid flattening (WGS84), for LatType and for Surface = "ellipsoid" |
| Surface | 1 x 1 | string ∈ ["sphere","ellipsoid"] | "sphere" | "sphere" \| "ellipsoid", as g.synthesis |
| SemiMajorAxis | 1 x 1 | double | 6378137 | ellipsoid semi-major axis [m], WGS84 |
| MaxMemGB | 1 x 1 | double | 4 | budget for the STACK itself; exceeding it is an error naming the size, not a silent swap-thrash |

| output | description |
|---|---|
| b | (1,1) struct as built by shLowLevel.gridHandover, the same bundle shCoefficients.toGrid returns, with Z (nlat x nlon x nEpochs) double, Epoch and C00 (nEpochs x 1) double, and FieldKind "mixed" when the epochs disagree |

*Example*

```
    b = ts.toGrid(-89.5:89.5, 0.5:359.5, Quantity = "ewh", kn = kn);
    size(b.Z)       % [180 360 nEpochs]
```

### shSeries.kalman

```
[out, rep] = ts.kalman(Name=Value)
```

Kalman-smoothed series (Kurtenbach/Kvas), solution mode. [TSK, REP] = ts.kalman(ModelSeries = TSM) runs shLowLevel.kalmanChain with TS as the observations: the VAR process model is estimated from TSM, every epoch of TS is one observation weighted by its formal sigmas (or NoiseCov), and a forward Kalman filter plus an RTS smoother return the smoothed series. shLowLevel.kalmanChain documents the chain, its references and its memory cost.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| ModelSeries | required | — | required | (1 x 1) shSeries process realization at this series' nmax, e.g. GAX/AOD1B or a hydrology model |
| Order | lane default | — | required | VAR order p |
| Structure | lane default | — | required | coupling structure of the VAR model |
| Shrink | lane default | — | required | diagonal loading of the VAR estimate |
| CondFun | lane default | — | required | covariance conditioning |
| Epochs | lane default | — | required | target epoch grid [decimal years] |
| Tolerance | lane default | — | required | epoch matching tolerance [yr] |
| NoiseCov | lane default | — | required | observation noise covariance |
| Climatology | lane default | — | required | reduce the series by its own climatology around the filter, and restore it after |
| Smoother | lane default | — | required | run the RTS pass |
| QC | lane default | — | required | innovation-based quality control |
| QCAlpha | lane default | — | required | false-alarm level of that test |
| StoreCov | lane default | — | required | where the smoother's covariances live |

| output | description |
|---|---|
| out | (1 x 1) shSeries smoothed series on the epoch grid, with formal 1-sigma stacks |
| rep | (1,1) struct as returned by shLowLevel.kalmanChain, which documents every field |

*Example*

```
    [tsk, rep] = ts.kalman(ModelSeries = tsm, Order = 1);
```

### shSeries.eof

`[modes, pcs, varExplained, info] = ts.eof(Name=Value)`

Empirical orthogonal functions of the series. [MODES, PCS, VE, INFO] = ts.eof() decomposes the series into spatial modes and unit-variance principal components by an SVD of the time-centred coefficient stack - shLowLevel.eofAnalysis, which documents the scaling: MODES{k} times PCS(:,k) reconstructs mode k's contribution exactly. It is usually run on a residual, after ts.climatology. The options go to shLowLevel.eofAnalysis only when given; "(lane default)" is that function's.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |

| input | size | type | default | description |
|---|---|---|---|---|
| NModes | lane default | — | required | number of modes returned |
| Center | lane default | — | required | remove the temporal mean first |

| output | description |
|---|---|
| modes | (1 x K) cell shCoefficients, the spatial patterns |
| pcs | (T x K) double principal components, unit variance |
| varExplained | (K x 1) double fraction of the variance per mode |
| info | (1,1) struct as returned by shLowLevel.eofAnalysis, which documents every field |

*Example*

```
[~, resid] = ts.climatology();
[modes, pcs, ve] = resid.eof(NModes = 3);
```

### shSeries.write

`ts.write(folder, Name=Value)`

Export every epoch to its own ICGEM .gfc file. ts.write(FOLDER) writes epoch k with g.write, through ts.at(k), so each file is what g.write writes for that epoch. The files are named after the series' names: NAMES(k) + ".gfc", unless FileNames says otherwise.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| folder | 1 x 1 | string | required | string existing folder to write into |
| FileNames | : x 1 | string | string.empty(0, 1) | ) (T x 1) string one file name per epoch, relative to FOLDER; empty uses NAMES + ".gfc" |
| Comment | 1 x 1 | string | "" | forwarded to g.write |
| Sidecar | 1 x 1 | logical | false | forwarded to g.write. Deliberately false here, where g.write defaults to true: a folder of epochs is meant to be read back as a series, and the default read ... |

*Example*

```
ts.write(outFolder, FileNames = "GSM-2_" + spans + "_mine.gfc");
```

### shSeries.gradient

`[dq, info] = ts.gradient(latDeg, lonDeg, Name=Value)`

North/east/up gradient of every epoch, stacked. [DQ, INFO] = ts.gradient(LAT, LON, ...) is g.gradient of each epoch g = ts.at(k), with DQ.north, DQ.east and DQ.up stacked along the THIRD dimension, page k for epoch k, as ts.toGrid stacks its fields: (nlat x nlon x T) on a grid and (npts x 1 x T) for Mode = "points". DQ.up stays [] for the surface quantities, which have no radial law. INFO is the first epoch's; it depends only on the options, which every epoch shares. The options go to g.gradient only when given,...

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| latDeg | 1 x : | double | required | double latitudes [deg] |
| lonDeg | 1 x : | double | required | double longitudes [deg] |
| Quantity | method default | — | required | any kernelFactors quantity |
| kn | method default | — | required | load Love numbers k'_n |

| input | size | type | default | description |
|---|---|---|---|---|
| hn | method default | — | required | load Love numbers h'_n |
| Nmin | method default | — | required | degrees below Nmin are zeroed |
| Nmax | method default | — | required | truncation degree |
| Height | method default | — | required | evaluation height [m] |
| Surface | method default | — | required | "sphere" or "ellipsoid" |
| SemiMajorAxis | method default | — | required | ellipsoid semi-major axis [m] |
| rho_ave | method default | — | required | mean Earth density [kg/m^3] |
| rho_water | method default | — | required | water density [kg/m^3] |
| Mode | method default | — | required | a grid, or pairwise points |
| Units | method default | — | required | per metre or per radian |
| MaxMemGB | method default | — | required | Legendre memory budget [GB] |
| LatType | method default | — | required | how the latitudes are read |
| Flattening | method default | — | required | ellipsoid flattening |

| output | description |
|---|---|
| dq | (1,1) struct as returned by shCoefficients.gradient, which documents every field, each component stacked along the third dimension |
| info | (1,1) struct as returned by shCoefficients.gradient for the first epoch, which documents every field |

*Example*

```
[dq, info] = ts.gradient(-89:2:89, 0:2:358, Quantity = "ewh", kn = kn);
size(dq.north)        % 90 x 180 x T
```

### shSeries.leakageCorrect

`[mCorr, info] = ts.leakageCorrect(latDeg, lonDeg, Name=Value)`

Forward-model leakage correction of every epoch. [M, INFO] = ts.leakageCorrect(LAT, LON, Filter = F, kn = KN) synthesizes each epoch on the grid (g.synthesis, geocentric, Quantity "ewh" unless told otherwise) and runs shLowLevel.leakageCorrect on it with the series' GM and R: the iterative forward model that finds the field which, pushed through the chain the data saw, reproduces the filtered field. M is the corrected stack, page k for epoch k. shLowLevel.leakageCorrect documents the method, its stopping rule, a...

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| latDeg | 1 x : | double | required | double geocentric latitudes [deg] |
| lonDeg | 1 x : | double | required | double longitudes [deg] |
| Filter | required | — | required | the filter the DATA saw: a string such as "gauss300" or "DDK3", or a W struct |
| Quantity | 1 x 1 | string | "ewh" | the synthesized quantity; its unit is M's |

| input | size | type | default | description |
|---|---|---|---|---|
| kn | n x m | double | [] | load Love numbers for the mass quantities (never hardcoded) |
| Mask | lane default | — | required | pixels allowed to hold mass |
| Nmax | lane default | — | required | degree of the intermediate analysis |
| Gain | lane default | — | required | relaxation factor |
| NoiseLevel | lane default | — | required | noise RMS of the field |
| ResidualRegion | lane default | — | required | where the residual is measured |
| Tau | lane default | — | required | discrepancy-principle safety factor |
| MaxIter | lane default | — | required | iteration cap |
| Tol | lane default | — | required | fallback step criterion |
| Quiet | lane default | — | required | suppress the per-epoch summary |

| output | description |
|---|---|
| mCorr | (nLat x nLon x T) double corrected fields, in the unit of Quantity |
| info | (1 x T) struct one per epoch, as returned by shLowLevel.leakageCorrect, which documents every field |

*Example*

```
    [m, info] = ts.leakageCorrect(-89:89, 0:359, Filter = "gauss300", ...
        kn = kn, Mask = unionMask, Quiet = true);
```

### shSeries.leakageDataDriven

`[fHat, info] = ts.leakageDataDriven(latDeg, lonDeg, Name=Value)`

Model-free leakage correction of a catchment mean. [FHAT, INFO] = ts.leakageDataDriven(LAT, LON, Mask = R, Filter = F, kn = KN) synthesizes every epoch on the grid (g.synthesis, geocentric, Quantity "ewh" unless told otherwise) and runs shLowLevel.leakageDataDriven on the stack with the series' GM and R - Vishwakarma's method of deviation, which repairs the catchment average from the filtered fields alone. FHAT is the corrected catchment mean, one value per epoch. shLowLevel.leakageDataDriven documents the metho...

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| latDeg | 1 x : | double | required | double geocentric latitudes [deg] |
| lonDeg | 1 x : | double | required | double longitudes [deg] |
| Filter | required | — | required | the filter the DATA saw: a string such as "gauss300" or "DDK3", or a W struct |
| Quantity | 1 x 1 | string | "ewh" | the synthesized quantity; its unit is FHAT's |
| kn | n x m | double | [] | load Love numbers for the mass quantities (never hardcoded) |
| Mask | lane default | — | required | the catchment function, 1 inside and 0 outside; required in practice |
| Nmax | lane default | — | required | degree of the analysis/synthesis chain |

| output | description |
|---|---|
| fHat | (T x 1) double corrected catchment average, in the unit of Quantity |
| info | (1,1) struct as returned by shLowLevel.leakageDataDriven, which documents every field |

*Example*

```
    [f, info] = ts.leakageDataDriven(lat, lon, Mask = basin, ...
        Filter = "gauss300", kn = kn);
```

### shSeries.disp

`ts.disp()`

Compact display: months, span, gaps, processing history. disp(TS) prints the product type, the number of epochs, the maximum degree and the covered epoch range, followed by the processing history. Called automatically when an shSeries object is shown without a semicolon.

*Example*

```
    ts = shSeries.read("ITSG-Grace2018_n60_*.gfc");
    disp(ts)
```

### shSeries.read (static)

`obj = shSeries.read(files, Name=Value)`

Read a series of gfc files (pattern or list), sorted by epoch. TS = shSeries.read("GSM-2_*.gfc") or shSeries.read(fileList).

| input | size | type | default | description |
|---|---|---|---|---|
| files | n x m | — | required | folder, file list or glob of monthly .gfc solutions |
| Truncate | 1 x 1 | double | NaN | truncate every field to this nmax while reading (NaN: keep the file resolution) |

| output | description |
|---|---|
| obj | (1 x 1) shSeries wildcard-read, epoch-sorted stack |

### shSeries.fromFolder (static)

`obj = shSeries.fromFolder(folder, Name=Value)`

Read every matching gfc file in a folder as a series. TS = shSeries.fromFolder("itsg_series") TS = shSeries.fromFolder(dest, Pattern="*n96*.gfc*", Truncate (NaN)=60) Convenience wrapper around shSeries.read (sorted by epoch); pairs with shLowLevel.fetchITSG, which downloads ITSG monthly solutions into tests/test_data/itsg_series on demand.

| input | size | type | default | description |
|---|---|---|---|---|
| folder | 1 x 1 | string | required | folder path |
| Pattern | 1 x 1 | string | "*.gfc*" | filename glob the folder is scanned with |
| Truncate | 1 x 1 | double | NaN | truncate solutions at this degree |

| output | description |
|---|---|
| obj | (1 x 1) shSeries all gfc(.gz) files of a folder, epoch-sorted |

## shClimatology class

Deterministic climatology of an SH series. Bias + trend + annual + semi-annual model per Stokes coefficient, as fitted by shSeries.climatology (least squares or Huber-robust): x(t) = bias + trend*(t-t0) + cosA*cos(2*pi*(t-t0)) + sinA*sin(...) + cosSA*cos(4*pi*(t-t0)) + sinSA*sin(4*pi*(t-t0)) + sum_k extra cos/sin terms of period p_k (v2.1) All phases and the bias refer to the reference epoch t0. Extra periods (Periods= option) typically hold the GRACE tidal alias terms S2 (161 d), K2 (3.66 yr), K1 (7.48 yr); access them via periodic(k). Per-coefficient 1-sigma significance from the fit is sto

| property | size | type | default | notes |
|---|---|---|---|---|
| t0 | 1 x 1 | double | NaN | |
| GM | 1 x 1 | double | 3.986004415e14 | |
| R | 1 x 1 | double | 6378136.3 | |
| biasC | — | double | — | |
| biasS | — | double | — | |
| trendC | — | double | — | |
| trendS | — | double | — | |
| cosAnnC | — | double | — | |
| cosAnnS | — | double | — | |
| sinAnnC | — | double | — | |
| sinAnnS | — | double | — | |
| cosSemiC | — | double | — | |
| cosSemiS | — | double | — | |
| sinSemiC | — | double | — | |
| sinSemiS | — | double | — | |
| periods | 1 x : | double | double.empty(1,0) | extra periods [yr] (v2.1) |
| extraCosC | — | double | [] | n1 x n1 x K stacks for the extra periods |
| extraCosS | — | double | [] | |
| extraSinC | — | double | [] | |
| extraSinS | — | double | [] | |
| coefSigma | — | double | [] | (6+2K) x 2*Nc per-coefficient 1-sigma (v2.1) |
| history | — | string | string.empty(0,1) | |
| nmax | — | — | — | |

### shClimatology.withNote

```
obj = clim.withNote(txt)
```

Append a provenance note to the history (v2.2). OBJ = clim.withNote(TXT) - the only supported way to annotate from outside the class (history is SetAccess=private).

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| txt | n x m | — | required | free-text provenance note appended to the history |

| output | description |
|---|---|
| obj | (1 x 1) shClimatology copy with the note appended to its history |

### shClimatology.removeGIA

```
out = clim.removeGIA(gia)
```

Subtract a GIA rate model from the fitted trend. OUT = clim.removeGIA(GIA): GIA is an shCoefficients holding RATE coefficients [1/yr] (user-supplied model file via shCoefficients.read). Only trendC/trendS change; trend sigmas are kept - the model is treated as exact (documented limitation; GIA model spread often dominates regional trend error budgets - compare several models).

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| gia | 1 x 1 | shCoefficients | required | GIA rate field (shCoefficients, struct or file) removed from the trend component |

| output | description |
|---|---|
| out | (1,1) shClimatology copy with the GIA trend removed from the trend component; history appended |

### shClimatology.eval

```
g = clim.eval(epoch)
```

Evaluate the full climatology at an epoch -> shCoefficients.

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| epoch | 1 x 1 | double | required | epoch [decimal years] |

| output | description |
|---|---|
| g | (1 x 1) shCoefficients model field evaluated at the epoch |

### shClimatology.bias

```
g = clim.bias()
```

Field at the reference epoch t0.

| output | description |
|---|---|
| g | (1 x 1) shCoefficients bias component with sigmas |

### shClimatology.trend

```
g = clim.trend()
```

Linear trend field, units of the coefficients per year.

| output | description |
|---|---|
| g | (1 x 1) shCoefficients trend component [units/yr] with sigmas |

### shClimatology.cosAnnual

```
g = clim.cosAnnual()
```

Cosine annual component (sigmas attached).

| output | description |
|---|---|
| g | (1 x 1) shCoefficients cosine annual component |

### shClimatology.sinAnnual

```
g = clim.sinAnnual()
```

Sine annual component (sigmas attached).

| output | description |
|---|---|
| g | (1 x 1) shCoefficients sine annual component |

### shClimatology.cosSemiannual

```
g = clim.cosSemiannual()
```

Cosine semiannual component (sigmas attached).

| output | description |
|---|---|
| g | (1 x 1) shCoefficients cosine semiannual component |

### shClimatology.sinSemiannual

```
g = clim.sinSemiannual()
```

Sine semiannual component (sigmas attached).

| output | description |
|---|---|
| g | (1 x 1) shCoefficients sine semiannual component |

### shClimatology.periodic

```
[gc, gs] = clim.periodic(k)
```

Cos/sin components of the k-th extra period (v2.1). [GC, GS] = clim.periodic(K) returns shCoefficients for the cos and sin terms of period obj.periods(K) [yr] - e.g. the S2/K2/K1 tidal alias terms when fitted via ts.climatology(Periods=[161/365.25, 3.66, 7.48]).

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| k | 1 x 1 | double | required | harmonic index: 1 = annual, 2 = semiannual, ... |

| output | description |
|---|---|
| gc | (1 x 1) shCoefficients cosine component of the k-th extra period |
| gs | (1 x 1) shCoefficients sine component |

### shClimatology.amplitudeMap

```
[A, lat, lon, phase] = clim.amplitudeMap(which, latVec, lonVec, Name=Value)
```

Pointwise seasonal amplitude (and phase) map. [A, LAT, LON, PHASE] = clim.amplitudeMap(WHICH, ... with WHICH "annual" | "semiannual" | k (index into clim.periods, v2.1), LATVEC, LONVEC, Quantity ("geoid")=..., kn ([])=...) synthesizes the cos and sin component fields and combines them pointwise: A = sqrt(fc.^2 + fs.^2) PHASE = atan2(fs, fc) [rad, relative to t0] (amplitude of a harmonic is NOT a linear functional of the coefficients, so it must be formed in the space domain).

| input | size | type | default | description |
|---|---|---|---|---|
| obj | n x m | — | required | the object the method is called on |
| which | 1 x 1 | — | required | selection of items to act on |
| latVec | 1 x : | double | required | geocentric latitude vector of the grid [deg] |
| lonVec | 1 x : | double | required | longitude vector of the grid [deg], [0, 360) |
| Quantity | 1 x 1 | string | "geoid" | functional of the field, see shLowLevel.kernelFactors |
| kn | n x m | double | [] | load Love numbers, (degree, kn) table or column vector - always user-supplied, no reference frame is assumed |

| output | description |
|---|---|
| A | (nlat x nlon) double amplitude of the harmonic in the requested quantity |
| lat | (1 x nlat) double latitudes |
| lon | (1 x nlon) double longitudes |
| phase | (nlat x nlon) double phase [rad] of the harmonic |

### shClimatology.disp

```
clim.disp()
```

Compact display: fitted components, reference epoch, note. disp(CLIM) prints the maximum degree and the reference epoch t0, the list of fitted components (bias, trend, annual and semi-annual cosine/sine, plus any extra Periods) and the processing history including the GIA note. Called automatically when an shClimatology object is shown without a semicolon.

*Example*

```
clim = ts.climatology();
disp(clim)
```

### shClimatology.fromCoef (static)

`obj = shClimatology.fromCoef(coef, t0, series, Name=Value)`

Build from shLowLevel.fitDeterministicModel output (internal). OBJ = shClimatology.fromCoef(COEF, T0, SERIES, Periods (double.empty(1,0))=[], CoefSigma=[]) with COEF (6+2K) x (2*Nc) over the flattened [C(:); S(:)] stack of SERIES; rows 7:end are the cos/sin pairs of the K extra Periods [yr].

| input | size | type | default | description |
|---|---|---|---|---|
| coef | P x K | double | required | fitted component coefficients in shIndex ordering |
| t0 | 1 x 1 | double | required | reference epoch of the fit [decimal years] |
| series | 1 x 1 | shSeries | required | shSeries the fit came from (history carrier) |
| Periods | 1 x : | double | double.empty(1,0) | (1 x K2) extra harmonic periods [yr] |
| CoefSigma | n x m | double | [] | (P x K) 1-sigma uncertainties of coef |

| output | description |
|---|---|
| obj | (1 x 1) shClimatology rebuilt from a coefficient table with the series' layout |

## Package +shLowLevel

### Readers, writers & data management

### shLowLevel.shReadGFC

`model = shLowLevel.shReadGFC(filename)`

Read an ICGEM ASCII .gfc gravity field model file. MODEL = SHREADGFC(FILENAME) parses header key:value pairs and the gfc/gfct data block of a standard ICGEM-format file (GRACE/GRACE-FO monthly solutions, static models, GOCE, EGM2008, etc.). Gzip- compressed files (.gfc.gz) are transparently decompressed.

| input | size | type | default | description |
|---|---|---|---|---|
| filename | — | — | required | — |

| output | description |
|---|---|
| model | (1,1) struct fields C, S, sigmaC, sigmaS (nmax+1 x nmax+1) double coefficients and their formal errors GM, R (1,1) double the file's reference constants nmax (1,1) double maximum degree read modelname (1,1) string the model's own name header (1,1) struct the parsed file header, verbatim - this is where the TIDE SYSTEM lives, as header.tide_system when the file declares one. There is no model.tide field; shCoefficients.read lifts it out of the header variableTerms (1,:) struct array trend/annual/semiannual terms; empty when the file carries none |

### shLowLevel.shEvalGFCT

`[Ct, St] = shLowLevel.shEvalGFCT(model, epoch)`

Evaluate time-variable Stokes coefficients at a given epoch. [CT, ST] = SHEVALGFCT(MODEL, EPOCH) reconstructs C(t) = C0 + trnd*(t-t0) + sum_k [acos_k*cos(2*pi*(t-t0)) ... + asin_k*sin(2*pi*(t-t0))] (and analogously for S) from MODEL.C / MODEL.S (constant part) and MODEL.variableTerms (from SHREADGFC), at EPOCH (same units/convention as the t0 values in the file -- typically decimal years).

| input | size | type | default | description |
|---|---|---|---|---|
| model | — | — | required | — |
| epoch | — | — | required | — |

| output | description |
|---|---|
| Ct | (nmax+1 x nmax+1) double cosine coefficients at the epoch |
| St | (nmax+1 x nmax+1) double sine coefficients at the epoch |

### shLowLevel.readTN13

```
tn = shLowLevel.readTN13(filename)
```

Read a GRACE/GRACE-FO TN-13 geocenter (degree-1) file. TN = shLowLevel.readTN13(FILENAME) parses the official TN-13 product (JPL/CSR/GFZ, e.g. TN-13_GEOC_CSR_RL06.txt): data lines GRCOF2 n m Clm Slm Clm_sig Slm_sig begin end with n=1, m=0/1 and begin/end epochs yyyymmdd.hhmm. Line pairs (m=0, m=1) sharing the same interval are combined into one monthly record with the interval mid-point as epoch.

| input | size | type | default | description |
|---|---|---|---|---|
| filename | n x m | — | required | char/string path (.txt, or .gz which is gunzipped) |

| output | description |
|---|---|
| tn | (1,1) struct fields epoch (K,1) double interval mid-points [decimal years] C10, C11, S11 (K,1) double degree-1 Stokes coefficients sigC10, sigC11, sigS11 (K,1) double 1-sigma t0, t1 (K,1) double interval bounds [decimal years] |

### shLowLevel.readTN14

```
tn = shLowLevel.readTN14(filename)
```

Parse a TN-14 style SLR C20/C30 replacement table. TN = shLowLevel.readTN14(FILENAME) reads the CSR/GFZ "TN-14" technical-note text format: an arbitrary text header followed by data lines with 10 whitespace-separated numeric columns MJD_begin yearfrac_begin C20 C20-mean*1e10 sigC20*1e10 ... C30 C30-mean*1e10 sigC30*1e10 MJD_end yearfrac_end C30 columns contain 'NaN' for months where no SLR C30 is provided (pre GRACE-FO accelerometer degradation).

| input | size | type | default | description |
|---|---|---|---|---|
| filename | n x m | — | required | char/string |

| output | description |
|---|---|
| tn | (1,1) struct one row per table line (T rows), fields epochStart, epochStop (T x 1 double) span bounds [decimal years] epoch (T x 1 double) span mid-point [decimal years] C20, C30 (T x 1 double) SLR replacement coefficients; C30 is NaN for months the file leaves blank sigmaC20, sigmaC30 (T x 1 double) 1-sigma, scaled out of the file's 1e-10 units |

### shLowLevel.readSINEX

```
snx = shLowLevel.readSINEX(filename, Name=Value)
```

Read gravity-field SINEX files (solutions, covariances, NEQs). SNX = shLowLevel.readSINEX(FILENAME) parses the SINEX blocks used by gravity-field solution providers (ITSG, COST-G, ...): +SOLUTION/ESTIMATE parameter list: type CN/SN, degree, order, value, std +SOLUTION/MATRIX_ESTIMATE L|U COVA covariance (lower/upper, up to 3 values per line) +SOLUTION/NORMAL_EQUATION_MATRIX L|U normal-equation matrix .gz files are gunzipped transparently.

| input | size | type | default | description |
|---|---|---|---|---|
| filename | n x m | — | required | output/input file path |
| Index | n x m | struct | struct([]) | shLowLevel.shIndex struct for reordering |
| Output | 1 x 1 | string ∈ ["raw","covariance"] | "raw" | raw returns the parsed blocks; covariance assembles it from a NEQ solution |

| input | size | type | default | description |
|---|---|---|---|---|
| Only | 1 x 1 | string ∈ ["all","estimate"] | "all" | estimate streams only the SOLUTION/ESTIMATE block of large files |

| output | description |
|---|---|
| snx | (1,1) struct the parsed solution, fields n, m, cs (Q x 1) double the parameter list; cs is 0 for C, 1 for S, in file order or in IDX order when Index was given x, sig (Q x 1) double estimates and formal sigmas; sig is NaN where the file carries none M (Q x Q) double covariance or normal-equation matrix, per Output=; [] when absent kind (1,:) char 'COVA' \| 'NEQ' \| '' epoch (1,1) double the solution's reference epoch stats (1,1) struct the parsed +SOLUTION/STATISTICS block (v3.20): keys and values verbatim, plus nobs, nunk, dof, wsos (l'Pl, also matched from "SQUARE SUM OF RESIDUALS") and vfactor where present. struct([]) when the block is absent, and ALWAYS struct([]) on the streaming Only="estimate" path, which stops before it - shLowLevel.neqCombine's VCE feeds on nobs and wsos, so that path cannot supply it |

### shLowLevel.readDDK

```
W = shLowLevel.readDDK(source, Name=Value)
```

Load a DDK (Kusche) anisotropic decorrelation filter. W = shLowLevel.readDDK(SOURCE) loads a DDK filter into the toolbox's order-block container, ready for shLowLevel.applyDDK / g.applyDDK / ts.applyDDK. The DDK filters (Kusche 2007; Kusche et al. 2009, DDK1..DDK8) are the standard published anisotropic alternative to Gaussian smoothing and the usual benchmark against tvANS.

| input | size | type | default | description |
|---|---|---|---|---|
| source | n x m | — | required | DDK filter file or folder the coefficients are read from |
| Nmax | n x m | double | [] | truncate the filter to degrees <= Nmax (leading submatrix per block - identical to the reference filterSH behavior for inputs with nmax below the filter's Lm... |

| output | description |
|---|---|
| W | (1,1) struct fields nmax (1,1) double maximum degree the filter covers name (1,1) string the DDK label, e.g. "DDK3" blocks (1 x 241) struct array one per (order, cs) block, each with m (1,1 double), cs ('c'\|'s'), n (1,B double, the degrees) and M (B x B double, the square block matrix) |

### shLowLevel.readMascon

```
mas = shLowLevel.readMascon(filename)
```

Read a GRACE/GRACE-FO mascon netCDF (JPL / CSR / GSFC style). MAS = shLowLevel.readMascon(FILENAME) reads gridded mascon solutions for comparison against the SH chain (base MATLAB: ncread/ncinfo, no toolboxes). Variable names are auto-detected among the layouts used by JPL RL06 ('lwe_thickness'), CSR RL06 ('lwe_thickness'), and generic 'lwe'/'water_thickness'; time units "days since YYYY-MM-DD" are converted to decimal years.

| input | size | type | default | description |
|---|---|---|---|---|
| filename | n x m | — | required | char/string .nc path |

| output | description |
|---|---|
| mas | (1 x 1) struct fields lat (nlat,1) double [deg] as stored - typically GEODETIC; convert with shLowLevel.geodetic2geocentric before synthesizing SH fields on this grid! lon (nlon,1) double [deg, 0..360) epoch (T,1) double decimal years ewh (nlat,nlon,T) double [cm or m AS STORED - check mas.units] units, name (1,1) string |

### shLowLevel.readLoveNumbers

```
[LN, info] = shLowLevel.readLoveNumbers(filename, Name=Value)
```

Read load Love numbers from a user-supplied table file. LN = shLowLevel.readLoveNumbers(FILE) parses a plain-text load-Love-number table (comment lines starting with %, #, ! or // are skipped; an optional header line naming the columns is recognized). The toolbox never hardcodes Love numbers - this reader turns YOUR

loading-model table (PREM, Gutenberg-Bullen, ak135, ...) into the kn/hn/ln vectors the synthesis and deformation routines expect.

| input | size | type | default | description |
|---|---|---|---|---|
| filename | n x m | — | required | char/string path of the file to read/write (gzipped .gz accepted where documented) |
| Columns | 1 x 1 | string | "auto" | explicit column meaning, tokens from {n,k,h,l}, e.g. "n k h l" (overrides header detection) |
| Nmax | 1 x 1 | double | [] | verify coverage of degrees 0..Nmax and truncate; with sparse tables combine with Interp ("none") |
| Interp | 1 x 1 | string ∈ ["none", "pchip"] | "none" | "none" \| "pchip": fill degree gaps by shape-preserving pchip in log(1+n) - Farrell-style sparse tables (n = ...,32,56,100,...) become dense. Python-validate... |
| InFrame | 1 x 1 | string ∈ ["", "CE", "CF", "CM"] | "" | degree-1 reference frame of the FILE, and |
| OutFrame | 1 x 1 | string ∈ ["", "CE", "CF", "CM"] | "" | requested output frame: "CE" \| "CF" \| "CM". Both or neither. Only degree 1 changes (Blewitt 2003 isomorphic frames): all of h1,l1,k1 shift by the same consta... |

| output | description |
|---|---|
| LN | (1,1) struct fields n (D x 1) degrees, ascending kn (D x 1) potential load Love numbers k'_n hn (D x 1) vertical (NaN if absent from file) ln (D x 1) horizontal (NaN if absent from file) |
| info | (1,1) struct fields source (1,1) string the file it was read from columns (1,:) string the columns actually used frame (1,1) string the degree-1 frame declared interpolated (D x 1) logical which degrees were filled in rather than read |

*Example*

```
LN = shLowLevel.readLoveNumbers("prem_load.txt", Columns="n h l k", ...
         Nmax=96, Interp="pchip", InFrame="CE", OutFrame="CF");
[up, no, ea] = g.deformation(latPts, lonPts, kn=LN.kn, hn=LN.hn, ...
         ln=LN.ln, Mode="points");
```

### shLowLevel.listICGEM

`[T, info] = shLowLevel.listICGEM(Name=Value)`

List gravity field models available at ICGEM (GFZ). T = shLowLevel.listICGEM() returns a table of the STATIC global models from https://icgem.gfz.de/tom_longtime (name, year, max degree, data sources, direct .gfc download URL) - feed a row's name into shLowLevel.fetchICGEM to download.

| input | size | type | default | description |
|---|---|---|---|---|
| Type | 1 x 1 | string ∈ ["static","temporal"] | "static" | "static" \| "temporal" |
| Series | 1 x 1 | string | "" | temporal only: catalogue path of one series to list its individual files (see above) |
| Source | 1 x 1 | string | "" | local HTML file instead of the live page (offline/testing; the parsers are fixture-tested) |
| Timeout | 1 x 1 | double | 60 | per-request timeout [s] |

| output | description |
|---|---|
| T | table: static: name, year, degree, data, url temporal: group, center, series, path, url, zip temporal+Series: name, url |
| info | (1,1) struct fields source (1,1) string where the catalogue came from retrieved (1,1) string when it was read note (1,1) string what the listing covers |

### shLowLevel.fetchITSG

```
[files, info] = shLowLevel.fetchITSG(months, Name=Value)
```

Download ITSG monthly OR daily solutions from TU Graz. The fetch family, one function per source and product: fetchITSG and fetchICGEM for solution series, fetchGAX for the AOD1B GAA/GAB/GAC/GAD monthly means (ICGEM/GFZ pages), fetchITSGSINEX for the ITSG monthly normal-equation SINEX, fetchITSGBackground for the ITSG monthly background models, plus fetchTN, fetchDDK, fetchLoveNumbers for auxiliaries.

| input | size | type | default | description |
|---|---|---|---|---|
| months | n x m | — | "" | numeric years (each expands to 12 months) OR string array "YYYY-MM" |
| Dest | 1 x 1 | string | "" | (fullfile(shLowLevel.dataFolder(), "series", "itsg", "monthly")) target folder (set a shared location once via shLowLevel.dataFolder(path)) |
| Nmax | 1 x 1 | double | NaN | monthly: 60 \| 96 \| 120 (server folder monthly_nXX; NaN resolves to 96). Daily solutions are n40 only and NaN resolves to 40; any other value errors |
| Product | 1 x 1 | string ∈ ["monthly", "daily"] | "monthly" | product identifier to download |
| Timeout | 1 x 1 | double | 60 | [s] per file |
| Proxy | 1 x 1 | string | "" | per-call proxy URL, e.g. "http://proxy:8080" (empty: MATLAB Web Preferences) |
| Update | 1 x 1 | logical | false | re-download/overwrite files that already exist |
| Quiet | 1 x 1 | logical | false | suppress per-file progress |
| Release | 1 x 1 | string | "" | product release identifier |
| BaseURL | 1 x 1 | string | "https://ftp.tugraz.at/pub/ITSG/GRACE" | ("https://ftp.tugraz.at/pub/ITSG/GRACE") server base URL, or a local mirror folder for offline use |
| Catalog | 1 x : | double | [] | row numbers of the shLowLevel.listITSG catalogue to fetch completely |

| output | description |
|---|---|
| files | (1,:) string paths of all present files (new + existing) |
| info | (1,1) struct fields fetched (1,:) string files downloaded now updated (1,:) string files replaced by a newer copy skipped (1,:) string files already present missing (1,:) string months the server did not offer dest (1,1) string target folder base (1,1) string server base URL product (1,1) string the ITSG product actually requested |

### shLowLevel.fetchICGEM

```
[file, info] = shLowLevel.fetchICGEM(model, Name=Value)
```

Download a static gravity field model from ICGEM. FILE = shLowLevel.fetchICGEM("EGM2008") resolves the model name against shLowLevel.listICGEM (case-insensitive exact match, else unique prefix) and downloads the .gfc into <dataFolder>/icgem. Existing files are skipped. Load with shCoefficients.read(FILE).

| input | size | type | default | description |
|---|---|---|---|---|
| model | n x m | — | required | ICGEM model name as listed by shLowLevel.listICGEM |
| Dest | 1 x 1 | string | "" | "" = the v3.16 layout: static models under dataFolder/static, series under dataFolder/series/<group_center_series> |
| Timeout | 1 x 1 | double | 300 | , |
| List | n x m | table | table() | ) ([]) pass a pre-fetched listICGEM table (avoids re-listing in loops / enables the offline fixture in tests) |
| Proxy | 1 x 1 | string | "" | per-call proxy URL, e.g. "http://proxy:8080" (empty: MATLAB Web Preferences) |

| input | size | type | default | description |
|---|---|---|---|---|
| Update | 1 x 1 | logical | false | refresh existing files (safe swap: verified before replacing) |
| Quiet | 1 x 1 | logical | false | suppress progress output ([k/K] counter, per-file timing, failure summary) |
| Pause | 1 x 1 | double | 3 | seconds between models in bulk mode - the ICGEM server RATE-LIMITS rapid requests (HTTP 429), which looks like a stall |
| Retries | 1 x 1 | double | 3 | retry attempts on 429/timeout, with 30/60/120 second backoff |
| Type | 1 x 1 | string ∈ ["static", "temporal"] | "static" | "static" \| "temporal": which catalogue numeric selections and names resolve against (mirrors shLowLevel.listICGEM) |
| Files | 1 x 1 | string | "*.gfc*" | filename filter in series mode |
| Mode | 1 x 1 | string ∈ ["auto", "archive", "files"] | "auto" | series download strategy: "auto" fetches the server's whole-series ZIP in ONE request (the rate limiter punishes hundreds of per-file requests) and falls bac... |
| FileList | n x m | table | table() | ) series mode: use this file table (from listICGEM(Series=...)) instead of querying - for subsetting or offline mirrors |

| output | description |
|---|---|
| file | string local path(s): one per model, or every file of the series in temporal mode (bulk selections are concatenated) |
| info | (1,1) struct fields url (1,:) string the URL(s) used skipped (1,1) logical the file was already present updated (1,:) string files replaced by a newer copy failed (1,1) logical the download did not complete mode (1,1) string which path actually ran: "model" (single static model) \| "archive" (whole series as one ZIP) \| "files" (per-file fallback) \| "present" (nothing to do). A script tells a fresh download from a no-op by this field |

### shLowLevel.fetchDDK

```
[files, info] = shLowLevel.fetchDDK(which, Name=Value)
```

Download DDK filter matrices (Wbd binaries) on demand. FILES = shLowLevel.fetchDDK(1:8) fetches the released anisotropic DDK decorrelation filters DDK1..DDK8 (Kusche 2007; Kusche et al. 2009; ~9.8 MB each) from the MIT-licensed strawpants/GRACE-filter repository into <dataFolder>/DDK. Existing files are skipped unless Update (false)=true, which re-downloads them with a safe swap (the fresh file is parse-verified by shLowLevel.readDDK before replacing the old one). DDK3 additionally ships as a toolbox sample (res...

| input | size | type | default | description |
|---|---|---|---|---|
| which | 1 x : | double | required | double subset of 1..8 |
| Dest | 1 x 1 | string | "" | (fullfile(shLowLevel.dataFolder(), "DDK")), Timeout (120), Quiet (false) |
| Timeout | 1 x 1 | double | 120 | per-request timeout [s] |
| Proxy | 1 x 1 | string | "" | per-call proxy URL, e.g. "http://proxy:8080" (empty: MATLAB Web Preferences) |
| Update | 1 x 1 | logical | false | re-download/overwrite files that already exist |
| Quiet | 1 x 1 | logical | false | suppress progress output |

| output | description |
|---|---|
| files | (1,:) string present file paths (new + existing) |
| info | (1,1) struct fields fetched (1,:) string files downloaded now updated (1,:) string files replaced by a newer copy skipped (1,:) string files already present names (1,:) string the DDK names resolved url (1,1) string server base URL used dest (1,1) string folder the files were resolved against |

### shLowLevel.fetchTN

```
[files, info] = shLowLevel.fetchTN(Name=Value)
```

Download the TN-13/TN-14 low-degree completion files on demand. FILES = shLowLevel.fetchTN() fetches the GRACE/GRACE-FO technical notes that complete the standard processing chain into <dataFolder>/TN: TN-14 GSFC SLR C20/C30 replacement (shLowLevel.readTN14/applyTN14) TN-13 geocenter (degree 1) per provider (shLowLevel.readTN13/addDegree1) Source: the GFZ ISDC open document server (URLs verified 2026-08-07; the CSR/JPL/GFZ RL06.3 files and the GSFC TN-14 also ship as test fixtures, so the toolbox works ...

| input | size | type | default | description |
|---|---|---|---|---|
| Providers | 1 x : | string ∈ ["GFZ", "CSR", "JPL"] | ["GFZ", "CSR", "JPL"] | TN-13 providers to fetch |
| TN14 | 1 x 1 | logical | true | also fetch the GSFC TN-14 file |
| Release | 1 x 1 | string | "RL06.3" | TN-13 release tag in the filename |
| Update | 1 x 1 | logical | false | refresh existing files (safe swap) |
| Dest | 1 x 1 | string | "" | (fullfile(shLowLevel.dataFolder(), "TN")) |
| BaseURL | 1 x 1 | string | "" | override source: an https base or a LOCAL MIRROR FOLDER (institute mirrors, offline tests) |
| Proxy | 1 x 1 | string | "" | per-call proxy, e.g. "http://proxy.uni.de:8080" (websave otherwise honours MATLAB Web Preferences) |
| Timeout | 1 x 1 | double | 60 | , Quiet (false) |
| Quiet | 1 x 1 | logical | false | suppress progress output |

| output | description |
|---|---|
| files | (1,:) string verified present file paths (new + updated + kept existing) |
| info | (1,1) struct fields fetched (1,:) string files downloaded now updated (1,:) string files replaced by a newer copy skipped (1,:) string files already present failed (1,:) string files that did not arrive urls (1,:) string the URLs used, in the same order dest (1,1) string target folder |

*Example*

```
shLowLevel.fetchTN(Update = true);           % monthly refresh, safe swap
tn13 = shLowLevel.readTN13(fullfile(shLowLevel.dataFolder(), "TN", ...
          "TN-13_GEOC_CSR_RL06.3.txt"));
g1 = g.applyTN14(tn14).addDegree1(tn13);
```

### shLowLevel.dataFolder

```
folder = shLowLevel.dataFolder(newFolder)
```

Get or set the persistent user data folder of the toolbox. F = shLowLevel.dataFolder() returns the current data folder (created on demand). Default: <toolbox root>/data. F = shLowLevel.dataFolder(PATH) sets and persists a user folder (MATLAB preference, survives sessions: setpref('shAnalysis','dataFolder')). F = shLowLevel.dataFolder("reset") back to the default.

| input | size | type | default | description |
|---|---|---|---|---|
| newFolder | 1 x 1 | string | "" | char/string new persistent data folder (omit to query the current one) |

| output | description |
|---|---|
| folder | (1 x 1) string current data folder (created on demand) |

### shLowLevel.ddkNames

```
names = shLowLevel.ddkNames()
```

Released Wbd filenames for DDK1..DDK8 (strong -> weak). NAMES = shLowLevel.ddkNames() - single source of truth for the mapping (verified against the strawpants/GRACE-filter repository listing). Claude (Fable 5), 2026-08-07 (v2.4.1).

| output | description |
|---|---|
| names | (1 x 8) string Wbd file names for DDK1..DDK8 |

### shLowLevel.parseGraceFilename

```
meta = shLowLevel.parseGraceFilename(filename)
```

Product type and mid-epoch from a GRACE L2 filename. META = shLowLevel.parseGraceFilename(FILENAME) parses the standard GRACE / GRACE-FO Level-2 naming convention PPP-2_YYYYDDD-YYYYDDD_*.gfc[.gz] e.g. GSM-2_2024032-2024060_GRFO_UTCSR_BA01_0600.gfc, where PPP is one of GSM, GAA, GAB, GAC, GAD and YYYYDDD are year + day-of-year of the coverage span. Best-effort: fields are NaN/"unknown" when the pattern does not match (static models, renamed files); callers can always override via explicit options.

| input | size | type | default | description |
|---|---|---|---|---|
| filename | n x m | — | required | char/string path or bare filename |

| output | description |
|---|---|
| meta | (1,1) struct fields productType (1,1) string "GSM"\|"GAA"\|"GAB"\|"GAC"\|"GAD", or "unknown" when no prefix matches epochStart, epochStop (1,1) double coverage span [decimal years], NaN when no date pattern matches epoch (1,1) double span mid-point [decimal years], NaN with the span |

### shLowLevel.icgemDate2Year

```
y = shLowLevel.icgemDate2Year(d)
```

Convert ICGEM/GRACE yyyymmdd(.hhmm) epochs to decimal years. Y = shLowLevel.icgemDate2Year(D) converts date codes of the form yyyymmdd or yyyymmdd.hhmm (as used in ICGEM gfct files, TN-13/TN-14) to decimal years, using day-of-year / (days in year) with proper leap years. Values that do not look like date codes (< 1.8e7, e.g. already-decimal years in legacy synthetic files) are returned UNCHANGED - this rule keeps pre-v2.1 files evaluating identically.

| input | size | type | default | description |
|---|---|---|---|---|
| d | n x m | double | required | dates as yyyymmdd(.hhmm) numbers or decimal years (passed through) |

| output | description |
|---|---|
| y | (same size as input) double decimal years |

### shLowLevel.writeGFC

```
shLowLevel.writeGFC(filename, C, S, GM, R, Name=Value)
```

Write Stokes coefficients to an ICGEM ASCII .gfc file. shLowLevel.writeGFC(FILENAME, C, S, GM, R) writes a static ICGEM-format gravity field file readable by shLowLevel.shReadGFC (round-trip tested to full double precision: coefficients are written with %22.14e).

| input | size | type | default | description |
|---|---|---|---|---|
| filename | n x m | — | required | char/string C, S (nmax+1)x(nmax+1) double fully normalized coefficients |
| C | n x m | double | required | cosine Stokes coefficients, C(n+1, m+1) indexing |
| S | n x m | double | required | sine Stokes coefficients, S(n+1, m+1) indexing |
| GM | 1 x 1 | double | required | double [m^3/s^2] |
| R | 1 x 1 | double | required | double [m] opts.SigmaC, opts.SigmaS same layout, formal errors ([] -> zeros, errors flag written as 'no') |
| SigmaC | n x m | double | [] | 1-sigma uncertainties of C, same indexing |
| SigmaS | n x m | double | [] | 1-sigma uncertainties of S, same indexing |
| ModelName | n x m | — | "shAnalysis_export" | string default "shAnalysis_export" |

| input | size | type | default | description |
|---|---|---|---|---|
| TideSystem | n x m | — | "unknown" | string default "unknown" |
| Comment | n x m | — | "" | string extra header comment lines ("" = none) Outputs: none (file on disk; plus <file>.provenance.json sidecar |
| Sidecar | 1 x 1 | logical | true | write the metadata sidecar next to the file |

### shLowLevel.writeGrid

```
shLowLevel.writeGrid(filename, grid, latDeg, lonDeg, Name=Value)
```

Export gridded fields to CF-style netCDF (base MATLAB). shLowLevel.writeGrid('ewh.nc', GRID, LAT, LON, Name ("field")="lwe_thickness", Units ("")="m", Epoch=2010.29) writes a 2-D field or a 3-D stack (nlat x nlon x T with Epochs ([])=) using nccreate/ncwrite - directly readable back by shLowLevel.readMascon (roundtrip-tested), by GMT, panoply, xarray, etc. Time is stored as "days since 2002-01-01" like the mascon products.

| input | size | type | default | description |
|---|---|---|---|---|
| filename | n x m | — | required | char/string .nc target (existing file is replaced; missing parent folders are created, v2.4.1) |
| grid | nlat x nlon | double | required | or (nlat,nlon,T) double |
| latDeg | : x 1 | double | required | double [deg] (document your latitude type - geocentric unless you converted; attribute 'lat_type') |
| lonDeg | : x 1 | double | required | double [deg, 0..360) |
| Name | 1 x 1 | string | "field" | , Units (""), Epoch/Epochs (decimal years, for 3-D), |
| Units | 1 x 1 | string | "" | unit label used for annotation |
| Epochs | : x 1 | double | [] | epochs [decimal years] |
| LatType | 1 x 1 | string | "geocentric" | , Description ("") |
| Description | 1 x 1 | string | "" | free-text description written to the sidecar |
| Sidecar | 1 x 1 | logical | true | write <file>.provenance.json alongside the output |

### shLowLevel.writeAnimation

```
shLowLevel.writeAnimation(ts, filename, Name=Value)
```

Export a series as a map animation, drawn by geoMap. shLowLevel.writeAnimation(TS, 'ewh.avi', Quantity ("ewh")=..., kn ([])=...) synthesizes every epoch of TS onto one grid, draws each frame with geo.map, and writes them to a video container with a provenance sidecar.

| input | size | type | default | description |
|---|---|---|---|---|
| ts | 1 x 1 | shSeries | required | shSeries the series to animate |
| filename | 1 x 1 | string | required | string .mp4 (Windows/macOS) or .avi (portable) |
| Quantity | 1 x 1 | string | "ewh" | any kernelFactors quantity |
| kn | n x m | double | [] | load Love numbers k'_n (mass quantities; never hardcoded) |
| hn | n x m | double | [] | load Love numbers h'_n (deformation_up only) |
| lat | 1 x : | double | -89:2:89 | latitude grid [deg] |
| lon | 1 x : | double | 0:2:358 | longitude grid [deg] |
| LatType | 1 x 1 | string ∈ ["geocentric", "geodetic"] | "geodetic" | "geodetic" \| "geocentric"; a map wants geodetic |
| FrameRate | 1 x 1 | double | 4 | frames per second |
| Sidecar | 1 x 1 | logical | true | write <file>.provenance.json alongside the output |

| input | size | type | default | description |
|---|---|---|---|---|
| Size | 1 x 2 | double | [900 470] | STARTING figure size in pixels, [width height]. geo.map fits the figure to the map from there, so the frames are usually taller than this; they are identical... |
| CRS | 1 x 1 | string | "equirectangular" | forwarded to geo.map as its projection |
| Basemap | 1 x 1 | struct | struct() | ) forwarded to geo.map's Basemap element. The stack-wide CLim and the Divergent flag are inserted into it unless you set them yourself, so overriding is del... |
| GeoArgs | n x m | cell | {} | cell of name-value pairs forwarded to geo.map verbatim |

*Example*

```
    shLowLevel.writeAnimation(ts, fullfile(tempdir, 'ewh.avi'), ...
        Quantity = "ewh", kn = kn, FrameRate = 6);

  See also SHSERIES/TOGRID, SHLOWLEVEL/WRITEFRAMES, REQUIREGEOMAP.
```

### Legendre, spectral tools & indexing

#### shLowLevel.legendreALF

```
P = shLowLevel.legendreALF(nmax, lat)
```

Fully normalized (4-pi) associated Legendre functions Pbar_nm. P = LEGENDREALF(NMAX, LAT) computes fully normalized associated Legendre functions up to degree/order NMAX for each latitude in LAT (radians, geocentric, scalar or vector), using the scaled forward column recursion of Holmes & Featherstone (2002): the u^m factor of each column is carried separately with a 1e-280 scale factor, which keeps the recursion overflow/underflow-free up to at least degree 2190 (EGM2008/XGM2019e) at all latitudes including nea...

| input | size | type | default | description |
|---|---|---|---|---|
| nmax | — | — | required | — |
| lat | — | — | required | — |

| output | description |
|---|---|
| P | (NMAX+1)x(NMAX+1)xnumel(LAT) double P(n+1,m+1,k) = Pbar_{n,m}(sin(lat(k))), upper triangle 0 |

#### shLowLevel.legendreALFDeriv

```
[P, D, D2] = shLowLevel.legendreALFDeriv(nmax, latRad)
```

Fully normalized ALFs and their latitude derivatives. [P, D] = shLowLevel.legendreALFDeriv(NMAX, LATRAD) returns the 4pi-fully- normalized associated Legendre functions P (via shLowLevel.legendreALF, scaled Holmes-Featherstone, stable to degree 2190) and their exact first derivatives with respect to LATITUDE phi,

| input | size | type | default | description |
|---|---|---|---|---|
| nmax | 1 x 1 | double | required | double maximum degree |
| latRad | 1 x : | double | required | double [rad] geocentric latitudes |

| output | description |
|---|---|
| P | (nmax+1, nmax+1, nlat) double Pbar_nm(sin lat) |
| D | (nmax+1, nmax+1, nlat) double dPbar_nm/dphi [1/rad] D2 (nmax+1, nmax+1, nlat) double d2Pbar_nm/dphi2 [1/rad^2]: the SAME column stencil applied to D - exact, because the first-derivative identity holds degree-wise as a function of phi and differentiates term by term (Python-validated vs numerical second differences, rel ~2e-7 = FD floor; v2.4) |

#### shLowLevel.legendreCached

```
P = shLowLevel.legendreCached(nmax, latRad)
```

Cached fully normalized Legendre functions. P = shLowLevel.legendreCached(NMAX, LATRAD) returns shLowLevel.legendreALF(NMAX, LATRAD), serving repeated requests for the same (NMAX, LATRAD) from a process-wide cache. Cache hits are verified with an EXACT isequal comparison of the stored latitude vector -- a mismatched latVec can never silently return the wrong P (this removes the documented footgun of shSynthesis' raw 'P' passthrough).

| input | size | type | default | description |
|---|---|---|---|---|
| nmax | — | — | required | — |
| latRad | — | — | required | — |

| output | description |
|---|---|
| P | (nmax+1)x(nmax+1)xnumel(latRad) double |

### shLowLevel.ylm

```
Y = shLowLevel.ylm(latRad, lonRad, idx)
```

4-pi normalized SH basis vectors at arbitrary points. Y = shLowLevel.ylm(LATRAD, LONRAD, IDX) evaluates the basis at nPts points (geocentric latitude/longitude in radians, equal-length vectors), so that f(points) = Y * x for a coefficient vector x in IDX ordering. Vectorized over points via one shLowLevel.legendreALF call.

| input | size | type | default | description |
|---|---|---|---|---|
| latRad | : x 1 | double | required | double geocentric latitudes [rad] |
| lonRad | : x 1 | double | required | double longitudes [rad] |
| idx | 1 x 1 | struct | required | struct from shLowLevel.shIndex |

| output | description |
|---|---|
| Y | (nPts x idx.P) double |

### shLowLevel.shIndex

```
idx = shLowLevel.shIndex(Lmax, Name=Value)
```

Canonical order-major index for SH coefficient vectors. idx = shLowLevel.shIndex(Lmax) builds the index table used by all shLowLevel functions: order-major ordering, C before S per (n,m), degrees n = Nmin..Lmax (default Nmin = 2; degree 0/1 and the SLR-C20 replacement are assumed to be handled upstream).

| input | size | type | default | description |
|---|---|---|---|---|
| Lmax | 1 x 1 | double | required | maximum spherical-harmonic degree of the index |
| Nmin | 1 x 1 | double | 2 | minimum spherical harmonic degree |

| output | description |
|---|---|
| idx | (1,1) struct the canonical vector ordering of the toolbox, fields n, m, cs (P x 1) double degree, order and 0=cosine/1=sine of every element, in vector order P (1,1) double number of elements Lmax (1,1) double maximum degree (the LMAX argument) minDegree (1,1) double first degree included (the Nmin option; the FIELD keeps the older spelling) pos (Lmax+1 x Lmax+1 x 2) double reverse map, the vector position of (n, m, cs) or 0 when absent |

### shLowLevel.vecFromCS

```
x = shLowLevel.vecFromCS(cnm, snm, idx)
```

Pack lower-triangular Cnm/Snm matrices into a shLowLevel vector. x = shLowLevel.vecFromCS(cnm, snm, idx) with cnm, snm of size (Lmax+1)x(Lmax+1), cnm(n+1,m+1) = Cnm, snm(n+1,m+1) = Snm.

| input | size | type | default | description |
|---|---|---|---|---|
| cnm | n x m | double | required | cosine coefficient matrix, C(n+1, m+1) indexing |
| snm | n x m | double | required | sine coefficient matrix, same indexing |

| input | size | type | default | description |
|---|---|---|---|---|
| idx | 1 x 1 | struct | required | index struct from shLowLevel.shIndex (fields n, m, cs, P, Lmax, minDegree, pos) |

| output | description |
|---|---|
| x | (P x 1) double coefficients gathered in idx ordering |

### shLowLevel.csFromVec

```
[cnm, snm] = shLowLevel.csFromVec(x, idx)
```

Unpack a shLowLevel vector into lower-triangular Cnm/Snm matrices. [cnm, snm] = shLowLevel.csFromVec(x, idx). Coefficients outside the index range (n < Nmin) are zero.

| input | size | type | default | description |
|---|---|---|---|---|
| x | : x 1 | double | required | coefficient stack, (P x T) in index ordering |
| idx | 1 x 1 | struct | required | index struct from shLowLevel.shIndex (fields n, m, cs, P, Lmax, minDegree, pos) |

| output | description |
|---|---|
| cnm | (nmax+1 x nmax+1) double cosine coefficients, C(n+1, m+1) |
| snm | (nmax+1 x nmax+1) double sine coefficients, S(:, 1) = 0 |

### shLowLevel.shDegreeRMS

```
spec = shLowLevel.shDegreeRMS(C, S, Name=Value)
```

Spectral (degree-domain) analysis of spherical harmonic coefficients. SPEC = SHDEGREERMS(C, S) computes, per degree n = 0..nmax: .degVariance(n+1) = sum_m (Cnm^2 + Snm^2) .degRMS(n+1) = sqrt(degVariance) .degAmplitude(n+1) = R * degRMS [geoid-height-equivalent, m] .cumAmplitude(n+1) = R * sqrt(cumsum(degVariance)) [cumulative, m]

| input | size | type | default | description |
|---|---|---|---|---|
| C | n x m | double | required | cosine Stokes coefficients, C(n+1, m+1) indexing |
| S | n x m | double | required | sine Stokes coefficients, S(n+1, m+1) indexing |
| R | 1 x 1 | double | 6378136.3 | (1,1) double reference radius [m] |
| Nmin | 1 x 1 | double | 0 | (1,1) double lowest degree to include; coefficients below it are zeroed before summation (Nmin=2 drops degree 0 and 1). Renamed from n0 in v4.0.0. |
| SigmaC | n x m | double | [] | formal 1-sigma of C, same layout |
| SigmaS | n x m | double | [] | formal 1-sigma of S, same layout |

| output | description |
|---|---|
| spec | (1,1) struct fields degree (nmax+1 x 1) double the degree axis degVariance, degRMS, degAmplitude (nmax+1 x 1) double per-degree signal measures cumVariance, cumRMS, cumAmplitude (nmax+1 x 1) double their cumulative counterparts R (1,1) double the radius the amplitudes use domain (1,:) char 'degree' (consumed by plotSHSpectrum) errVariance, errRMS, errAmplitude (nmax+1 x 1) double the same for the formal errors; present only when sigmas were supplied |

### shLowLevel.shOrderRMS

```
spec = shLowLevel.shOrderRMS(C, S, Name=Value)
```

Order-domain spectral diagnostics (striping direction). SPEC = shLowLevel.shOrderRMS(C, S) sums coefficient power per ORDER m - the complementary view to shLowLevel.shDegreeRMS and the natural axis for GRACE striping (correlated errors live at high orders):

| input | size | type | default | description |
|---|---|---|---|---|
| C | n x m | double | required | cosine Stokes coefficients, C(n+1, m+1) indexing |
| S | n x m | double | required | sine Stokes coefficients, S(n+1, m+1) indexing |
| R | 1 x 1 | double | 6378136.3 | [m], Nmin (0) zero degrees below it first, SigmaC, SigmaS ([]) formal errors |
| Nmin | 1 x 1 | double | 0 | minimum spherical harmonic degree |
| SigmaC | n x m | double | [] | 1-sigma uncertainties of C, same indexing |
| SigmaS | n x m | double | [] | 1-sigma uncertainties of S, same indexing |

| output | description |
|---|---|
| spec | (1,1) struct fields order (nmax+1 x 1) double the order axis ordVariance, ordRMS, ordAmplitude (nmax+1 x 1) double per-order signal measures cumVariance, cumRMS, cumAmplitude (nmax+1 x 1) double their cumulative counterparts R (1,1) double the radius the amplitudes use domain (1,:) char 'order' (consumed by plotSHSpectrum) errVariance, errRMS, errAmplitude (nmax+1 x 1) double the same for the formal errors; present only when sigmas were supplied |

### shLowLevel.shSpectralCrossover

```
[nCrossover, degreeInterp] = shLowLevel.shSpectralCrossover(spec)
```

Degree at which the error spectrum first exceeds the signal (degree-amplitude) spectrum -- a standard guide for choosing a truncation degree or Gaussian filter radius.

| input | size | type | default | description |
|---|---|---|---|---|
| spec | — | — | required | — |

| output | description |
|---|---|
| nCrossover | (1,1) double first degree with err >= signal (NaN if none) |
| degreeInterp | (1,1) double linearly interpolated crossing degree |

### shLowLevel.gaussLegendre

```
[x, w] = shLowLevel.gaussLegendre(n)
```

Gauss-Legendre nodes and weights on [-1, 1]. [X, W] = shLowLevel.gaussLegendre(N) via Golub-Welsch (symmetric eigenvalue problem of the Jacobi matrix). Exact for polynomials up to degree 2N-1: integral f(t) dt = sum(W .* f(X)).

| input | size | type | default | description |
|---|---|---|---|---|
| n | 1 x 1 | double | required | double number of nodes |

| output | description |
|---|---|
| x | (n,1) double nodes, ascending |
| w | (n,1) double weights, sum(w) = 2 |

## Synthesis & analysis

### shLowLevel.shSynthesis

```
[grid, lat, lon, P] = shLowLevel.shSynthesis(C, S, GM, R, latVec, lonVec, Name=Value)
```

Spherical harmonic synthesis of a global/regional grid. [GRID, LAT, LON] = SHSYNTHESIS(C, S, GM, R, LATVEC, LONVEC) synthesizes the disturbing-potential-derived quantity (default: geoid height) on the grid LATVEC x LONVEC (degrees) from fully normalized (ICGEM) Stokes coefficients C, S at the reference sphere r = R - or, with Surface = "ellipsoid", on the reference ellipsoid (v5.5.0).

| input | size | type | default | description |
|---|---|---|---|---|
| C | n x m | double | required | cosine Stokes coefficients, C(n+1, m+1) indexing |
| S | n x m | double | required | sine Stokes coefficients, S(n+1, m+1) indexing |

| input | size | type | default | description |
|---|---|---|---|---|
| GM | n x m | double | required | gravitational constant times Earth mass [m^3/s^2] (default 3.986004415e14, overridable) |
| R | n x m | double | required | reference radius [m] (default 6378136.3, overridable) |
| latVec | n x m | double | required | geocentric latitude vector of the grid [deg] |
| lonVec | n x m | double | required | longitude vector of the grid [deg], [0, 360) |
| Quantity | n x m | — | 'geoid' | one of ('none' = dimensionless passthrough, kernel 1, no GM/R/kn enter - the clean way to synthesize raw coefficient fields such as kernels or masks): 'geoid... |
| Nmin | 0 | — | 0 | , Nmax (size(C,1)-1) degree range to include. Renamed from nmin/nmax in v4.0.0. |
| Nmax | n x m | — | size(C,1)-1 | maximum spherical harmonic degree |
| kn | n x 1 | — | [] | load Love number vector k_n (index 1 = degree 0), required for "ewh"; NOT supplied by default -- see note below. |
| rho_ave | 5517 | — | 5517 | average Earth density [kg/m^3] (used for "ewh") |
| rho_water | 1000 | — | 1000 | water density [kg/m^3] (used for "ewh") |
| P | n x m | — | [] | precomputed Legendre array from a prior call/LEGENDREALF, for the identical latVec and nmax -- skips recomputing the recursion. No validation is done that la... |
| Method | n x m | — | 'auto' | "auto" / "direct" / "fft": FFT-along-longitude synthesis for uniform full-circle longitude grids; 'auto' picks it whenever applicable. It is not always faste... |
| MaxMemGB | 4 | — | 4 | Legendre-block memory budget before banding |
| hn | n x m | — | [] | load Love numbers h'_n, for deformation quantities |
| Height | 0 | — | 0 | evaluation height [m] above the evaluation surface: above R on the sphere, above r(phi) on the ellipsoid |
| Surface | 1 x 1 | string ∈ ["sphere", "ellipsoid"] | "sphere" | "sphere" \| "ellipsoid". "sphere" evaluates at r = R + Height, bitwise as before v5.5.0. "ellipsoid" evaluates at r(phi) + Height, r(phi) the geocentric radiu... |
| SemiMajorAxis | 1 x 1 | double | 6378137 | semi-major axis of the ellipsoid [m], WGS84; used only with Surface = "ellipsoid" |
| Flattening | 1 x 1 | double | 1/298.257223563 | flattening of the ellipsoid, WGS84; used only with Surface = "ellipsoid" |

| output | description |
|---|---|
| grid | (nlat x nlon) double synthesized field in the units of the requested quantity |
| lat | (1 x nlat) double geocentric latitude grid used [deg] |
| lon | (1 x nlon) double longitude grid used [deg] |
| P | (nmax+1 x nmax+1 x nlat) double Legendre array as returned by LEGENDREALF; pass it back via 'P' to skip the recursion. EMPTY in latitude-banded mode (MaxMemGB exceeded): the blocks are streamed and discarded, so request the 4th output only with a memory budget that fits the full array. |

### shLowLevel.synthesisMatrix

```
[Y, w, grid] = shLowLevel.synthesisMatrix(idx, Name=Value)
```

Dense SH synthesis matrix on a Gauss-Legendre grid. [Y, W, GRID] = shLowLevel.synthesisMatrix(IDX) returns Y (Ngrid x P) with f = Y*x for coefficient vectors x in IDX ordering (shLowLevel.shIndex), and quadrature weights W

(Ngrid x 1), normalized such that the discrete analysis A = Y' * diag(W) is EXACT for band-limited fields: A*Y = eye(P) to machine precision (asserted in the test suite).

| input | size | type | default | description |
|---|---|---|---|---|
| idx | 1 x 1 | struct | required | struct from shLowLevel.shIndex |
| NLat | 1 x 1 | double | idx.Lmax + 1 | number of latitude rings [default Lmax+1] |
| NLon | 1 x 1 | double | 2*idx.Lmax + 2 | number of longitudes [default 2*idx.Lmax + 2] |

| output | description |
|---|---|
| Y | (NLat*NLon x idx.P) double synthesis matrix |
| w | (NLat*NLon x 1) double quadrature weights, sum(w) = 1 |
| grid | (1,1) struct fields theta (NLat x 1) double co-latitude of each ring [rad] lambda (NLon x 1) double longitude of each column [rad] latDeg (NLat x 1) double the same latitudes [deg] lonDeg (NLon x 1) double the same longitudes [deg] |

### shLowLevel.shSynthesisDeformation

```
[up, north, east] = shLowLevel.shSynthesisDeformation(C, S, R, latDeg, lonDeg, Name=Value)
```

Elastic load deformation: up / north / east. [UP, NORTH, EAST] = shLowLevel.shSynthesisDeformation(C, S, R, LAT, LON, kn (""")=..., hn (""")=..., ln (""")=...) synthesizes the three components of the elastic surface deformation caused by the load expressed in the (residual) Stokes coefficients C, S (Wahr et al. 1998; Kusche & Schrama 2005 - the GRACE <-> GNSS loading comparison):

| input | size | type | default | description |
|---|---|---|---|---|
| C | n x m | double | required | cosine Stokes coefficients, C(n+1, m+1) indexing |
| S | n x m | double | required | sine Stokes coefficients, S(n+1, m+1) indexing |
| R | 1 x 1 | double | required | double [m] |
| latDeg | 1 x : | double | required | double [deg] geocentric latitude |
| lonDeg | 1 x : | double | required | double [deg] |
| kn | n x m | double | required | double load Love numbers k'_n (REQUIRED) |
| hn | n x m | double | required | double load Love numbers h'_n (REQUIRED) |
| ln | n x m | double | required | double load Love numbers l'_n (REQUIRED) |
| Nmin | 1 x 1 | double | 1 | double = 1 degrees below nmin are zeroed (degree 0 carries no deformation; include degree 1 ONLY if your coefficients carry geocenter motion in the CF fra... |
| Mode | 1 x 1 | string ∈ ["grid","points"] | "grid" | "grid"\|"points" = "grid" grid: nlat x nlon outer product; points: equal-length LAT/LON vectors evaluated pairwise (GNSS stations) |

| output | description |
|---|---|
| up, north, east | (nlat,nlon) double for Mode="grid", (np,1) for Mode="points"; vertical, north and east elastic deformation [m]. EAST is NaN at \|lat\| = 90, where the direction is undefined. |

### shLowLevel.shSynthesisGradientTensor

```
[comps, info] = shLowLevel.shSynthesisGradientTensor(C, S, GM, R, latDeg, lonDeg, Name=Value)
```

Full gravity gradient tensor in the local NEU frame. [COMPS, INFO] = shLowLevel.shSynthesisGradientTensor(C, S, GM, R, LAT, LON, Height (250e3)=250e3) synthesizes all six independent components of the disturbing gravity gradient tensor (GOCE-style diagnostics) in the local north-east-up frame at radius r = R + Height:

| input | size | type | default | description |
|---|---|---|---|---|
| C | n x m | double | required | cosine Stokes coefficients, C(n+1, m+1) indexing |
| S | n x m | double | required | sine Stokes coefficients, S(n+1, m+1) indexing |

| input | size | type | default | description |
|---|---|---|---|---|
| GM | 1 x 1 | double | required | gravitational constant times Earth mass [m^3/s^2] (default 3.986004415e14, overridable) |
| R | 1 x 1 | double | required | reference radius [m] (default 6378136.3, overridable) |
| latDeg | 1 x : | double | required | , lonDeg (1,nlon) double [deg], geocentric |
| lonDeg | 1 x : | double | required | longitudes [deg], lon in [0, 360) |
| Height | 1 x 1 | double | 250e3 | [m] evaluation altitude above R |
| Nmin | 1 x 1 | double | 2 | zero degrees below nmin |

| output | description |
|---|---|
| comps | (1,1) struct the six independent components of the symmetric tensor, each (nlat,nlon) double [1/s^2]; 1 Eotvos = 1e-9 1/s^2. Fields uu, nn, ee the diagonal (up, north, east) un, ue, ne the off-diagonal terms. Every component involving east is NaN at \|lat\| = 90 |
| info | (1,1) struct fields maxTraceResidual (1,1) double max \|trace\| / max\|G\| over the grid, poles excluded - the built-in Laplace self-check r (1,1) double evaluation radius [m] Height (1,1) double the altitude r was formed from, echoed back [m] |

### shLowLevel.shAnalysisGrid

```
[C, S, info] = shLowLevel.shAnalysisGrid(grid, latVec, lonVec, nmax, Name=Value)
```

Spherical harmonic analysis: estimate Stokes coefficients from gridded or scattered data (the inverse of shLowLevel.shSynthesis).

| input | size | type | default | description |
|---|---|---|---|---|
| grid | n x m | double | required | field values on the (lat, lon) grid |
| latVec | n x m | double | required | geocentric latitude vector of the grid [deg] |
| lonVec | n x m | double | required | longitude vector of the grid [deg], [0, 360) |
| nmax | 1 x 1 | double | required | maximum spherical harmonic degree |
| Method | 1 x 1 | string ∈ ["auto","rings","ls"] | "auto" | "auto" (default) \| "rings" \| "ls" |
| Quantity | n x m | — | 'geoid' | 'geoid' (default) \| 'potential' \| 'gravity_anomaly' \| 'gravity_disturbance' \| 'ewh' - the grid's physical Quantity; the degree-dependent kernel (shLowLevel.... |
| GM | 1 x 1 | double | 3.986004415e14 | gravitational constant times Earth mass [m^3/s^2] (default 3.986004415e14, overridable) |
| R | 1 x 1 | double | 6378136.3 | reference radius [m] (default 6378136.3, overridable) |
| kn | n x m | double | [] | load Love numbers, required for Quantity='ewh' rho_ave, rho_water EWH densities (5517, 1000) |
| hn | n x m | double | [] | vertical-deformation Love numbers, degrees 0..nmax (user-supplied; only reached through kernelFactors) |
| rho_ave | 1 x 1 | double | 5517 | mean Earth density [kg/m^3] (default 5517, overridable) |
| rho_water | 1 x 1 | double | 1000 | water density [kg/m^3] (default 1000, overridable) |
| Weights | 1 x 1 | string ∈ ["none","coslat"] | "none" | "none" (default) \| "coslat" - area weighting of rings/ points (recommended for equiangular ring grids: rows are scaled by sqrt(cos lat), a diagonal approxima... |

| input | size | type | default | description |
|---|---|---|---|---|
| Kaula | 1 x 1 | double | 0 | 0 (default: off). If > 0, Tikhonov regularization toward 0 with per-degree standard deviation Kaula*1e-5/n^2 (Kaula's rule; Kaula=1 is the classic value). RE... |
| ChunkSize | 1 x 1 | double | 2000 | points per accumulation chunk for "ls" (default 2000) |
| LatType | 1 x 1 | string ∈ ["geocentric", "geodetic"] | "geocentric" | "geocentric" (default) \| "geodetic": how LATVEC is interpreted; geodetic values are converted with Flattening before evaluation |
| Flattening | 1 x 1 | double | 1/298.257223563 | flattening of the reference ellipsoid for that conversion (WGS84; overridable, never silently assumed) |

| output | description |
|---|---|
| C | (nmax+1 x nmax+1) double estimated cosine coefficients |
| S | (nmax+1 x nmax+1) double estimated sine coefficients |
| info | (1,1) struct fields method (1,:) char operator actually used, 'rings' or 'ls' nPoints (1,1) double observations entering the estimate unobservedDegrees (1,:) double degrees with a zero kernel, returned as 0 in C and S regularized (1,1) logical whether Kaula > 0 was applied condEst (1,1) double condition estimate of the system (worst per-order value for 'rings') residRMS (1,1) double RMS of the post-fit residuals in the units of the input grid |

### shLowLevel.kernelFactors

```
[kernel, radialExp, units] = shLowLevel.kernelFactors(quantity, nmax, GM, R, Name=Value)
```

Degree-dependent spectral factors for SH synthesis/analysis. KERNEL = shLowLevel.kernelFactors(QUANTITY, NMAX, GM, R, kn ([])=..., ...) returns the (NMAX+1)x1 factor f_n such that a field of QUANTITY is q(lat,lon) = sum_n f_n * sum_m Pbar_nm (C_nm cos + S_nm sin). Shared by shLowLevel.shSynthesis (multiply) and shLowLevel.shAnalysisGrid (divide); the single source of truth for the quantity definitions.

| input | size | type | default | description |
|---|---|---|---|---|
| quantity | n x m | — | required | char/string see above |
| nmax | 1 x 1 | double | required | double GM, R (1,1) double [m^3/s^2], [m] |
| GM | 1 x 1 | double | required | gravitational constant times Earth mass [m^3/s^2] (default 3.986004415e14, overridable) |
| R | 1 x 1 | double | required | reference radius [m] (default 6378136.3, overridable) |
| kn | n x m | double | [] | double load Love numbers k'_n (NEVER hardcoded - user-supplied) |
| hn | n x m | double | [] | double load Love numbers h'_n (deformation_up only; user-supplied) |
| rho_ave | 1 x 1 | double | 5517 | double = 5517 [kg/m^3] |
| rho_water | 1 x 1 | double | 1000 | double = 1000 [kg/m^3] |
| Height | 1 x 1 | double | 0 | double = 0 [m] evaluation height above R: multiplies the potential-type kernels by the upward- continuation attenuation (R/r)^p with r = R+Height (p = n... |

| output | description |
|---|---|
| kernel | (nmax+1,1) double (zeros where the quantity carries no information, e.g. n=1 for gravity_anomaly) radialExp (nmax+1,1) double the upward-continuation exponent p_n in f_n(r) = kernel_n (R/r)^p_n, or EMPTY for a quantity that has no radial law (a surface concept). This is the same p_n the Height branch below applies, returned rather than restated: shLowLevel.shSynthesisGradient needs it for d/dr = -(p_n/r) f_n, and a second copy of these exponents would be a fact written twice. |

| output | description |
|---|---|
| units | (1,1) string the SI unit of the quantity ("" for the dimensionless passthrough). Same reason as radialExp: the unit belongs to the quantity, so it is returned from where the quantity is defined rather than restated in every bridge, plot label and export header. Consumed by shCoefficients.toGrid. |

### shLowLevel.normalFieldCS

```
[Cn, info] = shLowLevel.normalFieldCS(nmax, Name=Value)
```

Even zonal harmonics of an ellipsoidal normal field. [CN, INFO] = shLowLevel.normalFieldCS(NMAX, System="WGS84") returns the fully normalized even zonal coefficients Cbar_{2n,0} of the normal gravity field, COMPUTED from the defining constants (GM (NaN), a (NaN), f (NaN), omega (NaN)) via the closed-form theory (Heiskanen & Moritz 1967, sect. 2-9) - no coefficient tables anywhere:

| input | size | type | default | description |
|---|---|---|---|---|
| nmax | 1 x 1 | double | required | double maximum degree of the output |
| System | 1 x 1 | string ∈ ["WGS84","GRS80"] | "WGS84" | "WGS84" \| "GRS80": presets for the DEFINING constants below (citable definitions, not data) GM, a, f, omega override any defining constant. GRS80 defines J... |
| GM | 1 x 1 | double | NaN | gravitational constant times Earth mass [m^3/s^2] (default 3.986004415e14, overridable) |
| a | 1 x 1 | double | NaN | semi-major axis of the normal ellipsoid [m] |
| f | 1 x 1 | double | NaN | flattening of the normal ellipsoid |
| omega | 1 x 1 | double | NaN | angular velocity of the normal ellipsoid [rad/s] |

| output | description |
|---|---|
| Cn | (nmax+1,1) double Cbar_{n,0} (zero at odd/omitted degrees; Cn(1) = 1 represents the central GM/r term - subtract it ONLY together with a matching degree-0 of the model, see subtractNormalField) |
| info | (1,1) struct the defining constants actually used, fields GM (1,1) double geocentric gravitational constant a (1,1) double semi-major axis [m] f (1,1) double flattening omega (1,1) double angular velocity [rad/s] J2 (1,1) double dynamic form factor, COMPUTED from the four above, not tabulated e2 (1,1) double first eccentricity squared m (1,1) double omega^2 a^2 b / GM q0 (1,1) double the auxiliary Somigliana quantity system (1,1) string "WGS84" \| "GRS80" |

### shLowLevel.rescaleGMR

```
[C2, S2, sig] = shLowLevel.rescaleGMR(C, S, GM1, R1, GM2, R2, sigmaC, sigmaS)
```

Convert Stokes coefficients between (GM, R) reference values. [C2, S2] = shLowLevel.rescaleGMR(C, S, GM1, R1, GM2, R2) re-expresses coefficients given w.r.t. (GM1, R1) in the reference (GM2, R2):

| input | size | type | default | description |
|---|---|---|---|---|
| C | n x m | double | required | cosine Stokes coefficients, C(n+1, m+1) indexing |
| S | n x m | double | required | sine Stokes coefficients, S(n+1, m+1) indexing |
| GM1 | 1 x 1 | double | required | GM the coefficients are currently scaled to [m^3/s^2] |
| R1 | 1 x 1 | double | required | R the coefficients are currently scaled to [m] |
| GM2 | 1 x 1 | double | required | target GM [m^3/s^2] |
| R2 | 1 x 1 | double | required | target R [m] |
| sigmaC | n x m | double | [] | 1-sigma uncertainties of C, same indexing |
| sigmaS | n x m | double | [] | 1-sigma uncertainties of S, same indexing |

| output | description |
|---|---|
| C2, S2 | (n1,n1) double coefficients w.r.t. (GM2, R2) |
| sig | (1,1) struct fields C (n1,n1) double or [] rescaled cosine sigmas, empty when sigmaC was not supplied S (n1,n1) double or [] rescaled sine sigmas, empty when sigmaS was not supplied |

### shLowLevel.geodetic2geocentric

```
latGC = shLowLevel.geodetic2geocentric(latGD, Name=Value)
```

Convert geodetic to geocentric latitude. LATGC = shLowLevel.geodetic2geocentric(LATGD) converts geodetic (ellipsoidal) latitudes to geocentric latitudes via tan(latGC) = (1 - f)^2 * tan(latGD) All spherical-harmonic routines in this toolbox (synthesis, analysis, Legendre functions) expect GEOCENTRIC latitude; user grids from maps, GIS or mascon products are almost always GEODETIC. Forgetting this conversion biases mid-latitude values by up to ~0.19 deg of latitude (~21 km) - a classic silent error source.

| input | size | type | default | description |
|---|---|---|---|---|
| latGD | n x m | double | required | double array geodetic latitudes [deg] |
| Flattening | 1 x 1 | double | 1/298.257223563 | double = 1/298.257223563 (WGS84; override for other reference ellipsoids - a DEFAULT, not physics baked in) |

| output | description |
|---|---|
| latGC | double array geocentric latitudes [deg], poles/equator exact |

### shLowLevel.geocentric2geodetic

```
latGD = shLowLevel.geocentric2geodetic(latGC, Name=Value)
```

Convert geocentric to geodetic latitude. LATGD = shLowLevel.geocentric2geodetic(LATGC) inverts shLowLevel.geodetic2geocentric: tan(latGD) = tan(latGC) / (1 - f)^2.

| input | size | type | default | description |
|---|---|---|---|---|
| latGC | n x m | double | required | double array geocentric latitudes [deg] |
| Flattening | 1 x 1 | double | 1/298.257223563 | double = 1/298.257223563 (WGS84 default) |

| output | description |
|---|---|
| latGD | double array geodetic latitudes [deg] |

### shLowLevel.evalMask

```
[mask, grid] = shLowLevel.evalMask(idx, region, Name=Value)
```

Evaluate a region definition on the toolbox quadrature grid. [MASK, GRID] = shLowLevel.evalMask(IDX, REGION) evaluates REGION on the Gauss-Legendre quadrature grid of shLowLevel.synthesisMatrix(IDX) and returns MASK (Ngrid x 1, in [0,1]) plus the GRID struct. Shared by shLowLevel.basinKernel and shLowLevel.slepianBasis.

| input | size | type | default | description |
|---|---|---|---|---|
| idx | 1 x 1 | struct | required | index struct from shLowLevel.shIndex (fields n, m, cs, P, Lmax, minDegree, pos) |
| region | n x m | — | required | region as polygon [lat lon] deg, mask, or @(lat, lon) handle (see evalMask) |
| BufferKm | 1 x 1 | double | 0 | >0 grows, <0 shrinks the region by great-circle distance [km] to the region boundary points, computed exactly in latitude chunks (O(N*Nregion); fine to Lmax ... |
| R | 1 x 1 | double | 6378136.3 | sphere radius [m] for the km conversion |
| OverSample | 1 x 1 | double | 2 | refinement factor of the quadrature grid the mask is evaluated on; the Gauss rule uses OverSample*(Lmax+1) latitude rings, so a higher value resolves the reg... |

| output | description |
|---|---|
| mask | (Ngrid,1) double in [0,1] |

| output | description |
|---|---|
| grid | struct from shLowLevel.synthesisMatrix (latDeg, lonDeg per ring) |

### Filtering & operators

#### shLowLevel.shGaussianWeights

```
Wn = shLowLevel.shGaussianWeights(nmax, radius_km, Name=Value)
```

Degree-domain weights for isotropic Gaussian smoothing. WN = SHGAUSSIANWEIGHTS(NMAX, RADIUS_KM) computes the Jekeli (1981) Gaussian averaging function weights Wn, n = 0..NMAX, for an averaging radius RADIUS_KM [km] (half-width at which the spatial weighting function drops to 1/2). Standard GRACE/GRACE-FO mass-change smoothing.

| input | size | type | default | description |
|---|---|---|---|---|
| nmax | 1 x 1 | double | required | maximum spherical harmonic degree |
| radius_km | 1 x 1 | double | required | — |
| R | 1 x 1 | double | 6378.1363 | (1,1) double sphere radius [km] in the b formula |

| output | description |
|---|---|
| Wn | (nmax+1 x 1) double Jekeli (1981) weights, W(1) = 1, monotone non-increasing |

#### shLowLevel.shGaussianFilter

```
[Cf, Sf, Wn] = shLowLevel.shGaussianFilter(C, S, radius_km, varargin)
```

Apply isotropic Gaussian smoothing to Stokes coefficients. [CF, SF] = SHGAUSSIANFILTER(C, S, RADIUS_KM) multiplies each degree n row of C, S by the Jekeli (1981) weight Wn(n), see SHGAUSSIANWEIGHTS. Standard pre-processing step for GRACE/GRACE-FO mass-change maps before spatial synthesis, used to suppress high-degree noise.

| input | size | type | default | description |
|---|---|---|---|---|
| C | — | — | required | — |
| S | — | — | required | — |
| radius_km | — | — | required | — |
| varargin | — | — | required | — |

| output | description |
|---|---|
| Cf | (nmax+1 x nmax+1) double smoothed cosine coefficients |
| Sf | (nmax+1 x nmax+1) double smoothed sine coefficients |
| Wn | (nmax+1 x 1) double Jekeli degree weights applied |

#### shLowLevel.shFanFilter

```
[Cf, Sf] = shLowLevel.shFanFilter(C, S, rDegKm, rOrdKm, Name=Value)
```

Han fan filter: separable degree x order Gaussian smoothing. [CF, SF] = shLowLevel.shFanFilter(C, S, RDEGKM, RORDKM) applies the fan filter of Han et al. (2005): the product of a Jekeli Gaussian in DEGREE and one in ORDER, W_nm = W_n(rDeg) * W_m(rOrd), damping high orders (the striping direction) harder than an isotropic Gaussian of equal degree radius - the cheap anisotropic benchmark between Jekeli and DDK/tvANS. Common choice: rOrd = rDeg (original fan) or rOrd < rDeg for stronger destriping.

| input | size | type | default | description |
|---|---|---|---|---|
| C | n x m | double | required | cosine Stokes coefficients, C(n+1, m+1) indexing |
| S | n x m | double | required | sine Stokes coefficients, S(n+1, m+1) indexing |
| rDegKm | 1 x 1 | double | required | double degree half-response radius [km] |
| rOrdKm | 1 x 1 | double | required | double order half-response radius [km] |

| input | size | type | default | description |
|---|---|---|---|---|
| R | 1 x 1 | double | 6378136.3 | double = 6378136.3 [m] |

| output | description |
|---|---|
| Cf, Sf | (n1,n1) double filtered coefficients |

### shLowLevel.shDestripe

```
[Cf, Sf] = shLowLevel.shDestripe(C, S, Name=Value)
```

Remove correlated (north-south striping) noise from Stokes coefficients, following Swenson & Wahr (2006).

| input | size | type | default | description |
|---|---|---|---|---|
| C | n x m | double | required | cosine Stokes coefficients, C(n+1, m+1) indexing |
| S | n x m | double | required | sine Stokes coefficients, S(n+1, m+1) indexing |
| MinOrder | 1 x 1 | double | 6 | lowest order to filter (below this, real signal dominates and destriping mostly removes signal; the value is model/application dependent -- verify against yo... |
| PolyOrder | 1 x 1 | double | 3 | polynomial order fit and removed per order/parity (the classic Swenson & Wahr cubic fit) |
| WindowLength | n x m | double | [] | if empty, fit a single polynomial over the full available degree range per order/parity (the original Swenson & Wahr formulation). If set to an odd integer >... |

| output | description |
|---|---|
| Cf | (nmax+1 x nmax+1) double destriped cosine coefficients |
| Sf | (nmax+1 x nmax+1) double destriped sine coefficients |

### shLowLevel.applyDDK

```
[Cf, Sf] = shLowLevel.applyDDK(C, S, W)
```

Apply an order-block-diagonal (DDK-style) filter to coefficients. [CF, SF] = shLowLevel.applyDDK(C, S, W) applies the filter container from shLowLevel.readDDK: within each (order m, cs) block, the degree vector c(n1..n2) is replaced by W.blocks(k).M * c. Degrees present in C but not covered by any block pass through UNCHANGED (typically n < 2). User-facing wrappers: g.applyDDK(W), ts.applyDDK(W).

| input | size | type | default | description |
|---|---|---|---|---|
| C | n x m | double | required | cosine Stokes coefficients, C(n+1, m+1) indexing |
| S | n x m | double | required | sine Stokes coefficients, S(n+1, m+1) indexing |
| W | 1 x 1 | struct | required | struct from shLowLevel.readDDK (nmax, blocks(m,cs,n,M)) |

| output | description |
|---|---|
| Cf | (nmax+1 x nmax+1) double DDK-filtered cosine coefficients |
| Sf | (nmax+1 x nmax+1) double DDK-filtered sine coefficients |

### shLowLevel.tvANSFilter

```
[Xf, op, info] = shLowLevel.tvANSFilter(X, tYears, idx, Name=Value)
```

Time-variable anisotropic Wiener filtering of a SH series. [XF, OP, INFO] = shLowLevel.tvANSFilter(X, TYEARS, IDX) runs the tvANS chain on the coefficient series X (P x T, IDX ordering, shLowLevel.shIndex):

| input | size | type | default | description |
|---|---|---|---|---|
| X | P x T | double | required | double coefficient vectors per epoch |

| input | size | type | default | description |
|---|---|---|---|---|
| tYears | : x 1 | double | required | double decimal years |
| idx | 1 x 1 | struct | required | struct from shLowLevel.shIndex |
| NoiseCov | P x P | double | [] | double full covariance (else empirical) |
| Constraints | P x q | double | [] | double constraint kernels |
| SignalMode | n x m | — ∈ {'isotropic','inhomogeneous'} | 'isotropic' | 'isotropic' (default) \| 'inhomogeneous' |
| NIterSignal | 1 x 1 | double | 3 | double signal-covariance iterations, 3 |
| Robust | 1 x 1 | logical | false | logical robust deterministic fit, false |
| Shrinkage | 1 x 1 | double | 0.1 | double empirical noise-cov shrinkage, 0.1 |
| VCEMinDegree | 1 x 1 | double | round(2/3 * idx.Lmax) | double default round(2/3 * idx.Lmax) |
| VCEBands | 1 x : | double | [] | (1,:) double order-band edges, e.g. [0 16 33 61]: one monthly VCE factor per order band instead of a single s(t) for the whole field. Requires the block pat... |
| Blocks | n x m | — ∈ {'auto','on','off'} | 'auto' | 'auto' (default) \| 'on' \| 'off' Block-diagonal fast path: with SignalMode='isotropic', empirical noise covariance and no constraints, both S and N are block-... |

| output | description |
|---|---|
| Xf | (P,T) double filtered series (deterministic part restored) |
| op | (1,1) struct the filter operator, for shLowLevel.opApply, shLowLevel.basinDeconvolve and shLowLevel.resolutionMap. Fields layout (1,:) char 'full' \| 'blocks' - which of the two forms below is filled Ut, V (P x P) double the eigenbasis; layout 'full' blocks (1,K) struct per-block eigenbases; layout 'blocks' sBlocks (K x T) double per-block variance factors; 'blocks' lam (P x 1) double generalized eigenvalues s (T x 1) double per-month variance factors Ac, SA double the constraint operator and its covariance action; empty when unconstrained M double the constraint normal matrix idx (1,1) struct the shIndex ordering everything uses tYears (T x 1) double the epochs model (P x T) double the deterministic part removed Xres (P x T) double the residuals filtered Xfres (P x T) double the filtered residuals, filled by the caller detLeverage, detResVar double the deterministic fit's leverage and residual variance, carried so shLowLevel.basinDeconvolve can form the exact basin sigma |
| info | (1,1) struct noise, signal and VCE diagnostics, fields noise (1,1) struct the buildNoiseCov info signal (1,1) struct the buildSignalCov info vce (T x 1) double the per-month variance factors s lam (P x 1) double the generalized eigenvalues sigmaNote (1,:) char which posterior-sigma formula was used; empty when the filter is unconstrained sigmaXfres (P x T) double 1-sigma posterior uncertainty of the filtered residual per coefficient and month, diag((I-W_t)S) = (V.^2)*(s_t*lam./(lam+s_t)), validated to 3.5e-16 against the direct matrix product. With constraints (v2.5) this is the EXACT diagonal of (W-I)S(W-I)' + s*W*N*W'; the earlier unconstrained formula underestimated along constrained directions |

### shLowLevel.buildNoiseCov

```
[N, info] = shLowLevel.buildNoiseCov(Xres, idx, Name=Value)
```

Monthly-solution noise covariance (shape matrix). [N, info] = shLowLevel.buildNoiseCov(Xres, idx) estimates the noise covariance from the residual coefficient series Xres (P x T, residuals about the deterministic model). Two modes:

| input | size | type | default | description |
|---|---|---|---|---|
| Xres | P x T | double | required | double residual coefficient vectors (shIndex ordering) |
| idx | 1 x 1 | struct | required | struct shLowLevel.shIndex matching Xres |
| Mode | n x m | — ∈ {'empirical','full'} | 'empirical' | 'empirical' \| 'full' estimate from Xres, or wrap the user-supplied dense covariance in FullCov |
| FullCov | n x m | double | [] | (P x P) double dense covariance for Mode='full' |

| input | size | type | default | description |
|---|---|---|---|---|
| Shrinkage | 1 x 1 | double | 0.1 | (1 x 1) double gamma in C <- (1-gamma)*C + gamma*diag(diag(C)); Mode='empirical' only |
| Assemble | n x m | — ∈ {'full','blocks'} | 'full' | 'full' \| 'blocks' container layout of N |

| output | description |
|---|---|
| N | (P x P) double for Assemble='full'; for Assemble='blocks' a (1,1) struct with fields layout ('blocks'), blocks (cell of row index vectors), mats (cell of the dense blocks), diagN (P x 1 double) and P (1,1 double) |
| info | (1,1) struct which fields are present depends on the path taken, fields mode (1,:) char 'empirical' \| 'full' \| 'full-blocks' nBlocks (1,1) double number of (order, C/S, parity) blocks; absent for mode 'full' T (1,1) double epochs used; mode 'empirical' only shrinkage (1,1) double the gamma applied; 'empirical' only |

### shLowLevel.buildSignalCov

```
[S, info] = shLowLevel.buildSignalCov(Xres, N, idx, Name=Value)
```

Iterative, data-driven signal covariance. [S, info] = shLowLevel.buildSignalCov(Xres, N, idx) estimates the signal covariance from the residual series itself - no hydrology prior.

| input | size | type | default | description |
|---|---|---|---|---|
| Xres | P x T | double | required | double residual coefficient vectors (shIndex ordering) |
| N | P x P | — | required | double or (1,1) struct noise covariance, dense or the block container from buildNoiseCov(Assemble='blocks'); the block form supports Mode='isotropic' only |
| idx | 1 x 1 | struct | required | struct shLowLevel.shIndex matching Xres |
| Mode | n x m | — ∈ {'isotropic','inhomogeneous'} | 'isotropic' | 'isotropic' \| 'inhomogeneous' signal model |
| NIter | 1 x 1 | double | 3 | (1 x 1) double re-estimation iterations |
| FloorRel | 1 x 1 | double | 1e-8 | (1 x 1) double relative variance floor |
| MapSmooth | 1 x 1 | logical | true | (1 x 1) logical cap the variance map dynamic range to [0.1, 10] before renormalizing it |

| output | description |
|---|---|
| S | (P x P) double signal covariance in idx ordering; sparse diagonal when N is the block container |
| info | (1,1) struct fields c (P x NIter+1) double per-coefficient variances, column 1 the initialization, one column per iteration varMap (Ngrid x 1) double spatial variance map, weighted global mean 1; Mode='inhomogeneous' only |

### shLowLevel.vceRescale

```
s = shLowLevel.vceRescale(N, Xres, idx, Name=Value)
```

Per-month variance factors for the noise covariance shape. s = shLowLevel.vceRescale(N, Xres, idx) estimates one variance factor per month such that N_t = s(t) * N. The factor is the robust (median-based, chi-square(1) corrected) ratio between the observed squared residuals and the formal variances, evaluated over noise-dominated coefficients (n >= Nmin, default round(2/3 * idx.Lmax)). This is the single-component Helmert VCE specialization; it absorbs the month-to-month noise level changes (orbit geometry, beta...

| input | size | type | default | description |
|---|---|---|---|---|
| N | n x m | — | required | spherical harmonic degree |
| Xres | n x m | double | required | residual coefficient stack after the deterministic fit |
| idx | 1 x 1 | struct | required | index struct from shLowLevel.shIndex (fields n, m, cs, P, Lmax, minDegree, pos) |
| Nmin | 1 x 1 | double | round(2/3 * idx.Lmax) | ) (1 x 1) lower bound of the noise-dominated degree band |

| input | size | type | default | description |
|---|---|---|---|---|
| Rows | 1 x 1 | logical | true(0, 1) | ) logical row mask restricting the variance-component estimation to a subset of the coefficient vector. Empty: no restriction. This is how tvANSFilter's VC... |

| output | description |
|---|---|
| s | (T x 1) double one noise variance factor per month |

### shLowLevel.opApply

```
Yout = shLowLevel.opApply(op, Z, t, mode)
```

Apply the month-t filter operator W_t (or W_t') to columns of Z. Yout = shLowLevel.opApply(op, Z, t) computes W_t * Z Yout = shLowLevel.opApply(op, Z, t, 'transp') computes W_t' * Z

| input | size | type | default | description |
|---|---|---|---|---|
| op | 1 x 1 | struct | required | linear operator/kernel the map is derived from |
| Z | n x 1 | double | required | coefficient vector/stack in index ordering the operator acts on |
| t | 1 x 1 | double | required | target point or epoch of the evaluation |
| mode | 1 x n | — ∈ {'forward','transp'} | 'forward' | char 'forward' (default) \| 'transp' - forward or transposed operator action |

| output | description |
|---|---|
| Yout | (P x T) double operator applied to every column of Y |

### shLowLevel.resolutionMap

```
res = shLowLevel.resolutionMap(op, t, latDeg, lonDeg, Name=Value)
```

Filter-kernel half-widths: per-point, per-azimuth resolution. RES = shLowLevel.resolutionMap(OP, T, LATDEG, LONDEG) evaluates, for each query point, the effective smoothing kernel of month T: xf(P) = y_P' * W_t * x => kernel coefficients k = W_t' * y_P and finds the great-circle distance psi_1/2 at which the kernel drops to half its peak, along NAz (8) azimuths. Replaces the single "equivalent Gaussian radius" with a spatially and azimuthally resolved resolution product (N-S vs E-W half-width ratio = stripin...

| input | size | type | default | description |
|---|---|---|---|---|
| op | 1 x 1 | struct | required | struct from shLowLevel.tvANSFilter |
| t | 1 x 1 | double | required | month index |
| latDeg | : x 1 | double | required | query latitudes [deg] |
| lonDeg | : x 1 | double | required | query longitudes [deg] |
| NAz | 1 x 1 | double | 8 | azimuth count, default 8 |
| PsiMax | 1 x 1 | double | 20 | radial search range [deg], default 20 |
| NPsi | 1 x 1 | double | 120 | radial samples, default 120 |
| RadiusKm | 1 x 1 | double | 6371 | (1,1) sphere radius used to turn the half-width from degrees into kilometres [km]. The default is the MEAN Earth radius, which is the right choice for a gre... |

| output | description |
|---|---|
| res | (1,1) struct one row per query point (nPts rows), fields psiHalfDeg (nPts x NAz) double half-width per azimuth [deg] azDeg (1 x NAz) double azimuths evaluated [deg] kmMean, kmMin, kmMax (nPts x 1) double half-width across the azimuths [km] anisotropy (nPts x 1) double kmMax ./ kmMin, the striping fingerprint (1 = isotropic kernel) |

## Basins, uncertainty & analysis tools

### shLowLevel.basinKernel

```
[b, info] = shLowLevel.basinKernel(idx, region, Name=Value)
```

Band-limited basin kernel(s) from a region definition. [B, INFO] = shLowLevel.basinKernel(IDX, REGION) builds the SH coefficient vector of the band-limited indicator function of REGION by exact Gauss-Legendre quadrature, b = Y' * (w .* mask), ready for shSeries.basinAverage / shLowLevel.basinDeconvolve (P x 1, IDX ordering). Replaces the error-prone manual Y'*(w.*mask) construction.

| input | size | type | default | description |
|---|---|---|---|---|
| idx | 1 x 1 | struct | required | index struct from shLowLevel.shIndex (fields n, m, cs, P, Lmax, minDegree, pos) |
| region | n x m | — | required | region as polygon [lat lon] deg, mask, or @(lat, lon) handle (see evalMask) |
| BufferKm | 1 x 1 | double | 0 | grow (>0) / shrink (<0) the region by great-circle distance before quadrature - counters leakage by matching the kernel to the filter's resolution |
| TaperKm | 1 x 1 | double | 0 | >0: multiply the kernel spectrally by Jekeli Gaussian weights of this radius (shLowLevel.shGaussianWeights) - soft edges reduce ringing/leakage of the trunca... |
| R | 1 x 1 | double | 6378136.3 | sphere radius [m] for the km conversions |
| OverSample | 1 x 1 | double | 2 | quadrature refinement factor: indicator masks are not band-limited; the default doubles the ring/ meridian count to tame edge quantization (~18% cap- area er... |

| output | description |
|---|---|
| b | (P,1) double kernel in IDX ordering |
| info | (1,1) struct fields areaFraction (1,1) double quadrature area of the mask / 4pi nGridPoints (1,1) double grid points the mask was evaluated on buffered (1,1) logical BufferKm was non-zero tapered (1,1) logical TaperKm was positive |

### shLowLevel.basinDeconvolve

```
[avgHat, out] = shLowLevel.basinDeconvolve(B, op, Name=Value)
```

Unbiased basin averages by exact operator deconvolution. [avgHat, out] = shLowLevel.basinDeconvolve(B, op) removes filter attenuation and inter-basin leakage from basin averages, exploiting that W_t is a KNOWN linear operator (no hydrology-model scale factors).

| input | size | type | default | description |
|---|---|---|---|---|
| B | n x m | double | required | basin-kernel coefficient vector from basinKernel |
| op | 1 x 1 | struct | required | linear operator/kernel the map is derived from |
| Ridge | 1 x 1 | double | 0 | Tikhonov parameter for ill-conditioned basin sets, RELATIVE to the spectral norm of A (v2.1): c = (A'A + lam^2 I) \ (A' v) with lam = Ridge*||A||_2. An abs... |

| output | description |
|---|---|
| avgHat | (K x T) double deconvolved basin averages, ((B'B)*c + B'model) ./ diag(B'B) |
| out | (1,1) struct fields c (K x T) double span coefficients sigma (K x T) double 1-sigma of avgHat, deterministic term included (v2.5) avgNaive (K x T) double naive averages of the filtered field (attenuated and leaky) for comparison attn (K x T) double diag(A_t) ./ diag(B'B) attenuation factors condA (T x 1) double cond(A_t) per month |

### shLowLevel.basinScaling

```
[k, info] = shLowLevel.basinScaling(op, b, tsModel, Name=Value)
```

Leakage/attenuation scaling factors by forward modelling. [K, INFO] = shLowLevel.basinScaling(OP, B, TSMODEL) estimates the classic basin scaling (gain) factor: push a MODEL series (hydrology model, mascon-derived field, synthetic pattern) through the SAME filter operator OP that processed the data, and regress true against filtered basin averages: k = sum(aTrue .* aFilt) / sum(aFilt.^2) Scaled data series: k * (b' xFiltered)/(b'b). Complements shLowLevel.basinDeconvolve: deconvolution inverts the operator on a ...

| input | size | type | default | description |
|---|---|---|---|---|
| op | n x m | — | required | operator, one of struct from shLowLevel.tvANSFilter (applied via shLowLevel.opApply) (P,P) double static matrix operator (Gaussian, DDK, fan as a matrix in... |
| b | : x 1 | double | required | basin kernel (shLowLevel.basinKernel, idx order) |
| tsModel | 1 x 1 | shSeries | required | shSeries model series matched by nearest epoch within 0.05 yr against opts.tYears ([]) / op.tYears |
| Idx | n x m | struct | struct() | ) required for matrix/handle operators (shLowLevel.shIndex); structs carry their own |
| TYears | 1 x : | double | [] | required for matrix/handle operators |
| PerMonth | 1 x 1 | logical | false | also return monthly factors aTrue./aFilt |

| output | description |
|---|---|
| k | (1,1) double regression scaling factor |
| info | (1,1) struct fields sigmaK (1,1) double 1-sigma of the regression factor kMonthly (T x 1) double per-month factors; present only with PerMonth matched (T x 1) logical which epochs matched between the two series nMatched (1,1) double nnz(matched), the epochs the regression actually used aTrue (T x 1) double basin average of the model field aFilt (T x 1) double basin average after filtering |

### shLowLevel.fitDeterministicModel

`[model, Xres, coef, A, coefSigma, resVar, ar1] = shLowLevel.fitDeterministicModel(X, tYears, Name=Value)`

Bias/trend/(semi-)annual (+extra periods) per SH coefficient. [MODEL, XRES, COEF, A, COEFSIGMA, RESVAR] = shLowLevel.fitDeterministicModel(X, TYEARS, ...) fits, for each coefficient time series (rows of X, P x T), the model x(t) = a + b*tc + c*cos(2*pi*tc) + d*sin(2*pi*tc) + e*cos(4*pi*tc) + f*sin(4*pi*tc) + sum_k [ g_k*cos(2*pi*tc/p_k) + h_k*sin(2*pi*tc/p_k) ] with tc = t - T0 (NaN) and optional extra periods p_k [years] - e.g. the GRACE tidal alias periods S2 = 161/365.25, K2 = 3.66, K1 = 7.48.

| input | size | type | default | description |
|---|---|---|---|---|
| X | n x m | double | required | coefficient stack, (P x T) in index ordering |
| tYears | : x 1 | double | required | time stamps as decimal years, (T x 1) |
| Robust | 1 x 1 | logical | false | Huber IRLS per coefficient |
| HuberK | 1 x 1 | double | 1.5 | , MaxIter (10) |
| MaxIter | 1 x 1 | double | 10 | iteration cap |
| T0 | 1 x 1 | double | NaN | ) reference epoch [decimal years] |
| Periods | 1 x : | double | double.empty(1,0) | ) ([]) extra periods p_k [years], row vector |
| Weights | : x 1 | double | [] | T x 1 per-epoch weights (e.g. 1./s from VCE); weighted LS with sqrt(w)-scaled rows |
| ARCorrect | 1 x 1 | logical | false | inflate coefSigma by sqrt((1+r1)/(1-r1)) with the Kendall bias-corrected r1' = r1 + (1+3r1)/T (v2.5) where the lag-1 autocorrelation of each residual series ... |
| Breaks | 1 x : | double | double.empty(1,0) | ) epochs [decimal years] of continuous piecewise-linear trend hinges; each adds a max(0, t - tb) column to the design matrix |

| output | description |
|---|---|
| model | P x T fitted deterministic part |
| Xres | P x T X - model |
| coef | ncol x P [bias; trend; cosA; sinA; cosSA; sinSA; cosP1; sinP1; ...], ncol = 6 + 2*numel(Periods) |
| A | T x ncol design matrix (unweighted) |
| coefSigma | ncol x P 1-sigma per coefficient: sqrt(resVar_p * diag(inv(A'WA))); OLS formula, validated by Monte Carlo (ratios 0.99-1.02). For Robust=true the final IRLS weights enter W (approximation, documented). |
| resVar | 1 x P residual variance rss/(T - ncol) |
| ar1 | 1 x P bias-corrected lag-1 residual auto- correlation actually applied (clamped to [0, 0.99]; 0 when ARCorrect=false) |

### shLowLevel.mcPropagate

```
out = shLowLevel.mcPropagate(fun, g, Name=Value)
```

Monte-Carlo uncertainty propagation through arbitrary chains. OUT = shLowLevel.mcPropagate(FUN, G) draws coefficient samples consistent with the formal errors of the shCoefficients G, pushes each through FUN, and returns empirical statistics - uncertainty propagation for ANY functional (filtered grids, basin averages, amplitude maps, ...) without deriving analytic formulas, and the standard cross-check for every analytic sigma in this toolbox.

| input | size | type | default | description |
|---|---|---|---|---|
| fun | 1 x 1 | function_handle | required | function handle y = fun(gs), gs an shCoefficients sample; y numeric (any fixed size, column-ized) |
| g | 1 x 1 | shCoefficients | required | shCoefficients with sigmas (or Cov supplied) |
| N | 1 x 1 | double | 500 | double = 500 number of samples |
| Cov | n x m | double | [] | double = [] full covariance (P x P, Idx ordering) |
| Idx | n x m | struct | struct([]) | struct = struct([]) shLowLevel.shIndex for Cov |
| Seed | n x m | double | [] | double = [] rng seed for reproducibility |
| KeepSamples | 1 x 1 | logical | false | logical = false |

| output | description |
|---|---|
| out | (1,1) struct Monte-Carlo moments of the functional, fields mean size(fun(g)) double sample mean sigma size(fun(g)) double sample 1-sigma (N-1 normalization) N (1,1) double number of samples drawn samples (K x N) double only when KeepSamples=true; K = numel(fun(g)), one column-ized draw per column |

### shLowLevel.errorMap

```
[sig, latDeg, lonDeg] = shLowLevel.errorMap(M, idx, latDeg, lonDeg, Name=Value)
```

Analytic formal-error maps from a full coefficient covariance. [SIG, LAT, LON] = shLowLevel.errorMap(M, IDX, LAT, LON, Quantity ("geoid")="ewh", kn ([])=kn) computes the pointwise 1-sigma of a synthesized field from a full P x P covariance (e.g. shLowLevel.readSINEX(..., Output="covariance").M, reordered via Index=idx).

| input | size | type | default | description |
|---|---|---|---|---|
| M | P x P | double | required | double covariance in IDX ordering (symmetric PSD) |
| idx | 1 x 1 | struct | required | struct shLowLevel.shIndex (analysis-style Nmin matching M) |
| latDeg | 1 x : | double | required | , lonDeg (1,nlon) double [deg] geocentric |
| lonDeg | 1 x : | double | required | longitudes [deg], lon in [0, 360) |
| Quantity | 1 x 1 | string | "geoid" | , kn, hn ([]), GM (3.986004415e14), R (6378136.3), |

| input | size | type | default | description |
|---|---|---|---|---|
| kn | n x m | double | [] | load Love numbers, (degree, kn) table or column vector - always user-supplied, no reference frame is assumed |
| hn | n x m | double | [] | load Love numbers h_n, same layout as kn |
| GM | 1 x 1 | double | 3.986004415e14 | gravitational constant times Earth mass [m^3/s^2] (default 3.986004415e14, overridable) |
| R | 1 x 1 | double | 6378136.3 | reference radius [m] (default 6378136.3, overridable) |
| Height | 1 x 1 | double | 0 | - as in shLowLevel.kernelFactors |

| output | description |
|---|---|
| sig | (nlat,nlon) double 1-sigma of the field [unit of quantity] |
| latDeg | (1,nlat) double latitude grid used [deg], as passed in |
| lonDeg | (1,nlon) double longitude grid used [deg], as passed in |

### shLowLevel.combineCenters

```
[tsComb, info] = shLowLevel.combineCenters(tsCell, Name=Value)
```

VCE-weighted combination of multi-center monthly series. [TSCOMB, INFO] = shLowLevel.combineCenters({tsITSG, tsCSR, tsGFZ}) combines monthly solutions from several processing centers into one series with one variance factor per (center, month), estimated by Foerstner variance-component iteration with PARTIAL REDUNDANCIES:

| input | size | type | default | description |
|---|---|---|---|---|
| tsCell | 1 x : | cell | required | 1 x C cell of shSeries, equal nmax, >= 2 centers |
| Tolerance | 1 x 1 | double | 0.05 | [yr] epoch matching (centers date mid-months slightly differently) |
| AllowMissing | 1 x 1 | logical | true | months missing at some centers are combined from the available ones (weights renormalize); false: only fully-covered months are kept |
| Robust | 1 x 1 | logical | false | per-month factor = median over blocks of the unbiased block estimates q_b/r_b instead of the global ratio (outlier-month protection; Python-validated design) |
| MaxIter | 1 x 1 | double | 6 | , Tol (1e-3) VCE iteration control |
| Tol | 1 x 1 | double | 1e-3 | convergence tolerance |

| output | description |
|---|---|
| tsComb | shSeries combined stacks on the reference epochs (first center), sigmaCs/Ss = posterior sqrt(diag(H)) per month, provenance in history |
| info | (1,1) struct fields s2 (C x T) double variance factors per centre and month, NaN where that centre has no solution weights (C x T) double normalized 1/s2 redundancy (C x T) double partial redundancy per centre interCenterCorr (C x C) double empirical residual correlation - the honesty diagnostic: VCE assumes independence, and GRACE centres share data and background models nIter (1,1) double VCE iterations used epochs (T x 1) double the combined epoch grid centers (C x 1) string centre labels, in the row order of s2/weights/redundancy note (1,1) string what the combination actually did |

### shLowLevel.eofAnalysis

```
[modes, pcs, varExplained, info] = shLowLevel.eofAnalysis(ts, Name=Value)
```

Empirical orthogonal functions of a coefficient series. [MODES, PCS, VAREXPLAINED, INFO] = shLowLevel.eofAnalysis(TS) decomposes the (typically residual) series into spatial modes and principal components via SVD of the flattened, time-centered coefficient stack: X = U S V'; MODES{k} carries U(:,k)*S(k,k)/sqrt(T-1) as an shCoefficients (synthesize for the spatial pattern), PCS(:,k) = sqrt(T-1)*V(:,k) is the unit-variance temporal amplitude, so mode-k field * pc reconstructs the mode's contribution exactly.

| input | size | type | default | description |
|---|---|---|---|---|
| ts | 1 x 1 | shSeries | required | shSeries (NaN-free; use dropNaN across the gap) |
| NModes | 1 x 1 | double | 6 | modes returned |
| Center | 1 x 1 | logical | true | remove the temporal mean first |

| output | description |
|---|---|
| modes | 1 x K cell of shCoefficients (spatial patterns) |
| pcs | (T,K) double, unit variance columns |
| varExplained | (K,1) double, fraction in [0,1] |
| info | (1,1) struct fields singularValues (K x 1) double singular values of the centred data matrix totalVariance (1,1) double total variance before truncation T (1,1) double epochs used centered (1,1) logical whether the temporal mean was removed (= the Center option) |

### shLowLevel.slepianBasis

```
[G, lam, info] = shLowLevel.slepianBasis(idx, region, Name=Value)
```

Spatially concentrated (Slepian) basis for a region. [G, LAM, INFO] = shLowLevel.slepianBasis(IDX, REGION) solves the spherical concentration problem of Simons et al. (2006): find band-limited functions g = Y*x maximizing the energy fraction inside REGION, lambda = (x' K x) / (x' x), K = Y' diag(w .* mask) Y, by symmetric eigendecomposition of the localization kernel K (exact Gauss-Legendre quadrature). Columns of G are the Slepian taper coefficient vectors, orthonormal (G'G = I), sorted by concentration LAM (...

| input | size | type | default | description |
|---|---|---|---|---|
| idx | 1 x 1 | struct | required | struct shLowLevel.shIndex (P x P eig: fine to Lmax ~ 60-96) |
| region | n x m | — | required | handle \| polygon \| mask (see shLowLevel.evalMask) |
| NKeep | n x 1 | double | [] | ) columns of G to return; [] = all P |
| BufferKm | 1 x 1 | double | 0 | , R (6378136.3) passed to shLowLevel.evalMask |
| R | 1 x 1 | double | 6378136.3 | reference radius [m] (default 6378136.3, overridable) |
| OverSample | 1 x 1 | double | 2 | refinement factor of the quadrature grid the mask is evaluated on; the Gauss rule uses OverSample*(Lmax+1) latitude rings, so a higher value resolves the reg... |

| output | description |
|---|---|
| G | (P,NKeep) double taper coefficient vectors, G'G = I |
| lam | (NKeep,1) double concentrations, descending in [0,1] |
| info | (1,1) struct fields shannon (1,1) double the Shannon number, areaFraction*P, exact by the addition theorem areaFraction (1,1) double the region's area / 4pi lamAll (P x 1) double every concentration eigenvalue, not only the retained ones |

### shLowLevel.seaLevelFingerprint

```
[S, grid, info] = shLowLevel.seaLevelFingerprint(loadRegion, ocean, idx, Name=Value)
```

Elastic sea-level fingerprint (sea-level equation). [S, GRID, INFO] = shLowLevel.seaLevelFingerprint(LOADREGION, OCEAN, IDX, kn (“”)=kn, hn (“”)=hn) solves the elastic sea-level equation for a land load: the ocean does NOT rise uniformly - it follows the perturbed geoid minus the deformed crust under global mass conservation (Farrell & Clark 1976, elastic limit):

| input | size | type | default | description |
|---|---|---|---|---|
| loadRegion | n x m | — | required | handle f(latDeg,lonDeg) -> surface density CHANGE [kg/m^2] (negative = mass loss), polygon (Kx2 with opts.LoadValue (1)), or Ngrid x 1 values on the solver grid |
| ocean | n x m | — | required | ocean region: handle \| polygon \| mask (shLowLevel.evalMask forms). The load is restricted to land (1 - ocean). |
| idx | 1 x 1 | struct | required | shLowLevel.shIndex(Lmax, Nmin = 0) - Nmin MUST be 0 (mass conservation lives in degree 0/1) kn, hn (REQUIRED) load Love numbers k'_n, h'_n (user-supplied) |
| kn | n x m | double | required | load Love numbers, (degree, kn) table or column vector - always user-supplied, no reference frame is assumed |
| hn | n x m | double | required | load Love numbers h_n, same layout as kn |
| LoadValue | 1 x 1 | double | 1 | kg/m^2 for polygon loads |
| rho_water | 1 x 1 | double | 1000 | , rho_ave (5517), R (6378136.3) |
| rho_ave | 1 x 1 | double | 5517 | mean Earth density [kg/m^3] (default 5517, overridable) |
| R | 1 x 1 | double | 6378136.3 | reference radius [m] (default 6378136.3, overridable) |
| OverSample | 1 x 1 | double | 2 | quadrature refinement (mask fidelity) |
| MaxIter | 1 x 1 | double | 50 | maximum sea-level-equation iterations |
| Tol | 1 x 1 | double | 1e-8 | convergence threshold on max\|dS\|, RELATIVE to \|eustatic\| (the absolute bound is Tol * \|eustatic\|) |

| output | description |
|---|---|
| S | (Ngrid,1) double relative sea-level change over the ocean [m per unit load], 0 on land |
| grid | (1,1) struct the solver grid, as returned by shLowLevel.synthesisMatrix (latDeg/lonDeg rings) |
| info | (1,1) struct fields iterations (1,1) double sweeps of the sea-level equation actually run dS (1,1) double last change in the solution converged (1,1) logical dS fell below the tolerance massResidual (1,1) double [kg/m^2 sphere mean] - the conservation check eustatic (1,1) double uniform equivalent rise [m] kappa (1,1) double the rotational feedback factor applied areaFractionOcean (1,1) double ocean area / sphere area S2D (nlat x nlon) double the sea-level change field |

### shLowLevel.pctile

```
v = shLowLevel.pctile(x, p)
```

Percentile without the Statistics Toolbox (base MATLAB only). V = shLowLevel.pctile(X, P) returns the P-th percentile(s) (0..100) of the finite values of X(:), using the same linear interpolation between order statistics as the Statistics Toolbox prctile: sample k maps to percentile 100*(k-0.5)/n. P may be a vector (v2.5.1); V has the same size as P. NaN/Inf are ignored; empty input yields NaN(size(P)).

| input | size | type | default | description |
|---|---|---|---|---|
| x | n x m | double | required | double, any shape |
| p | 1 x : | double | required | double in [0, 100], scalar or vector |

| output | description |
|---|---|
| v | (size(p)) double percentile value per requested p |

### Comparison & validation metrics (v2.6.0)

### shLowLevel.compareSolutions

```
[rep, h] = shLowLevel.compareSolutions(g1, g2, Name=Value)
```

Full spectral + spatial comparison of two solutions. REP = shLowLevel.compareSolutions(G1, G2) compares two shCoefficients on a COMMON BASIS - both are truncated to the smaller nmax and G2 is rescaled to G1's GM/R if they differ (otherwise the comparison measures processing chains, not solutions) - and returns the standard metric set: spectral: difference degree amplitude vs both signal spectra, per-degree correlation, crossover degree of agreement (shLowLevel.diffSpectrum) spatial: area-weighted bias/RMSD/cent...

| input | size | type | default | description |
|---|---|---|---|---|
| g1 | 1 x 1 | shCoefficients | required | first coefficient set |
| g2 | 1 x 1 | shCoefficients | required | second coefficient set |
| LatDeg | 1 x : | double | -89:2:89 | , LonDeg (0:2:358) comparison grid |
| LonDeg | 1 x : | double | 0:2:358 | longitudes [deg], lon in [0, 360) |
| Quantity | 1 x 1 | string | "geoid" | synthesis quantity; "ewh" etc. need kn ([]) |
| kn | n x m | double | [] | load Love numbers (user-supplied, 0..nmax) |
| Mask | n x m | — | [] | nlat x nlon logical region restriction |
| Names | 1 x 2 | string | ["solution 1", "solution 2"] | (["solution 1","solution 2"]) |
| Plot | 1 x 1 | logical | false | produce the diagnostic figure |

| output | description |
|---|---|
| rep | (1,1) struct fields: .spectral (1,1) struct shLowLevel.diffSpectrum output .spatial (1,1) struct shLowLevel.spatialStats output .chi2dof (1,1) double error-realism statistic (NaN when either solution lacks sigmas) .nmax (1,1) double common maximum degree used .quantity (1,1) string compared quantity .names (1,2) string solution labels .rescaled (1,1) logical true when GM/R were unified |
| h | (1,1) graphics handle figure handle (Plot = true only) |

*Example*

```
rep = shLowLevel.compareSolutions(g, g.gaussian(350), Names = ["raw", "G350"]);
fprintf("crossover n = %d, spatial corr = %.3f\n", ...
    rep.spectral.ncross, rep.spatial.corr)
```

### shLowLevel.compareSeries

`[rep, h] = shLowLevel.compareSeries(tsList, Name=Value)`

Temporal + component comparison of N solution series. REP = shLowLevel.compareSeries({TS1, TS2, ...}) compares N >= 2 shSeries against the FIRST (the reference) on a COMMON BASIS: epochs are matched to the reference within MatchTolerance (0.02) (unmatched epochs are dropped and reported - never interpolated across gaps), all series are truncated to the smallest nmax, and GM/R are unified. Per solution k >= 2 the standard temporal metric set is computed on a scalar comparison series y_k(t) - a basin average when ...

| input | size | type | default | description |
|---|---|---|---|---|
| tsList | 1 x : | cell | required | cell of shSeries the series to compare; the FIRST is the reference |
| Names | 1 x 1 | string | strings(1, 0) | ) ([]) string array, one per series |
| MatchTolerance | 1 x 1 | double | 0.02 | epoch matching tolerance [yr] |
| Basin | n x 1 | double | [] | , Idx ([]) P x 1 basin vector + shLowLevel.shIndex struct: y = Basin' * coefficient vector per epoch |
| Idx | n x m | — | [] | index struct from shLowLevel.shIndex (fields n, m, cs, P, Lmax, minDegree, pos) |
| LatDeg | 1 x : | double | -88:4:88 | , LonDeg (0:6:354), Quantity ("geoid"), kn ([]), |
| LonDeg | 1 x : | double | 0:6:354 | longitudes [deg], lon in [0, 360) |
| Quantity | 1 x 1 | string | "geoid" | functional of the field, see shLowLevel.kernelFactors |

| input | size | type | default | description |
|---|---|---|---|---|
| kn | n x m | double | [] | load Love numbers, (degree, kn) table or column vector - always user-supplied, no reference frame is assumed |
| Mask | n x m | — | [] | synthesis path when Basin is not given |
| Plot | 1 x 1 | logical | false | produce the diagnostic figure |

| output | description |
|---|---|
| rep | (1,1) struct fields: .names (1,N) string .epochs (T,1) double common epochs used .nDropped (1,N) double reference epochs unmatched per series .y (T,N) double comparison series (col 1 = reference) .nse (1,N) double NSE vs reference (NaN for col 1) .corr (1,N) struct-array-free: Pearson r per series .pcorr (1,N) double AR(1)-corrected two-sided p .neff (1,N) double effective sample sizes .trendDiff (1,N) double trend - reference trend [units/yr] .trendSigma (1,N) double combined 1-sigma of trendDiff .trendZ (1,N) double significance z of trendDiff .ampRatio (1,N) double annual amplitude / reference .phaseLagDays (1,N) double annual phase lag vs reference .chi2dof (1,N) double error realism vs reference .trendDiffFields (1,N) cell shCoefficients (empty for col 1) .tch (1,1) struct shLowLevel.threeCorneredHat output (N >= 3) .nmax (1,1) double common maximum degree |
| h | (1,1) graphics handle figure handle (Plot = true only) |

*Example*

```
rep = shLowLevel.compareSeries({tsCSR, tsGFZ, tsJPL}, ...
    Names = ["CSR", "GFZ", "JPL"]);
fprintf("TCH noise: %s\n", sprintf("%.3g ", rep.tch.sigma))
```

### shLowLevel.diffSpectrum

`spec = shLowLevel.diffSpectrum(C1, S1, C2, S2)`

Spectral comparison of two coefficient sets. SPEC = shLowLevel.diffSpectrum(C1, S1, C2, S2) compares two solutions in the spectral domain: the difference degree amplitude localizes the disagreement in wavelength, the degree correlation separates "same pattern, different amplitude" from genuine disagreement, and the crossover of the difference against the signal spectrum gives the effective resolution of agreement as a single scalar. Inputs must share the same size; truncate beforehand for mixed degrees (the shLo...

| input | size | type | default | description |
|---|---|---|---|---|
| C1 | n x m | double | required | cosine Stokes coefficients, C(n+1, m+1) indexing |
| S1 | n x m | double | required | sine Stokes coefficients, S(n+1, m+1) indexing |
| C2 | n x m | double | required | second cosine set, same indexing as C |
| S2 | n x m | double | required | second sine set, same indexing as S |

| output | description |
|---|---|
| spec | (1,1) struct fields: .n (nmax+1 x 1) double degree axis 0..nmax .diffAmp (nmax+1 x 1) double degree amplitude of (1)-(2) .amp1 (nmax+1 x 1) double signal degree amplitude of (1) .amp2 (nmax+1 x 1) double signal degree amplitude of (2) .degCorr (nmax+1 x 1) double per-degree correlation (NaN where a degree is all-zero) .ncross (1,1) double first degree where diffAmp exceeds amp1 (NaN if never; agreement to nmax) |

*Example*

```
gF = g.gaussian(350);
spec = shLowLevel.diffSpectrum(g.C, g.S, gF.C, gF.S);
semilogy(spec.n, [spec.amp1, spec.diffAmp]); grid on
```

### shLowLevel.spatialStats

`stats = shLowLevel.spatialStats(A, B, latDeg, lonDeg, Name=Value)`

Area-weighted spatial comparison of two grids. STATS = shLowLevel.spatialStats(A, B, LATDEG, LONDEG) compares two nlat x nlon grids with proper cos(latitude) area weighting - unweighted RMS on a lat/lon grid over-counts the poles and is simply wrong. Returns bias, RMSD, the CENTERED pattern statistics (centered RMSD, weighted correlation, standard deviations) that feed a Taylor diagram, and verifies the Taylor identity

crmsd^2 = stdA^2 + stdB^2 - 2 stdA stdB corr internally.

| input | size | type | default | description |
|---|---|---|---|---|
| A | n x m | double | required | semi-major axis of the normal ellipsoid [m] |
| B | n x m | double | required | basin-kernel coefficient vector from basinKernel |
| latDeg | 1 x : | double | required | geocentric latitudes [deg] |
| lonDeg | 1 x : | double | required | longitudes [deg], lon in [0, 360) |
| Mask | n x m | — | [] | nlat x nlon logical: restrict to a region (e.g. land/ocean); default all points |
| Weights | n x m | double | [] | custom nlat x nlon weights (e.g. quadrature); default cos(latDeg) rings, normalized over the mask |

| output | description |
|---|---|
| stats | (1,1) struct fields: .bias (1,1) double weighted mean of A - B .rmsd (1,1) double weighted RMS of A - B .crmsd (1,1) double centered (bias-free) RMSD .corr (1,1) double weighted pattern correlation .stdA (1,1) double weighted std of A .stdB (1,1) double weighted std of B .ratio (1,1) double stdB / stdA .nUsed (1,1) double number of grid points used |

*Example*

```
A1 = g.synthesis(lat, lon); A2 = gF.synthesis(lat, lon);
st = shLowLevel.spatialStats(A1, A2, lat, lon);
fprintf("rmsd %.3g, corr %.3f\n", st.rmsd, st.corr)
```

### shLowLevel.nashSutcliffe

`nse = shLowLevel.nashSutcliffe(ref, model)`

Nash-Sutcliffe efficiency of a model against a reference. NSE = shLowLevel.nashSutcliffe(REF, MODEL) computes the hydrology-standard skill score NSE = 1 - sum((model-ref).^2) / sum((ref-mean(ref)).^2) column-wise. NSE = 1 is perfect, 0 means "no better than the reference mean", negative is worse than the mean. More informative than correlation because bias and amplitude errors are punished. NaN pairs are removed per column.

| input | size | type | default | description |
|---|---|---|---|---|
| ref | n x m | double | required | reference series |
| model | n x m | double | required | model series scored against the reference |

| output | description |
|---|---|
| nse | (1,K) double efficiency per column (K = 1 for vectors) |

*Example*

```
nse = shLowLevel.nashSutcliffe(avg(1,:), avg(2,:));   % e.g. SH vs mascon basin
```

### shLowLevel.effectiveCorr

`out = shLowLevel.effectiveCorr(x, y)`

Correlation with AR(1)-corrected significance. OUT = shLowLevel.effectiveCorr(X, Y) computes the Pearson correlation of two time series together with a significance test that accounts for serial correlation: monthly GRACE series are strongly autocorrelated, so the effective sample size Neff = T*(1-r1*r2)/(1+r1*r2) (r1, r2 = lag-1 autocorrelations) is well below T, and the naive t-test badly over-states significance (Monte-Carlo: ~30% false positives at phi = 0.7 vs ~6% corrected; validated in Python). The two-si...

| input | size | type | default | description |
|---|---|---|---|---|
| x | : x 1 | double | required | coefficient stack, (P x T) in index ordering |
| y | : x 1 | double | required | series the lag-1 autocorrelation is estimated from |

| output | description |
|---|---|
| out | (1,1) struct fields: .r (1,1) double Pearson correlation .neff (1,1) double effective sample size (clamped [4, T]) .t (1,1) double t statistic on neff-2 dof .p (1,1) double two-sided p-value .n (1,1) double finite pairs used |

*Example*

```
ec = shLowLevel.effectiveCorr(avg(1,:)', avg(2,:)');   % two basin series
fprintf("r = %.3f (p = %.3g, Neff = %.0f)\n", ec.r, ec.p, ec.neff)
```

### shLowLevel.threeCorneredHat

`out = shLowLevel.threeCorneredHat(X)`

Per-solution noise levels from N >= 3 series. OUT = shLowLevel.threeCorneredHat(X) with X (T x N, N >= 3) estimates the individual noise standard deviation of each series by the classic (generalized) three-cornered hat: pairwise difference variances V(i,j) = var(x_i - x_j) cancel the common signal, and under the assumption of mutually independent noises V(i,j) = s_i^2 + s_j^2; the N unknowns are solved from the N*(N-1)/2 pairs by least squares (exact Grubbs solution for N = 3). This answers the question pairwise...

| input | size | type | default | description |
|---|---|---|---|---|
| X | : x : | double | required | coefficient stack, (P x T) in index ordering |

| output | description |
|---|---|
| out | (1,1) struct fields: .sigma (1,N) double noise standard deviation per series .variance (1,N) double raw LS variance estimates (may be < 0) .pairVar (P,3) double [i j var(x_i - x_j)] per pair .clipped (1,N) logical true where variance was clipped to 0 |

*Example*

```
out = shLowLevel.threeCorneredHat(Y3);   % T x 3 basin series (CSR/GFZ/JPL)
fprintf("noise levels: %s\n", sprintf("%.3g ", out.sigma))
```

### shLowLevel.taylorDiagram

`h = shLowLevel.taylorDiagram(refStd, stds, corrs, Name=Value)`

Taylor diagram (std / correlation / centered RMSD). H = shLowLevel.taylorDiagram(REFSTD, STDS, CORRS) draws the standard one-figure comparison of solutions against a reference: radius is the (weighted) standard deviation, the azimuth is acos(correlation), and by the law of cosines the distance to the reference point equals the centered RMSD - so pattern amplitude, pattern correlation and pattern error are read off a single plot. Feed it the .stdA/.stdB/ .corr fields of shLowLevel.spatialStats, or temporal statis...

| input | size | type | default | description |
|---|---|---|---|---|
| refStd | 1 x 1 | double | required | standard deviation of the reference field |
| stds | 1 x : | double | required | standard deviations of the compared fields |
| corrs | 1 x : | double | required | correlations of the compared fields with the reference |
| Labels | 1 x 1 | string | strings(1, 0) | ) ([]) string array, one per point |
| Title | 1 x 1 | string | "Taylor diagram" | figure title |
| Normalize | 1 x 1 | logical | false | divide all stds by REFSTD (radius 1 = reference) |

| output | description |
|---|---|
| h | (1,1) graphics handle the axes handle |

*Example*

```
st = shLowLevel.spatialStats(grid, 0.8 * grid, lat, lon);
shLowLevel.taylorDiagram(st.stdA, st.stdB, st.corr, Labels = "damped");
```

## Plotting

### shLowLevel.plotSHSpectrum

```
h = shLowLevel.plotSHSpectrum(specs, Name=Value)
```

Log-log degree-amplitude spectrum plot. PLOTSHSPECTRUM(SPEC) plots SPEC.degree vs SPEC.degAmplitude (from SHDEGREERMS) on a log-log scale, in mm geoid-height-equivalent.

| input | size | type | default | description |
|---|---|---|---|---|
| specs | n x m | — | required | struct or struct array spectrum struct(s) from shLowLevel.shDegreeRMS (overlaid when several) |
| Names | n x m | — | {} | cell array of legend labels |
| Units | n x m | — | 'mm' | 'mm' or 'm' for the y-axis |
| Field | n x m | — | '' | LEGACY override: plot this spec field directly and ignore Quantity. Empty means "not supplied" - the sentinel that replaces inputParser's UsingDefaults. (als... |
| Ax | n x m | — | [] | axes handle to plot into; empty uses the current axes |
| Kaula | n x m | — | [] | scalar K: overlay the Kaula-rule reference degree amplitude R*sqrt(2n+1)*K/n^2 (typical K ~ 1e-5) as a dashed grey line - the sanity yardstick for signal decay |
| MarkCrossover | false | — | false | with exactly TWO spectra, mark the signal/error crossover degree found by shLowLevel.shSpectralCrossover with a vertical line |
| Quantity | n x m | — | 'amplitude' | "amplitude" ([m]) \| "rms" \| "variance" \| "cumamplitude" \| "cumrms" \| "cumvariance" - which spectral quantity to draw; error overlays ride along when the spec... |

| output | description |
|---|---|
| h | (1 x 1) graphics handle log-log spectrum axes |

### shLowLevel.plotSHCoeffTriangle

```
h = shLowLevel.plotSHCoeffTriangle(C, S, Name=Value)
```

Coefficient-triangle diagnostic plot. PLOTSHCOEFFTRIANGLE(C, S) shows |Cnm| (left half) and |Snm| (right half) as a single degree/order "butterfly" triangle image on a log color scale -- the standard first diagnostic for spotting striping, leakage, or bad coefficients before spectral/spatial analysis. x-axis: order m (mirrored: Snm on the LEFT, Cnm on the RIGHT), y-axis: degree n with the apex (n=0) at the TOP - the triangle's peak points upwards. The data aspect ratio is fixed to 1:1, so the axes box is 2:1 (x ...

| input | size | type | default | description |
|---|---|---|---|---|
| C | n x m | double | required | cosine Stokes coefficients, C(n+1, m+1) indexing |
| S | n x m | double | required | sine Stokes coefficients, S(n+1, m+1) indexing |
| Nmax | n x m | — | size(C,1)-1 | -1) max degree to display |
| CLim | n x m | — | [] | [min max] log10(\|value\|) colour limits; empty is auto |
| Ax | n x m | — | [] | axes handle to plot into; empty uses the current axes |
| RefC | n x m | — | [] | , RefS ([]) reference coefficients: plot the SIGNED difference C-RefC / S-RefS on a linear divergent scale, symmetric about zero - the before/after diagnost... |

| input | size | type | default | description |
|---|---|---|---|---|
| RefS | n x m | — | [] | — |

| output | description |
|---|---|
| h | (1 x 1) graphics handle butterfly triangle image (abs log10 or signed diff with RefC/RefS) |

### shLowLevel.plotBasinSeries

```
h = shLowLevel.plotBasinSeries(tYears, c, sigma, Name=Value)
```

Basin time series with 1-sigma band, gaps, trend. H = shLowLevel.plotBasinSeries(TYEARS, C, SIGMA) draws the standard basin- average figure: shaded 1-sigma band (fill), data line, the GRACE <-> GRACE-FO gap (or any gap > GapThreshold (0.3) years) hatched grey, and an annotated trend fitted with the toolbox climatology design (bias/ trend/annual/semi-annual, AR(1)-corrected sigma).

| input | size | type | default | description |
|---|---|---|---|---|
| tYears | : x 1 | double | required | double epochs [decimal years] |
| c | : x 1 | double | required | double basin averages (e.g. out.c from shLowLevel.basinDeconvolve) |
| sigma | T x 1 | double | [] | double 1-sigma; [] for no band |
| Label | 1 x 1 | string | "basin average" | , Units (""), GapThreshold (0.3) [yr], |
| Units | 1 x 1 | string | "" | unit label used for annotation |
| GapThreshold | 1 x 1 | double | 0.3 | gap length that breaks the plotted line [yr] |
| Trend | 1 x 1 | logical | true | fit + annotate trend +/- sigma (AR(1)-corrected), |
| Ax | n x m | — | [] | target axes handle ([] creates a new figure) |

| output | description |
|---|---|
| h | (1,1) graphics handle axes of the basin series plot |

### shLowLevel.plotCovariance

```
h = shLowLevel.plotCovariance(M, idx, Name=Value)
```

Covariance/correlation image with order-block boundaries. H = shLowLevel.plotCovariance(M, IDX) shows a P x P covariance (e.g. from shLowLevel.readSINEX with Index=IDX) as an image in shIndex ordering, with the (order, C/S) block boundaries drawn - the one-look check of how good the block-diagonal approximation (tvANS block path, DDK structure) is for THIS matrix.

| input | size | type | default | description |
|---|---|---|---|---|
| M | P x P | double | required | double covariance in IDX ordering |
| idx | 1 x 1 | struct | required | struct shLowLevel.shIndex |
| Type | 1 x 1 | string ∈ ["correlation", "covariance"] | "correlation" | "correlation" (normalized, [-1,1], divergent scale) \| "covariance" (log10\|value\|) |
| BlockLines | 1 x 1 | logical | true | draw order/cs boundaries |
| Ax | n x m | — | [] | target axes handle ([] creates a new figure) |

| output | description |
|---|---|
| h | (1 x 1) graphics handle image of the covariance/correlation structure |

## Utilities

### shLowLevel.antarcticaChain

```
[gt, rep] = shLowLevel.antarcticaChain(folder, gravisFolder, Name=Value)
```

The validated GravIS Antarctica ice-trend chain. [GT, REP] = shLowLevel.antarcticaChain(FOLDER, GRAVISFOLDER, kn=KN) runs the same chain as greenlandChain with the 25 GravIS AIS drainage-basin polygons as the mask (guide chapter V7). With the tested defaults and COST-G RL02.1 this returns -125.7 Gt/yr against the

AWI joint-inversion -146.9 - read V7 before interpreting the 14%: it is the documented distance between map-based estimators and the Sasgen forward-modelling class (the official TU Dresden kernel grid s...

| input | size | type | default | description |
|---|---|---|---|---|
| folder | 1 x 1 | string | required | string monthly GSM-2_*.gfc folder |
| gravisFolder | 1 x 1 | string | "" | string GravIS aux folder; "" (default) uses the shipped data/gravis copies (see gravisL2B) |
| kn | n x m | double | [] | load Love numbers (degree,kn) - REQUIRED, the empty default errors: no frame is assumed |
| BasinFile | 1 x 1 | string | "basins_AIS.json" | GravIS AIS GeoJSON (https://gravis.gfz.de/basins/AIS; [lon lat] rings, converted and dateline-recentred internally) |
| NeighbourBoxes | n x m | double | [] | Antarctica has no land neighbours |
| GIAFile | 1 x 1 | string | "GRAVIS-2B_COSTG_0200_GIA_ICE-6G_D_VM5a_0001.gz" | ("GRAVIS-2B_COSTG_0200_GIA_ICE-6G_D_VM5a_0001.gz") |
| Filter | 1 x 1 | string | "gauss445" | as in greenlandChain |
| SpanEnd | 1 x 1 | double | 2023.099 | keep epochs <= SpanEnd [decimal yr] |
| GridStep | 1 x 1 | double | 1 | synthesis graticule step [deg] |
| NoiseLevel | 1 x 1 | double | NaN | sigma_trend policy, see greenlandChain |
| OceanMask | n x m | — | [] | REQUIRED unless NoiseLevel is numeric; must be false over land (see greenlandChain) |
| MaxIter | 1 x 1 | double | 400 | , TrendGrid ([]) |
| TrendGrid | n x m | — | [] | overlay the fitted trend line |
| GM | 1 x 1 | double | 3.986004415e14 | , R (6378136.3) |
| R | 1 x 1 | double | 6378136.3 | reference radius [m] (default 6378136.3, overridable) |
| Quiet | 1 x 1 | logical | false | suppress progress output |

| output | description |
|---|---|
| gt | (1,1) double total AIS trend over the basin union [Gt/yr] |
| rep | (1,1) struct as returned by shLowLevel.gravisRegionChain - the same fields as greenlandChain, with basins filled: a table of name and gt integrated per GravIS polygon |

*Example*

```
    [gt, rep] = shLowLevel.antarcticaChain("E:/series/COSTG", ...
        "E:/GravIS", kn = kn, OceanMask = @(la,lo) abs(la) < 60);
    rep.basins
```

### shLowLevel.buildCondFun

`[f, info] = shLowLevel.buildCondFun(idx, Name=Value)`

Covariance conditioner in the EWH domain (Kvas 2019, 2.4). [F, INFO] = shLowLevel.buildCondFun(IDX, kn=KN) builds the CondFun handle for shLowLevel.estimateVAR that implements the covariance conditioning of Kvas (2019), Sec. 2.4: the spectral covariance is propagated to equivalent-water-height values on an exact Gauss-Legendre grid, conditioned there, and propagated back: Sigma~ = Fq * ( Z .* T .* (G Sigma G') ) * Fq' with G = Y * D spectral -> EWH grid (D the per-degree EWH kernel from shLowLevel.kern...

| input | size | type | default | description |
|---|---|---|---|---|
| idx | 1 x 1 | struct | required | struct from shLowLevel.shIndex - fixes Lmax and the coefficient ordering the conditioner acts on |

| input | size | type | default | description |
|---|---|---|---|---|
| kn | required | double | required | (>= Lmax+1 x 1) double load Love numbers k'_n, kn(1) is degree 0 (kernelFactors contract) |
| Psi0Km | 1 x 1 | double | Inf | (1 x 1) double taper scale psi0 [km] of exp(-psi/psi0); Inf disables the taper. Kvas' choice corresponds to a half-width of ~1100 km |
| Regions | n x 1 | — | [] | region assignment for the block mask: a function handle id = fun(latDeg, lonDeg) (array-valued, numeric ids; e.g. @(la, lo) la >= 0) or an (M x 1) numeric ve... |
| GM | 1 x 1 | double | 3.986004415e14 | (1 x 1) double [m^3/s^2] (overridable) |
| R | 1 x 1 | double | 6378136.3 | (1 x 1) double [m] reference radius; also converts Psi0Km to spherical distance (geocentric sphere) |
| NLat | 1 x 1 | double | [] | (1 x 1) double Gauss-Legendre rings (default Lmax+1, the exactness minimum) |
| NLon | 1 x 1 | double | [] | (1 x 1) double longitudes (default 2*Lmax+2) |

| output | description |
|---|---|
| f | function handle Sigma~ = f(Sigma) for (P x P) covariances in IDX ordering - pass as estimateVAR(..., CondFun=f) |
| info | (1,1) struct fields grid (1,1) struct the synthesisMatrix grid struct M (1,1) double number of grid points region (M x 1) double or [] the region weight, empty when none was given W (M x M) double the elementwise spatial weight Z .* T memGB (1,1) double storage of the precomputed operators |

*Example*

```
   kn  = shLowLevel.readLoveNumbers(lovePath);
   idx = shLowLevel.shIndex(40);
   land = @(la, lo) double(landMaskFun(la, lo));   % your land/ocean rule
   cf  = shLowLevel.buildCondFun(idx, kn=kn, Psi0Km=1100, Regions=land);
   model = shLowLevel.estimateVAR(X, Order=1, CondFun=cf);

Memory/cost: precomputes G, Fq (M x P each) and W (M x M); Lmax 40
gives M = 3362, ~180 MB total; every f(Sigma) costs two M x P x P
products. Numerics pre-validated in Python
(tools/dev/validate_condfun.py).

Reference: Kvas, TU Graz PhD thesis (2019), Sec. 2.4, eqs.
(2.117)-(2.120); Schur product theorem e.g. Horn & Johnson.
```

### shLowLevel.chi2Quantile

`x = shLowLevel.chi2Quantile(p, k)`

Chi-square quantile without toolboxes (Wilson-Hilferty). X = shLowLevel.chi2Quantile(P, K) approximates the chi-square quantile chi2inv(P, K) using the Wilson-Hilferty cube transform x = k * (1 - 2/(9k) + z * sqrt(2/(9k)))^3, z = sqrt(2) * erfinv(2p - 1) in base MATLAB (erfinv is core; chi2inv is Statistics Toolbox - same reason shLowLevel.pctile exists instead of prctile).

| input | size | type | default | description |
|---|---|---|---|---|
| p | any size | double | required | double probability level(s) in (0, 1) |
| k | 1 x 1 | double | required | double degrees of freedom, k >= 1 |

| output | description |
|---|---|
| x | (size of p) double approximate chi-square quantile(s) |

*Example*

```
    thr = shLowLevel.chi2Quantile(1 - 1e-3, 1677);   % QC threshold

  Reference: Wilson & Hilferty (1931), PNAS 17.
```

### shLowLevel.designFilter

`[W, info] = shLowLevel.designFilter(SigmaC, SigmaS, Name=Value)`

Build a DDK-class anisotropic filter from user covariances. W = shLowLevel.designFilter(SIGMAC, SIGMAS) constructs the Wiener- type decorrelation filter W = (N + Alpha * inv(S))^-1 * N per order/parity block - the same construction as the published DDK filters, but matched to YOUR solution's error information instead of a fixed release. N is the normal (inverse noise covariance) matrix built from the formal sigmas (diagonal) or from a full covariance (Noise=); S is the signal covariance from a Kaula-type degree ...

| input | size | type | default | description |
|---|---|---|---|---|
| SigmaC | nmax+1 x nmax+1 | double | required | double formal sigmas of the cosine coefficients, C(n+1, m+1) layout (e.g. g.sigmaC) |
| SigmaS | nmax+1 x nmax+1 | double | required | double same for the sine coefficients |
| Alpha | 1 x 1 | double | 1 | regularization weight: larger = stronger smoothing (DDK1-like), smaller = weaker (DDK7+) |
| Kaula | 1 x 1 | double | 1e-5 | signal RMS per coefficient K / n^2 (ignored when DegVar or Signal is given) |
| DegVar | n x m | double | [] | (nmax+1 x 1) double per-coefficient signal VARIANCE per degree (overrides Kaula) |
| Noise | n x m | double | [] | (P x P) double full noise covariance in shLowLevel.shIndex ordering (overrides sigmas) |
| Signal | n x m | double | [] | (P x P) double full signal covariance, same ordering (overrides Kaula/DegVar) |
| Idx | n x m | — | [] | shLowLevel.shIndex struct, required with Noise/Signal |
| Nmin | 1 x 1 | double | 2 | first filtered degree; below it W passes through |
| Name | 1 x 1 | string | "designFilter" | stored in W.name |

| output | description |
|---|---|
| W | (1,1) struct identical layout to shLowLevel.readDDK, fields nmax (1,1) double maximum degree the filter covers name (1,:) char the filter's label blocks (1,K) struct one per (order, cs) block, each with m (1,1 double), cs ('c'\|'s'), n (1,B double, the degrees) and M (B x B double, the block matrix) |
| info | (1,1) struct fields alpha (1,1) double the regularisation actually used mode (1,1) string "diagonal" \| "fullcov" gainRange (1,2) double min and max eigenvalue over all blocks |

*Example*

```
    W = shLowLevel.designFilter(g.sigmaC, g.sigmaS, Alpha = 1);
    gF = g.applyDDK(W);               % apply like any DDK
```

### shLowLevel.ellipsoidRadius

`r = shLowLevel.ellipsoidRadius(latDeg, Name=Value)`

Geocentric radius of a reference ellipsoid at a latitude. R = shLowLevel.ellipsoidRadius(LATDEG) is the distance from the Earth's centre to the surface of the WGS84 ellipsoid at each GEOCENTRIC latitude in LATDEG:

| input | size | type | default | description |
|---|---|---|---|---|
| latDeg | n x m | double | required | double array GEOCENTRIC latitudes in degrees, -90..90 |

| input | size | type | default | description |
|---|---|---|---|---|
| SemiMajorAxis | 1 x 1 | double | 6378137 | semi-major axis a [m]. The WGS84 value - the same a shLowLevel.normalFieldCS uses for its WGS84 system, and a test pins that the two agree. Deliberately NOT ... |
| Flattening | 1 x 1 | double | 1/298.257223563 | flattening f, WGS84 as everywhere else in the toolbox. 0 is allowed and gives a sphere of radius SemiMajorAxis, which is what makes the sphere a special case... |

| output | description |
|---|---|
| r | double array geocentric radius [m], same size as latDeg |

*Example*

```
    % gravity disturbance at stations on the ellipsoid, 150 m up
    r = shLowLevel.ellipsoidRadius(lat) + 150;
    dg = shLowLevel.synthesisPoints(C, S, GM, R, lat, lon, r, ...
        Quantity = "disturbance");

  See also shLowLevel.shSynthesis, shLowLevel.synthesisPoints,
  shLowLevel.geodetic2geocentric, shLowLevel.normalFieldCS.
```

### shLowLevel.eofSeparate

`[circ, noiseR, info] = shLowLevel.eofSeparate(R, w, Name=Value)`

Separate coherent modes from noise in a residual stack. [CIRC, NOISER, INFO] = shLowLevel.eofSeparate(R, W) splits a time-centred residual stack into a coherent low-mode part (for ocean applications: residual circulation - interannual dynamics left after the trend + seasonal fit) and a noise remainder, using an area-weighted EOF decomposition with the North et al. (1982) rule: leading modes are kept while their eigenvalue is separated from the next one by more than its sampling uncertainty lambda * sqrt(2/T), ex...

| input | size | type | default | description |
|---|---|---|---|---|
| R | : x : | double | required | double residual stack, one row per (masked) pixel - |
| w | : x 1 | double | required | double positive area weights of the pixels |
| NKeep | 1 x 1 | double | [] | (1 x 1) fixed number of modes; [] applies the North rule |
| MaxModes | 1 x 1 | double | 10 | (1 x 1) hard cap on kept modes |

| output | description |
|---|---|
| circ | (Q x T) double coherent (circulation) part noiseR (Q x T) double time-centred residual minus CIRC |
| info | (1,1) struct fields lam (M x 1) double variance per mode nKeep (1,1) double modes kept as coherent bulkEdge (1,1) double the Marchenko-Pastur bulk edge a retained mode must exceed northSeparated (:,1) logical per leading mode, whether North's rule separates it from its neighbour varExplained (1,1) double fraction of variance in the kept modes rmsNoise (1,1) double area-weighted RMS of NOISER |

*Example*

```
    % after [out, rep] = oceanChain(...): rebuild the pixel residuals
    % R (Q x T), weights wA, then
    [circ, nz, inf1] = shLowLevel.eofSeparate(R, wA);
    inf1.nKeep              % modes the North rule accepts
    inf1.rmsNoise           % the de-circulated noise level
```

### shLowLevel.erodeMask

`[mk2, dmin] = shLowLevel.erodeMask(mk, lat, lon, bufferKm)`

Erode a region mask by a great-circle boundary buffer. MK2 = shLowLevel.erodeMask(MK, LAT, LON, KM) keeps only mask pixels whose great-circle distance to the nearest OUTSIDE pixel exceeds BUFFERKM. Distances are computed against the boundary pixels of the complement only (coast), so the cost is nCoast x nGrid.

| input | size | type | default | description |
|---|---|---|---|---|
| mk | — | — | required | — |
| lat | — | — | required | — |
| lon | — | — | required | — |
| bufferKm | — | — | required | — |

| output | description |
|---|---|
| mk2 | (nLat x nLon) logical eroded mask |
| dmin | (nLat x nLon) double distance to the nearest outside coast pixel [km] (Inf where no outside pixel exists) |

*Example*

```
    oc = abs(LA) <= 66 & ~landBoxes;              % crude ocean mask
    oc300 = shLowLevel.erodeMask(oc, lat, lon, 300);
    nnz(oc) - nnz(oc300)                          % coast pixels shed
```

### shLowLevel.estimateDegree1

`[d1, info] = shLowLevel.estimateDegree1(ts, ocean, Name=Value)`

Geocentre coefficients from GRACE itself (Swenson 2008). [D1, INFO] = shLowLevel.estimateDegree1(TS, OCEAN) estimates the degree-1 coefficients C10, C11, S11 for every epoch of the series TS, using the method of Swenson, Chambers & Wahr (2008).

| input | size | type | default | description |
|---|---|---|---|---|
| ts | 1 x 1 | shSeries | required | shSeries the solutions, degrees 2+ (a GSM series; any existing degree 1 is IGNORED, not added to) |
| ocean | nLat x nLon | — | required | logical, or a function handle f(latDeg, lonDeg) -> logical. true = ocean. This is user-supplied, like Love numbers: a wrong ocean domain changes the answer s... |
| kn | n x m | double | [] | load Love numbers, degrees 0..nmax (REQUIRED - the degree-1 value is frame dependent and must not be assumed; see shLowLevel.readLoveNumbers for the frame co... |
| OceanModel | n x m | double | [] | (nLat x nLon x T) double expected ocean mass in EWH [m] per epoch, or (nLat x nLon) for a static field, or [] for none. OMITTING IT BIASES THE RESULT: whate... |
| LatDeg | 1 x : | double | -89:2:89 | , LonDeg (0:2:358) the evaluation grid. It only has to resolve the ocean domain and the three smooth degree-1 patterns, so a coarse grid is fine and fast |
| LonDeg | 1 x : | double | 0:2:358 | longitudes [deg], lon in [0, 360) |
| Nmax | 1 x 1 | double | NaN | truncate the observed field before evaluation (NaN: the series' own nmax) |

| output | description |
|---|---|
| d1 | (1,1) struct exactly the layout shLowLevel.readTN13 returns - sigmas included, which shCoefficients.addDegree1 requires - so it drops straight into shSeries.addDegree1. Fields epoch, t0, t1 (T,1) double epoch and span bounds [decimal years] C10, C11, S11 (T,1) double the geocentre coefficients sigC10, sigC11, sigS11 (T,1) double their 1-sigma, from the ocean-fit residual propagated through the least-squares solution |

| output | description |
|---|---|
| info | (1,1) struct fields cond (T,1) double condition number of the column-equilibrated design matrix per epoch - equilibrated so the number reflects the GEOMETRY of the ocean domain, not the units of the columns. A rising value warns that the domain is too small or too lopsided BEFORE the noise shows it oceanFraction (1,1) double area fraction used nPixels (1,1) double ocean pixels that entered the fit hasModel (1,1) logical whether an ocean model was supplied residualRMS (T,1) double of the ocean fit, in EWH [m] |

*Example*

```
    LN = shLowLevel.readLoveNumbers("loadLoveNumbers.txt");
    d1 = shLowLevel.estimateDegree1(ts, isOcean, kn = LN.kn);
    ts = ts.addDegree1(d1);            % same interface as TN-13

  See also shLowLevel.readTN13, shSeries.addDegree1,
  shLowLevel.oceanRMS.
```

### shLowLevel.estimateVAR

`model = shLowLevel.estimateVAR(X, Name=Value)`

Empirical VAR(p) process model from an SH state series. MODEL = shLowLevel.estimateVAR(X) estimates the stochastic process model of Kurtenbach et al. (2012) from a centred state realization: the vector autoregressive model x_t = Phi_1 x_{t-1} + ... + Phi_p x_{t-p} + w, w ~ N(0, Q) determined by the Yule-Walker equations from the empirical (cross-)covariances Sigma(h) = 1/(T-h) sum x_i x_{i-h}' of the series (Kvas 2019, Sec. 2.4). For Order = 1 this is EXACTLY the closed form of Kurtenbach (2012), eqs. (3.84)-(3...

| input | size | type | default | description |
|---|---|---|---|---|
| X | : x : | double | required | double centred state realization, one shIndex-ordered |
| Order | 1 x 1 | double | 1 | (1 x 1) VAR order p >= 1. Kurtenbach: 1. Higher orders capture longer temporal correlation (Kvas) at p-fold state dimension in the filter. |
| Shrink | 1 x 1 | double | 0 | (1 x 1) diagonal loading factor: Sigma(0) + Shrink * trace(Sigma(0))/P * I. Stabilizes the Yule-Walker solve when T is not >> P. |
| CondFun | n x m | — | [] | function handle C -> C applied to every Sigma(h) before the solve, for covariance conditioning a la Kvas Sec. 2.4: land/ocean block masking and the distance-... |
| Structure | 1 x 1 | string ∈ ["full","diagonal","orderblock"] | "full" | coupling structure of the estimate: "full" dense Yule-Walker solve (Kurtenbach/Kvas) - needs T >> P; the right choice for daily sampling with condition... |
| Blocks | n x 1 | cell | {} | cell of index vectors partitioning 1:P, required for Structure="orderblock" - e.g. from shIndex: one block per (m, C/S) pair |

| output | description |
|---|---|
| model | (1,1) struct the process model, fields Phi (1 x p) cell of (P x P) double coefficient matrices Q (P x P) double symmetric PSD process-noise covariance Sigma0 (P x P) double empirical auto-covariance Sigma(0): the stationary state covariance, and the filter's initial covariance order (1,1) double the VAR order p P (1,1) double state dimension specRadius (1,1) double spectral radius of the companion matrix; < 1 is stable, and anything else raises a warning because filtering an unstable model diverges over long gaps |

*Example*

```
    [~, resid] = tsModel.climatology;             % reduce long-term parts
    idx = shLowLevel.shIndex(tsModel.nmax);
    X = zeros(idx.P, resid.nEpochs);
    for k = 1:resid.nEpochs
        g = resid.at(k);
        X(:, k) = shLowLevel.vecFromCS(g.C, g.S, idx);
    end
    model = shLowLevel.estimateVAR(X, Order=1);

  Reference: Kurtenbach, DGK C-683 (2012), Sec. 3.2; Kvas, TU Graz
  PhD thesis (2019), Secs. 2.3-2.4; Luetkepohl (2005).
  Numerics pre-validated in Python (tools/dev/validate_kalman.py).
```

### shLowLevel.fetchGAX

`[files, info] = shLowLevel.fetchGAX(dest, Name=Value)`

Download AOD1B GAX monthly means (GAA/GAB/GAC/GAD) from ICGEM. FILES = shLowLevel.fetchGAX(DEST) downloads the monthly-mean dealiasing products needed for full ocean-mass restoration (Chambers & Willis 2010) as ICGEM-converted .gfc files into DEST/<PRODUCT>/. The AOD1B products are identical across processing centres, so the GFZ series pages (which carry all four products from 2002 on) serve every GSM series including COST-G. Listing and download go through the toolbox's Retry-After-aware fetch layer; files alre...

| input | size | type | default | description |
|---|---|---|---|---|
| dest | 1 x 1 | string | "" | string destination folder ("" = the v3.16 default dataFolder/series/GAX); product subfolders are created as needed |
| Products | 1 x : | string | ["GAD", "GAA"] | (1 x k string) any of GAA/GAB/GAC/GAD; GAD restores the model ocean signal, the ocean mean of GAA removes the atmospheric land-ocean mass term |
| Series | 1 x : | string | ["01_GRACE/GFZ/GFZ Release 06", "01_GRACE/GFZ/GFZ Release 06.3 (GFO)"] | (["01_GRACE/GFZ/GFZ Release 06", "01_GRACE/GFZ/GFZ Release 06.3 (GFO)"]) (1 x s string) ICGEM catalogue paths "group/center/series" fed to shLowLevel.listICGEM |
| Update | 1 x 1 | logical | false | (1 x 1) re-download files that already exist |
| MaxFiles | 1 x 1 | double | Inf | (1 x 1) stop after this many downloads per product - bounded acceptance runs, not a user knob |
| Downloader | 1 x 1 | string ∈ ["websave", "httpFetch"] | "websave" | (1 x 1) "websave" (webread network stack, matches listICGEM) or "httpFetch" (matlab.net.http with Retry-After support); the stacks differ in proxy/TLS behavi... |
| BudgetSec | 1 x 1 | double | Inf | (1 x 1) wall-clock budget; on expiry the fetch stops CLEANLY, keeps what it has, and reports the remainder (info.nRemaining) - never a hung call |
| MaxFailures | 1 x 1 | double | 5 | (1 x 1) consecutive-failure cap per product; on hitting it the product loop stops with a warning instead of failing every remaining file one by one |
| PauseSec | 1 x 1 | double | 1.5 | (1 x 1) polite pause between downloads [s] - the ICGEM server rate-limits bursts (HTTP 429; observed live) |
| RetryAfterCap | 1 x 1 | double | 30 | (1 x 1) on a 429 the download is retried ONCE after this wait [s]; a second 429 counts as a failure (websave does not expose the Retry-After header, so a fix... |
| Quiet | 1 x 1 | logical | false | (1 x 1) suppress progress output |

| output | description |
|---|---|
| info | (1,1) struct fields products (1 x nProducts) struct array, one per product, with nListed, nDownloaded, nSkipped, nFailed and nRemaining (what the budget cut off) series (1,:) string the series actually used |

*Example*

```
    f = shLowLevel.fetchGAX("E:/DATAPOOL/GravityField/GAX");
    [out, rep] = shLowLevel.oceanChain(ser, kn = kn, OceanMask = oc, ...
        GADFolder = "E:/DATAPOOL/GravityField/GAX/GAD", ...
        GAAFolder = "E:/DATAPOOL/GravityField/GAX/GAA");

  See fetchITSGSINEX and fetchITSGBackground for the ITSG-side products;
  the ITSG background models are per-release means, NOT the AOD1B
  GAX split.
```

### shLowLevel.fetchITSGBackground

`[files, info] = shLowLevel.fetchITSGBackground(months, Name=Value)`

Download ITSG monthly-mean background models. FILES = shLowLevel.fetchITSGBackground("2018-06") downloads the monthly means of the background models the ITSG processing reduced from the observations, as .gfc (about 1 MB each): "dealiasing" (atmosphere + ocean - the ITSG counterpart of restoring AOD), "oceanTide", "earthTide", "poleTide" and "oceanPoleTide". Release routing follows fetchITSG: months before 2017-07 from ITSG-Grace2018, from 2018-06 on from ITSG-Grace_operational.

| input | size | type | default | description |
|---|---|---|---|---|
| months | n x 1 | — | required | (1 x k string \| numeric years) "YYYY-MM" strings or year vectors, as in fetchITSG |
| Products | 1 x : | string | "dealiasing" | (1 x k string) product selection, one subfolder per product. Availability is release-dependent (verified live 2026-08-12): BOTH eras carry dealiasing, earthT... |
| Dest | 1 x 1 | string | "" | destination folder; "" = dataFolder/series/itsg/background (v3.16) |
| Release | 1 x 1 | string | "" | "" routes by month; a release name pins it |
| Update | 1 x 1 | logical | false | re-download files that already exist |
| MaxFiles | 1 x 1 | double | Inf | downloads cap per product |
| BudgetSec | 1 x 1 | double | Inf | wall-clock cap with clean partial stop |
| MaxFailures | 1 x 1 | double | 5 | consecutive-failure cap, stops loudly |
| PauseSec | 1 x 1 | double | 1.5 | polite pause between downloads [s] |
| RetryAfterCap | 1 x 1 | double | 30 | single capped retry wait on HTTP 429 [s] |
| Downloader | 1 x 1 | string ∈ ["websave", "httpFetch"] | "websave" | "websave" \| "httpFetch" |
| BaseURL | 1 x 1 | string | "https://ftp.tugraz.at/pub/ITSG/GRACE" | ("https://ftp.tugraz.at/pub/ITSG/GRACE") server base |
| Quiet | 1 x 1 | logical | false | suppress progress output |

| output | description |
|---|---|
| info | (1,1) struct fields products (1 x nProducts) struct array, one per product, with nListed, nDownloaded, nSkipped, nFailed, nRemaining missing (1,:) string months the server did not offer releases (1,:) string releases actually used |

*Example*

```
    f = shLowLevel.fetchITSGBackground(["2018-06", "2018-07"]);
    g = shCoefficients.read(f(1));   % plain gfc, C(n+1, m+1)
```

### shLowLevel.fetchITSGSINEX

```
[files, info] = shLowLevel.fetchITSGSINEX(months, Name=Value)
```

Download ITSG monthly normal-equation SINEX from TU Graz. FILES = shLowLevel.fetchITSGSINEX("2018-06") downloads the monthly normal-equation/solution SINEX of the ITSG series - the only public per-month SINEX source (COST-G combines on the normal-equation level internally but does NOT distribute per-month SINEX or covariances). Release routing follows fetchITSG: months before 2017-07 come from ITSG-Grace2018, months from 2018-06 on from ITSG-Grace_operational. Feed the files to shLowLevel.readSINEX (verified aga...

| input | size | type | default | description |
|---|---|---|---|---|
| months | n x 1 | — | required | (1 x k string \| numeric years) "YYYY-MM" strings or year vectors, as in fetchITSG |
| Dest | 1 x 1 | string | "" | destination folder; "" = dataFolder/series/itsg/sinex (v3.16 layout) |
| Nmax | 1 x 1 | double | 96 | (1 x 1) 96 or 120 - the two server variants |
| Release | 1 x 1 | string | "" | "" routes by month (fetchITSG rule); "ITSG-Grace2018" or "ITSG-Grace_operational" pin one release |
| Update | 1 x 1 | logical | false | re-download files that already exist |
| MaxFiles | 1 x 1 | double | Inf | downloads cap - bounded acceptance runs |
| BudgetSec | 1 x 1 | double | Inf | wall-clock cap with clean partial stop |
| MaxFailures | 1 x 1 | double | 5 | consecutive-failure cap, stops loudly |
| PauseSec | 1 x 1 | double | 1.5 | polite pause between downloads [s] |
| RetryAfterCap | 1 x 1 | double | 30 | single capped retry wait on HTTP 429 [s] |
| Downloader | 1 x 1 | string ∈ ["websave", "httpFetch"] | "websave" | "websave" \| "httpFetch" (Retry-After-aware, with its own websave transport fallback) |
| BaseURL | 1 x 1 | string | "https://ftp.tugraz.at/pub/ITSG/GRACE" | ("https://ftp.tugraz.at/pub/ITSG/GRACE") server base |
| Quiet | 1 x 1 | logical | false | suppress progress output |

| output | description |
|---|---|
| info | (1,1) struct fields nListed, nDownloaded, nSkipped, nFailed, nRemaining (1,1) double the download tally; nRemaining is what the budget cut off missing (1,:) string requested months absent on the server releases (1,:) string releases actually used |

*Example*

```
f = shLowLevel.fetchITSGSINEX(["2018-06", "2018-07"], Nmax = 96);
snx = shLowLevel.readSINEX(f(1), Only = "estimate");
```

### shLowLevel.fetchLoveNumbers

```
[files, info] = shLowLevel.fetchLoveNumbers(names, Name=Value)
```

Download load/deformation Love number files (GROOPS set). FILES = shLowLevel.fetchLoveNumbers() downloads the standard load Love number file (Gegout97, kn for degrees 0..1024) from the GROOPS data collection hosted by TU Graz into <dataFolder>/loveNumbers, with the usual safe-swap semantics. The toolbox deliberately never hardcodes Love numbers - this fetcher automates obtaining a PUBLISHED set, which you then pass explicitly (kn = ...) where required.

| input | size | type | default | description |
|---|---|---|---|---|
| names | 1 x 1 | string | "loadLoveNumbers_Gegout97.txt" | display names, one per series/solution |
| Dest | 1 x 1 | string | "" | target folder; default <dataFolder>/loveNumbers |

| input | size | type | default | description |
|---|---|---|---|---|
| BaseURL | 1 x 1 | string | "https://ftp.tugraz.at/pub/ITSG/groops/data/loading" | ("https://ftp.tugraz.at/pub/ITSG/groops/data/loading") (GROOPS/TU Graz loading folder) or a local mirror folder |
| Timeout | 1 x 1 | double | 30 | , Proxy (""), Update (false), Quiet (false) |
| Proxy | 1 x 1 | string | "" | proxy server URL (" = direct) |
| Update | 1 x 1 | logical | false | re-download/overwrite files that already exist |
| Quiet | 1 x 1 | logical | false | suppress progress output |

| output | description |
|---|---|
| files | (1,K) string local paths |
| info | (1,1) struct fields fetched, updated, skipped, failed (1,K) string the tally of file names per outcome parsed (1,K) struct one per file, with name, n (N x 1) and kn (N x 1); empty for deformation files |

*Example*

```
    [f, inf] = shLowLevel.fetchLoveNumbers();
    kn = inf.parsed(1).kn(1:61);        % degrees 0..60 for an n60 field
```

### shLowLevel.filterSpec

`[spec, msg] = shLowLevel.filterSpec(F, Name=Value)`

Read a filter specification - the one parser of the vocabulary. SPEC = shLowLevel.filterSpec(F) reads F, a filter specification in the form every lane with a Filter option takes, and says what it means:

| input | size | type | default | description |
|---|---|---|---|---|
| F | n x m | — | required | text scalar or struct the specification |
| Allowed | 1 x : | string ∈ ["none", "gauss", "ddk", "W"] | ["none", "gauss", "ddk", "W"] | (["none","gauss","ddk","W"]) the kinds the caller admits. The GravIS grid chains admit only "none" and "gauss" |

| output | description |
|---|---|
| spec | (1,1) struct fields kind (1,1) string "none" \| "gauss" \| "ddk" \| "W", or "" when F is refused radiusKm (1,1) double the Gaussian radius [km]; NaN otherwise ddk (1,1) double the DDK number, 1 to 8; NaN otherwise W struct \| [] the filter for kind "W"; [] otherwise name (1,1) string the label the lanes report: W.name for a struct ("custom W" when it has none), the text itself otherwise |
| msg | (1,1) string "" when F is accepted, else the reason |

*Example*

```
    spec = shLowLevel.filterSpec("gauss300");    % kind "gauss", radiusKm 300
    [s, msg] = shLowLevel.filterSpec("DDK3", Allowed = ["none", "gauss"]);
    % s.kind is "" and msg says why

  See also shLowLevel.shGaussianWeights, shLowLevel.readDDK,
  shLowLevel.ddkNames, shLowLevel.designFilter.
```

### shLowLevel.gravisDataFolder

`p = shLowLevel.gravisDataFolder()`

Path of the shipped GravIS auxiliary data. P = shLowLevel.gravisDataFolder() returns the absolute path of the data/gravis folder shipped with the toolbox: frozen (2026-08-12) copies of the GravIS COST-G Level-2B auxiliary files and the GravIS basin polygons that gravisL2B, greenlandChain, antarcticaChain and twsChain use when no gravisFolder is given. The correction tables trail the monthly solutions - for epochs after the freeze fetch fresh copies (see data/gravis/README.md) and pass gravisFolder=.

| output | description |
|---|---|
| p | (1,1) string absolute folder path |

*Example*

```
dir(shLowLevel.gravisDataFolder())
```

### shLowLevel.gravisL2B

`[ts, rep] = shLowLevel.gravisL2B(folder, gravisFolder, Name=Value)`

Build a GravIS-Level-2B-equivalent series from monthly fields. [TS, REP] = shLowLevel.gravisL2B(FOLDER, GRAVISFOLDER) reproduces the GravIS Level-2B correction chain (Dahle & Murboeck 2025) on a folder of monthly .gfc solutions, exactly as validated against the GravIS Greenland, Antarctica and TWS Level-3 products (guide chapters V1-V8): 1. read all monthly solutions (shSeries.fromFolder) 2. replace C20, C30, C21, S21 from the GravIS GRACE+SLR low-degree table, matched on the BEGIN epoch parsed from the GSM-2_YY...

| input | size | type | default | description |
|---|---|---|---|---|
| folder | 1 x 1 | string | required | string folder of monthly GSM-2_*.gfc(.gz) |
| gravisFolder | 1 x 1 | string | "" | string folder holding the GravIS aux files. "" (default) uses the frozen copies shipped in data/gravis (see shLowLevel.gravisDataFolder). The tables trail t... |
| LowDegreeFile | 1 x 1 | string | "GRAVIS-2B_COSTG_0200_GRACE+SLR_LOW_DEGREES_0001.dat" | ("GRAVIS-2B_COSTG_0200_GRACE+SLR_LOW_DEGREES_0001.dat") |
| GeocenterFile | 1 x 1 | string | "GRAVIS-2B_COSTG_0200_GEOCENTER_0001.dat" | ("GRAVIS-2B_COSTG_0200_GEOCENTER_0001.dat") |
| MeanFile | 1 x 1 | string | "GRAVIS-2B_COSTG_0200_MEAN_2002095-2020091_NFIL_0001.gz" | ("GRAVIS-2B_COSTG_0200_MEAN_2002095-2020091_NFIL_0001.gz") |
| GIAFile | 1 x 1 | string | "" | SHM rate file; "" skips step 5. The GravIS product is GRAVIS-2B_COSTG_0200_GIA_ICE-6G_D_VM5a_0001.gz |
| GIAEpoch | 1 x 1 | double | 2011 | reference epoch of the GIA rate (GravIS centres ICE-6G_D on 2011) |
| Nmax | 1 x 1 | double | 90 | truncation degree |
| Pattern | 1 x 1 | string | "GSM-2_*.gfc*" | file pattern |
| Tolerance | 1 x 1 | double | 2e-3 | max \|begin difference\| [yr] for a table match |
| OnMissing | 1 x 1 | string ∈ ["drop","error"] | "drop" | "drop" \| "error" for uncovered epochs |
| SpanEnd | 1 x 1 | double | Inf | keep epochs <= SpanEnd only (decimal years); the published ice trends pin 2023.099 (guide V3) |
| Quiet | 1 x 1 | logical | false | suppress progress output |

| output | description |
|---|---|
| ts | (1,1) shSeries corrected anomaly series (nmax = Nmax) |
| rep | (1,1) struct fields steps (1,K) string the stages applied, with parameters epochs (T,1) double the epoch grid [decimal years] begins (T,1) double each month's interval start nDropped (1,1) double epochs dropped for lack of coverage nmax (1,1) double the degree read version (1,1) string shLowLevel.version at run time created (1,1) string when the run happened |

*Example*

```
    [ts, rep] = shLowLevel.gravisL2B("E:/series/COSTG", ...   % shipped aux data
        GIAFile = "GRAVIS-2B_COSTG_0200_GIA_ICE-6G_D_VM5a_0001.gz");
```

### shLowLevel.gravisRegionChain

```
[gt, rep] = shLowLevel.gravisRegionChain(region, folder, gravisFolder)
```

Shared engine of greenlandChain / antarcticaChain. [GT, REP] = shLowLevel.gravisRegionChain(REGION, FOLDER, GRAVISFOLDER, OPTS) is the common implementation behind the two public ice chains; call those instead. REGION is "greenland" or "antarctica"; OPTS is the (already argument-validated) option struct of the wrappers. Stages: gravisL2B (with GIA) -> EWH synthesis on a GridStep grid -> pixel-wise bias + trend + annual + semiannual fit -> open-ocean residual RMS -> sigma_trend = sigma_monthly / sqrt(Sxx) unless ...

| input | size | type | default | description |
|---|---|---|---|---|
| region | 1 x 1 | string ∈ ["greenland"," antarctica"] | required | string "greenland" \| "antarctica" |
| folder | 1 x 1 | string | required | string monthly solution folder |
| gravisFolder | 1 x 1 | string | required | string GravIS aux folder |
| opts | 1 x 1 | struct | required | struct see greenlandChain / antarcticaChain |

| output | description |
|---|---|
| gt | (1,1) double basin-mask mass trend [Gt/yr] |
| rep | (1,1) struct fields trendGrid (nlat x nlon) double the trend field [cm/yr EWH] lat, lon (1,:) double the grid it is on [deg] sigTrend (nlat x nlon) double its 1-sigma residRMS (1,1) double RMS of the post-fit residual mask (nlat x nlon) logical the region unionMask (nlat x nlon) logical region plus the buffer the leakage step needed m (nlat x nlon) double the inverted (leakage corrected) map iterations (1,1) double leakage iterations run stoppedBy (1,1) string why the iteration stopped basins table per-polygon integrals; filled for antarctica, empty otherwise steps (1,K) string the stages applied, with parameters l2b (1,1) struct the shLowLevel.gravisL2B report version (1,1) string shLowLevel.version at run time created (1,1) string when the run happened |

*Example*

```
    % called for you by the wrappers; direct use mirrors them:
    % [gt, rep] = shLowLevel.gravisRegionChain("greenland", ser, gd, opts)
```

### shLowLevel.greenlandChain

```
[gt, rep] = shLowLevel.greenlandChain(folder, gravisFolder, Name=Value)
```

The validated GravIS Greenland ice-trend chain, one call. [GT, REP] = shLowLevel.greenlandChain(FOLDER, GRAVISFOLDER, kn=KN) reproduces guide chapters V1-V6 end to end: gravisL2B corrections with the ICE-6G_D GIA rate, EWH synthesis on a global grid, a pixel-wise trend + annual + semiannual fit, and the regularised leakage inversion stopped on the discrepancy principle with the TREND-matched noise level sigma_monthly/sqrt(Sxx) (the V4c lesson). With the tested defaults and the COST-G RL02.1 series this returns -...

| input | size | type | default | description |
|---|---|---|---|---|
| folder | 1 x 1 | string | required | string monthly GSM-2_*.gfc folder |
| gravisFolder | 1 x 1 | string | "" | string GravIS aux folder; "" (default) uses the frozen data/gravis copies shipped with the toolbox (see gravisL2B), which also hold BasinFile |
| kn | n x 1 | double | [] | load Love numbers (degree,kn) or column vector - REQUIRED, the empty default errors: no frame is assumed for you |
| BasinFile | 1 x 1 | string | "basins_GIS.json" | GravIS GeoJSON of the target basins ([lon lat] rings; converted internally - see https://gravis.gfz.de/basins/GIS) |
| NeighbourBoxes | n x m | double | [60 84 232 300; 63 67 335 347; 76 81 10 34] | ([60 84 232 300; 63 67 335 347; 76 81 10 34]) the guide-V2 Canadian Arctic, Iceland and Svalbard boxes, [latMin latMax lonMin lonMax] rows, lon in [0, 360) e... |

| input | size | type | default | description |
|---|---|---|---|---|
| GIAFile | 1 x 1 | string | "GRAVIS-2B_COSTG_0200_GIA_ICE-6G_D_VM5a_0001.gz" | ("GRAVIS-2B_COSTG_0200_GIA_ICE-6G_D_VM5a_0001.gz") |
| Filter | 1 x 1 | string | "gauss445" | filter DECLARED to the inversion and applied to the synthesis ("none" \| "gaussN") |
| SpanEnd | 1 x 1 | double | 2023.099 | published-trend span (guide V3) |
| GridStep | 1 x 1 | double | 1 | synthesis grid step [deg] |
| NoiseLevel | 1 x 1 | double | NaN | NaN: derive sigma_trend from the open-ocean residual RMS and the regressor power (V4c); a number overrides it |
| OceanMask | n x m | — | [] | USER-SUPPLIED open-ocean mask for the sigma_trend policy (logical grid or @(lat,lon) handle; see oceanRMS - no coastline is assumed). REQUIRED unless NoiseLe... |
| MaxIter | 1 x 1 | double | 400 | iteration cap |
| TrendGrid | n x m | — | [] | precomputed struct from a previous REP (fields trendGrid, lat, lon, sigTrend) - skips the synthesis/fit stage for sensitivity studies |
| GM | 1 x 1 | double | 3.986004415e14 | , R (6378136.3) |
| R | 1 x 1 | double | 6378136.3 | reference radius [m] (default 6378136.3, overridable) |
| Quiet | 1 x 1 | logical | false | suppress progress output |

| output | description |
|---|---|
| gt | (1,1) double ice-mass trend over the basin mask [Gt/yr] |
| rep | (1,1) struct as returned by shLowLevel.gravisRegionChain, which this function delegates to and which documents every field: trendGrid, lat, lon, sigTrend, residRMS, mask, unionMask, m, iterations, stoppedBy, steps, l2b, version, created |

*Example*

```
kn = readmatrix("loadLoveNumbers_Gegout97.txt", FileType="text", ...
    NumHeaderLines=2);
[gt, rep] = shLowLevel.greenlandChain("E:/series/COSTG", ...
    "E:/GravIS", kn = kn, OceanMask = @(la,lo) abs(la) < 60);
```

### shLowLevel.gridHandover

`b = shLowLevel.gridHandover(Z, lat, lon, units, latType, quantity, productType, name, epoch, c00, history, nmax, nmin)`

Assemble the metadata that travels with a synthesized grid. B = shLowLevel.gridHandover(Z, LAT, LON, UNITS, LATTYPE, QUANTITY, PRODUCTTYPE, NAME, EPOCH, C00, HISTORY, NMAX, NMIN) packs a synthesized field together with what it is, for handover to a cartographic toolbox. Called by shCoefficients.toGrid and shSeries.toGrid; it exists so the two bridges cannot describe the same field differently, which is the failure they were built to prevent in the first place.

| input | size | type | default | description |
|---|---|---|---|---|
| Z | n x m | double | required | double synthesized values |
| lat | 1 x : | double | required | double [deg] as given by the caller |
| lon | 1 x : | double | required | double [deg] |
| units | 1 x 1 | string | required | string from shLowLevel.kernelFactors |
| latType | 1 x 1 | string | required | string "geodetic" \| "geocentric" |

| input | size | type | default | description |
|---|---|---|---|---|
| quantity | 1 x 1 | string | required | string |
| productType | 1 x 1 | string | required | string e.g. "GSM", "GAD", "unknown" |
| name | 1 x 1 | string | required | string model label, may be "" |
| epoch | : x 1 | double | required | double decimal year(s), NaN if unset |
| c00 | : x 1 | double | required | double degree-0 coefficient per epoch |
| history | n x 1 | string | required | string processing history nmax, nmin (1,1) double degree range actually used |
| nmax | 1 x 1 | double | required | maximum spherical harmonic degree |
| nmin | 1 x 1 | double | required | minimum spherical harmonic degree |

| output | description |
|---|---|
| b | (1,1) struct Z, lat (nlat x 1) double, lon (1 x nlon) double, Units, LatType, Quantity, ProductType, Name, Source, FieldKind (1,1) string, Epoch (: x 1) double, C00 (: x 1) double, Zmin, Zmax, Zmean (1,1) double, Nmax, Nmin (1,1) double, History (: x 1) string |

*Example*

```
    b = shLowLevel.gridHandover(Z, lat, lon, "m", "geodetic", "ewh", ...
            "GSM", "ITSG-Grace2018", 2008.29, 0, string.empty(0,1), 60, 0);
    b.FieldKind        % "anomaly"

  See also SHCOEFFICIENTS/TOGRID, SHSERIES/TOGRID, KERNELFACTORS.
```

### shLowLevel.gridScaling

`[k, info] = shLowLevel.gridScaling(model, latDeg, lonDeg, Name=Value)`

Per-pixel leakage/attenuation scaling factors from a model. [K, INFO] = shLowLevel.gridScaling(MODEL, LATDEG, LONDEG, Filter = ...) computes the classic gridded scaling factors: push a MODEL field series through the SAME processing chain the data saw and regress, per pixel, the true value on the filtered one,

| input | size | type | default | description |
|---|---|---|---|---|
| model | nLat x nLon x T | double | required | double UNFILTERED model fields, one slice per epoch, in the same quantity as the data |
| latDeg | n x m | double | required | double geocentric latitudes [deg] |
| lonDeg | n x m | double | required | double longitudes [deg] |
| Filter | n x m | — | "gauss300" | the chain the DATA saw: "none" \| "gauss<radiusKm>" \| "DDKn" \| a W struct from shLowLevel.readDDK / shLowLevel.designFilter |
| Nmax | 1 x 1 | double | NaN | expansion degree (NaN: floor((nLat - 1) / 2)) |
| MinSignal | 1 x 1 | double | 1e-3 | threshold as a fraction of the strongest pixel's summed POWER (so 1e-3 is about 3% in amplitude). Pixels below it in filtered power get k = NaN; the same thr... |
| Clip | n x m | double | [] | (1 x 2) double clamp k to [lo hi] ([]: no clamping). Factors far from 1 usually mean the model does not describe that pixel, not that the signal is huge |
| GM | 1 x 1 | double | 3.986004415e14 | reference constants used to convert the grid to coefficients and back. They CANCEL in the chain, so the result does not depend on them; they exist so an unus... |
| R | 1 x 1 | double | 6378136.3 | reference radius [m], see GM |
| Quiet | 1 x 1 | logical | false | suppress the coverage summary |

| output | description |
|---|---|
| k | (nLat x nLon) double scaling factors; NaN where the model carries too little signal to define one |
| info | (1,1) struct fields covered (1,1) double fraction of the grid with a finite k kMedian, kMin, kMax (1,1) double over the pixels the MODEL gives mass to - deliberately NOT over all finite pixels, where pure-leakage k = 0 would dominate onSignal (1,1) double how many pixels those are clipped (1,1) double pixels whose k was clamped nmax (1,1) double degree the scaling was built at filter (1,1) string the filter it was built for epochs (1,1) double epochs that entered the regression |

*Example*

```
    k = shLowLevel.gridScaling(modelEWH, -89:89, 0:359, ...
            Filter = "gauss300");
    corrected = k .* filteredEWH;      % NaN outside the model's reach

  See also shLowLevel.leakageCorrect, shLowLevel.basinScaling.
```

### shLowLevel.httpFetch

```
info = shLowLevel.httpFetch(url, dest, Name=Value)
```

Download a URL to a file with Retry-After-aware retries. INFO = shLowLevel.httpFetch(URL, DEST) downloads URL to the file DEST. On transient failures (HTTP 429, 500, 502, 503, 504, or a network error) it retries with exponential backoff and jitter; when the server sends a Retry-After header (seconds or an HTTP-date), that value takes precedence over the backoff. Built on matlab.net.http because websave discards the response headers on error - which is exactly why polite retrying was never possible with it. Motiv...

| input | size | type | default | description |
|---|---|---|---|---|
| url | 1 x 1 | string | required | string the URL to download (https recommended) |
| dest | 1 x 1 | string | required | string target file path (parent folder must exist) |
| MaxTries | 1 x 1 | double | 4 | (1 x 1) total attempts before giving up |
| BaseDelay | 1 x 1 | double | 2 | (1 x 1) first backoff delay [s]; attempt k waits BaseDelay * 2^(k-1) plus up to 25% jitter |
| MaxDelay | 1 x 1 | double | 60 | (1 x 1) cap on any single wait [s], also applied to server-sent Retry-After values |
| Timeout | 1 x 1 | double | 60 | (1 x 1) per-attempt timeout [s] |
| WebsaveFallback | 1 x 1 | logical | true | (1 x 1) after all matlab.net.http attempts fail WITHOUT ever receiving an HTTP status (a transport-level failure: proxy, TLS, or a stalled connection), try O... |
| Quiet | 1 x 1 | logical | true | (1 x 1) suppress the per-retry progress line |

| output | description |
|---|---|
| info | (1 x 1) struct with fields status (1 x 1) double final HTTP status code tries (1 x 1) double attempts used waited (1 x 1) double total seconds slept between attempts bytes (1 x 1) double bytes written to DEST transport (1 x 1) string present ONLY when the websave fallback ran (value "websave-fallback"); the normal path leaves the field absent, so test with isfield |

*Example*

```
    info = shLowLevel.httpFetch("https://isdc-data.gfz.de/...", ...
        "C:/data/aux.dat", MaxTries = 5);
```

### shLowLevel.httpRetryDelay

```
w = shLowLevel.httpRetryDelay(attempt, retryAfter, Name=Value)
```

Wait time for a retry: server hint first, else backoff. W = shLowLevel.httpRetryDelay(ATTEMPT, RETRYAFTER) returns the wait [s] before retry number ATTEMPT: when the server sent a finite Retry-After value that value wins (capped at MaxDelay); otherwise exponential backoff BaseDelay * 2^(ATTEMPT-1) with up to 25% uniform jitter, capped at MaxDelay. Pure function - the testable half of shLowLevel.httpFetch.

| input | size | type | default | description |
|---|---|---|---|---|
| attempt | 1 x 1 | double | required | double 1-based attempt counter |
| retryAfter | 1 x 1 | double | required | double server Retry-After [s], NaN if absent |
| BaseDelay | 1 x 1 | double | 2 | (1 x 1) first backoff delay [s] |
| MaxDelay | 1 x 1 | double | 60 | (1 x 1) cap on the returned wait [s] |
| Jitter | 1 x 1 | double | 0.25 | (1 x 1) relative jitter range, 0 disables |

| output | description |
|---|---|
| w | (1 x 1) double wait before this retry [s] |

*Example*

```
    w = shLowLevel.httpRetryDelay(3, NaN)        % ~8 s (+jitter)
    w = shLowLevel.httpRetryDelay(1, 30)         % 30 s (server wins)
```

### shLowLevel.hydroExtremeIndex

`[Z, info] = shLowLevel.hydroExtremeIndex(X, epochs, Name=Value)`

Flood/drought indices per grid cell or basin. [Z, INFO] = shLowLevel.hydroExtremeIndex(X, EPOCHS) turns a TWS anomaly stack (grid cells, basin series, or a single series) into a standardized hydrological-extreme index. Three published modes:

| input | size | type | default | description |
|---|---|---|---|---|
| X | n x m | double | required | (nLat x nLon x T \| K x T \| T x 1) double TWS anomalies [any unit]; grids as from twsChain, basins from shSeries.basinAverage |
| epochs | : x 1 | double | required | double decimal years |
| Mode | 1 x 1 | string ∈ ["DSI", "WSDI", "StorageDeficit"] | "DSI" | "DSI" \| "WSDI" \| "StorageDeficit" |
| Detrend | 1 x 1 | string ∈ ["linear", "none", "default"] | "default" | "linear" \| "none" \| "default"; "default" resolves to "linear" for DSI/WSDI and to "none" for StorageDeficit (the physical Reager convention) |
| Sigma | 1 x 1 | string ∈ ["std", "robust"] | "std" | "std" \| "robust" (1.4826 * MAD - one bad month corrupts std by 6x, MAD by nothing: measured in the pre-validation) |
| MinYears | 1 x 1 | double | 5 | (1 x 1) calendar months sampled by fewer years get NaN and a warning, never a quietly unstable sigma |
| PrecipGrid | n x m | double | [] | same shape as X [same unit]: enables the full FPI in StorageDeficit mode |
| Climatology | 1 x 1 | string ∈ ["auto", "monthly", "doy"] | "auto" | "monthly" \| "doy" \| "auto" (daily data - median epoch spacing < 20 days - selects "doy"). The DOY mode is stage 2: daily Kalman solutions (fetchITSG Product=... |
| WindowDays | 1 x 1 | double | 31 | (1 x 1) circular DOY window width [days] |

| output | description |
|---|---|
| Z | same shape as X the index (DSI/WSDI z-scores; Sdef in the unit of X; FPI dimensionless in [-inf, 1] when PrecipGrid) info (1 x 1) struct mode, detrended, clim (12 x cells, DSI), sigma, trendPerYr, category (same shape as Z, int8 -5..+5, DSI only; edges [-2 -1.6 -1.3 -0.8 -0.5 .5 .8 1.3 1.6 2]), categoryNames (11 x 1 string, "D4".."W4"), nPerMonth |

*Example*

```
    [tws, rep] = shLowLevel.twsChain(ser, kn = kn);       % fields
    [Z, inf1] = shLowLevel.hydroExtremeIndex(tws.grid, tws.epochs);
    imagesc(inf1.category(:, :, end))    % current drought map
```

### shLowLevel.kalmanChain

`[ts, rep] = shLowLevel.kalmanChain(obsIn, Name=Value)`

Kalman-smoothed SH series in one call (Kurtenbach/Kvas). [TS, REP] = shLowLevel.kalmanChain(OBSIN, ModelSeries=TSM) is the single point of access to the Kalman/VAR module: it turns noisy per-epoch gravity information plus a geophysical model series into a temporally smoothed shSeries, encoding the correctly ordered chain of Kurtenbach (2012) and Kvas (2019): 1. reduce the model series TSM by its own climatology (mean, trend, annual, semi-annual) - the residual approximates the short-term process (Kurtenbach eq. ...

| input | size | type | default | description |
|---|---|---|---|---|
| obsIn | 1 x 1 | — | required | shSeries \| (1 x 1) string observations: series (solution mode) or SINEX folder (neq mode) |
| ModelSeries | 1 x 1 | shSeries | required | (1 x 1) shSeries process realization at the target sampling and nmax (truncate first if needed); e.g. GAX/AOD1B, ESA ESM or a hydrology model as SH series |
| Order | 1 x 1 | double | 1 | (1 x 1) VAR order p (see estimateVAR) |
| Shrink | 1 x 1 | double | 0 | (1 x 1) diagonal loading (see estimateVAR) |
| Structure | 1 x 1 | string ∈ ["full","diagonal","orderblock"] | "full" | "full" \| "diagonal" \| "orderblock": coupling structure of the VAR estimate (v3.28; see estimateVAR for the measured monthly table). "orderblock" builds the c... |
| CondFun | n x m | — | [] | covariance conditioning (see estimateVAR) |
| Epochs | : x 1 | double | [] | (K x 1) double target epoch grid [decimal years]. [] uses the observation epochs as-is. A grid entry with no observation within Tolerance becomes a gap epoc... |
| Tolerance | 1 x 1 | double | 0.5/365.25 | (1 x 1) max \|epoch difference\| [yr] to match an observation to a grid entry |
| NoiseCov | n x m | double | [] | solution mode: (P x P) double full R, (P x 1) double diagonal, or (P x T) double per-epoch diagonals; [] derives diagonals from the series' formal sigmas |
| Climatology | 1 x 1 | logical | true | solution mode: reduce OBSIN by its own climatology before filtering and restore it afterwards |
| Smoother | 1 x 1 | logical | true | run the RTS pass (false: filter only, e.g. for NRT-style one-way runs a la Kvas Ch. 3; then the internal filter stores only diagonals and memory stays flat) |
| QC | 1 x 1 | string ∈ ["none","flag","reject"] | "none" | "none" \| "flag" \| "reject": innovation-based per-epoch quality control (Kvas Sec. 3.3; see kalmanFilter). "reject" keeps blunder epochs out of the recursion ... |
| QCAlpha | 1 x 1 | double | 1e-3 | false-alarm level of the test |
| StoreCov | 1 x 1 | string ∈ ["auto","full","matfile"] | "auto" | where the filter keeps the full covariances the smoother needs: "auto" = "full" (RAM) with the smoother, "diag" without; "matfile" streams them through a v7.... |
| Pattern | 1 x 1 | string | "*.snx*" | neq mode: file pattern inside the folder(s) |
| NeqWeights | : x 1 | double | [] | multi-center neq mode: (F x 1) double fixed weights per center (skips per-epoch VCE); [] runs VCE, which requires STATISTICS on every file and errors loudly ... |

| output | description |
|---|---|
| ts | (1 x 1) shSeries smoothed series on the epoch grid, formal 1-sigma stacks filled, every step in the history |
| rep | (1,1) struct fields mode (1,1) string "solution" \| "neq" order (1,1) double the VAR order used nEpochs (1,1) double epochs on the output grid nGaps (1,1) double epochs with no data qcStat (1 x K) double innovation statistic per epoch; NaN when QC="none" nRejected (1,1) double epochs QC rejected centers (F x 1) string contributor labels; "neq" mode sigma2 (F x K) double per-epoch VCE variance factors; NaN where a centre has no file, and in fixed-weight mode specRadius (1,1) double spectral radius of the companion matrix; < 1 is stable meanContribution (1 x K) double mean data share per epoch (Kurtenbach Sec. 3.3.2); NaN in gaps epochs (K,1) double the epoch grid gap (1 x K) logical which epochs were gaps model (1,1) struct the estimateVAR struct, for reuse memGB (1,1) double covariance storage actually used |

*Example*

```
    tsm = shSeries.fromFolder(gaxFolder).truncate(40);
    [ts, rep] = shLowLevel.kalmanChain(tsObs, ModelSeries=tsm, Order=1);

  Memory: the smoother stores two full (pP x pP) covariances per
  epoch: n40/p1/T365 ~ 16 GB. For long daily series run in windows
  or set Smoother=false; REP.memGB reports the actual footprint.

  Reference: Kurtenbach, DGK C-683 (2012); Kvas, TU Graz PhD thesis
  (2019). Numerics pre-validated in Python
  (tools/dev/validate_kalman.py).
```

### shLowLevel.kalmanFilter

`filt = shLowLevel.kalmanFilter(model, obs, Name=Value)`

Forward Kalman filter for SH series (Kurtenbach/Kvas). FILT = shLowLevel.kalmanFilter(MODEL, OBS) runs the forward Kalman filter of Kurtenbach (2012, Sec. 3.3) with the VAR(p) process model from shLowLevel.estimateVAR and one observation record per epoch. Two observation modes, decided per epoch by the fields present: solution l = x + v, v ~ N(0, R): a (noisy) coefficient-vector solution with its error covariance - e.g. a GFC series with formal sigmas, or a SINEX COVA solution neq N x = b: an unreduced no...

| input | size | type | default | description |
|---|---|---|---|---|
| model | 1 x 1 | struct | required | struct from shLowLevel.estimateVAR: Phi, Q, Sigma0, order, P |
| obs | 1 x : | struct | required | struct per-epoch records with fields l (P x 1 double) and R (P x P double, or P x 1 diagonal) for solution mode, or N (P x P double) and b (P x 1 double) fo... |
| StoreCov | 1 x 1 | string ∈ ["full","diag","matfile"] | "full" | where the predicted and filtered covariances live (rtsSmoother needs them in full): "full" in RAM: 2 T (pP)^2 doubles - n40, p=1, T=365: ~16 GB; keep T or... |
| Contribution | 1 x 1 | logical | true | record diag(K) (solution) / diag(P+ N) (neq): the share of the estimate coming from the DATA rather than the process model - Kurtenbach Sec. 3.3.2 |
| QC | 1 x 1 | string ∈ ["none","flag","reject"] | "none" | innovation-based quality control per epoch (Kvas 2019, Sec. 3.3): the statistic solution T = d' S^-1 d, d = l - x-, S = P-(1:P,1:P) + R, dof = P neq T... |
| QCAlpha | 1 x 1 | double | 1e-3 | false-alarm level; threshold from shLowLevel.chi2Quantile(1-QCAlpha, dof) (Wilson-Hilferty, essentially exact at dof = P; see its accuracy note) |

| output | description |
|---|---|
| filt | (1,1) struct companion-space filter results. The PHYSICAL state is the first block, xf(1:P, :). Fields xf, xp (pP x T) double filtered and predicted states Pf, Pp (pP x pP x T) double covariances, StoreCov="full" dPf, dPp (pP x T) double their diagonals instead, for StoreCov="diag" or "matfile" covFile (1,1) string path of the v7.3 MAT holding Pf/Pp, StoreCov="matfile" only, else "" - delete it when you are done with it B (pP x P) double the companion selection matrix contrib (P x T) double data share per coefficient, NaN in gaps gap (1 x T) logical which epochs had no data qcStat (1 x T) double innovation statistic; NaN when QC="none" or in a gap qcDof (1 x T) double its degrees of freedom qcReject (1 x T) logical epochs rejected; always all-false unless QC="reject" P (1,1) double block size order (1,1) double the VAR order storeCov (1,1) string which of the three layouts above is filled |

*Example*

```
    model = shLowLevel.estimateVAR(X, Order=1);
    obs = repmat(struct('l',[],'R',[],'N',[],'b',[]), 1, numel(ep));
    for t = 1:numel(ep)          % fill l/R (or N/b), leave gaps empty
        obs(t).l = L(:, t);  obs(t).R = sig(:, t).^2;
    end
    filt = shLowLevel.kalmanFilter(model, obs);

  Reference: Kurtenbach, DGK C-683 (2012), Sec. 3.3; Kvas, TU Graz
  PhD thesis (2019), Secs. 2.2-2.3.
  Numerics pre-validated in Python (tools/dev/validate_kalman.py).
```

### shLowLevel.leakageCorrect

`[mCorr, info] = shLowLevel.leakageCorrect(grid, latDeg, lonDeg, Name=Value)`

Iterative forward modelling of filter leakage/attenuation. [MCORR, INFO] = shLowLevel.leakageCorrect(GRID, LATDEG, LONDEG, Filter = ...) recovers the mass field that, pushed through the SAME processing chain the data saw, reproduces the observed filtered GRID. The chain F is analysis -> filter -> synthesis, and the correction is the fixed-point iteration

| input | size | type | default | description |
|---|---|---|---|---|
| grid | nLat x nLon | double | required | double observed FILTERED field, in the units you want back (EWH [m], mass, ...) |
| latDeg | n x m | double | required | double geocentric latitudes [deg] |
| lonDeg | n x m | double | required | double longitudes [deg] |
| Filter | n x m | — | "gauss300" | the chain the DATA saw: "none" \| "gauss<radiusKm>" \| "DDKn" \| a W struct from shLowLevel.readDDK / shLowLevel.designFilter. Getting this wrong is the one way... |
| Mask | n x m | — | [] | (nLat x nLon) logical confine the solution to these pixels ([]: unconstrained, global) |
| Nmax | 1 x 1 | double | NaN | expansion degree of the intermediate analysis (NaN: floor((nLat - 1) / 2), the highest degree the grid supports) |
| Gain | 1 x 1 | double | 1 | relaxation factor. Values up to about 3 accelerate convergence; large values DIVERGE (validated: 5 diverges on the reference problem), which is detected rath... |
| NoiseLevel | 1 x 1 | double | 0 | RMS noise of GRID, in GRID's units. The standard estimate is the RMS of the field over the open ocean far from any signal (> 1000 km from the coast), which i... |
| ResidualRegion | n x m | — | [] | where the discrepancy principle measures the residual. [] uses MASK when one is given, and the whole grid otherwise. This matters: with a mask the model desc... |

| input | size | type | default | description |
|---|---|---|---|---|
| Tau | 1 x 1 | double | 1.2 | safety factor in the discrepancy principle; stop when rms(residual) <= Tau * NoiseLevel. Values of 1.2 to 1.5 land nearest the optimum, 1.0 slightly overfits |
| MaxIter | 1 x 1 | double | 200 | iteration cap. Small regions under a strong filter converge slowly - a 6-degree disc under a 500 km Gaussian needs about 260 iterations at Gain = 1 and about... |
| Tol | 1 x 1 | double | 1e-4 | fallback stopping rule, used ONLY when NoiseLevel is 0. Stop when the relative CHANGE OF THE SOLUTION between two iterations falls below this. Not the residu... |
| GM | 1 x 1 | double | 3.986004415e14 | reference constants used to convert the grid to coefficients and back. They CANCEL in the chain, so the result does not depend on them; they exist so an unus... |
| R | 1 x 1 | double | 6378136.3 | reference radius [m], see GM |
| Quiet | 1 x 1 | logical | false | suppress the per-run summary |

| output | description |
|---|---|
| mCorr | (nLat x nLon) double leakage-corrected field, same units and grid as GRID |
| info | (1,1) struct fields iterations (1,1) double iterations actually run residual (1,1) double final relative residual; with a mask this floors above Tol, and that is correct history (1,K) double the residual per iteration - plot it, a rising curve means Gain is too large step (1,K) double the relative change of the solution per iteration; this is what Tol tests stoppedBy (1,1) string "discrepancy" \| "step" \| "maxIter" \| "zeroField". ALWAYS check this: it says whether the result was regularised residualRMS (1,1) double over ResidualRegion residualRMSGlobal (1,1) double over the whole grid; a large gap between the two means unmodelled signal elsewhere noiseLevel (1,1) double as given converged (1,1) logical whether the criterion was met nmax (1,1) double the degree it worked at filter (1,1) string the filter it inverted masked (1,1) logical whether a mask was in play |

*Example*

```
    ewh = g.synthesis(-89:89, 0:359, Quantity = "ewh", kn = kn);
    [m, info] = shLowLevel.leakageCorrect(ewh, -89:89, 0:359, ...
        Filter = "gauss300", Mask = basinMask);
    fprintf("%d iterations, residual %.2e\n", info.iterations, info.residual);

  See also shLowLevel.gridScaling, shLowLevel.basinDeconvolve,
  shLowLevel.basinScaling.
```

### shLowLevel.leakageDataDriven

`[fHat, info] = shLowLevel.leakageDataDriven(gridSeries, latDeg, lonDeg, Name=Value)`

Model-free leakage/damage correction of a catchment mean. FHAT = shLowLevel.leakageDataDriven(GRIDSERIES, LATDEG, LONDEG, Mask = R, Filter = F) repairs the catchment average of a FILTERED time series using correction terms built from the filtered fields alone - Vishwakarma's "method of deviation". No hydrological model is involved, which is the whole point: it is the model-independent counterpart of shLowLevel.leakageCorrect's forward model.

| input | size | type | default | description |
|---|---|---|---|---|
| gridSeries | nLat x nLon x T | double | required | double ALREADY-FILTERED fields, one per epoch, in any unit that is linear in mass (EWH is the usual one). The unit passes straight through. |
| latDeg | 1 x nLat | double | required | double geocentric latitudes in degrees |
| lonDeg | 1 x nLon | double | required | double longitudes in degrees |

| input | size | type | default | description |
|---|---|---|---|---|
| Mask | n x m | double | [] | (nLat x nLon) double the catchment function R: 1 inside, 0 outside. Fractional edge values are allowed and are used as given. The default is empty and the f... |
| Filter | n x m | — | "gauss300" | the SAME filter the data already saw - "none" \| "gauss<radiusKm>" \| "DDKn" \| a W struct, exactly as shLowLevel.leakageCorrect takes it. Getting this wrong do... |
| Nmax | 1 x 1 | double | NaN | degree of the analysis/synthesis chain; default floor((nLat-1)/2) |
| GM | 1 x 1 | double | 3.986004415e14 | used identically on both sides of the chain, so it cancels |
| R | 1 x 1 | double | 6378136.3 | as GM |

| output | description |
|---|---|
| fHat | (T x 1) double corrected catchment average, same unit as gridSeries |
| info | (1,1) struct fields filtered (T x 1) double the uncorrected catchment average of the input - what a user would have had without this function leakage (T x 1) double lhat, the estimated true leakage that was removed deviation (T x 1) double dFhat, the estimated deviation integral that was removed leakageOnce (T x 1) double lbar, leakage from the once filtered fields (before extrapolation) leakageTwice (T x 1) double lbarbar, from the twice filtered fields deviationOnce (T x 1) double dFbar deviationTwice (T x 1) double dFbarbar phaseDeg (1,1) double phase difference applied to the leakage series, in degrees leakageRatio (1,1) double amplitude factor applied to the leakage series deviationRatio (1,1) double amplitude factor applied to the deviation integral areaFraction (1,1) double catchment area over the sphere; multiply by 5.1006e8 km^2 for a rough size, and compare against the ~65000 km^2 the method wants nmax (1,1) double degree the chain worked at filter (1,1) string the filter that was re-applied |

*Example*

```
    % ewh is (nLat x nLon x T) of Gaussian-300 filtered fields
    [fHat, info] = shLowLevel.leakageDataDriven(ewh, lat, lon, ...
        Mask = basin, Filter = "gauss300");
    fprintf("leakage removed: %.3f m RMS, phase %.1f deg\n", ...
        rms(info.leakage), info.phaseDeg);

  References
    Vishwakarma, B. D. (2017), Understanding and repairing the signal
      damage due to filtering of mass change estimates from the GRACE
      satellite mission, PhD thesis, Institute of Geodesy, University
      of Stuttgart - chapter 3, eqs. (3.19)-(3.29).
    Vishwakarma, Horwath, Devaraju, Groh & Sneeuw (2017), A data-driven
      approach for repairing the hydrological catchment signal damage
      due to filtering of GRACE products, Water Resour. Res. 53,
      9824-9844, doi:10.1002/2017WR021150.
    Phillips, Nerem, Fox-Kemper, Famiglietti & Rajagopalan (2012), The
      influence of ENSO on global terrestrial water storage using
      GRACE, Geophys. Res. Lett. 39, L16705, doi:10.1029/2012GL052495
      - the Hilbert phase estimator.
    Python-validated in tools/dev/validate_leakage_datadriven.py: the
      decomposition closes to 4e-15, the phase measure/shift pair are
      mutual inverses to 7e-15 deg, and a closed loop removes 63% of
      the filtering damage. testCorrectness measures 55.3% on its own
      fixture; the grids and the basin differ, the method does not.

  See also shLowLevel.leakageCorrect, shLowLevel.basinScaling,
  shLowLevel.gridScaling, shLowLevel.basinKernel.
```

### shLowLevel.listITSG

`[T, info] = shLowLevel.listITSG(Name=Value)`

Catalogue of ITSG gravity field series on the TU Graz server. T = shLowLevel.listITSG() enumerates the available ITSG releases and product folders (releases ITSG-Grace2014/2016/2018 and the continuously extended ITSG-Grace_operational; products monthly at n60/n96/n120 and daily Kalman solutions) by parsing the server

directory indices - so mixing files from different releases, the trap of guessing folder names, becomes impossible. The catalogue row number is the selection handle for shLowLevel.fetchITSG(Catalog=...

| input | size | type | default | description |
|---|---|---|---|---|
| BaseURL | 1 x 1 | string | "https://ftp.tugraz.at/pub/ITSG/GRACE" | ("https://ftp.tugraz.at/pub/ITSG/GRACE") server base, or a LOCAL MIRROR FOLDER (offline/testing: directory tree is walked instead of scraped) |
| Timeout | 1 x 1 | double | 30 | per-request timeout [s] |

| output | description |
|---|---|
| T | (K x 5) table columns: idx (1,1) double selection number for fetchITSG release (1,1) string e.g. "ITSG-Grace2018" product (1,1) string "monthly" \| "daily" \| "static" nmax (1,1) double 60/96/120 (NaN for daily/static) url (1,1) string folder URL or mirror path |
| info | (1,1) struct fields source (1,1) string where the catalogue came from nReleases (1,1) double releases found |

*Example*

```
    T = shLowLevel.listITSG();
    disp(T)     % pick rows, then fetch via Catalog= in fetchITSG
```

### shLowLevel.makeTutorials

`out = shLowLevel.makeTutorials(Name=Value)`

Generate Live Script tutorials from the demo registry. FILES = shLowLevel.makeTutorials() writes one tutorial per demo case into tutorials/ as a plain .m script with Live Editor section markers, and converts each to .mlx when the converter is available.

| input | size | type | default | description |
|---|---|---|---|---|
| Dest | 1 x 1 | string | "" | (fullfile(shLowLevel.version().Root, "tutorials")) output folder; created if absent |
| Cases | 1 x : | string | "all" | which demo cases to write ("all", "core", or an array of ids such as ["D01" "D05"]) |
| Convert | 1 x 1 | logical | true | also write .mlx. The converter is an internal MATLAB API and is not present in every release, so a failure is reported and the .m files are kept: those are t... |
| Quiet | 1 x 1 | logical | false | suppress progress output |

| output | description |
|---|---|
| out | (1,1) struct fields files (1,N) string the .m files written mlx (1,M) string the .mlx files written; empty when Convert is false or the conversion is unavailable dest (1,1) string the folder written to converted (1,1) logical whether the .mlx conversion ran |

*Example*

```
    shLowLevel.makeTutorials(Cases = ["D01" "D05"]);
    edit(fullfile("tutorials", "T_D01_read_and_spectra.m"))

  See also demo_shAnalysis, shLowLevel.version.
```

### shLowLevel.neqCombine

`[x, out] = shLowLevel.neqCombine(neqs, Name=Value)`

VCE-weighted combination on the normal-equation level. [X, OUT] = shLowLevel.neqCombine(NEQS) combines K normal equations into one solution the COST-G way - on the normal-equation level, BEFORE solving - with one variance component per contribution, estimated by Foerstner/Koch iteration with partial redundancies: N = sum_i (1/s_i^2) N_i, b = sum_i (1/s_i^2) b_i x = N^-1 b Om_i = ltpl_i - 2 x' b_i + x' N_i x (= v_i' P_i v_i) r_i = nobs_i - (1/s_i^2) tr(N^-1 N_i) (partial redundancy) s_i^2 = Om_i ...

| input | size | type | default | description |
|---|---|---|---|---|
| neqs | 1 x : | struct | required | struct one contribution each, in EITHER form: N (P x P double), b (P x 1 double), and for VCE also ltpl (1 x 1 double, l'Pl of the contribution) and nobs (1... |
| Weights | : x 1 | double | [] | (K x 1) double FIXED weights w_i = 1/s_i^2: skips the VCE iteration entirely (then ltpl/nobs are not needed); [] runs VCE, which REQUIRES ltpl and nobs on e... |
| MaxIter | 1 x 1 | double | 20 | (1 x 1) VCE iteration cap |
| Tol | 1 x 1 | double | 1e-6 | (1 x 1) relative change of all s_i^2 to stop |
| Covariance | 1 x 1 | logical | false | also return Cx = N^-1 (P x P inverse - costly for large P, off by default) |

| output | description |
|---|---|
| x | (P x 1) double combined solution |
| out | (1,1) struct fields sigma2 (K x 1) double final variance factors; the fixed 1./Weights when those were given w (K x 1) double = 1./sigma2 redundancy (K x 1) double partial redundancies; NaN in fixed mode, where nothing was estimated nIter (1,1) double VCE iterations used converged (1,1) logical whether it converged N (P x P) double the combined normal matrix Cx (P x P) double its inverse; Covariance=true only names (K x 1) string the contributor labels |

*Example*

```
   a = shLowLevel.readSINEX(fileA, Index = idx);
   b = shLowLevel.readSINEX(fileB, Index = idx);
   [x, out] = shLowLevel.neqCombine([a, b]);
   disp(out.sigma2')          % who carries how much noise power

 Numerics pre-validated in Python (tools/dev/validate_neqcombine.py,
 5 checks: fixed == stacked GLS, factor recovery, redundancy
 invariant, combined beats singles, equal-noise symmetry).

 Reference: Koch (1999) variance component estimation; Foerstner
 (1979); Kvas, TU Graz PhD thesis (2019), Secs. 2.1-2.2; the
 COST-G combination concept (Jaeggi et al.).
```

### shLowLevel.obpChain

`[out, rep] = shLowLevel.obpChain(folder, Name=Value)`

Ocean-bottom-pressure fields (GravIS Level-3 OBP recipe). [OUT, REP] = shLowLevel.obpChain(FOLDER, kn = KN, OceanMask = OC, GADFolder = GAD) produces gridded ocean-bottom-pressure anomalies in EWH following the GravIS/GFZ Level-3 OBP definition: gravisL2B corrections (C20/C30/C21/S21, degree 1, minus NFIL mean, GIA) -> GAD added back on the coefficient level -> filter (AFTER the GAD addition - the GravIS order) -> EWH synthesis -> anomalies against the mean over RefPeriod, land set to NaN.

| input | size | type | default | description |
|---|---|---|---|---|
| folder | 1 x 1 | string | required | string monthly .gfc folder (GSM-2_*, any centre) |
| kn | n x 1 | double | [] | (N x 2 \| N x 1) REQUIRED load Love numbers |
| OceanMask | n x m | — | [] | (fn handle \| nLat x nLon logical) REQUIRED ocean decision, false over land |
| GADFolder | 1 x 1 | string | "" | (1 x 1) string REQUIRED folder of GAD-2_*.gfc (shLowLevel.fetchGAX) - OBP without GAD is not OBP, so the empty default errors |
| GravisFolder | 1 x 1 | string | "" | GravIS aux folder; "" = shipped data/gravis |
| GIAFile | 1 x 1 | string | "GRAVIS-2B_COSTG_0200_GIA_ICE-6G_D_VM5a_0001.gz" | ("GRAVIS-2B_COSTG_0200_GIA_ICE-6G_D_VM5a_0001.gz") "" disables the GIA rate (see gravisL2B) |

| input | size | type | default | description |
|---|---|---|---|---|
| GIAEpoch | 1 x 1 | double | 2011 | (1 x 1) GIA reference epoch |
| GridStep | 1 x 1 | double | 1 | (1 x 1) synthesis graticule [deg] |
| SpanEnd | 1 x 1 | double | [] | (1 x 1) keep epochs <= SpanEnd [decimal yr] |
| Filter | 1 x 1 | string | "gauss445" | "none" \| "gaussN" [km], applied after GAD |
| RefPeriod | 1 x 2 | double | [2002.25, 2020.25] | (1 x 2) anomaly reference window [decimal years], the GravIS 2002/04-2020/03 convention |
| Quiet | 1 x 1 | logical | true | (1 x 1) suppress progress output |

| output | description |
|---|---|
| out | (1 x 1) struct with fields grid (nLat x nLon x T) OBP anomaly [cm EWH], land NaN lat, lon (nLat x 1), (nLon x 1) grid vectors [deg] epochs (T x 1) decimal years refMean (nLat x nLon) subtracted reference-period mean [cm] oceanMeanOBP (T x 1) area-weighted ocean mean of the anomaly [cm] - contains the atmospheric term by definition |
| rep | (1,1) struct fields steps (1,K) string the stages applied, with parameters version (1,1) string shLowLevel.version at run time nEpochs (1,1) double epochs processed nGadRestored (1,1) double months the GAD restore reached nRefEpochs (1,1) double epochs inside RefPeriod |

*Example*

```
kn = readmatrix("loadLoveNumbers_Gegout97.txt", FileType = "text", ...
    NumHeaderLines = 2);
[out, rep] = shLowLevel.obpChain(serFolder, kn = kn, ...
    OceanMask = oc, GADFolder = "E:/DATAPOOL/GravityField/GAX/GAD");
imagesc(out.lon, out.lat, out.grid(:, :, end)); axis xy
```

### shLowLevel.oceanChain

`[out, rep] = shLowLevel.oceanChain(folder, Name=Value)`

Global-ocean mass chain: barystatic series from GSM + GravIS. [OUT, REP] = shLowLevel.oceanChain(FOLDER, kn = KN, OceanMask = OC) runs the ocean sibling of the validated chains (roadmap item 7): gravisL2B corrections (C20/C30/C21/S21, degree 1, minus NFIL mean, GIA rate ON by default - the ocean-floor GIA correction matters for barystatic trends) -> global EWH synthesis on a GridStep graticule -> area-weighted ocean-mean series [cm] -> trend + annual fit. The residual after the per-pixel trend+seasonal fit is wh...

| input | size | type | default | description |
|---|---|---|---|---|
| folder | 1 x 1 | string | required | string monthly .gfc folder (GSM-2_*, any centre) |
| kn | n x 1 | double | [] | (N x 2 \| N x 1) REQUIRED load Love numbers |
| OceanMask | n x m | — | [] | (fn handle \| nLat x nLon logical) REQUIRED ocean decision @(lat, lon) -> logical, MUST be false over land (oceanRMS contract; the chains' documented 5 Gt/yr ... |
| GravisFolder | 1 x 1 | string | "" | GravIS aux folder; "" = shipped data/gravis |
| GIAFile | 1 x 1 | string | "GRAVIS-2B_COSTG_0200_GIA_ICE-6G_D_VM5a_0001.gz" | ("GRAVIS-2B_COSTG_0200_GIA_ICE-6G_D_VM5a_0001.gz") "" disables the GIA rate; the default ocean-floor correction raises the barystatic trend by about +1 mm/yr... |
| GIAEpoch | 1 x 1 | double | 2011 | (1 x 1) GIA reference epoch |
| Filter | 1 x 1 | string | "gauss445" | "none" \| "gaussN" [km]; the ocean MEAN is nearly filter-invariant, but sigMon describes the filtered world - unfiltered, the pixel residuals are stripe-domin... |
| GridStep | 1 x 1 | double | 1 | (1 x 1) synthesis graticule [deg] |
| SpanEnd | 1 x 1 | double | [] | (1 x 1) keep epochs <= SpanEnd [decimal years] |

| input | size | type | default | description |
|---|---|---|---|---|
| GADFolder | 1 x 1 | string | "" | folder of GAD-2_*.gfc; when set, epochs with a begin-matched GAD file get it added back on the coefficient level and REP records covered and uncovered epochs... |
| GAAFolder | 1 x 1 | string | "" | folder of GAA-2_*.gfc; when set, the ocean MEAN of GAA is subtracted per epoch (Chambers & Willis 2010) - removes the atmospheric land-ocean mass term that G... |
| SeparateCirculation | 1 x 1 | logical | false | split the pixel residuals (after the North-multiplet + Marchenko-Pastur selection, capped at 3 modes: the MP edge assumes i.i.d. noise and turns liberal unde... |
| CoastBufferKm | 1 x 1 | double | 300 | erode the ocean mask by this great-circle distance from every coast [km]. Quantified (Python, gauss445 point-source experiment): 300 km removes 54% of the co... |
| RemoveLandLeakage | 1 x 1 | logical | false | iterative support separation of ALL sources outside the ocean mask (two-sided alternating projections, Papoulis-Gerchberg style): land and ocean components a... |
| LandLeakIter | 1 x 1 | double | 1 | (1 x 1) separation sweeps. Machine-measured trend ladder with the 300 km buffer: 0 sweeps +1.557, 1 sweep +1.524, 5 sweeps +1.321 mm/yr - deeper iteration at... |
| Quiet | 1 x 1 | logical | true | (1 x 1) suppress progress output |

| output | description |
|---|---|
| out | (1 x 1) struct with fields epochs (T x 1) decimal years oceanMean (T x 1) area-weighted ocean-mean EWH [cm] trend (1 x 1) barystatic trend [mm/yr] trendGt (1 x 1) the same as mass rate [Gt/yr] over the mask trendSigma (1 x 1) OLS 1-sigma of the trend [mm/yr] (no AR correction - stated, not hidden) ampAnnual (1 x 1) annual amplitude of the ocean mean [mm] sigMon (1 x 1) open-ocean residual RMS after the per-pixel trend+seasonal fit [m] (the honest noise proxy) oceanArea (1 x 1) mask area [m^2] circulationRMS (1 x 1) area-weighted RMS of the separated residual circulation [m] (NaN unless SeparateCirculation) nModes (1 x 1) EOF modes the North rule kept (0 unless SeparateCirculation) lamModes (K x 1) variance per EOF mode ([] unless SeparateCirculation) |
| rep | (1,1) struct fields steps (1,K) string the stages applied, with parameters, including any dropped or uncovered epochs version (1,1) string shLowLevel.version at run time nEpochs (1,1) double epochs processed nGadRestored (1,1) double months the GAD restore reached nGaaApplied (1,1) double months the GAA correction reached |

*Example*

```
kn = readmatrix("loadLoveNumbers_Gegout97.txt", FileType = "text", ...
    NumHeaderLines = 2);
oc = @(la, lo) abs(la) <= 66 & ~yourLandFcn(la, lo);
[out, rep] = shLowLevel.oceanChain("E:/series/COSTG", kn = kn, ...
    OceanMask = oc);
plot(out.epochs, out.oceanMean)   % barystatic curve [cm]
```

### shLowLevel.oceanRMS

`[rmsVal, info] = shLowLevel.oceanRMS(grid, latDeg, lonDeg, ocean, Name=Value)`

Open-ocean RMS of a field - the standard GRACE noise metric. RMSVAL = shLowLevel.oceanRMS(GRID, LATDEG, LONDEG, OCEAN) returns the area-weighted RMS of GRID over the OPEN ocean: ocean points more than MinDistanceKm (default 1000) from the nearest non-ocean point.

| input | size | type | default | description |
|---|---|---|---|---|
| grid | nLat x nLon | double | required | double the field to measure |

| input | size | type | default | description |
|---|---|---|---|---|
| latDeg | n x m | double | required | double geocentric latitudes [deg] |
| lonDeg | n x m | double | required | double longitudes [deg] |
| ocean | nLat x nLon | — | required | logical, or a function handle f(latDeg, lonDeg) -> logical/[0,1] evaluated on the grid. true = ocean |
| MinDistanceKm | 1 x 1 | double | 1000 | erode the ocean mask by this great-circle distance, i.e. keep only points farther than this from any non-ocean point. 1000 km is the common convention; 0 use... |
| Weighted | 1 x 1 | logical | true | weight by cos(latitude). Grid cells are not equal in area, and an unweighted RMS over a lat/lon grid over-counts the polar rows. On white noise the two agree... |
| Subsample | 1 x 1 | double | 1 | use every n-th boundary point for the distance computation. The erosion costs O(nOcean x nLand); on fine grids raise this, at the cost of a distance error of... |

| output | description |
|---|---|
| rmsVal | (1,1) double open-ocean RMS, in GRID's units. NaN when the erosion leaves no points |
| info | (1,1) struct fields nPixels (1,1) double points that entered the RMS fractionUsed (1,1) double those as a fraction of the grid mask (nLat x nLon) logical the eroded region - plot it once before trusting the number minDistanceKm (1,1) double erosion distance applied [km] weighted (1,1) logical whether area weights were used |

*Example*

```
    isOcean = @(la, lo) ~inpolygon(lo, la, landLon, landLat);
    noise = shLowLevel.oceanRMS(ewh, -89:89, 0:359, isOcean);
    m = shLowLevel.leakageCorrect(ewh, -89:89, 0:359, ...
            Filter = "gauss300", Mask = basin, NoiseLevel = noise);

  See also shLowLevel.leakageCorrect, shLowLevel.spatialStats.
```

### shLowLevel.poleTideConvert

`varargout = shLowLevel.poleTideConvert(varargin)`

Convert C21/S21 between mean-pole (pole tide) conventions. G2 = shLowLevel.poleTideConvert(G, From="IERS2010", To="IERS2018") adjusts the C21/S21 coefficients of an shCoefficients object (epoch taken from G) from one mean-pole convention to another. Processing centers remove the (solid + ocean) pole tide using a MEAN POLE MODEL, and the IERS 2010 conventional model (cubic until 2010.0, linear after) differs from the 2018 secular-pole update (linear, adopted for GRACE/GRACE-FO RL06) by up to ~1e-10 in C21/S21 wit...

| input | size | type | default | description |
|---|---|---|---|---|
| varargin | — | — | required | — |

| output | description |
|---|---|
| G2 | (1,1) shCoefficients converted object (object input; history appended) |
| C2, S2 | (nmax+1 x nmax+1) double converted triangles (raw input) |

*Example*

```
    g18 = shLowLevel.poleTideConvert(g, From = "IERS2010", To = "IERS2018");
    dC21 = g18.C(3, 2) - g.C(3, 2)     % ~1e-10 level, trend-like in t
```

### shLowLevel.readSHM

`out = shLowLevel.readSHM(filename, Name=Value)`

Read a GRAVIS/GRACE SHM file (GRCOF2 fields, GRDOTA rates). OUT = shLowLevel.readSHM(FILENAME) reads the spherical-harmonic format used by the GRAVIS Level-2B products and by GRACE/GRACE-FO Level-2

files that are not in ICGEM layout:

| input | size | type | default | description |
|---|---|---|---|---|
| filename | 1 x 1 | — | required | string \| char path to the file; .gz is read transparently |
| Nmax | 1 x 1 | double | NaN | truncate while reading (NaN: the file's own maximum degree). A GIA model at degree 256 against a solution at 96 wastes memory and time |

| output | description |
|---|---|
| out | (1,1) struct fields: kind (1,1) string "GRCOF2" (field) \| "GRDOTA" (rate) C, S (nmax+1 x nmax+1) double coefficients in the C(n+1, m+1) house layout; for GRDOTA these are rates per year sigmaC, sigmaS (nmax+1 x nmax+1) double, or [] when the file carries none (GRDOTA never does) GM (1,1) double [m^3/s^2] from the header R (1,1) double [m] from the header nmax (1,1) double maximum degree actually read header (1,1) string the raw YAML header, kept so provenance is not lost on the way in |

*Example*

```
    gia = shLowLevel.readSHM("GRAVIS-2B_..._GIA_ICE-6G_D_VM5a_0001.gz", ...
        Nmax = 96);
    g = shCoefficients(gia.C, gia.S, GM = gia.GM, R = gia.R);
    ts = shLowLevel.standardChain(folder, GIA = g);   % rates in 1/yr

  See also shLowLevel.shReadGFC, shLowLevel.standardChain.
```

### shLowLevel.requireGeoMap

```
loc = shLowLevel.requireGeoMap()
```

Assert that geoMap is on the path; say where it is. LOC = shLowLevel.requireGeoMap() returns the path of geo.map, or raises shLowLevel:requireGeoMap:geoMapRequired with a message that says what to do instead of reporting an undefined function three frames deeper.

| output | description |
|---|---|
| loc | (1,:) char full path of geo.map, for provenance records |

*Example*

```
    loc = shLowLevel.requireGeoMap();   % errors if geoMap is absent

  See also SHCOEFFICIENTS/MAP, SHCOEFFICIENTS/TOGRID, WRITEANIMATION.
```

### shLowLevel.rtsSmoother

```
smo = shLowLevel.rtsSmoother(filt, Name=Value)
```

Rauch-Tung-Striebel backward pass for kalmanFilter output. SMO = shLowLevel.rtsSmoother(FILT) runs the fixed-interval RTS smoother (Kurtenbach 2012, Sec. 3.3.3) over the forward-filter result of shLowLevel.kalmanFilter: G_t = Pf_t B' Pp_{t+1}^-1 xs_t = xf_t + G_t (xs_{t+1} - xp_{t+1}) Ps_t = Pf_t + G_t (Ps_{t+1} - Pp_{t+1}) G_t' so the estimate at epoch t uses ALL observations, past and future. With the stationary initialization of kalmanFilter this reproduces the joint least-squares adjustment over all epochs ...

| input | size | type | default | description |
|---|---|---|---|---|
| filt | 1 x 1 | struct | required | struct from shLowLevel.kalmanFilter with |
| Lag | 1 x 1 | double | Inf | (1 x 1) double smoothing window in epochs: the estimate at t uses data up to t+Lag; 0 returns the pure filter estimates, Inf the full RTS pass. The filt req... |

| output | description |
|---|---|
| smo | (1,1) struct smoothed results, PHYSICAL block only. Fields xs (P x T) double smoothed states sig (P x T) double formal 1-sigma per coefficient, the square root of the smoothed covariance diagonal PsLast (P x P) double the full smoothed covariance of the LAST epoch, which equals the filtered one there P (1,1) double block size order (1,1) double the VAR order |

*Example*

```
    filt = shLowLevel.kalmanFilter(model, obs);   % StoreCov="full"
    smo  = shLowLevel.rtsSmoother(filt);
    plot(ep, smo.xs(1, :))                        % first coefficient

  Reference: Rauch, Tung & Striebel (1965); Kurtenbach, DGK C-683
  (2012), Sec. 3.3.3; Kvas, TU Graz PhD thesis (2019), Sec. 2.3.
  Numerics pre-validated in Python (tools/dev/validate_kalman.py).
```

### shLowLevel.safeMove

`n = shLowLevel.safeMove(src, dst, Name=Value)`

Move a file into place, retrying while a scanner still holds it. SAFEMOVE(SRC, DST) renames SRC to DST, overwriting DST, and retries with exponential backoff while the move fails. Every shLowLevel.fetch* function downloads to "<target>.part", verifies the fresh file by parsing it, and only then swaps it into place - this function is that swap.

| input | size | type | default | description |
|---|---|---|---|---|
| src | 1 x 1 | string | required | string existing file to move (e.g. "x.gfc.part") |
| dst | 1 x 1 | string | required | string destination path; overwritten if present |
| Retries | 1 x 1 | double | 5 | number of RETRIES after the first attempt; 0 gives a single attempt and the behaviour of a plain movefile |
| Pause | 1 x 1 | double | 0.25 | seconds before the first retry. Each further wait doubles, capped at 4 s, so the default schedule is 0.25/0.5/1/2/4 s - about 7.75 s of patience in total, wh... |

| output | description |
|---|---|
| n | (1,1) double number of attempts used (1 = the move succeeded immediately, which is the normal case) |

*Example*

```
    % download, verify, then swap - the fetcher pattern
    tmpf = "ITSG-Grace2018_n60_2008-04.gfc.part";
    shLowLevel.shReadGFC(tmpf);              % verify BEFORE the swap
    shLowLevel.safeMove(tmpf, "ITSG-Grace2018_n60_2008-04.gfc");

  See also movefile, shLowLevel.fetchICGEM, shLowLevel.fetchITSG.
```

### shLowLevel.sensitivityKernel

`[k, info] = shLowLevel.sensitivityKernel(idx, region, Name=Value)`

Basin kernel with an explicit leakage/noise trade-off. [K, INFO] = shLowLevel.sensitivityKernel(IDX, REGION) builds an averaging kernel for REGION that balances two things a plain indicator kernel cannot: signal leaking in from OUTSIDE the region, and GRACE error propagating into the average.

| input | size | type | default | description |
|---|---|---|---|---|
| idx | 1 x 1 | struct | required | struct from shLowLevel.shIndex |
| region | n x m | — | required | the region: a function handle f(latDeg,lonDeg) -> [0,1], a K x 2 polygon [latDeg lonDeg], or a mask - anything shLowLevel.basinKernel accepts |

| input | size | type | default | description |
|---|---|---|---|---|
| Alpha | 1 x 1 | double | 1 | the trade-off weight. Larger = less noise, more leakage, at unchanged unit gain. There is no universally right value: sweep it and look at INFO.leakage again... |
| Noise | n x 1 | double | [] | (P x P) double error covariance in IDX ordering, or a (P x 1) vector of formal sigmas, or [] to use a degree- dependent default that mimics GRACE error grow... |
| FarField | n x m | double | [] | (P x 1) double per-coefficient weight for the leakage term ([]: 1/(n+1), which emphasises the low degrees where a compact basin's leakage lives). The benefi... |
| BufferKm | 1 x 1 | double | 0 | , TaperKm (0), R (6378136.3), OverSample (2) passed to shLowLevel.basinKernel for the exact indicator |
| TaperKm | 1 x 1 | double | 0 | cosine taper width at the region edge [km] |
| R | 1 x 1 | double | 6378136.3 | reference radius [m] (default 6378136.3, overridable) |
| OverSample | 1 x 1 | double | 2 | quadrature-grid refinement factor (boundary resolution grows linearly, cost quadratically) |

| output | description |
|---|---|
| k | (P,1) double the kernel, IDX ordering, ready for shSeries.basinAverage or b' * x like any other kernel |
| info | (1,1) struct fields leakage (1,1) double ||k - kExact||_M noise (1,1) double sqrt(k' N k) alpha (1,1) double the trade-off actually used kExact (P,1) double the indicator it started from areaFraction (1,1) double the region's area / 4pi gain (1,1) double k' kExact / (kExact' kExact) - how much of the region's own signal survives. The constraint holds it at 1; a value drifting off 1 means the constraint could not be applied, e.g. a degenerate region |

*Example*

```
    idx = shLowLevel.shIndex(60);
    for a = logspace(-2, 4, 20)                % sweep, then choose
        [~, s(end+1)] = shLowLevel.sensitivityKernel(idx, basin, ...
            Alpha = a, Noise = sigmaVec);
    end
    loglog([s.leakage], [s.noise])             % the L-curve

  See also shLowLevel.basinKernel, shLowLevel.basinDeconvolve,
  shLowLevel.leakageCorrect.
```

### shLowLevel.shSynthesisGradient

`[dq, info] = shLowLevel.shSynthesisGradient(C, S, GM, R, latDeg, lonDeg, Name=Value)`

North/east/up gradient of any synthesized quantity. [DQ, INFO] = shLowLevel.shSynthesisGradient(C, S, GM, R, LAT, LON, Quantity ("geoid")=..., Units ("perMeter")=..., Mode ("grid")=...) returns the spatial gradient of the field that shLowLevel.shSynthesis would produce from the SAME coefficients and the SAME options, resolved in the local north-east-up frame:

| input | size | type | default | description |
|---|---|---|---|---|
| C | n x m | double | required | cosine Stokes coefficients, C(n+1, m+1) indexing |
| S | n x m | double | required | sine Stokes coefficients, S(n+1, m+1) indexing |
| GM | 1 x 1 | double | required | double [m^3/s^2] |
| R | 1 x 1 | double | required | double [m] reference radius |
| latDeg | 1 x : | double | required | double [deg] GEOCENTRIC latitude |
| lonDeg | 1 x : | double | required | double [deg] longitude |

| input | size | type | default | description |
|---|---|---|---|---|
| Quantity | 1 x 1 | string | "geoid" | any quantity of shLowLevel.kernelFactors: "none", "geoid", "potential", "gravity_anomaly", "gravity_disturbance", "gravity_gradient_rr", "ewh", "surface_dens... |
| kn | n x m | double | [] | load Love numbers k'_n, degrees 0..nmax; REQUIRED by the mass quantities, never hardcoded |
| hn | n x m | double | [] | load Love numbers h'_n (deformation_up only) |
| Nmin | 1 x 1 | double | 0 | degrees below nmin are zeroed. Degree 0 and 1 have no horizontal gradient beyond the degree-1 tilt, but they DO contribute to G.up - so this is not a free ch... |
| Nmax | 1 x 1 | double | NaN | truncation degree; NaN uses all supplied degrees |
| Height | 1 x 1 | double | 0 | evaluation height [m] above the evaluation surface; r enters both the kernel attenuation and the metric factors. Rejected for surface quantities, by kernelFa... |
| Surface | 1 x 1 | string ∈ ["sphere", "ellipsoid"] | "sphere" | "sphere" \| "ellipsoid", as in shLowLevel.shSynthesis: "ellipsoid" evaluates at r(phi) + Height with r(phi) from shLowLevel.ellipsoidRadius, and needs a quant... |
| SemiMajorAxis | 1 x 1 | double | 6378137 | ellipsoid semi-major axis [m], WGS84; used only with Surface = "ellipsoid" |
| Flattening | 1 x 1 | double | 1/298.257223563 | ellipsoid flattening, WGS84; used only with Surface = "ellipsoid" |
| rho_ave | 1 x 1 | double | 5517 | mean Earth density [kg/m^3] (mass kernels) |
| rho_water | 1 x 1 | double | 1000 | water density [kg/m^3] (ewh kernel) |
| Mode | 1 x 1 | string ∈ ["grid", "points"] | "grid" | "grid" evaluates the nlat x nlon outer product; "points" takes equal-length LAT/LON and evaluates pairwise - the mode a particle advection needs when it step... |
| Units | 1 x 1 | string ∈ ["perMeter", "perRadian"] | "perMeter" | "perMeter" is the true gradient [Q]/m. "perRadian" multiplies every component by r, giving dQ/dphi, (1/cos phi) dQ/dlambda and dQ/d(ln r) - the form a lat/lo... |
| MaxMemGB | 1 x 1 | double | 4 | memory budget [GB] for the Legendre stack; latitudes are processed in bands so nmax is not limited by it |

| output | description |
|---|---|
| dq | (1,1) struct each field is (nlat x nlon) double for Mode="grid" or (npts x 1) double for "points". Fields north the north component of the gradient east the east component. NaN where \|lat\| = 90 - the east direction does not exist at a pole, and the cos(phi) division there returns a finite, meaningless number if it is not caught up the radial component; [] for surface quantities (see above) |
| info | (1,1) struct fields quantity (1,1) string what was differentiated units (1,1) string the units of dq r (1,nr) double evaluation radius [m]: nr = 1 on the sphere; on the ellipsoid one radius per latitude (Mode="grid") or per point (Mode="points") surface (1,1) string "sphere" \| "ellipsoid", as used nmin, nmax (1,1) double the degree range used hasUp (1,1) logical whether dq.up is filled lat (1 x nlat) double latitudes actually used lon (1 x nlon) double longitudes actually used latType (1,1) string always "geocentric" here: the latitudes this function takes are geocentric, and Surface moves the radius, not the latitude. shCoefficients.gradient overwrites lat and latType when it was handed geodetic input, so the convention travels with the numbers instead of being remembered by the caller |

*Example*

```
    % a zonal degree-2 pattern: the east gradient must vanish exactly
    C = zeros(3); S = zeros(3); C(3,1) = 1;              % C_20
    lat = -80:10:80; lon = 0:20:340;
    dq = shLowLevel.shSynthesisGradient(C, S, 3.986004415e14, ...
            6378136.3, lat, lon, Quantity = "none", Units = "perRadian");
    max(abs(dq.east(:)))            % 0
    quiver(lon, lat, dq.east, dq.north)

  References
    Wahr, Molenaar & Bryan (1998), JGR 103(B12) - the degree factors.
    Holmes & Featherstone (2002), J Geod 76 - the Legendre recursion
    whose exact latitude derivative shLowLevel.legendreALFDeriv carries.

  Python-validated before implementation (tools/dev/validate_gradient.py):
  all three components against 5-point finite differences of an
  independent scalar synthesis at 1e-12 (the difference floor), the
  radial identities d(potential)/dr = -gravity_disturbance and
  d(gravity_disturbance)/dr = -gravity_gradient_rr at 4e-16, and
  single-coefficient closed forms at 2e-16.

  See also SHSYNTHESIS, SHSYNTHESISDEFORMATION, SHSYNTHESISGRADIENTTENSOR,
  KERNELFACTORS, LEGENDREALFDERIV.
```

### shLowLevel.signalVarianceKaula

`[ab, info] = shLowLevel.signalVarianceKaula(Cs, Ss, epochs, Name=Value)`

Cyclostationary Kaula signal variance (VDK input). [AB, INFO] = shLowLevel.signalVarianceKaula(CS, SS, EPOCHS) estimates the degree-dependent signal-variance approximation sigma_M(l) = a * l^b (Horvath et al. 2018, Eq. 2) per CALENDAR month from a coefficient series - the cyclostationary signal model of the VDK/VADER decorrelation filter. Feed a PRE-FILTERED series (e.g. DDK4, as in the paper): the estimator reads signal, so the input must not be stripe-dominated.

| input | size | type | default | description |
|---|---|---|---|---|
| Cs | : x : x : | double | required | stacked cosine coefficients, C(n+1, m+1, T) |
| Ss | : x : x : | double | required | stacked sine coefficients, S(n+1, m+1, T) |
| epochs | : x 1 | double | required | decimal years (calendar month via the fraction) |
| LRange | 1 x 2 | double | [10, 60] | (1 x 2) degree range of the log-log fit; the lower bound excludes the low degrees where real signal is not Kaula-like and the chi-square dof are small |

| output | description |
|---|---|
| ab | (12 x 2) double [a, b] per calendar month (Jan..Dec); evaluate as sigmaM = ab(mo, 1) * l.^ab(mo, 2) info (1 x 1) struct nPerMonth (12 x 1 epochs used), LRange, abAll (T x 2 per-epoch fits, for inspection) |

*Example*

```
    [ab, ~] = shLowLevel.signalVarianceKaula(ts.Cs, ts.Ss, ts.epochs);
    sigM = ab(4, 1) * (2:96)'.^ab(4, 2);   % April signal model
```

### shLowLevel.slepianProject

`[a, rec] = shLowLevel.slepianProject(Cs, Ss, G, idx)`

Project coefficient sets onto a Slepian basis (and back). [A, REC] = shLowLevel.slepianProject(CS, SS, G, IDX) projects one or many coefficient sets onto the Slepian tapers G from shLowLevel.slepianBasis: A = G' * X where X stacks the (C, S) coefficients in IDX ordering. This is the missing application half of the concentration problem (roadmap item 8): a regional time series analysis estimates only the ~Shannon leading Slepian coefficients instead of all P spherical-harmonic ones - the well-posed alternative to...

| input | size | type | default | description |
|---|---|---|---|---|
| Cs | n+1 x n+1 x T | double | required | cosine coefficients (T = 1 for a single set) |
| Ss | n+1 x n+1 x T | double | required | sine coefficients |

| input | size | type | default | description |
|---|---|---|---|---|
| G | : x : | double | required | Slepian taper matrix from slepianBasis (any K) |
| idx | 1 x 1 | struct | required | struct the SAME shLowLevel.shIndex the basis was built with (P must match size(G, 1)) |

| output | description |
|---|---|
| a | (K x T) Slepian-domain coefficient time series |
| rec | (1 x 1) struct with fields Cs (n+1 x n+1 x T) reconstructed cosine coefficients Ss (n+1 x n+1 x T) reconstructed sine coefficients |

*Example*

```
idx = shLowLevel.shIndex(30, Nmin = 0);
[G, lam, info] = shLowLevel.slepianBasis(idx, @(la, lo) la > 60);
K = round(info.shannon);
a = shLowLevel.slepianProject(ts.Cs, ts.Ss, G(:, 1:K), idx);
plot(ts.epochs, a')          % regional modes vs time
```

### shLowLevel.standardChain

`[ts, rep] = shLowLevel.standardChain(folder, Name=Value)`

Canonical GRACE/GRACE-FO post-processing in one call. [TS, REP] = shLowLevel.standardChain(FOLDER) encodes the standard chain as a single, correctly ordered entry point: 1. read all monthly solutions from FOLDER (shSeries.fromFolder) 2. replace C20/C30 with the SLR values (TN-14) 3. add the degree-1 (geocenter) coefficients (TN-13) 4. optionally subtract a GIA trend field 5. filter (Gaussian radius, a DDK, or a shLowLevel.designFilter W) The order matters (TN corrections BEFORE filtering, GIA on the corrected se...

| input | size | type | default | description |
|---|---|---|---|---|
| folder | 1 x 1 | string | required | string folder of monthly .gfc(.gz) files |
| Pattern | 1 x 1 | string | "*.gfc*" | file pattern for shSeries.fromFolder |
| Truncate | 1 x 1 | double | NaN | truncate to this nmax after reading |
| TN14 | 1 x 1 | logical | true | apply SLR C20/C30 |
| TN14File | 1 x 1 | string | "" | explicit TN-14 path (default: dataFolder copy) |
| Degree1 | 1 x 1 | string ∈ ["GFZ", "CSR", "JPL", "none"] | "GFZ" | TN-13 provider "GFZ" \| "CSR" \| "JPL" \| "none" |
| Degree1File | 1 x 1 | string | "" | explicit TN-13 path (default: dataFolder copy) |
| GIA | n x m | — | [] | shLowLevel trend field (shCoefficients, [1/yr]) subtracted as (t - GIAEpoch) * GIA |
| GIAEpoch | 1 x 1 | double | NaN | reference epoch of the GIA trend (NaN: mean epoch of the series) |
| Filter | n x m | — | "gauss300" | "none" \| "gaussN" (N = radius [km]) \| "DDKn" (n = 1..8, fetched on demand) \| a W struct from shLowLevel.readDDK / designFilter |
| Tolerance | 1 x 1 | double | 0.05 | maximum \|epoch difference\| [yr] accepted when matching a TN-13/TN-14 record to a solution |
| OnMissing | 1 x 1 | string ∈ ["drop", "error"] | "drop" | what to do with solution epochs the correction tables do not reach. The tables ALWAYS trail the solutions - a provider publishes TN-13 weeks after the monthl... |
| Quiet | 1 x 1 | logical | false | suppress progress output |

| output | description |
|---|---|
| ts | (1,1) shSeries the processed series (full history) |

| output | description |
|---|---|
| rep | (1,1) struct fields files (1,T) string the files read, in epoch order steps (1,K) string the stages applied, with parameters nmax (1,1) double the degree the chain ran at epochs (T,1) double the epoch grid [decimal years] version (1,1) string shLowLevel.version at run time created (1,1) string when the run happened |

*Example*

```
[ts, rep] = shLowLevel.standardChain("D:/grace/monthly", ...
    Filter = "DDK3", Degree1 = "GFZ");
disp(rep.steps')
```

### shLowLevel.synthesisPoints

`val = shLowLevel.synthesisPoints(C, S, GM, R, latDeg, lonDeg, r, Name=Value)`

Gravity field quantities at arbitrary 3-D points. VAL = shLowLevel.synthesisPoints(C, S, GM, R, LATDEG, LONDEG, RGEO) evaluates the spherical harmonic expansion POINTWISE at positions given by equal-length vectors of geocentric latitude [deg], longitude [deg] and geocentric radius [m] - satellite altitudes, scattered stations, profiles - including the upward continuation (R/r)^n that grid synthesis on the sphere cannot provide: potential T = GM/r sum_n (R/r)^n sum_m Pnm (C cos + S sin) disturbance ...

| input | size | type | default | description |
|---|---|---|---|---|
| C | n x m | double | required | cosine Stokes coefficients, C(n+1, m+1) indexing |
| S | n x m | double | required | sine Stokes coefficients, S(n+1, m+1) indexing |
| GM | 1 x 1 | double | required | gravitational constant times Earth mass [m^3/s^2] (default 3.986004415e14, overridable) |
| R | 1 x 1 | double | required | reference radius [m] (default 6378136.3, overridable) |
| latDeg | : x 1 | double | required | geocentric latitudes [deg] |
| lonDeg | : x 1 | double | required | longitudes [deg], lon in [0, 360) |
| r | : x 1 | double | required | reference radius [m] (default 6378136.3, overridable) |
| Quantity | 1 x 1 | string ∈ ["potential", "disturbance", "anomaly"] | "potential" | "potential" [m^2/s^2] \| "disturbance" \| "anomaly" (both [m/s^2]; multiply by 1e5 for mGal) |
| Nmin | 1 x 1 | double | 2 | first degree included (0 evaluates the full field including the central term) |
| Method | 1 x 1 | string ∈ ["auto", "direct", "clenshaw"] | "auto" | "auto" \| "direct" \| "clenshaw" - which summation to use; "auto" chooses "clenshaw" once the point count reaches ClenshawFrom |
| ClenshawFrom | 1 x 1 | double | 1 | point count at or above which "auto" switches to "clenshaw". The default is 1 because MATLAB has NO crossover: benchPointsCrossover timed both paths at the s... |
| MaxMemGB | 1 x 1 | double | 4 | memory budget for the "direct" path's Legendre block; the chunk size follows from it and from nmax. Ignored by "clenshaw", which never allocates one |

| output | description |
|---|---|
| val | (P,1) double quantity at each point |

*Example*

```
dg = shLowLevel.synthesisPoints(g.C, g.S, g.GM, g.R, ...
    [10; 12], [100; 101], (g.R + 450e3) * [1; 1], ...
    Quantity = "disturbance") * 1e5;   % mGal at 450 km
```

### shLowLevel.testDataDir

```
d = shLowLevel.testDataDir()
```

Resolve the sample/fixture data folder (relocatable). D = shLowLevel.testDataDir() returns, in order of precedence: 1. getenv('SHX_TESTDATA_FOLDER') if set and a folder 2. getpref('shAnalysis','TestDataFolder') if set and a folder 3. <toolbox>/tests/test_data the in-repo default Same contract as tests/shxTestDataDir (which the test suites use); this packaged twin exists because TOOLBOX code also consults the shipped samples - readDDK's "DDK3" name form and the demos - and must not depend on...

| output | description |
|---|---|
| d | (1 x :) char folder path (existence NOT enforced for the fallback - callers keep their own diagnostics) |

*Example*

```
    W = shLowLevel.readDDK("DDK3");   % resolves the shipped sample
    f = fullfile(shLowLevel.testDataDir(), 'test_static.gfc');
```

### shLowLevel.twsChain

```
[out, rep] = shLowLevel.twsChain(folder, gravisFolder, Name=Value)
```

The validated GravIS terrestrial-water-storage basin chain. [OUT, REP] = shLowLevel.twsChain(FOLDER, GRAVISFOLDER, kn=KN) reproduces guide chapter V8: gravisL2B corrections WITH the ICE-6G_D GIA rate (the GravIS TWS product carries this correction even though its Technical Note omits it - subtracting it took the trend RMS from 0.225 to 0.032 cm/yr), a DDK3 filter as the declared stand-in for GravIS' VDK5/VDK3 blend, and cos-weighted 1-degree basin means in cm EWH computed by per-basin bounding-box synthesis with...

| input | size | type | default | description |
|---|---|---|---|---|
| folder | 1 x 1 | string | required | string monthly GSM-2_*.gfc folder |
| gravisFolder | 1 x 1 | string | "" | string GravIS aux folder; "" (default) uses the shipped data/gravis copies (see gravisL2B); must also hold BasinFile and, for the default filter, the DDK bi... |
| kn | n x m | double | [] | load Love numbers (degree,kn) - REQUIRED, the empty default errors: no frame is assumed |
| BasinFile | 1 x 1 | string | "basins_rivbas.json" | GravIS GeoJSON of the target basins ([lon lat] rings, converted internally; https://gravis.gfz.de/basins/rivbas) |
| Basins | 1 x 1 | string | strings(0, 1) | ) feature names to process; the empty default processes every feature in BasinFile |
| Filter | n x m | — | "DDK3" | "none" \| "gaussN" \| "DDKn" (n = 1..8) \| a W struct from shLowLevel.readDDK |
| DDKFolder | 1 x 1 | string | "" | folder holding the Wbd_2-120.a_* binaries for the "DDKn" shorthand (default: GRAVISFOLDER, then <dataFolder>/DDK, where shLowLevel.fetchDDK stores them) |
| GIAFile | 1 x 1 | string | "GRAVIS-2B_COSTG_0200_GIA_ICE-6G_D_VM5a_0001.gz" | ("GRAVIS-2B_COSTG_0200_GIA_ICE-6G_D_VM5a_0001.gz") "" disables the GIA step - and reproduces the 7x larger trend RMS documented in V8 |
| SpanEnd | 1 x 1 | double | Inf | the TWS validation uses the full record |
| GridStep | 1 x 1 | double | 1 | basin-box synthesis step [deg] |
| GM | 1 x 1 | double | 3.986004415e14 | , R (6378136.3) |
| R | 1 x 1 | double | 6378136.3 | reference radius [m] (default 6378136.3, overridable) |
| Quiet | 1 x 1 | logical | false | suppress progress output |

| output | description |
|---|---|
| out | (1,1) struct fields name (K,1) string basin names, in the row order below epochs (T,1) double the epoch grid [decimal years] series (K x T) double basin means [cm EWH] trend (K,1) double per-basin trend [cm/yr] amplitude (K,1) double annual amplitude [cm] phase (K,1) double annual phase [rad] |
| rep | (1,1) struct fields steps (1,K) string the stages applied, with parameters filter (1,1) string the filter actually used l2b (1,1) struct the shLowLevel.gravisL2B report version (1,1) string shLowLevel.version at run time created (1,1) string when the run happened |

*Example*

```
kn = readmatrix("loadLoveNumbers_Gegout97.txt", FileType="text", ...
    NumHeaderLines=2);
[out, rep] = shLowLevel.twsChain("E:/series/COSTG", "E:/GravIS", ...
    kn = kn, Basins = ["Amazonas", "Congo", "Danube"]);
plot(out.epochs, out.series); legend(out.name)
```

### shLowLevel.vdkApply

`[xf, info] = shLowLevel.vdkApply(x, N, degOfPar, sigM, Name=Value)`

Apply the VDK/VADER decorrelation filter to one solution. XF = shLowLevel.vdkApply(X, N, DEGOFPAR, SIGM) computes

| input | size | type | default | description |
|---|---|---|---|---|
| x | : x 1 | double | required | double coefficient vector, same ordering as N |
| N | : x : | double | required | double normal-equation matrix (symmetric positive definite) |
| degOfPar | : x 1 | double | required | double spherical-harmonic degree of every parameter (idx.n from shLowLevel.shIndex) |
| sigM | n x m | double | required | double signal sigma per parameter, OR [a, b] evaluating sigma = a * l^b per parameter (one row of signalVarianceKaula's AB) |
| Alpha | 1 x 1 | double | 1 | (1 x 1) filter strength; larger = smoother (paper Table 1 maps alpha to mean Gaussian radii for the ITSG-Grace2014 case - the mapping is series-specific) |

| output | description |
|---|---|
| xf | (P x 1) double filtered coefficient vector info (1 x 1) struct alpha, P, cholOK (true when (N + alpha M) factorized cleanly) |

*Example*

```
idx = shLowLevel.shIndex(96, Nmin = 2);
snx = shLowLevel.readSINEX(f, Index = idx);
xf = shLowLevel.vdkApply(snx.x, snx.M, idx.n, ab(mo, :), Alpha = 1);
```

### shLowLevel.version

`v = shLowLevel.version()`

Toolbox metadata: name, version, date, root path, provenance. V = shLowLevel.version() returns the toolbox metadata as a struct. The version string and date are PARSED FROM Contents.m (the MATLAB toolbox convention line "% Version x.y.z ... dd-Mmm-yyyy"), so there is a single source of truth also honoured by MATLAB's own ver('shAnalysis'). Called without an output, prints a summary.

| output | description |
|---|---|
| v | (1,1) struct fields: .Name (1,1) string "shAnalysis" .Version (1,1) string e.g. "2.5.1" .Date (1,1) string release date from Contents.m .Root (1,1) string toolbox root folder .Provenance (1,1) string authorship line |

*Example*

```
    v = shLowLevel.version();
    fprintf("%s %s (%s)\n", v.Name, v.Version, v.Date)
```

### shLowLevel.writeFrames

`shLowLevel.writeFrames(frames, filename, Name=Value)`

Write ready-made RGB frames to a video container. shLowLevel.writeFrames(FRAMES, 'out.avi', FrameRate (4)=..., ...) writes an (H x W x 3 x T) uint8 stack to an MPEG-4 or Motion-JPEG AVI file and, by default, a provenance sidecar beside it.

| input | size | type | default | description |
|---|---|---|---|---|
| frames | H x W x 3 x T | uint8 | required | uint8 RGB frames, e.g. from getframe().cdata |
| filename | 1 x 1 | string | required | string .mp4 or .avi target path |
| FrameRate | 1 x 1 | double | 4 | frames per second |
| Sidecar | 1 x 1 | logical | true | write <file>.provenance.json alongside the output (nFrames, frameRate and frameSize are always recorded) |
| SidecarExtra | 1 x 1 | struct | struct() | ) extra fields merged into the sidecar |

*Example*

```
    F = uint8(zeros(120, 240, 3, 5));
    shLowLevel.writeFrames(F, fullfile(tempdir, 'demo.avi'), FrameRate = 2);

  See also SHLOWLEVEL/WRITEANIMATION, SHSERIES/TOGRID.
```

## Toolbox root

### setup_shAnalysis

`summary = setup_shAnalysis(Name=Value)`

One-call setup: path + staged data downloads. setup_shAnalysis adds the toolbox to the path for THIS session only setup_shAnalysis(Permanent (false) = true) also persists it (savepath) setup_shAnalysis(Download ("none") = "core") + TN-14 and TN-13 files setup_shAnalysis(Download = "filters") + DDK (3) filter matrices setup_shAnalysis(Download = "starter") + ITSG starter months setup_shAnalysis(Download = "all") + EVERY monthly ITSG solution setup_shAnalysis(DryRun (false) =...

| input | size | type | default | description |
|---|---|---|---|---|
| Permanent | 1 x 1 | logical | false | addpath + savepath; if savepath fails (no write permission on pathdef.m) a warning explains the startup.m alternative |
| Download | 1 x 1 | string ∈ ["none", "core", "filters", "starter", "all"] | "none" | "none" \| "core" \| "filters" \| "starter" \| "all" "all" (v3.27) extends "starter" to the FULL ITSG monthly archive (2002 -> present, both releases, at Nmax; se... |
| DDK | 1 x : | double | 3 | subset of 1..8 for the "filters" level |
| Providers | 1 x : | string ∈ ["GFZ", "CSR", "JPL"] | ["GFZ", "CSR", "JPL"] | TN-13 providers for "core" |
| Months | 1 x : | string | ["2008-04", "2025-12"] | months for "starter" |
| Nmax | 1 x 1 | double ∈ [60, 96] | 96 | degree of the ITSG starter files (60 or 96) |
| Docs | 1 x 1 | logical | false | builddocsearchdb on html/ so the toolbox pages appear in the MATLAB help-browser search. Covers the narrative pages and the generated html/ref/ reference pag... |
| DryRun | 1 x 1 | logical | false | report the plan only - zero side effects |

| input | size | type | default | description |
|---|---|---|---|---|
| DataFolder | 1 x 1 | string | "" | persistent data folder, applied BEFORE any fetcher runs |
| Proxy | 1 x 1 | string | "" | per-call proxy URL, e.g. "http://proxy:8080" (empty: MATLAB Web Preferences) |
| Update | 1 x 1 | logical | false | forward to the fetchers: refresh existing files with a safe, parse-verified swap (temporal files like TN-13/TN-14 grow monthly) |
| SeriesFolder | 1 x 1 | string | "" | folder of monthly .gfc files, exported as the SHX_SERIES_FOLDER environment variable AND persisted via setpref('shAnalysis', 'SeriesFolder', ...) - preferenc... |
| MasconFile | 1 x 1 | string | "" | a mascon .nc, exported as SHX_MASCON_FILE and persisted as preference 'MasconFile'. This enables the opt-in readMascon check against a REAL product: the synt... |
| GravisFolder | 1 x 1 | string | "" | GravIS aux folder for the validated chains, exported as SHX_GRAVIS_FOLDER + preference 'GravisFolder' (opt-in real-data tests) |
| DDKFolder | 1 x 1 | string | "" | DDK filter binaries folder, exported as SHX_DDK_FOLDER + preference 'DDKFolder' |
| TestDataFolder | 1 x 1 | string | "" | folder holding the test fixtures when they live outside the repository (exported as SHX_TESTDATA_FOLDER + preference; the test suites resolve it via tests/sh... |
| Quiet | 1 x 1 | logical | false | suppress progress output |

| output | description |
|---|---|
| root | (1,1) string toolbox root folder |
| pathAction | (1,1) string "already-on-path" \| "added-temporarily" \| "added-permanently" \| "planned" |
| dataFolder | (1,1) string download target root |
| plan | (1,:) string human-readable action list |
| fetched | (1,:) string files newly downloaded (verified) |
| skipped | (1,:) string files already present (re-verified) |
| failed | (1,:) string files that failed (download or parse) |
| testData | (1,:) string opt-in test-data variables that were set, if any (SHX_SERIES_FOLDER, SHX_MASCON_FILE). setenv lasts for the session only - put the same lines in startup.m to keep them |
| ok | (1,1) logical isempty(failed) |

*Example*

```
    setup_shAnalysis(Permanent = true, Download = "core")
    setup_shAnalysis(Download = "filters", DDK = [2 3 5])
    s = setup_shAnalysis(DryRun = true, Download = "starter")
    setup_shAnalysis(SeriesFolder = "D:/grace/itsg", ...
        MasconFile = "D:/grace/GRCTellus_JPL.nc")   % opt-in test data
```

### runAllTests

```
results = runAllTests(Name=Value)
```

Run the complete shAnalysis v2 validation suite. RESULTS = RUNALLTESTS runs, in order: tests/testCorrectness.m numerics vs. golden/legacy/analytic values tests/testScience.m regression against PUBLISHED values (defining constants, load Love numbers, the closed-form EWH kernel, geocenter amplitudes) - so a consistently wrong formula has somewhere to show up tests/testContract.m error IDs, immutability, dimensions tests/testRobustness.m edge cases, degenerate inputs, file I/O documented patch: kern...

| input | size | type | default | description |
|---|---|---|---|---|
| SkipPerformance | 1 x 1 | logical | false | logical, default false |
| LogFile | n x m | — | "" | text, default fullfile(tests,'runAllTests_results.txt') |

| output | description |
|---|---|
| results | (1,N) matlab.unittest.TestResult full result array (pass/fail/duration per test; log written to LogFile) |

### demo_shAnalysis

```
reg = demo_shAnalysis(cases, Name=Value)
```

Selectable demonstrations of every toolbox capability. demo_shAnalysis runs the "core" set (D01 D02 D04 D05 D15) demo_shAnalysis("all") runs every case demo_shAnalysis("list") prints the case table, returns registry demo_shAnalysis(["D04","D15"]) runs selected cases demo_shAnalysis(..., Visible (true)=false) hidden figures (CI/smoke tests) demo_shAnalysis(..., OutDir (string(tempdir))=tempdir) target for D16 file exports demo_shAnalysis(..., StopOnError (false)=true) rethrow ins...

| input | size | type | default | description |
|---|---|---|---|---|
| cases | 1 x : | string | "core" | string demo IDs to run, e.g. "D01" or ["D01" "D13"]; "core" expands to the curated smoke set, "all" runs the full registry in order |
| Visible | 1 x 1 | logical | true | logical show figures (false keeps them off-screen for batch/CI runs) |
| OutDir | 1 x 1 | string | string(tempdir) | string output folder for exported files; "" = a fresh folder under tempdir |
| StopOnError | 1 x 1 | logical | false | logical true stops at the first failing case; false (default) prints FAILED and continues - the fail-and-continue contract the suite exercises |

| output | description |
|---|---|
| reg | (D x 3) table demo registry: id, title, exercised API |

# Appendix

## A. Error-identifier conventions

Namespaces: `shLowLevel:<function>:<what>` for package functions, `shCoefficients:/shSeries:/shClimatology:` for class methods. Every guard has a unit test; every message names the offending quantity. Instrumentation asserts (nonFiniteVCE, nonFiniteEig, nonFiniteOperator, allZeroVariance) remain in the code as a permanent safety net after the v2.1 debugging campaign.

## B. Known limitations (documented, tested where applicable)

- TN-13 verified against the real GFZ, CSR and JPL RL06.3 files (v2.5; 256 paired months each, cross-provider C10 correlation 0.995); SINEX against ITSG monthly files and synthetic fixtures — COST-G/other SINEX dialects may need parser tweaks (drop files into tests/test_data, the discovery tests pick them up).
- DDK: all eight released Wbd binaries parsed and gain-ordering validated (v2.5); a discovery test re-checks whatever subset is present locally. Other BIN dialects untested.
- readMascon validated against a synthetic JPL-style fixture; real product files were not openly reachable at build time (auth/503) — verify against yours (units, GIA convention, CRI flag).
- GIA models treated as exact; AR(1) correction is first-order (Kendall-corrected $r_1$ since v2.5, ~3% residual optimism).
- Multi-center combination: common-mode errors invisible → posterior sigmas a lower bound.
- Basin sigma: deterministic-part errors correlated across epochs (pointwise 1-sigma reported).
- Streaming gz SINEX needs the JVM (fallback: gunzip to temp).
- ICGEM catalogue parsers are fixture-tested against real page captures (2026-08-07); a future site redesign would need refreshed fixtures (shLowLevel:listICGEM:parseFailed errors loudly).

## C. License & provenance

The toolbox is released under the MIT license (LICENSE in the package root; copyright Matthias Weigelt). The bundled DDK3 Wbd file carries its own MIT license from its upstream repository; ITSG/GFZ files in tests/test_data remain the property of their providers and are included as test fixtures only.

All code, tests, documentation and this guide: developed by Matthias Weigelt with the help of Claude (Fable 5), 2026-08-07 (v1 through v2.4.2 in working sessions the same week). Every numerical method was independently validated in Python before MATLAB implementation; MATLAB execution itself happens on the user's side with runAllTests as the acceptance gate (118/118 green at v2.1; 134 at v2.2.2; 167 at v2.4.1; 168 at v2.4.2; 179 at v2.5 — v2.5 adds the exact constrained posterior, the basin deterministic-sigma term, the Kendall AR(1) correction, shLowLevel.readLoveNumbers, setup_shAnalysis/shLowLevel.fetchTN with verified staged downloads, real CSR/JPL TN-13 + GSFC TN-14 fixtures and the DDK-pack validation).